

TECHNISCHE UNIVERSITÄT DRESDEN

FAKULTÄT INFORMATIK
INSTITUT FÜR SYSTEMARCHITEKTUR
PROFESSUR FÜR DATENBANKEN
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Aktive cache-bewusste Hardware-Adaption: Eine Cache-Engine für Trie-basierte Indexstrukturen

Benjamin-Elias Probst
(Geboren am 11. April 1996 in Potsdam)

Hochschullehrer: apl. Prof. Dr.-Ing. Dirk Habich
Betreuer: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, 14. August 2026

Aufgabenstellung

Aktive cache-bewusste Hardware-Adaption:

Eine Cache-Engine für Trie-basierte Indexstrukturen

Kontext und Motivation

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanksysteme. Die Professur für Datenbanken an der TU Dresden adressiert die hardware-nahe Optimierung solcher Systeme. Seit der Einführung des Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Techniken und verwandter Algorithmenfamilien vorgeschlagen — von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation —, doch nutzt fast jede Veröffentlichung eigene Datenstrukturen, verschiedene Lastprofile und inkonsistente Messmethodik. Ergebnisse sind dadurch über Veröffentlichungen hinweg schwer vergleichbar. Zuletzt sind die Gewichtungen über Verbesserungen einzelner Algorithmus-Bestandteile derzeit nicht trennbar.

Problemstellung

Suchverfahren — von ausgewogenen Suchbäumen (B-/B⁺-Bäume) über Tries und Radix-Bäume bis zu spezialisierten, eigenentwickelten Konstrukten — spannen einen großen, aber konzeptionell einheitlichen, *Entwurfsraum* (*design space*) auf: Bereits eine zwei- bzw. dreikomponentige Datenstruktur erlaubt astronomisch viele Entwürfe, und ein konkreter Algorithmus ist ein einzelner Punkt in diesem Raum, beschrieben durch viele einzelne Entwurfsentscheidungen — wie Knoten, Verweise und Daten repräsentiert, durchlaufen, im Speicher angeordnet, und alloziert werden [IZA⁺18, IZH⁺18]. Cache-line-bewusstes Verhalten entsteht dabei nicht an einer einzelnen dieser Stellen, sondern an *vielen verschiedenen strukturellen Stellen* eines Algorithmus bzw. dessen Speicherverwaltung und aus dem Zusammenspiel multipler konzeptioneller Schichten eines Algorithmus. Der Stand der Technik umfasst zahlreiche ausgereifte Verfahren, doch optimiert jedes nur einzelne dieser Stellen und belegt seinen Nutzen in einer eigenen Messumgebung, was den Fortschritt über die Verfahren hinweg schwer vergleichbar macht. Es fehlt ein gemeinsames, generisches Konzept und **Prinzip**, das cache-line-bewusstes Verhalten, neben anderen Messschwerpunkten, an den verschiedenen Stellen eines Suchalgorithmus — austauschbar, systematisch und fair — messen und auswerten kann.

Zielsetzung

Ziel der Arbeit ist es, ein solches Prinzip (ein Rahmenwerk) zu finden, zu entwerfen und prototypisch als Experimentier-Framework umzusetzen, das mit Konzentrationspunkt auf eine dynamische Zusammensetzung der verteilten Algorithmus-Konzepte: (i) cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegt, (ii) Algorithmen des Standes der Technik als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert, (iii) cache-line-bewusstes Verhalten sowohl an den einzelnen direkt und indirekt betroffenen Stellen als auch für den gesamten Algorithmus auf gemeinsamer CPU-only-Basis misst und auswertet und (iv) auf Basis der feingliedrigen Messergebnisse den besten Gesamtalgorithmus als eigenständige Binary hinter einem einheitlichen Suchalgorithmus-Interface ausliefert. Als Prüfling für die testweise Eingliederung einer neuen Technologie in den Stand der Technik trägt die Arbeit PRT-ART bei (einen Probst-Redirect-Tree-/ART-Hybriden), der als konzeptionell neuartiger Suchalgorithmus untersucht wird, allerdings nur einige neue Aspekte innerhalb der Klassifizierung von Suchalgorithmen liefert und keinen vollständigen Algorithmus.

Die hier geforderte Trennbarkeit der einzelnen Algorithmus-Bestandteile ist in der vorliegenden Arbeit konstruktiv herzustellen und messbar zu machen.

Teilaufgaben

1. Systematische Erfassung der allgemeinen Entwurfsbestandteile von Suchalgorithmen — insbesondere der cache-bewussten — und Identifikation der Stellen, an denen cache-line-bewusste und suchalgorithmus-spezifische Entscheidungen wirksam werden. Dazu zählt die systematische Untergliederung der Entwurfsbestandteile in Entwurfsebenen einer verallgemeinerten Algorithmus-Kategorisierung.

-
2. Rekonstruktion von Entwürfen des Stands der Technik — möglichst aus dem Originalcode der jeweiligen Veröffentlichung und Original Compiler Binary linking — (ART, HOT, Masstree, CoCotrie, START, B²-Tree, Wormhole, SuRF sowie weitere Verfahren) als explizite, vergleichbare Konfigurationen.
 3. Entwurf und Implementierung von PRT-ART als Prüfling mit mehreren eigenen Entwurfsbestandteilen, der gegen den erfassten Stand der Technik antritt.
 4. Erweiterung des Gesamtablaufs zu einer durchgehend deklarativ gesteuerten Kette: Diese beginnt mit einer XML-Beschreibung des Experiments, aus der für das betreffende System ein automatisierter, plattformbezogener System-Bau abgeleitet wird, der seinerseits alle erdenklichen Binary-Permutationen der Entwurfsbestandteile erzeugt. Deren Messergebnisse werden auf die beste gemessene System-Binary und System-Binary-Rekombination hin ausgewertet; eine finale Messung der einen Binary bzw. hybrider Binary-Rekombinationen erzeugt eine Auswertung über CSV-/xlsx-Dateien, die abschließend in LaTeX und Diagramme übersetzt wird. Dass deklarative Konfigurationsbeschreibungen den automatisierten Aufbau modularer Systemvarianten steuern, ist ein etabliertes Konstrukt [CE00, ABKS13].
 5. Aufbau einer reproduzierbaren Mess-Pipeline (Binary → CSV → LaTeX → Diagramm) mit profilbewusstem Workload-Routing aller im Stand der Technik gefundenen Workload-Frameworks über *alle* verfügbaren Workloads und Hardware-Counter-Erfassung.
 6. Systematische, möglichst vollständige Messung über die Konfigurationen in drei verpflichtenden Messreihen — (A) Prüfling gegen den Stand der Technik, (B) systematische Variation der Entwurfsentscheidungen, (C) Merge/Regression alter gegen neue Varianten —, wobei jede Reihe in drei Granularitäten erhoben wird: *Micro-Benchmarking* (jeder einzelne Entwurfsbestandteil wird — innerhalb der Suchalgorithmus-Hülle wie der Algorithmus-Kategorie-Hülle — in jeder gemeinsamen Interface-Funktion seiner Kategorie isoliert gegen Lastprofile gemessen; die Algorithmen der Kategorie werden je Interface-Funktion untereinander vergleichend ausgewertet, unter Einhaltung latenzbewusster Best Practices der Datenbankforschung), *Makro-Benchmarking* (die einzelnen Interface-Operationen des Suchalgorithmus als Ganzes — z. B. Einfügen, Punktsuche, Löschen — werden über die Interface-Funktionen der Algorithmus-Kategorie gegen Beispiellasten bzw. Test-Teilbereiche der Algorithmus-Bestandteile gemessen) und *Gesamt-Benchmarking* (etablierte YCSB-ähnliche Lastprofile und vergleichbare Frameworks über jede Rekombination der Entwurfsbestandteile).
 7. Auswertung, an welchen Stellen und in welchen Kombinationen cache-line-bewusste Entscheidungen für welche datentyp-spezifischen Lastmuster vorteilhaft oder nachteilig sind — ermittelt aus der Rangbildung über die drei Benchmarking-Granularitäten und aufbereitet als auswertbare Diagramme und Tabellen.
 8. Automatisierung der Systemerkennung und Binary-Auswahl des optimalsten cache-line aware Algorithmus, anhand von empirisch belegten Messwerten, zusammen mit einer anhand der Messwerte dynamisch gesteuerten cache-line aware Anpassung des Algorithmus-Verhaltens als statische heuristische System-Richtlinie für das Algorithmusverhalten unter verschiedenen Lasten und Operationen. Diese sogenannte Extraktion soll als XML-Lastprofil-Ergebnis für die testende Umgebung für einen Gesamtalgorithmus unter Referenz verschiedener angewendeter Lastprofile abgelegt werden, sie ist für jeden Architekturfokus wiederverwendbar, um automatisch Heuristiken zu erstellen.

Methodik und Evaluation

Die Evaluation umfasst neben dem CPU-only-Kern auch SIMD- und nebenläufige Multi-Threading-Pfade und vergleicht die Konfigurationen gegen etablierte Baselines, darunter eine nicht-baumartige

Hash-Tabelle (`flat_hash_map`) als Gegenprobe innerhalb derselben Gattung. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik — etwa `url`, `dna`, `protein`, `xml`, `tpcds-id` und `trec-terms` — unter variierten Datenvolumina, Keylängen, Präfixüberlappungen, lokaler Dichte, Skew, Treffer-/Fehlerquoten und Value-Größen. Gemessen werden Exact-/Prefix-Lookup, Prefix-Enumeration, Einfügen/Aktualisieren/Löschen unter allen Varianten der C++ standard-Interfaces bekannter `std::map` und `std::vector` Container-Hüllen, p50/p95/p99-Latenz, Durchsatz, Bytes pro Key sowie Hardwarezähler (Cycles, Instructions, L1/L2/LLC- und dTLB-Misses, Branch-Mispredictions). Zunächst wird das Single-Thread-Verhalten untersucht, danach folgen leseparallele Multi-Thread-Szenarien.

Faire Vergleichbarkeit wird durch feste Regeln gesichert: ein gemeinsamer Minimalmodus (Common-Denominator) wird vom PRT-ART-Native-Modus getrennt, alle Verfahren erhalten identische Schlüsselräume und Seeds, fehlende Fähigkeiten werden als N/A ausgewiesen statt verdeckt emuliert, und Compiler, Flags, ISA-Pfad, Allokator sowie Commit-Hash jeder Baseline werden protokolliert. Für die Reproduzierbarkeit wird jede Gesamt-Algorithmus-Konfiguration in mehreren getrennten Läufen vermessen, deren Streuung Teil der Aussage bleibt (Perzentile über HDR-Histogramme statt Mittelung); jeder Datensatz trägt eine Akte aus Quelle, Prüfsumme und Seed-Regel, um den Originalcode zweifelsfrei nachzuweisen.

Umfangsabgrenzung

Der Kernbeitrag ist CPU-fokussiert mit SIMD-Erweiterungen und Multi-Threading, beschränkt sich dabei aber auf leseparallele Verfahren mit serialisiertem Schreiben (multi-read, single-write). Die GPU liegt vorerst außerhalb des Umfangs (zukünftig erweiterbar); volle Schreibparallelität und eine Engine-/Optimierer-Integration sind optionale Erweiterungen. Alle Erweiterungen der Architekturen sind ihrerseits dynamische Erweiterungen gegen den CPU-Standard-Lauf und führen zu tieferen Mess-Permutationen mit multipler Laufzeit. Die systemoptimierte Binary lässt sich dabei wahlweise mit oder ohne Mess-Observer übersetzen — als Debug-, Mess- oder Release-Variante.

Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Aktive cache-bewusste Hardware-Adaption:
Eine Cache-Engine für Trie-basierte Indexstrukturen*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 14. August 2026

Benjamin-Elias Probst
Mat.-Nr. 4510512

Kurzfassung

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanken. Seit dem Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Optimierungen vorgeschlagen—von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation—, doch wurden sie nie in einem einzigen, fairen Messrahmen kombiniert und verglichen: Jede Veröffentlichung nutzt eigene Datenstrukturen, Lastprofile und Messmethodik, wodurch Ergebnisse über Veröffentlichungen hinweg schwer vergleichbar sind. Diese Arbeit stellt ein Cache-Engine-Framework vor, dessen Module auf Binärebene durch eine einzige, ABI-stabile C-Schnittstelle getrennt sind (je Modul genau ein exportiertes Lebewesen); dahinter liegt eine dreischichtige Abstraktion innerhalb der erzeugten Tier-Binary (CacheEngine → ExecutionEngine → SearchEngine), die die Such-API bereitstellt und vom Anatomie-Modell (Gattung, Lebewesen, Organe) abgegrenzt bleibt. Das Framework zerlegt cache-bewusste Suchalgorithmen mithilfe eines Planers und eines CacheEngineBuilder in austauschbare, verteilte (über den Code gestreute) Entwurfsentscheidungen eines großen Entwurfsraums (*design space*) [IZA⁺18, IZH⁺18] und setzt sie durch dynamische Zusammensetzung mittels Compile-Zeit-Permutation (keine Laufzeit-Adaption) wieder zusammen. Strukturell verschiedene Verfahren—von baumartigen Indizes (Tries, B⁺-Bäume) bis zu nicht-baumartigen Hash-Tabellen—werden als Punkte desselben Entwurfsraums hinter einem einheitlichen Interface ausgedrückt; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Entwurfsentscheidungen. Algorithmen des Standes der Technik werden als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert und auf drei Granularitäten gemessen—einzelne Entwurfsentscheidungen, einzelne Interface-Operationen und ganze Algorithmen—unter bekannten Workload-Framework-Lastprofilen (YCSB und weitere, von diesem einen Framework integrierte Frameworks), wobei jedes Modul wahlweise mit oder ohne Mess-Observer für Debug, Messung oder Release gebaut werden kann. Als Prüfling trägt zu dieser Arbeit der PRT-ART bei, ein Probst-Redirect-Tree-/ART-Hybrid mit acht eigenen Bausteinschichten; ein Mess-Treiber mit profilbewusstem Workload-Routing bewertet ihn unter drei verpflichtenden Messreihen gegen dreißig rekonstruierte Stand-der-Technik-Verfahren (acht etablierte Referenzverfahren mit Bausteinen und weitere); Fernziel ist die Auslieferung der besten Rekombination als eigenständige Binary.

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	6
1.3	Forschungsfragen	6
1.4	Zielsetzung und Beiträge	8
1.5	Aufbau der Arbeit	10
2	Suchbäume und Grundlagen	11
2.1	Konzept und Überblick über den Stand der Technik	11
2.2	Hardware: historische Anpassung an die Suchleistung	13
2.2.1	Cache-Organisation: Assoziativität, Fehlzugriffs-Klassen und Parallelität	14
2.2.2	Cache-line-orientierte Layout-Forschung	14
2.2.3	Prefetching	15
2.2.4	Hauptspeicher und Adressübersetzung: DRAM, NUMA und TLB	15
2.2.5	Schreibpfad, Ersetzung und Kohärenz	16
2.2.6	Cache-bewusste und cache-oblivious Datenstrukturen	17
2.3	Workloads und Benchmark-Konventionen	17
2.4	Software-Mittel und Zusammenführung zur Abstraktion	19
2.5	Heuristiken als Entscheidungsregeln	20
3	Konzepte eines cache-bewussten Mess-Systems	21
3.1	Problemstellung und Anforderungen an ein Mess-System	21
3.2	Stand der Technik als Achsen-Instanzen	23
3.3	Lösungskonzept: das Anatomie-Modell	31
3.3.1	Achsen-Komposition und die Lebewesen-Metapher	32
3.3.2	Das Drei-Ebenen-Modell und die drei Gattungen	33
3.3.3	Entwurfsmuster und Übersetzungszeit-Muster	35
3.3.4	Fünf Interface-Schichten: E4 bis E0	37
3.4	Das Mess-System	38
3.4.1	Die Dreiteilung: Organ-, System- und Mess-Achsen	39
3.4.2	Komplex-Achsen: die Ziel-ISA als feste Rekombination	41
3.4.3	Hardware-Erkennung als zweigeteilte Mess-Achse	42
3.4.4	Identitäts-Stempel: eine Zeile je Realm	43
3.4.5	Subsysteme und die Kette von der XML zur Tier-Binary	43
3.5	PRT-ART als Demonstrator	46
3.6	Schärfung durch Heuristiken	47
3.6.1	Heuristik-Schleife und gelernte Lastprofile	47

3.6.2	Die vier Betriebsmodi des Builders	48
3.6.3	Messkurven-Typsystem, Filterkette und Messfehler-Erkennung	49
4	Implementierung und konkrete Algorithmusbestandteile	51
4.1	Drei-Repository-Architektur	51
4.2	PRT-ART im Detail	52
4.3	Achsen-Implementierung und Mess-System-Ebenen	53
4.3.1	Die eine Architektur im Code	54
4.3.2	Vom Profil zur Tier-Binary: Planer, Builder und Mess-Pipeline	54
4.4	Qualitätssicherung: Contract-Tests je Schicht	59
4.5	Realm-Wurzeln im Code: System- und Mess-Achsen	60
5	Evaluation	65
5.1	Methodik	65
5.2	Ergebnisse und Analyse	68
6	Fazit und Ausblick	71
6.1	Beantwortung der Forschungsfragen	71
6.2	Limitierungen	74
6.3	Ausblick	75
	Literaturverzeichnis	77
	Abbildungsverzeichnis	93
	Tabellenverzeichnis	99
A	Vollständige Messreihen	101
A.1	Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)	101
A.2	Bias-Bruch-Matrix	101
A.3	Latenz-Surfaces je Schnittstellen-Funktion	104
A.4	Dreidimensionale Rohdaten-Ansicht	105
A.5	Arbeitsmengen-Sweep	105
A.6	Per-Achsen-Latenz-Attribution	105
A.6.1	Per-Achsen-Observer-Detail	105
A.7	Latenz-Verteilung	108
A.8	Referenzbezogene Auswertung	108
A.8.1	Verhältnis-Matrizen	109
A.8.2	Normalisierte Balken	110
A.9	Achsen-Austauschbarkeit	111
A.10	Limitierungen	113
B	Code-Struktur	121
C	Glossar	127

D	Bausteine-Matrix	131
D.1	T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)	131
D.2	T1 Cache-Speicher-Traversierung (Fanout-Lookup)	134
D.3	T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)	135
D.4	T3 Pfadkompression	136
D.5	T4 Knotentypen (Node-Type-Achse)	137
D.6	T5 Speicher-Layout (Memory Layout)	138
D.7	T6 Allokator-Achse (T6)	140
D.8	T7 Prefetch-Strategie	142
D.9	T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)	144
D.10	T9 Serialisierung (Kodierung und Kompression)	145
D.11	T10 Wert-Handle (Value-Handle)	146
D.12	T11 Index-Organisation	147
D.13	T12 I/O-Dispatch-Achse	148
D.14	T13 Migrations-Strategie-Achse	149
D.15	T14 Filter-Achse	151
D.16	T15 Pufferung Q1 (Operations-Puffer)	152
D.17	T16 Pufferung Q2 (Flush-Policy)	154
D.18	T17 Persistenz-Ziel (Persistence-Target)	155
D.19	System-Sphäre: Ziel-ISA (target_isa)	156
D.20	System-Sphäre: Betriebssystem (operating_system)	158
D.21	System-Sphäre: Erweiterungs-Hardware (external_utils)	160
D.22	Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse	161
D.23	System-Sphäre: Äußere Komplex-Klammer (build_target_complex) und Bau- Werkzeugkette (build_toolchain)	163
D.24	Mess-Sphäre: Mess-Kategorien (measurement_category)	164
D.25	Mess-Sphäre: Mess-Tooling/Collector (collector)	165
D.26	Mess-Sphäre: Telemetrie-Bausteine (konservierter Katalog)	166
D.27	Mess-Sphäre: Last-Rahmenwerk (load_framework) — Meta-Meta-Achse	167
D.28	Build-PG Seitentyp-Achse (PAGE-TYPE)	168
D.29	Build-HW — Plattform-Hardware-Strategie	169
D.30	A-Korpus Allokator-Korpus (A01-A23)	170
D.31	Achsen-Inventar aus den generierten Registries	173
E	Architekturentscheidungen	175
F	Vergleichsinterfaces: <code>std::map</code> und <code>std::vector</code>	179
F.1	<code>std::map</code> (geordnet-assoziativ)	179
F.2	<code>std::vector</code> (Sequenz)	182
F.3	Zusammengesetzte Funktionen und Konformitäts-Teilmenge	184

1 Einleitung

1.1 Motivation

Suchbäume und Indexstrukturen tragen moderne datenintensive Systeme — von In-Memory-Datenbanken über Dateisysteme bis zur Sprachverarbeitung. Zugleich werden heutige Prozessoren von der Speicher-Hierarchie dominiert: nicht die Zahl der Operationen, sondern ihr Cache-Verhalten bestimmt die Laufzeit. Aus diesem Spannungsfeld erwächst die Leitfrage der Arbeit. Im Zentrum stehen dabei trie-basierte Indizes und cache-line-optimierte Verarbeitung. Seit dem Adaptive Radix Tree (ART) [LKN13] von 2013 hat sich die Forschung ausdifferenziert: Trie-Höhen-Optimierung (HOT [BZP⁺18]), hybride B⁺-Strukturen (Masstree [MKM12], B²-Baum [SSL⁺21]), succinct-Repräsentationen (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]) und selbstoptimierende Ansätze (START [FJKN20]).

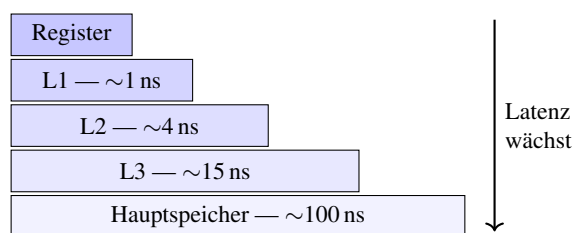


Abbildung 1.1: Die „Cache-Wand“: die Zugriffslatenz wächst über die Speicher-Hierarchie um rund zwei Größenordnungen.

Parallel dazu ist ein eigener Forschungskorpus zu Cache-Bewusstheit, Prefetching und Allokationsstrategien entstanden. Heuristiken wie Cache-Sensitive Search Trees, cache-oblivious Layouts und hierarchisches Prefetching adressieren jeweils einen separat betrachteten Aspekt der Suchalgorithmus-Architektur, wobei der Fokus meist auf empirisch gefundenen Verbesserungen und Messungen liegt. Weil so vieles auf cache-line-optimiertem Verhalten beruht, liegt eine Segmentierung der Suchalgorithmus-Bestandteile in *Achsen* nahe — eine rückwärtsgerichtete Abstraktion bekannter Bausteine in wiederverwendbare Module über die wichtigsten Arbeiten des Standes der Technik hinweg.

Diese Arbeit vollzieht diese Segmentierung mit einem Rahmenwerk, dessen Bausteine sich als austauschbare Entwurfsdimensionen eines großen Entwurfsraums (*design space*) auffassen lassen [IZA⁺18, IZH⁺18]. Strukturell verschiedene Suchverfahren — von baumartigen Indizes bis zu nicht-baumartigen Hash-Tabellen, samt Spezialeigenschaften wie Cache-Line-Bewusstheit — werden so als Punkte desselben Entwurfsraums vergleichbar; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Dimensionen über demselben Suchalgorithmus-Interface, als wären sie der einzige Fächer einer Baum-Wurzel. Als Prüfling für die Stand-der-Technik-Zugehörigkeit dient PRT-ART, ein Probst-Redirect-Tree-/ART-Hybrid und eine abstrakte, unvollständige Ansammlung neuartiger Suchalgorithmus-Bestandteile.

1.2 Problemstellung

Cache-Bewusstheit ist in heutigen Suchstrukturen entweder fest verdrahtet oder ganz ignoriert; vor allem aber lässt sich nicht *trennen*, welche einzelne Entwurfsentscheidung welchem Lastmuster nützt — das *Trennbarkeits-Problem*, der rote Faden dieser Arbeit.

Bestehende cache-bewusste Index-Entwürfe optimieren ein oder wenige Aspekte isoliert und berichten Ergebnisse, die sich über Veröffentlichungen hinweg schwer vergleichen lassen, da jede Arbeit eigene Datenstrukturen, Workloads und Mess-Methodiken verwendet. Die Gewichtungen der Verbesserungen einzelner Algorithmus-Bestandteile bleiben dabei untrennbar: Ein gemeldeter Leistungsgewinn lässt sich nicht eindeutig auf einen einzelnen Bestandteil (Seitentyp, Layout, Value-Ablage, Prefetching) zurückführen, weil die Bestandteile in den Original-Entwürfen verflochten bleiben. Es fehlt ein gemeinsamer Rahmen, der (i) solche Entwürfe in austauschbare Achsen zerlegt, (ii) publizierte Algorithmen als explizite Konfigurationen rekonstruiert und (iii) sowohl einzelne Achsen als auch ganze Algorithmen auf einer einheitlichen, fairen Basis vermessen kann.

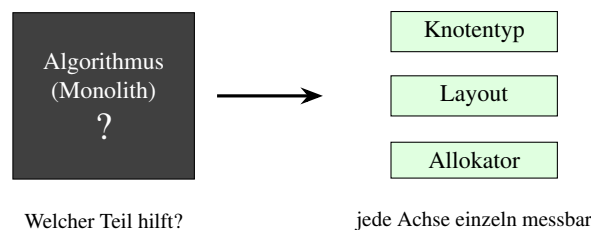


Abbildung 1.2: Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar.

1.3 Forschungsfragen

Aus der Aufgabenstellung und dem Ziel der Cache-Line-Optimierung ergeben sich — aufgrund der Allgegenwärtigkeit des Einflusses der Cache-Line-Einstellungen — eine Hauptfrage (FF0) und vier messbare Forschungsfragen (FF1–FF4); ergänzende Aspekte sind ihnen als vertiefende *Teilfragen* zugeordnet. Quer zu allen fünf Fragen liegt eine Kategorisierungs-Frage, die diese Arbeit als Begriffsprägung mitführt: Schon die Aufgabenstellung unterscheidet das Gemessene (die Entwurfsbestandteile des Algorithmus), die Plattform, auf der gebaut und gemessen wird, und das Messen selbst. Ob sich die Achsen des Entwurfsraums entlang dieser drei Belange sauber in *Organ-Achsen* (die austauschbaren Entwurfsbestandteile des Algorithmus), *System-Achsen* (die Belange der Bau- und Ziel-Plattform) und *Mess-Achsen* (die Belange des Messens selbst) trennen lassen — und wie eine solche Trennung zu bewerkstelligen ist, falls sie in einem Entwurf noch nicht vorhanden ist — bleibt an dieser Stelle bewusst offen; die Teilfragen sind darum entlang dieser drei *Achsen-Kategorien* benannt; die Antworten entwickeln die Kapitel 3 und 4.

FF0 (Hauptfrage) Wie stark verbessern *aktive, messgetriebene* cache-bewusste Entscheidungen — Seitentyp, Speicher-Layout, Value-Ablage und Prefetching — die Leistung trie-basierter Suchstrukturen gegenüber *passiven, statisch ausgelegten* Varianten auf unterschiedlichen CPU-Plattformen (Hybrid-CPU's, Sapphire Rapids)? *Erweiterung (Achsen-Kategorien)*: Lässt sich ein gemessener Gewinn überhaupt eindeutig zuordnen — der Entwurfsentscheidung selbst (Organ), der Bau- und

Ziel-Plattform (System) oder dem Mess-Aufbau (Mess) —, und welche Trennung der Achsen-Kategorien ist dafür nötig?

FF1 (Komposition) Wie lässt sich ein Suchalgorithmus-Entwurf kanonisch in orthogonale Achsen zerlegen und in einer Permutationsmatrix systematisieren, sodass die Stand-der-Technik-Verfahren möglichst aus ihrem Originalcode bias-frei als Profil-Konfigurationen rekonstruierbar und beliebig permutierbar werden? *Teilfrage (Kategorien-Schnitt):* Zerfällt die kanonische Zerlegung selbst entlang der drei Achsen-Kategorien — müssen Plattform- und Mess-Belange aus der Permutationsmatrix herausgehalten werden, damit die Zerlegung der Entwurfsbestandteile orthogonal bleibt —, und wie ist ein solcher Schnitt zu bewerkstelligen, falls er im Entwurf noch nicht angelegt ist? *Teilfrage (Organ-Kopplungen und System-Variation):* Welche Achsen-Kopplungen (Knotentyp $\times$ SIMD $\times$ Pfadkompression) begrenzen die Isolation einzelner Achsen, und wie verhält sich dieselbe achsen-konstante Struktur im Cache-Update- und Cache-Line-Füllverhalten, wenn allein die ISA-/SIMD-Achse (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) variiert wird?

FF2 (Mess-Methodik) Wie muss eine reproduzierbare Mess-Pipeline mit Hardware-Zähler-Erhebung gestaltet sein, damit jede Achsen-Permutation fair (unter sonst gleichen Bedingungen, also gleicher Compiler- und Flag-Basis) in drei Messreihen (A–C) über drei Granularitäten (Micro-, Makro- und Gesamt-Benchmarking) gemessen wird, und unter welchen plattform-kalibrierten Kriterien gilt eine Plattform als erfolgreich verifiziert? *Teilfrage (Mess-Achsen):* Müssen die Mess-Instrumente selbst — Zeitnahme, Zähler-Erhebung, Last-Erzeugung — als eigene Achsen geführt werden, die stets präsent sind, ohne je zu verändern, was gebaut und gemessen wird, und wie ist diese Trennung zu bewerkstelligen, falls sie noch nicht vorhanden ist? *Teilfrage (Organ-Achsen):* Wie lassen sich widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine Cache-Line bei CSS/CSB⁺ gegenüber bis zu sechzehn bei Hankins/Patel) durch achsen-isolierte, unter sonst gleichen Bedingungen permutierte Messung bias-frei nachmessen, statt sie aus inkompatiblen Original-Setups zu übernehmen? *Teilfrage (Erweiterbarkeit):* Wie erweiterbar ist eine solche Mess-Pipeline — wie gliedern sich neue Prüflinge und ihre Entwurfsbestandteile ein, wie werden sie abstrahiert, und ordnen sie sich bestehenden Algorithmus-Kategorien zu, begründen sie neue Kategorien oder eine Mischform; auf welcher konzeptionellen Ebene sollte dieser Schnitt verlaufen? Zentral ist dabei: Was ist überhaupt eine Erweiterung des Systems, und wie wirkt sich das Hinzufügen oder Wegschneiden einzelner Erweiterungs-Bausteine — etwa der aus einer einzelnen Veröffentlichung stammenden Entwurfsbestandteile — auf das erreichbare Maß an Cache-Line-Bewusstheit des Gesamtsystems aus? Braucht es dafür einen Betriebsmodus, der die Entwurfsbestandteile eines wählbaren Referenz-Entwurfs aus der Literatur als geschlossenes Achsen-Set isoliert anbietet, um genau diese eine Veröffentlichung — bei weiterhin freistehenden Mess- und System-Achsen — originalgetreu durchzumessen?

FF3 (Prüfling-Vergleich) Gewinnt der Prüfling PRT-ART gegenüber den acht Rang-1-Entwürfen im Stand der Technik (*State of the Art*, kurz SOTA; ART, HOT, Masstree, CoCo-Trie, START, B²-Baum, Wormhole, SuRF)¹ und der Rang-2/3-SOTA *nicht* durch asymptotische Neuheit,

¹Die *Ränge* ordnen die Stand-der-Technik-Veröffentlichungen nach Zentralität für diese Arbeit: Rang-1 umfasst die zehn wichtigsten Kern-Arbeiten — darunter die acht eigenständigen Suchentwürfe sowie ARTSync (P08) [LABN16] und LOUDS (P09) [Jac89], die als Organe eingehen —, Rang-2 die cache-bewussten Layout- und B⁺-Vorarbeiten und Rang-3 die Prefetching-, Synchronisations- und TU-Dresden-Hardware-Arbeiten.

sondern durch bessere Mikroarchitektur-Eigenschaften (höhere Cache-Line-Auslastung, weniger LLC- und dTLB-Misses, geringere Branch-Kosten, kleinere heiße Suchpfade) bei Exact-Lookup, Prefix-Lookup und Prefix-Enumeration — gemessen unter den drei Pflicht-Messreihen A (Prüfling vs. Stand der Technik), B (systematische Variation der Entwurfsentscheidungen) und C (Merge/Regression alt gegen neu)? *Teilfrage (Organ-Achsen)*: Welche lokale Seitendarstellung (8-/16-Bit-Punktseite gegenüber -Kompaktseite) ist je Verzweigungspunkt am günstigsten, und nach welchen plattform-kalibrierten Umschaltregeln — einschließlich der Reihenfolge $\text{Array}_{256} \rightarrow \text{Array}_{65535} \rightarrow \text{Vector}\langle u8, u8 \rangle \rightarrow \text{Vector}\langle u16, u16 \rangle$ je Füllstand und Suchtyp — soll adaptiv gewechselt werden? *Teilfrage (Compile-Time-Zwang)*: Erzwingen latenz-kritische Messungen, dass an sich zur Laufzeit wählbare Entscheidungen — etwa solche adaptiven Umschaltregeln — bereits zur Übersetzungszeit als Anforderungen fixiert werden, sodass sich jede Achse in eine übersetzungszeit-fixierte Haupt-Entscheidung und laufzeit-variable Unter-Entscheidungen gliedert — eine Zuspitzung, die FF4 aufnimmt?

FF4 (Compile-Time vs. Laufzeit) Welche Achsen-Eigenschaften müssen zur Übersetzungszeit fixiert werden, um plattform-optimierte, provenienz-nachweisbare Binaries je Permutation zu erzeugen, und welche können bzw. müssen zur Laufzeit gewählt werden (bit-codierter Permutations-Identifizier, adaptive Layout-Selection), ohne die wissenschaftliche Vergleichbarkeit (gleiche Compiler- und Flag-Basis) zu verletzen? Wird es durch die Messung möglich, ISA+OS-optimierte Binaries für eine Suchalgorithmus-Rekombination zu erzeugen, deren heuristische Optimierung perfekt auf ihre Laufzeitumgebung zugeschnitten ist? *Erweiterung (System-Achsen)*: Verläuft die Grenze zwischen Übersetzungszeit und Laufzeit entlang der Achsen-Kategorien — gehören insbesondere die Plattform-Eigenschaften (ISA, Betriebssystem, Hardware-Erweiterungen) als System-Belange in die Übersetzungszeit-Identität der Binary, während Mess-Wahlen laufzeit-variabel bleiben dürfen?

1.4 Zielsetzung und Beiträge

Die Zielsetzung folgt der Aufgabenstellung in vier Schritten: cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegen, Stand-der-Technik-Entwürfe als explizite Konfigurationen rekonstruieren, cache-line-bewusstes Verhalten sowohl je Bestandteil als auch für den ganzen Algorithmus auf gemeinsamer CPU-Basis messen und auswerten sowie den besten Gesamtalgorithmus nach einer autonomen Heuristik-Profil-Auswertung und Heuristik-Funktions-Modellierung als eigenständige Binary ausliefern. Der Code der Diplomarbeit ist dabei der *formale Bediener* der Cache-Engine: Er lädt den Prüfling PRT-ART, kompiliert ihn per Metaprogrammierung gegen die Kernbestandteile der Cache-Engine, liefert ABI-stabile *Lebewesen* (Suchalgorithmus-Instanzen) aus und treibt sie über das Prüf-Dock — das Mess-Interface der Gattung (Kapitel 3) — zur Messung. Das zugrunde liegende Architekturmodell — die drei Ebenen (Gattungs-Interface, Gattungs-Unterklasse, Achsen-Organe), die vier Subsysteme und die ABI-stabile Auslieferung der Suchalgorithmus-*Lebewesen* — entwickeln Kapitel 3 und Kapitel 4; das Konzept der Achsen-Architektur selbst ist aus einer Vorarbeit des Autors übernommen (zur Herkunft siehe Abschnitt 3.3.3), der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine mess-orientierte Realisierung. Die zentralen Beiträge sind:

- Eine ABI-stabile Modul-Schnittstelle: eine einheitliche C-Modulgrenze (`comdare_`

`create_anatomy() → IAnatomyBase`) exportiert je Permutation genau ein Lebewesen; dahinter liegt eine dreischichtige Abstraktionskette (`CacheEngine → ExecutionEngine → SearchEngine`) mit variadisch templatierten Hybrid-API-Spezialisierungen ($1 \text{ Parameter} \Rightarrow \text{std::vector-API}$; $2 \text{ Parameter} \Rightarrow \text{std::map-API}$; $N > 2 \Rightarrow \text{std::map}<K, \text{std::tuple}<V_1 \dots V_N>$).

- Eine Achsen-Permutationsmatrix in dreigeteilter Ordnung: achtzehn Organ-Achsen (Kompositions-Slots T0–T17) tragen die Binary-Permutation, drei System-Haupt-Achsen — Ziel-ISA, Betriebssystem und Erweiterungs-Hardware — beschreiben die Bau- und Ziel-Plattform, und ein eigenständiger Mess-Achsen-Realm hält die Mess-Belange von beidem getrennt; die frühere Neunzehn-Achsen-Zählung (Zwischenstand Juli 2026) ist eine historische Vorstufe dieser Ordnung. Umgesetzt ist die Matrix mit Boost.MP11-Compile-Time-Typlisten-Bausteinen (kein Runtime-Tag, kein `std::variant`; der `std::variant`-Katalog besteht nur als historische Elf-Achsen-Vorstufe) und Compile-Time-Fallback zwischen Prüfling-Stack und Stand-der-Technik-Stack.
- Dreißig SOTA-Algorithmus-Profile als persistierte XML-Konfigurationen (8 eigenständige Rang-1 + 22 Rang-2/3), die der `ExperimentDriver` automatisch in vorkompilierte C++23-Module überführt; perspektivisch soll der Profilsatz jede analysierte Veröffentlichung mit eigenständigem Suchentwurf abdecken.
- PRT-ART als Prüfling mit acht eigenen Bausteinschichten (4+1+2-Pool-Allokator, optimistisches Lock-Coupling mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3, Inline/External/ChainRef-ValueHandles, Signaling-Bit-Serialisierung); im Mess-Apparat überschreibt der Prüfling sechs Slots — die Build-Achse Seitentyp sowie Pfadkompression (Redirect), pfadorientiertes Prefetch, OLC-Nebenläufigkeit, Telemetrie und den ChainRef-Value-Handle-Slot —, während MultiLevel-Layout, Pool-Allokator und Serialisierung im Mess-Pfad derzeit über den Cache-Engine-Standard laufen (Abschnitt 3.5).
- Ein Mess-Treiber mit Defined-/Full-Modus-Unterscheidung, profilbewusstem Workload-Routing und drei Granularitäten — Micro-, Makro- und Gesamt-Benchmarking (einzelne Bestandteile, Interface-Operationen, ganze Algorithmen).
- Eine durchgehend deklarativ gesteuerte Bau- und Mess-Kette hinter diesem Treiber: ein *Experiment-Planer* löst die XML-Beschreibung des Experiments gegen die registrierten Achsen-Angebote auf, und der `CacheEngineBuilder` baut und vermisst daraus plattformbezogen die Binary-Permutationen — bis zur Auswertung über CSV-Tabellen und LaTeX-Diagramme.
- Eine bias-freie Vergleichs-Matrix, die *alle* Lastprofile gegen *alle* rekonstruierten Lebewesen fährt und so dem methodischen Heimspiel-Bias der Einzelveröffentlichungen begegnet.
- Eine generische Plattform-Probe, die Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) je Zielplattform ermittelt und als feste Größen in eine systemoptimierte Binary einfließen lässt; die beiden Produktionsmaschinen sollen dafür unter zwei Betriebssystemen (immutables Talos und root-Linux mit vollem Hardware-Zähler-Zugriff) vermessen werden — zur Abgabe liegt das root-Linux-Regime vor, der Talos-Spiegel steht aus.
- *Als Zielsetzung und Ausblick:* die Auslieferung des besten Gesamtalgorithmus unter dem Messaspekt des besten Cache-aware-Verhaltens und der Beschleunigung unter Cache-Line-Anpassung

— automatisch ausgewählt aus den feingliedrigen Messergebnissen — als eigenständige, ABI-stabile Binary hinter dem einheitlichen Suchalgorithmus-Interface; die Mess-Pipeline liefert die dafür nötige Rangbildung bereits, und ein erstes Inkrement der automatischen Auswahl und des Binary-Versands ist als eigenständiges Werkzeug (`best_binary_selector`) implementiert; die heuristik-gesteuerte Vollautomatisierung bleibt Ausblick.

1.5 Aufbau der Arbeit

Der Aufbau folgt dem Weg vom Allgemeinen zum Speziellen: Kapitel 2 legt — konzeptionell und bildhaft — die Grundlagen aus bekanntem Wissen: die Landschaft der Suchstrukturen, die Hardware-Eigenschaften der Speicherhierarchie, die Workload-Konventionen der Vergleichsliteratur, die Software-Mittel der Abstraktion und den Heuristik-Begriff. Kapitel 3 entwickelt daraus die Konzepte eines cache-bewussten Mess-Systems: die Problemstellung und ihre Anforderungen, den Stand der Technik als Achsen-Instanzen, das Anatomie-Modell mit seinen Schichten, das Mess-System selbst, den Prüfling PRT-ART als Demonstrator und die heuristische Auswertung samt ihrer Betriebsmodi. Kapitel 4 setzt dieses Konzept als Software-Architektur um — Repositorien, der Prüfling im Code, Achsen-Architektur und Übersetzungs-Pipeline, Schichten-Verträge mit Contract-Tests sowie die System-Achsen im Code. Kapitel 5 wertet entlang der Forschungsfragen aus, einschließlich des heuristischen Gegenbeweises; Kapitel 6 schließt mit Fazit und Ausblick. Die Anhänge vertiefen begleitend: die Messreihen-Dokumentation (Anhang A), die Code-Struktur (Anhang B), das Glossar (Anhang C), die vollständige Bausteine-Matrix (Anhang D), die Architekturentscheidungen (Anhang E) und die Vergleichs-Interfaces (Anhang F).

2 Suchbäume und Grundlagen

Dieses Kapitel führt vom vertrauten Gegenstand — den Suchbäumen — über die Hardware, die ihre Leistung prägt, und die Workloads, unter denen sie verglichen werden, zu den Software-Mitteln und zum Heuristik-Begriff, mit denen sich ihre Entwurfsentscheidungen abstrahieren und auswählen lassen. Alles bleibt bei *bekannten* Begriffen, Konzepten und Veröffentlichungen; die systematischen Kataloge und das eigene Begriffsgerüst folgen in Kapitel 3.

2.1 Konzept und Überblick über den Stand der Technik

Wir kartieren zunächst die Landschaft der Suchstrukturen — vergleichsbasierte Bäume, digitale und präfixbasierte Verfahren, Hashing, räumliche und flache Strukturen — und ordnen die bekannten Vertreter mit ihren Kernideen und Belegstellen ein. Die Gesamtheit der jeweils besten veröffentlichten Verfahren bezeichnet die Literatur als *Stand der Technik* (englisch *State of the Art*, kurz SOTA); gemeint sind damit in dieser Arbeit stets die publizierten Suchstruktur-Entwürfe, an denen sich ein neuer Entwurf messen lassen muss. Hier deutet sich bereits an, dass sich jeder Algorithmus in einem gemeinsamen Entwurfsraum verorten lässt; wie sich dieser Entwurfsraum systematisch aufspannt, entwickelt Kapitel 3. Suchstrukturen lassen sich danach einordnen, mit welchem Mechanismus sie einen Schlüssel

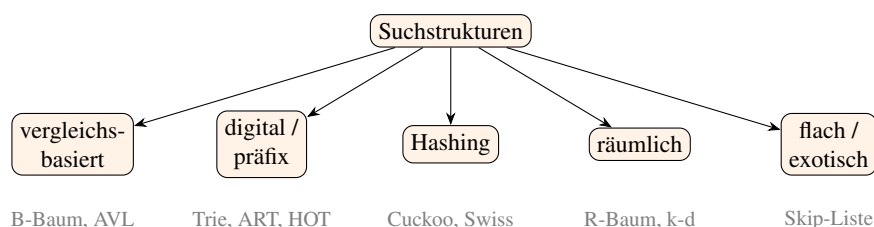


Abbildung 2.1: Die Landschaft der Suchstrukturen: fünf Klassen mit typischen Vertretern.

auf einen Datensatz abbilden [Knu98]. *Vergleichsbasierte* Strukturen lokalisieren den Schlüssel über Ordnungsvergleiche und setzen eine totale Ordnung voraus — von den selbstbalancierenden AVL-Bäumen [AVL62], deren Geschwister-Teilbäume sich in der Höhe um höchstens eins unterscheiden, und den Rot-Schwarz-Bäumen [Bay72, GS78], die dieselbe logarithmische Höhe über eine Knotenfärbung mit lockereren, günstiger zu wartenden Invarianten sichern, über die block- und externspeicherorientierten B-/B⁺-Bäume [BM72, Com79], deren Knoten typischerweise einer Cache- oder Speicherblock-Größe entsprechen [GL01, HP03] und die in der B⁺-Variante alle Nutzdaten in den Blättern halten und innere Knoten auf Wegweiser reduzieren, bis zum hauptspeicher-nahen T-Baum [LC86], der knotenlokale Wertebereiche mit der AVL-Balance verbindet und so die Zahl der Zeiger-Indirektionen je Suchschritt senkt. *Digitale* Strukturen verzweigen dagegen nach den Bytes des Schlüssels statt durch Vergleich, sodass die Suchpfadlänge von der Schlüssellänge und nicht von der Schlüsselanzahl abhängt: der *Trie* [Bri59, Fre60] als Grundform, in dem jeder Wurzelpfad einen Schlüssel buchstabiert und gemeinsame Präfixe geteilt

werden, die pfadkomprimierten Radix-/PATRICIA-Bäume [Mor68], die dünn belegte Ein-Kind-Ketten kollabieren, und der *Adaptive Radix Tree* (ART) [LKN13], der die Knotenrepräsentation an den tatsächlichen Verzweigungsgrad anpasst und dieser Arbeit durchgängig als Referenzentwurf dient.

Jenseits der baumartigen Verfahren bildet *Hashing* Schlüssel direkt über eine Hashfunktion ab und kennt keine inhärente Ordnung, weshalb Bereichsabfragen teuer sind [Knu98]; Cuckoo-Hashing [PR04], das im Gegensatz zu offener und verketteter Adressierung konstante Worst-Case-Lookups garantiert, und Tabellen wie SwissTable [Goo18], die dank SIMD-Gruppen-Probing hohe praktische Dichte erzielen, treten dieser Arbeit als nicht-baumartige Vergleichsgröße zur Seite. *Räumliche* Strukturen (k-d-Baum [Ben75], Quadtree [FB74], R-Baum [Gut84]) partitionieren einen mehrdimensionalen Schlüsselraum und liegen außerhalb des Umfangs, ebenso wie die theoretischen „Exoten“ — der van-Emde-Boas-Baum [EB75] und der y-fast Trie [Wil83] mit $O(\log \log u)$ über einem Universum der Größe u oder der die Vergleichsschranke unterschreitende Fusion Tree [FW93] — sowie die randomisierte Skip-Liste [Pug90], das flache, sortierte Array mit Binär- oder Interpolationssuche [Knu98] — streng genommen das einzelne Blatt eines Baumes — und das Suffix-Array, das dieses Layout auf String-Schlüsseln zu einem platzsparenden Volltext-Index verallgemeinert und dessen leichtgewichtige, speicher-budgetierte Konstruktion als Musterbeispiel des Algorithm-Engineerings gilt [MF02]. Diese Arbeit konzentriert sich auf die baumartigen Strukturen der vergleichs- und der digitalbasierten Klasse (Abbildung 2.1); so verschieden die Klassen äußerlich wirken, teilen sie doch wiederkehrende, voneinander trennbare Teilentscheidungen.

Quer zu dieser Suchalgorithmus-Sicht steht die etablierte Untergliederung der *Container* der C++-Standardbibliothek [ISO24]: Sequenz-Container (`std::vector`) ordnen nach Einfügeposition, geordnet-assoziative (`std::map`, `std::set`) nach Schlüsselordnung, ungeordnete (`std::unordered_map`) bilden über eine Hashfunktion ab. Diese genormten Schnittstellen-Verträge bilden die gemeinsame Basis, gegen die sich strukturell verschiedene Verfahren einheitlich vermessen lassen — die geordnete Map als gemeinsamer Nenner vergleichs- wie digitalbasierter Suchstrukturen, der `std::vector` als Sequenz-Sicht und die Hash-Tabelle als ungeordnete Gegenprobe; ihre Einordnung in das Begriffsgerüst dieser Arbeit entwickelt Kapitel 3.

Alle bisher genannten Verfahren sind *fest entworfene* Strukturen: ihr innerer Aufbau steht fest, bevor die erste Anfrage eintrifft. Eine etablierte Forschungslinie verlagert den Entwurf dagegen in die Laufzeit. *Database Cracking* [IKM07] baut den Index als Nebenprodukt der Anfrageverarbeitung adaptiv auf — jede Anfrage partitioniert die Daten ein Stück weiter —, und *Stochastic Cracking* [HIKY12] macht dieses Vorgehen gegen ungünstige Anfragefolgen robust; *gelernte Indizes* [KBC⁺18] ersetzen die Suchstruktur durch ein Modell der Schlüsselverteilung, das die Position eines Schlüssels näherungsweise vorhersagt. Jüngere Arbeiten führen diese Linie aktiv-tunend fort: VEGA justiert einen gelernten Index zur Laufzeit mit gruppenweiser Lern-Granularität nach [LCL⁺25], und AirIndex entwirft die Hierarchie eines Index automatisch aus dem Daten- und Speicher-I/O-Profil des Zielsystems [CLP23]. Diese Ansätze zeigen einen gemeinsamen Trend: weg von der fest verdrahteten Einzelstruktur, hin zu konfigurierbaren, sich anpassenden Entwürfen. Für die vorliegende Arbeit sind sie die nächstverwandten bekannten Ansätze, denn sie belegen, dass die Entwurfsentscheidungen einer Suchstruktur veränderlich gehalten werden können; wie sich solche Entscheidungen systematisch zerlegen und einzeln vermessen lassen, entwickelt Kapitel 3.

Für die cache-bezogenen Eigenschaften dieser Entwürfe hat die Literatur eine gestufte Begriffs-Taxonomie herausgebildet, die diese Arbeit durchgängig verwendet. Ihr Ausgangspunkt ist das *I/O-Modell* (External-

Memory-Modell) von Aggarwal und Vitter: ein zweistufiger Speicher, dessen Ebenen Daten nur in Blöcken fester Größe austauschen; Kostenmaß eines Algorithmus ist allein die Zahl der Blocktransfers [AV88]. Auf die Cache-Hierarchie übertragen heißt ein Entwurf *cache-aware*, wenn er die konkreten Parameter der Zielplattform — Blockgröße, Kapazität — kennt und explizit nutzt; *cache-conscious* bezeichnet seit Chilimbi, Hill und Larus dieselbe Haltung aus Sicht der Datenstruktur: ihr Layout wird durch Clustering, Färbung und Kompression gezielt an Cache-Blöcke angepasst [CHL99]. *Cache-oblivious* ist die Gegenposition von Frigo, Leiserson, Prokop und Ramachandran: Der Algorithmus kennt keinerlei Parameter; analysiert wird im *Ideal-Cache-Modell* — zwei Ebenen, Kapazität Z , Line-Länge L , volle Assoziativität, optimale Ersetzung —, und ein optimaler cache-oblivious Entwurf ist dann auf *jeder* Ebene einer mehrstufigen Hierarchie zugleich nahezu optimal [FLPR99]. *Cache-sensitive* schließlich ist die Wortwahl von Rao und Ross für ihre auf die Cache-Line zugeschnittenen Suchbäume (Abschnitt 2.2) und wird weitgehend synonym zu cache-aware geführt [RR99, RR00]; als deutschen Oberbegriff für die parameter-kennenden Spielarten verwendet diese Arbeit *cache-bewusst*.

Quer zu dieser Parameter-Frage liegt die Zeitpunkt-Frage: *Wann* fällt die cache-bezogene Entwurfsentscheidung? Ein *statischer* Entwurf fixiert sie vor der ersten Anfrage — sei es cache-bewusst auf ermittelte Parameter oder cache-oblivious parameterfrei —, während die *aktiven beziehungsweise adaptiven* Ansätze des vorigen Absatzes die Entscheidung in die Laufzeit verlagern und aus beobachtetem Verhalten nachführen. Beide Unterscheidungen — Parameter-Kenntnis und Entscheidungs-Zeitpunkt — sind voneinander unabhängig; der Heuristik-Begriff, der sie später verbindet, folgt in Abschnitt 2.5.

2.2 Hardware: historische Anpassung an die Suchleistung

Die Leistung von Suchstrukturen wird seit jeher von der Hardware getrieben. Wir zeichnen die historische Einführung cache-naher Hardware nach — Cache-Hierarchie, Cache-Line, Adressübersetzung, Kohärenz — mit Fokus auf die *statischen und übersetzungszeit-relevanten* Eigenschaften, die später die Abstraktion in Abschnitt 2.4 tragen. Moderne Prozessoren schalten mehrere Cache-Ebenen (L1, L2 und — nicht

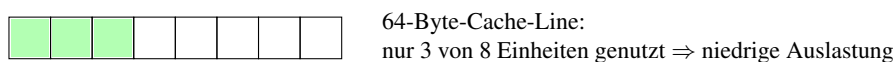


Abbildung 2.2: Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.

auf jeder Plattform — L3) zwischen die Register und den Hauptspeicher [HP19]; ein Zugriff, der sein Datum nahe findet, ist günstig, während ein Fehlzugriff bis zum Hauptspeicher einige hundert Zyklen kostet [Dre07], auf Mehr-Sockel- und Chiplet-Systemen zusätzlich abhängig vom besitzenden NUMA-Knoten. Die Übertragungseinheit zwischen den Ebenen ist die *Cache-Line* (Abbildung 2.2); ihre Größe ist keine universelle Konstante, sondern je Mikroarchitektur fest verdrahtet — 64 Byte auf x86-64 und gängigen ARM-Kernen (etwa Cortex-A76 oder Neoverse V2), 128 Byte etwa auf Apple-Silicon-L2 und System-Level-Cache [App24], schmalere 32 Byte auf älteren oder eingebetteten Kernen [HP19, Int24a, Adv23]. Programme können sie zur Laufzeit *auslesen* — über `CPUID` (x86), `CTR_EL0` (ARM) oder `riscv_hwprobe` (RISC-V; eingeschränkt, u. a. CBO-Blockgrößen) [Arm24, Int24b] —, aber nicht verändern; auch die Tiefe der Hierarchie variiert (einfache RISC-V-Kerne wie der SiFive U74 enden bei L2,

manche AMD-Prozessoren bringen einen gestapelten 3D-V-Cache-L3 mit). Für ein cache-line-bewusstes Layout ist die je Zielplattform ermittelte Line-Größe daher ein *Eingabeparameter der Übersetzung*, keine feste Annahme. Der Transfer einer Line erfolgt zudem nicht streng blockweise am Stück: *critical-word-first* liefert zuerst das angefragte Wort, und DDR5 unterteilt den Speicherkanal in zwei unabhängige 32-Bit-Sub-Kanäle [KAS21]; nicht-temporale bzw. Write-Combining-Schreibzugriffe umgehen überdies den normalen Write-Allocate-Pfad, statt die betroffene Cache-Line in die Cache-Hierarchie zu holen [Int24b].

2.2.1 Cache-Organisation: Assoziativität, Fehlzugriffs-Klassen und Parallelität

Wo eine Line im Cache liegen darf, bestimmt die *Assoziativität*: Der Cache ist in *Sets* gegliedert, von denen jedes n Ways (Line-Plätze) hält; die Speicheradresse zerfällt in Tag, Set-Index und Line-Offset, und jede Adresse kann nur in den n Plätzen *ihrer* Sets liegen — direkt abgebildet ($n = 1$) und voll-assoziativ (ein einziges Set) sind die Grenzfälle, gängige Daten-Caches liegen mit 4- bis 16-fach dazwischen [HP19, Int24a, Adv23]. Nach ihrer Ursache klassifizieren Hill und Smith Fehlzugriffe in drei Klassen (3-C-Modell): *compulsory* (Erstzugriff — die Line war nie im Cache), *capacity* (die Arbeitsmenge übersteigt die Kapazität) und *conflict* (mehr heiße Lines fallen auf ein Set, als es Ways hat) [HS89]; die Lehrbuch-Darstellung ergänzt in Mehrkern-Systemen *coherence*-Fehlzugriffe als vierte Klasse [HP19]. Die Klassifikation ist mehr als Buchhaltung, denn sie trennt die Gegenmittel: Compulsory-Misses adressiert Prefetching (Abschnitt 2.2.3), Capacity-Misses adressieren kompaktere Layouts, und Conflict-Misses entstehen gerade bei den regelmäßigen Adressmustern großer Knoten-Arrays — liegen heiße Elemente um Zweierpotenz-Abstände auseinander, konkurrieren sie um dieselben Sets [HP19, Dre07].

Ein Fehlzugriff blockiert einen modernen Cache zudem nicht: Seit Krofts *lockup-free*-Organisation führt der Cache je ausstehendem Miss ein Statusregister (*Miss-Status-Holding-Register*, MSHR) und bedient währenddessen weitere Zugriffe [Kro81]. Die kleine, feste Zahl dieser Register — bei Intel als *Line-Fill-Buffer* zwischen L1 und L2 realisiert — begrenzt, wie viele Speicherzugriffe gleichzeitig unterwegs sein können: die *Memory-Level-Parallelität* (MLP) [HP19, Int24a]. Ob eine Suchstruktur diese Parallelität überhaupt anbieten kann, ist eine Entwurfsfrage — der Suchpfad eines Baumes ist eine Kette voneinander abhängiger Zugriffe und nutzt sie ohne Weiteres nicht (Abschnitt 2.2.3).

Zwei weitere Eigenschaften wirken an den Line-Grenzen: *Ausrichtung* (Alignment) — ein Objekt, das eine Line-Grenze überspannt, kostet zwei Transfers statt einem, weshalb cache-bewusste Layouts Knoten an Line-Grenzen ausrichten — und *False Sharing*: Schreiben zwei Kerne auf *disjunkte* Felder derselben Line, wandert die Line trotz logischer Unabhängigkeit zwischen den Kernen hin und her, weil die Kohärenz (Abschnitt 2.2.5) stets ganze Lines verwaltet [Dre07, Int24a].

2.2.2 Cache-line-orientierte Layout-Forschung

Eben dieses cache-line-orientierte Layout ist der historische Leitgedanke der einschlägigen Forschung: Der CSS-Baum [RR99] führt den Cache-Line-Knoten ein, der CSB⁺-Baum [RR00] das zusammenhängende Geschwister-Cluster, Hankins und Patel [HP03] vermessen den Effekt der Knotengröße und Samuel et al. [SPB05] machen den Knoten prozessor-bewusst; der Adaptive Radix Tree [LKN13] staffelt mit Node4/16/48/256 die Knotengröße cache-line-orientiert, und Schmidt et al. [SKK⁺25] vermessen Stride-gegen Sequenz-Zugriffe auf High-Bandwidth-Memory. Als konkrete Layout-Optionen kennt die Literatur das dichte und das cache-line-ausgerichtete Array-of-Structs, das spaltenorientierte Struct-of-Arrays,

das bit-gepackte LOUDS [Jac89] und das SIMD-gekachelte AoSoA. Die mehrstufige Hierarchie selbst ist überdies Gegenstand software-/hardware-kooperativer Prefetch-Verfahren wie dem hierarchischen Instruktions-Prefetcher für Server-Anwendungen von Zhang et al. [ZGH⁺25], der in diese Arbeit als Prefetching-Instanz und nicht als datenstruktur-spezifischer Baum-Datenprefetch eingeht.

2.2.3 Prefetching

Prefetching holt Daten *vor* ihrer Verwendung in den Cache und ist das wichtigste Gegenmittel gegen Compulsory-Misses und unverdeckte Speicherlatenz (Abschnitt 2.2.1). Die Hardware-Prefetcher moderner Kerne bilden wenige, dokumentierte Klassen: Next-/Adjacent-Line-Prefetcher holen die Nachbar-Line jeder verfehlten Line, Stream-Prefetcher erkennen auf- oder absteigende Zugriffsfolgen und laufen ihnen voraus, Stride-Prefetcher lernen je Lade-Instruktion (am Instruction-Pointer erkannt) konstante Abstände, und komplexere raum- oder zeitkorrelierende Verfahren sind Gegenstand der Forschung; die Übersicht gibt Mittal [Mit16], die konkreten Prefetcher der L1-/L2-Stufen dokumentieren die Hersteller-Handbücher [Int24a, Adv23]. Bewertet wird ein Prefetcher an drei Größen: *Genauigkeit* (wie viele geholte Lines wurden genutzt), *Abdeckung* (wie viele Misses wurden vermieden) und *Rechtzeitigkeit* — zu späte Prefetches verdecken die Latenz nur teilweise, zu frühe verdrängen noch benötigte Lines [Mit16].

Ergänzend stellen die Architekturen *Software*-Prefetch-Instruktionen bereit (x86: `PREFETCHh`-Familie samt nicht-temporaler Variante; ARM: `PRFM`), mit denen das Programm eine Adresse samt Ziel-Ebene selbst ankündigt [Int24b, Arm24]. Ihre zentrale Stellgröße ist die *Prefetch-Distanz* — wie weit voraus angekündigt wird: groß genug, um die Speicherlatenz zu verdecken, klein genug, um die geholte Line nicht vor Gebrauch wieder zu verlieren; sie hängt von Latenz und Verarbeitungstempo ab und ist damit eine je Plattform zu kalibrierende Größe [Mit16]. In der Index-Forschung begründen Chen et al. genau darüber breitere B⁺-Knoten: Werden alle Lines eines mehrzeiligen Knotens parallel vorangekündigt, sinkt die effektive Knoten-Ladezeit, und die optimale Knotenbreite wächst über die eine Cache-Line hinaus [CGM01]; Fractal Prefetching überträgt dasselbe Prinzip ebenen-übergreifend auf Platten-Seiten [CGM02].

Seine Grenze findet Prefetching am *Pointer-Chasing*, dem Kern jeder Baumsuche: Die Adresse des nächsten Knotens steht erst fest, wenn die Line des aktuellen Knotens eingetroffen ist. Eine solche abhängige Ladekette bietet weder einem Stride- noch einem Stream-Prefetcher ein Muster, und sie serialisiert die Speicherzugriffe — die vorhandene Memory-Level-Parallelität (Abschnitt 2.2.1) bleibt ungenutzt [Mit16]. Die bekannten Gegenmittel setzen deshalb nicht am Vorhersagen, sondern am Entwurf an: mehr Nutzbytes je abhängigem Schritt (breitere, line-ausgerichtete Knoten [CGM01]) oder mehrere *unabhängige* Suchen, deren Zugriffe so verschränkt werden, dass die Ladekette der einen die Wartezeit der anderen füllt — Entwurfsantworten, die in Abschnitt 2.2.6 als Parameter wiederkehren.

2.2.4 Hauptspeicher und Adressübersetzung: DRAM, NUMA und TLB

Hinter dem letzten Cache liegt der DRAM-Hauptspeicher, dessen Kenngrößen die JEDEC-Norm für die aktuelle Generation DDR5 festlegt (JESD79-5) [JED20]. Zwei Größen werden dabei häufig vermengt: Die *Datenrate* in Megatransfers je Sekunde (MT/s; zwei Transfers je Takt) misst die Bandbreite des Interfaces, die *CAS-Latenz* (CL) zählt Taktzyklen bis zur ersten Antwort einer geöffneten Zeile — da die CL-Zyklenzahl mit der Nennrate mitwächst, senken steigende Datenraten die absolute Zugriffslatenz

kaum [HP19, KAS21]. Parallelität liefert der Aufbau: mehrere Speicherkanäle je System, die bereits erwähnte DDR5-Zweiteilung jedes Kanals in unabhängige 32-Bit-Sub-Kanäle sowie Bank-Groups innerhalb jedes Chips, deren verschränkte Ansteuerung Zeilen-Aktivierungszeiten überdeckt [JED20, KAS21]. Auf dem Modul hinterlegt sind die *JEDEC-Nennprofile* der Norm; daneben existieren herstellerspezifische Übertaktungsprofile (Intel XMP, AMD EXPO) jenseits der Norm-Raten — welche Rate eine Maschine tatsächlich fährt, ist darum eine dokumentationspflichtige Eigenschaft der Zielplattform, keine aus dem Modul ablesbare Konstante.

In Mehr-Sockel- und Chiplet-Systemen kommt die Topologie hinzu: Bei *NUMA* (Non-Uniform Memory Access) besitzt jeder Knoten eigenen lokalen Speicher, und Zugriffe auf entfernte Knoten kosten spürbar mehr als lokale. Das Betriebssystem platziert Seiten standardmäßig nach der *First-Touch*-Politik — eine Seite landet auf dem Knoten desjenigen Threads, der sie zuerst beschreibt [Lam13]. Für Messungen an Suchstrukturen ist das eine stille Vorbedingung: Wer die Struktur auf einem anderen Knoten aufbaut, als er sie vermisst, misst die Interconnect-Latenz mit.

Neben der Daten-Cache-Hierarchie existiert eine zweite Lokaltätsgrenze: der *Translation Lookaside Buffer* (TLB) cacht Virtuell-zu-Physisch-Abbildungen, und ein TLB-Miss löst einen teuren Page-Walk aus [HP19]. Sein *Reach* — Eintragszahl mal Seitengröße — begrenzt den ohne Page-Walk adressierbaren Speicherbereich. Prozessoren trennen aus Lokaltätsgründen den Instruktions- vom Daten-TLB (dTLB) — da Instruktionszugriffe deutlich lokaler sind als Datenzugriffe — und legen ihn mehrstufig aus (L1-dTLB plus gemeinsamer L2-STLB) [Int24b, Dre07]; für Suchstrukturen ist der dTLB die zweite, eigenständige Grenze neben der Cache-Line, denn große oder weit gestreute Knoten erhöhen den dTLB-Druck unabhängig von der Cache-Auslastung. Im Stand der Technik tritt der dTLB vorwiegend als *Mess-Ziel* auf; algorithmenseitig adressieren ihn große Seiten (Huge-Pages, 2 MiB/1 GiB), die den TLB-Reach vergrößern, von mehreren Allokatoren (mimalloc, jemalloc, TCMalloc, snmalloc) unterstützt werden und die Schmidt et al. [SKK⁺25] mit 2-MiB-Seiten empirisch nutzen — eine eigenständige dTLB-Optimierung bleibt in den Suchalgorithmus-Papern jedoch unterbelegt, eine Beobachtung, auf die die Forschungslücke in Kapitel 3 (Abschnitt 3.1) zurückkommt.

2.2.5 Schreibpfad, Ersetzung und Kohärenz

Auch das Schreib- und Update-Verhalten der Caches ist überwiegend mikroarchitektonisch fixiert. Schreibzugriffe folgen *Write-Through*, das unmittelbar bis in die nächste Ebene durchschreibt, oder *Write-Back*, das die Änderung zunächst nur in der Cache-Line vermerkt (*dirty*) und erst bei deren Verdrängung zurückschreibt — was Bus-Verkehr spart und die Schreiblatenz senkt; orthogonal dazu legt die Allokations-Politik (Write-Allocate, typisch bei Write-Back, bzw. No-Write-Allocate, typisch bei Write-Through) die Behandlung eines Schreib-Fehlzugriffs fest; ARMv8 macht diese Wahl über Seiten-Attribute — Write-Through, Write-Back oder Non-Cacheable samt Read- bzw. Read-Write-Allocate-Hinweisen — explizit [Arm24], während x86 im Normalfall Write-Back mit Write-Allocate verwendet [Int24b]. Welche Line verdrängt wird, entscheidet eine heuristische Ersetzungsstrategie von LRU/Pseudo-LRU bis zur zwei-Bit-sparsamen, scan- und thrash-resistenten *Re-Reference Interval Prediction* (dynamische Variante DRRIP) [JTJE10], die auf der adaptiven Insertions-Politik [QJP⁺07] aufbaut; wie viele Ebenen eine Line zugleich führen, regelt die Inklusions-Politik — ein *inklusive* Cache hält jede Line der höheren Ebenen redundant, ein *exklusiver* (Victim-Cache) schließt sie aus, ein *nicht-inklusive* lässt beides zu —, und in Mehrkern-Systemen sichert ein invalidierungsbasiertes Kohärenzprotokoll die Konsistenz: das verbreitete MESI führt jede Line in

einem der Zustände *Modified*, *Exclusive*, *Shared* oder *Invalid*, herstellerseitig um Varianten wie MOESI (AMD [Adv23]) oder MESIF (Intel [Int24a]) erweitert [HP19]. Als konkrete Instanz dieser fixierten Eigenschaften variiert etwa die L3-Inklusions-Politik zwischen den Generationen — inklusiv bei früheren Intel-Client-Generationen (etwa Nehalem), nicht-inklusiv bei Intel-Servern ab Skylake-SP/Xeon Scalable und überwiegend victim-artig bei AMD Zen [Int24a, Adv23]. Da all dies zur Laufzeit überwiegend nicht frei wählbar ist, sind diese Eigenschaften je Zielplattform gegeben und nur durch Messung zugänglich; ihre Wirkung — insbesondere das *Cache-Line-Ping-Pong* bei konkurrierenden Schreibzugriffen auf dieselbe Line — muss ein faires Messverfahren eigens sichtbar machen (Kapitel 3).

2.2.6 Cache-bewusste und cache-oblivious Datenstrukturen

Mit der Begriffs-Taxonomie aus Abschnitt 2.1 und den nun eingeführten Parametern lässt sich der Stand der Technik der cache-nahen Datenstrukturen ordnen. Den empirischen Anstoß der cache-bewussten Linie gaben Ailamaki et al., deren Zeit-Zerlegung kommerzieller Datenbanksysteme zeigte, dass rund die Hälfte der Ausführungszeit in Speicher-Wartezyklen liegt — vor allem in L2-Daten- und L1-Instruktions-Fehlzugriffen [ADHW99]. Die Antworten der Folgejahre binden je einen Teil der Parameter aus den vorigen Abschnitten explizit in den Entwurf ein: die Cache-Line-Größe (CSS-/CSB⁺-Baum [RR99, RR00]), die Prefetch-Distanz samt mehrzeiliger Knoten (pB⁺-Baum [CGM01], ebenen-übergreifend Fractal Prefetching [CGM02]), Seitengröße und SIMD-Breite zugleich (FAST, ein architektur-sensitives Baum-Layout mit verzahnter Seiten-, Line- und SIMD-Blockung [KCS⁺10]) oder den Verzweigungsgrad je Cache-Line (ART [LKN13]); die Layout-Techniken des Clustering und der Kompression gehen auf Chilimbi et al. zurück [CHL99]. Die cache-oblivious Gegenposition ist ebenso instanziiert: Bender et al. begründen den cache-oblivious B-Baum über das van-Emde-Boas-Layout [BDFC05], aufbauend auf dem Order-Maintenance-/Packed-Memory-Primitiv [BCD⁺02], der cache-oblivious Traversierung geordneter Mengen mit lokalitätserhaltender $O(k/B)$ -Schranke [BCDFC02] und dem Baum-Layout fester Topologie, das die erwartete Zahl der Blocktransfers in einem mehrstufigen Speicher-Modell auch bei unbekannter Block-Größe nahezu optimal minimiert [BDFC02, BDFC00]; Saikkonen und Soisalon-Soininen halten eine cache-sensitive Layout-Invariante auch unter Updates [SSS08, SSS16], und Graefe/Larson [GL01] vergleichen beide Ansätze über die CPU-Cache-Ebenen, während sämtliche praktischen Trie- und B⁺-Indizes cache-bewusst ausgelegt sind. Tabelle 2.1 ordnet die genannten Entwürfe danach, welche Hardware-Parameter sie explizit in den Entwurf binden.

Zwei Beobachtungen an der Tabelle tragen weiter. Erstens bindet jeder Entwurf eine *Handvoll* Parameter fest ein — kein bekannter Entwurf behandelt die Parameter-Menge der vorigen Abschnitte als offenen Eingaberaum. Zweitens lassen beide Philosophien dieselbe Frage offen: Wie lässt sich *gemessenes* Lokalitätswissen der konkreten Zielplattform — deren Cache-Eigenschaften überwiegend fest verdrahtet und nur zum kleineren Teil konfigurierbar sind (Abschnitt 2.2.5) — systematisch mit den tatsächlich wählbaren cache-nahen Einstellungen verbinden? Diese Frage nimmt Kapitel 3 (Abschnitt 3.1) wieder auf.

2.3 Workloads und Benchmark-Konventionen

Ob eine Suchstruktur die Hardware-Eigenschaften aus Abschnitt 2.2 gut nutzt, zeigt sich erst unter Last; dafür haben sich feste Begriffe und Benchmark-Konventionen etabliert. Eine *Last* (Operation)

Tabelle 2.1: Cache-nahe Struktur-Entwürfe und die Hardware-Parameter, die sie explizit in den Entwurf binden.

Entwurf	Cache-Line	Seite/TLB	SIMD-Breite	Prefetch-Distanz	parameterfrei
CSS-Baum [RR99]	•	—	—	—	—
CSB ⁺ -Baum [RR00]	•	—	—	—	—
pB ⁺ -Baum [CGM01]	•	—	—	•	—
Fractal Prefetching [CGM02]	•	•	—	•	—
FAST [KCS ⁺ 10]	•	•	•	—	—
ART [LKN13]	○	—	•	—	—
Cache-oblivious B-Baum [BDFC05]	—	—	—	—	•
Knotengrößen-Studie Hankins/Patel [HP03]	•	—	—	—	—

Legende: • = expliziter Entwurfsparameter der Originalarbeit; ○ = indirekt berücksichtigt (ART staffelt die Knotengrößen cache-line-orientiert, ohne die Line-Größe zu parametrisieren); — = nicht adressiert. Einordnung dieser Arbeit auf Basis der zitierten Originalarbeiten.

ist ein einzelner Aufruf der Such-Schnittstelle — eine Punktanfrage, ein Einfügen, ein Löschen oder ein Bereichs-Scan. Ein *Workload* (in dieser Arbeit synonym: *Lastprofil*) ist ein strukturiertes Profil solcher Operationen: ein Operations-Mix (etwa 50 % Lesen / 50 % Aktualisieren) über einer Schlüssel-Zugriffsverteilung (gleichverteilt, Zipf-verteilt, jüngste Werte) auf einem Datensatz definierter Größe und Schlüssel-Charakteristik. Realistische Workloads sind dabei Folgen vieler aufeinanderfolgender Interface-Aufrufe — nicht isolierte, unrealistische Einzelzugriffe.

Die verbreitetste Workload-Konvention ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]. Er definiert eine Familie von Schlüssel-Wert-Workloads: YCSB-A (50/50 Lesen/Aktualisieren, Zipf-verteilt), YCSB-B (95/5 Lesen/Aktualisieren), YCSB-C (nur Lesen), YCSB-D (jüngste Werte), YCSB-E (kurze Bereichs-Scans) und YCSB-F (Read-Modify-Write). Daneben stützt sich die Literatur auf weitere etablierte Familien: den Index-Benchmark *SOSD* [KMR⁺19], die Benchmarks der *TPC*-Familie (transaktional TPC-C, analytisch TPC-DS) [Tra24], die Integration in reale Systeme (etwa SuRF [ZLL⁺18] als Bloom-Filter-Ersatz in RocksDB), reale String-Korpora (URLs, DNA, Wörterbuch-Terme) sowie — bei den hardware-nahen Arbeiten — *SPEC CPU* [Sta17] und *CloudSuite* [FAK⁺12]. Diese Frameworks erzeugen unterschiedliche, untereinander nicht unmittelbar vergleichbare Lasten.

Das *Ziel der Workload-Variation* ist der Bruch eines methodischen Bias: jede Veröffentlichung wählt typischerweise das Lastprofil, unter dem ihr eigener Algorithmus am besten abschneidet. Erst wenn alle Lastprofile gegen alle Vergleichskandidaten laufen, lässt sich bias-reduzierte Vergleichbarkeit gewinnen. Für diese Arbeit liefern die genannten Konventionen das Rohmaterial eines gemeinsamen Lastprofil-Satzes; wie dieser Satz aufgebaut ist und über alle Kandidaten gefahren wird, entwickelt Kapitel 3, die zugehörige Mess-Matrix gehört zur Evaluationsmethodik (Kapitel 5).

Beobachtet wird unter Last mit einem etablierten Satz von Kenngrößen. Auf Cache-Ebene zählen die Fehlzugriffe je Ebene, ausgedrückt als *Miss-Rate* (Fehlzugriffe je Zugriff) oder — robuster beim Vergleich verschieden zugriffsintensiver Programme — als *MPKI* (Misses je tausend Instruktionen); auf Kern-Ebene messen *IPC* beziehungsweise *CPI* (Instruktionen je Takt und dessen Kehrwert) die erreichte Ausführungsdichte, und Stall-Zyklen-Zerlegungen weisen aus, welcher Anteil der Taktzyklen auf wartende Speicherzugriffe entfällt [HP19]; dass dieser Anteil bei Datenbank-Lasten dominieren kann, war gerade der Befund von Ailamaki et al. [ADHW99] (Abschnitt 2.2.6). Für cache-bewusste Layouts kommt die *Cache-Line-Auslastung* hinzu — der Anteil tatsächlich genutzter Bytes je geladener Line (Abbildung 2.2) —, die unmittelbar misst, was ein Layout an der Übertragungseinheit gewinnt.

Erhoben werden diese Größen über die *Performance-Monitoring-Counter* (PMC) der Prozessoren: wenige programmierbare Zählregister je Kern, denen per Event-Auswahl ein Hardware-Ereignis (etwa L2-Miss oder dTLB-Miss) zugewiesen wird; zugänglich machen sie Werkzeuge wie `perf` [Mel10] oder VTune [Rei05]. Ihre Grenzen sind ebenso dokumentiert: Zählerstände sind über Wiederholungen nicht streng deterministisch und zählen je nach Ereignis systematisch über [WM08], und sollen mehr Ereignisse beobachtet werden, als Register existieren, multiplext das Werkzeug die Zähler — das Ergebnis ist dann eine Hochrechnung, keine Zählung. Ein belastbarer Vergleich braucht darum Wiederholungen und eine ehrliche Streuungs-Angabe [HB15, WM08]. Welche dieser Kenngrößen diese Arbeit erhebt und wie sie zu Mess-Kategorien geordnet werden, entwickelt Kapitel 3.

2.4 Software-Mittel und Zusammenführung zur Abstraktion

Mit welchen Software-System-Mitteln — Übersetzungszeit-Metaprogrammierung, statische Nutzung der Hardware-Eigenschaften aus Abschnitt 2.2 — lässt sich die dort offen gebliebene Abstraktion *umsetzen*? Zur Verfügung stehen die etablierten Mittel des software-technischen Entwurfs: die benannten Entwurfsmuster [GHJV94], der statische Polymorphismus des *Curiously Recurring Template Pattern* samt *Policy-based Design* [Ale01] sowie die Komposition aus austauschbaren Bausteinen, wie sie die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03], die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92] beschreiben; C++23-Module strukturieren die Übersetzungseinheiten solcher Baustein-Kompositionen, garantieren für sich genommen aber noch keine portable ABI-Stabilität [Xu24] — relevant, sobald vorübersetzte Bausteine ausgetauscht werden sollen.

Damit aus austauschbaren Bausteinen ein konkretes System wird, braucht es neben den Kompositions-Techniken eine *Beschreibungsform* für die jeweils gewählte Konfiguration. Die etablierte Antwort sind *deklarative Konfigurationsmodelle*: Feature-Modelle beschreiben seit der FODA-Studie konfigurierbare Variantenräume als Merkmal-Hierarchien mit Auswahl-Constraints [KCH⁺90], die generative Programmierung erzeugt aus solchem Konfigurationswissen im Problemraum die Systemvariante im Lösungsraum [CE00], und die Produktlinien-Literatur systematisiert die zugehörigen Bindungszeitpunkte von der Übersetzungszeit bis zur Laufzeit [ABKS13]. Der industrielle Werkzeugbau folgt demselben Prinzip: Mavens *Project Object Model* ist die deklarative — als XML-Datei gehaltene — Manifestation dessen, was ein modulares Java-System ausmacht, während der Build-Lebenszyklus das *Wann* und *Wie* übernimmt [Apa26]; CMake erzeugt aus einer deklarativen Beschreibung über austauschbare *Generatoren* je Plattform ein natives Buildsystem [Kit26]; und Mokhov, Mitchell und Peyton Jones zeigen, dass sich Build-Systeme insgesamt als Kombination weniger Scheduler- und Rebuilder-Bausteine über deklarativen Abhängigkeitsbeschreibungen wissenschaftlich fassen und neu zusammensetzen lassen [MMP18]. Dass deklarative Modelle ein System über die Bauzeit hinaus auch im Betrieb begleiten können, entwickelt die `models@run.time`-Linie [BBF09]; Götz zeigt in der TU-Dresdner MQuAT-Dissertation, wie deklarative (Konfigurations-)Modelle die Selbstoptimierung komponentenbasierter Systeme durch Kontrakt-Verhandlung steuern [Göt13].

Für diese Arbeit bilden beide Stränge zusammen den Werkzeugkasten: die Kompositions-Techniken, mit denen sich die je Zielplattform gemessenen Hardware-Eigenschaften zur Übersetzungszeit in ein konkretes Layout umsetzen lassen, und die deklarativen Konfigurationsmodelle, mit denen sich die gewählte

Konfiguration beschreiben und reproduzieren lässt; wie beide dafür kombiniert werden, entwickelt Kapitel 3.

2.5 Heuristiken als Entscheidungsregeln

Sowohl die adaptiven und gelernten Indizes (Abschnitt 2.1) als auch die Ersetzungsstrategien der Caches (Abschnitt 2.2) treffen ihre Entscheidungen *heuristisch*. Eine *Heuristik* ist eine Entscheidungsregel ohne Optimalitätsgarantie: Sie bildet beobachtbare Merkmale — etwa die je Zielplattform ermittelten Cache-Eigenschaften (Abschnitt 2.2) oder das gemessene Lastprofil (Abschnitt 2.3) — auf eine konkrete Konfiguration ab, ohne deren Bestmöglichkeit zu beweisen. Sie tritt überall dort an die Stelle einer exakten Lösung, wo diese unbekannt oder zu teuer zu bestimmen ist; die heuristischen Ersetzungsstrategien des Caches selbst (LRU, RRIP; Abschnitt 2.2) sind das hardwareseitige Vorbild dieser Denkweise [JTJE10]. Heuristiken lassen sich danach unterscheiden, *wann* ihre Entscheidung fällt. Eine *statische Heuristik* ist zur Übersetzungszeit fixiert: Die gewählte Konfiguration wird fest in das übersetzte Programm einkompiliert und ändert sich zur Laufzeit nicht mehr. Eine *dynamische Heuristik* wählt die Konfiguration hingegen zur Laufzeit, abhängig vom tatsächlich beobachteten Verhalten, und kann sie im Betrieb anpassen. Beide Spielarten setzen dieselbe *Heuristik-Funktion* voraus, verschieben aber den Zeitpunkt ihrer Auswertung. Formal ist diese Heuristik-Funktion ein *Mapping* von einem Messergebnis beziehungsweise Lastprofil auf eine Konfiguration: $h : \mathcal{M} \rightarrow \mathcal{K}$, wobei $\mathcal{M}$ den Raum der erhobenen Merkmale (Messwerte, Lastprofil-Kennzahlen) und $\mathcal{K}$ den Raum der zulässigen Konfigurationen bezeichnet. Die vorliegende Arbeit misst diesen Zusammenhang zunächst systematisch aus (Kapitel 3); die *automatisierte* Bestimmung von h selbst — der Mechanismus, der aus den Messergebnissen die jeweils beste Konfiguration ableitet — ist Zielsetzung und Ausblick dieser Arbeit (Abschnitt 6.3).

Schließlich sind die in der Literatur teils synonym gebrauchten Begriffe abzugrenzen. *Adaptiv* heißt eine Heuristik, die ihre Entscheidung an veränderliche Bedingungen anpasst; *dynamisch* bezieht sich enger auf den Zeitpunkt — die Auswertung zur Laufzeit statt zur Übersetzungszeit; und *messwertgetrieben* (*measurement-driven*) betont die Datenquelle — die Entscheidung gründet auf erhobenen Messwerten und nicht auf festen Modellannahmen. Eine adaptive Heuristik ist somit stets dynamisch, eine dynamische nicht notwendig adaptiv, und beide können, müssen aber nicht messwertgetrieben sein.

Damit sind die Grundlagen beisammen: die Landschaft der Suchstrukturen, die Hardware-Eigenschaften, die ihre Leistung prägen, die Workload-Konventionen, unter denen sie verglichen werden, die Software-Mittel der Abstraktion und der Heuristik-Begriff, der Messwissen mit Konfigurationswahl verbindet. Wie sich mit genau diesen Mitteln ein Suchalgorithmus in orthogonale, einzeln vermessbare Entwurfsentscheidungen zerlegen lässt, entwickelt Kapitel 3; die konkrete Umsetzung der dort eingeführten Bausteine zeigt Kapitel 4.

3 Konzepte eines cache-bewussten Mess-Systems

Auf den Grundlagen aus Kapitel 2 aufbauend entwickelt dieses Kapitel zunächst die Problemstellung und die Anforderungen an ein Mess-System und als deren *Lösung* das Anatomie-Modell und das darauf aufbauende Mess-System — demonstriert am Prüfling PRT-ART.

3.1 Problemstellung und Anforderungen an ein Mess-System

Den Ausgangspunkt liefert der Befund der Grundlagen (Kapitel 2): Über die Leistung cache-bewusster Suchstrukturen entscheidet nicht die asymptotische Komplexität, sondern das Cache-Line-Verhalten unter realer Last. Dieses Verhalten ist jedoch das Ergebnis vieler ineinandergreifender Entwurfsentscheidungen — wie Knoten verzweigen, wie Seiten im Speicher angeordnet werden, wie alloziert, vorgeladen und synchronisiert wird —, und in den publizierten Verfahren sind diese Entscheidungen monolithisch verwachsen: Wer ein Verfahren vermisst, vermisst stets das ganze Bündel, nie die einzelne Entscheidung. Genau das ist die Problemstellung dieser Arbeit, das *Trennbarkeits-Problem*: Der Beitrag jeder einzelnen Entwurfsentscheidung zum Cache-Line-Verhalten soll isoliert sichtbar werden — je Entscheidung wie für den ganzen Algorithmus.

Der Stand der Technik verschärft dieses Problem in zwei Richtungen. Inhaltlich optimiert jede der untersuchten Arbeiten nur wenige Entwurfsentscheidungen isoliert, und im untersuchten Korpus belegt *kein einziges Verfahren alle Entwurfsdimensionen gemeinsam*: Manche Dimensionen sind ausschließlich mit Baseline- oder Lehrbuch-Varianten besetzt (das Mapping zwischen Algorithmus- und Cache-Traversierung), andere treten nur als *Mess-Ziel* auf (der dTLB), die räumliche Such-Klasse fehlt ganz, und Entscheidungen wie Ein-/Ausgabe, Migration, Filter und Pufferung sind zwar je einzeln, aber nie gemeinsam mit den Such-Entscheidungen in einem Entwurf belegt. Methodisch kommt eine zweite Lücke hinzu: Ergebnisse werden in nicht vergleichbaren Settings berichtet, und die häufige Wahl szenariospezifischer, oft günstig zugeschnittener Lastprofile erschwert einen fairen Quervergleich; es fehlt eine bias-reduzierte Vermessung *aller* Lastprofile gegen *alle* Verfahren auf einer gemeinsamen Basis.

Aus dieser doppelten Lücke folgen vier Anforderungen an eine Lösung. Sie muss cache-bewusste Suchalgorithmen erstens in austauschbare Entwurfsbestandteile *zerlegen*, sodass jede Entscheidung einzeln fassbar wird; zweitens die Stand-der-Technik-Entwürfe als explizite Konfigurationen dieser Bestandteile *rekonstruieren*, damit Bekanntes und Neues im selben Raum vergleichbar werden; drittens das cache-line-bewusste Verhalten je Bestandteil wie für den ganzen Algorithmus auf gemeinsamer Basis *messen* — alle Lastprofile gegen alle Verfahren, bias-reduziert; und viertens aus den Messungen den besten Gesamtalgorithmus ableiten und systemoptimiert *ausliefern*. Der cache-aware-Fokus ist dabei keine Randbedingung, sondern die Leitgröße: Genau das Cache-Line-Verhalten, das über die Leistung entscheidet, soll je Entwurfsentscheidung isoliert messbar werden.

Die Mess-Anforderung präzisiert sich an der Gegenüberstellung cache-bewusster und cache-oblivious Entwürfe (Abschnitt 2.2) zu einem dritten, dazwischenliegenden Weg: Die tatsächlichen Cache-Eigenschaften

(Line-Größe, Hierarchie-Tiefe, Topologie) werden je Zielplattform *ermittelt* und gehen als feste Größen in eine systemoptimierte Binary ein, während die *konfigurierbaren* cache-nahen Einstellungen (Prefetch-Distanz, Stapel- und Inline-Schwellen, Pool-Budgets sowie, wo Architektur und Betriebssystem es erlauben, der Hardware-Prefetcher) als Achsen-Parameter gegen eine bereits geladene Binary permutiert werden — abgestuft nach dem, was Hardware und Betriebssystem erlauben, von voll laufzeit-konfigurierbar (mutables Linux mit MSR-Zugriff) über boot- bzw. übersetzungsstatisch (immutable Betriebssysteme) bis fixiert (read-only-Cache-Register auf ARM und RISC-V). So wird cache-oblivious-Verhalten um *gemessenes* Lokaliätswissen ergänzt.

Die vier Anforderungen verdichten sich zu einem Zielbild, dem *Entwurfsraum*: Jede Teilentscheidung — etwa Knotentyp, Speicher-Layout oder Allokator — spannt eine eigene Achse auf, und jeder Stand-der-Technik-Algorithmus ist ein Punkt im kartesischen Produkt dieser Achsen (Abbildung 3.1).

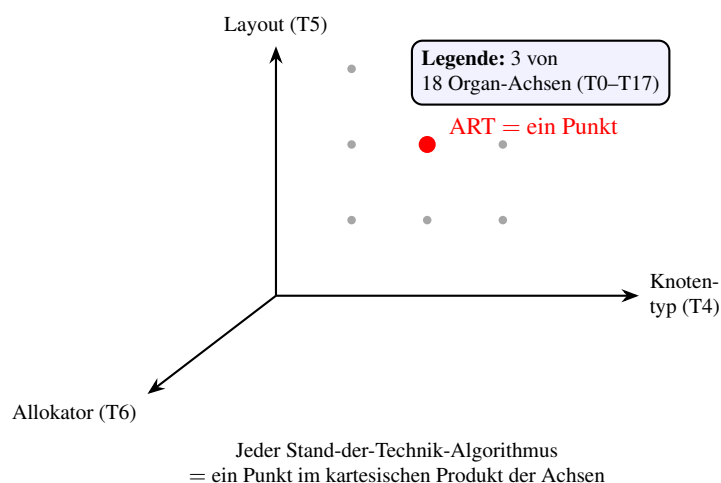


Abbildung 3.1: Der Entwurfsraum: die Achsen spannen ein kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der achtzehn Organ-Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat achtzehn Organ-Dimensionen.

Ist der Entwurfsraum so aufgespannt, wird das Trennbarkeits-Problem lösbar: Eine Entwurfsentscheidung zu variieren heißt, sich entlang genau einer Achse zu bewegen, während alle übrigen fixiert bleiben — ihr Beitrag wird isoliert messbar. Was diese Arbeit beiträgt, ist folglich die systematische, orthogonale Achsen-Zerlegung der bekannten Entwürfe samt einer *permutierten Mess-Architektur*, die sie als explizite Konfigurationen rekonstruiert und auf gemeinsamer Basis vermisst; denn cache-aware Algorithmen sind ohne eine Aufteilung der das Caching maßgeblich bestimmenden Eigenschaften nicht differenzierbar. Die vier Anforderungen beschreiben dabei zugleich eine Verarbeitungs-Kette — von der Definition einer Achsen-Belegung über ihre Erzeugung und Vermessung bis zur Auswertung —, und ein System dieser Größe bleibt nur beherrschbar, wenn diese Stationen als getrennte Schichten mit festen Verträgen entwickelt werden; das Schichtenmodell dazu entwickelt Abschnitt 3.3.4. Welche der katalogisierten Instanzen das Framework übernimmt und welche es als überholt ersetzt, entwickelt der folgende Abschnitt 3.2 dialektisch; das Lösungskonzept (Abschnitt 3.3) und das darauf aufbauende Mess-System (Abschnitt 3.4) sind die Lösung, die die doppelte Lücke schließt.

3.2 Stand der Technik als Achsen-Instanzen

Die aus Kapitel 2 bekannten Verfahren erscheinen nun systematisch als Achsen-Kompositionen; die vollständigen Kataloge dieser Sezierung versammelt der vorliegende Abschnitt. Diese Aneignung verläuft *dialektisch* (These–Antithese–Synthese): je Achse wird ein fremdes Organ *übernommen* (These) oder eine überholte Annahme *ersetzt* (Antithese), woraus der eigene Beitrag als Synthese hervorgeht. Leitend ist der „dritte Weg“ aus Abschnitt 3.1 — gemessenes Lokaliätswissen statt fest verdrahteter oder ignorierter Cache-Bewusstheit —, der jede Achse als orthogonales, permutierbares und einzeln messbares Organ explizit macht. Bei vielen Achsen ist die Antithese keine überholte Fremd-Instanz, sondern die implizite Annahme, die Entscheidung gar nicht als wählbar zu betrachten; die Synthese besteht dann darin, sie überhaupt als Achse explizit zu machen (in der Ersetzen-Spalte als „—“ markiert). Zwei Cluster-Zuordnungen sind dabei bewusst Näherungen: Die Cache-Traversal-Achse (T1) bezieht ihre Anregungen sowohl aus der Layout-Theorie als auch aus dem Prefetching, und die Reklamations-Unterachse des Allokators (T6) teilt ihre Quellen mit der Synchronisation (T8) — eine Inter-Organ-Verdrahtung, keine Cluster-Verletzung. Tabelle 3.1 fasst diese Aneignung je Kompositions-Achse zusammen.

Tabelle 3.1: Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in kanonischer Slot-Reihenfolge T0–T17; die frühere Zählung T0–T18 führte Telemetrie und ISA noch als Kompositions-Achsen — beide sind in die System-Sphäre umgezogen, Abschnitt 3.4.1. Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware/Messung/Engineering)

Achse		übernommen (These)	ersetzt (Antithese)	Synthese-Kern
T0	Search-Algorithm (A)	ART, HOT, START, SuRF, Wormhole; k-äre/Interpolations-Suche	Monolith „Struktur ist Verfahren“	Such-Paradigma als permutierbare Achse
T1	Cache-Traversal (B)	Cache-Line-Stride (Chen), cache-oblivious (Bender), HBM (Schmidt)	implizite Lehrbuch-Linearsuche	Algorithmus- und Cache-Sicht getrennt
T2	Mapping (B)	lineare 1:1-Abbildung; Hash-Redirect, Permutations-Index	—	Direct- vs. Pool-Relative-Mapping explizit
T3	Path-Compression (A)	byte-weise (ART), Patricia (HOT), Macro-Node (CoCo), Multi-Byte (START)	unkomprimierter Rohpfad	Kompressions-Granularität wählbar
T4	Node-Type (A)	ART Node4/16/48/256	feste Knotengröße	adaptive Knotenklasse + Storage-Delegation
T5	Memory-Layout (B)	AoS-aligned, SoA, LOUDS-bitgepackt, CSS/CSB ⁺ , Ao-SoA	—	vier Layout-Dimensionen permutierbar
T6	Allocator (C)	A01–A23 (Hoard, mimalloc, jemalloc, TCMalloc, ...) + RCU/Hazard	algorithmus-fest verdrahteter Allokator	Allokation als gemeinsames Interface-Organ (sieben Sub-Achsen)
T7	Prefetch (D)	Distance-Estimator (ART), Hardware (Wormhole), Stride (Chen), adaptiv	Prefetch nicht als Achse isoliert	Path-Oriented Bundle-Prefetch (eigen)
T8	Concurrency (E)	OLC (ART), RCU, Hazard-Pointer; Lock-/Wait-Free	grobes Sperren als einzige Annahme	Pattern $\perp$ Reclamation als zwei Sub-Achsen
T9	Serialization (F)	Raw-Binary, Varint (ART), succinct (LOUDS/FST), LZ77	—	Kodierung/Kompression wählbar

Achse	übernommen	ersetzt	Synthese-Kern
T10 Value-Handle (F)	Inline, External-Pool, Immutable-Shared (RCU), Versioned	implizit fest eingebetteter Wert	Platzierung, Ownership und Versionierung (+ Chain-Ref)
T11 Index-Organization (B)	Clustered, IOT, Non-Clustered, Heap	—	Storage-Order als eigene Achse
T12 I/O-Dispatch (F)	Buffered, mmap, Direct-IO	mmap als Anti-Pattern	Persistenz von der Suchstruktur entkoppelt
T13 Migration (F)	Hot/Cold, Multi-Tier NVM, LeCaR-ML	Tiering nie mit Suchachsen kombiniert	Migrationslogik als messbare Achse
T14 Filter (F)	Bloom, Cuckoo, SuRF-Range, Xor	Filter nie mit Suchachsen integriert	Filter als orthogonales Organ
T15 Queuing-q1 (F)	Disruptor-Ring, Delta-Chain, CoW, Epoch-RCU, SPSC/MPMC	Operations-Puffer nie als Achse	Puffer mit Zugriffsmuster-Klassen
T16 Queuing-q2 (F)	Eager/Lazy, Watermark (LSM), Timed	Flush-Policy implizit	adaptive EWMA-Flush-Policy (eigen)
T17 Persistence-Target (F)	In-Memory-Ablage als Standard-Ziel	implizite In-Memory-Annahme	Persistenz-Ziel als eigene Achse (im Vermessungskatalog auf In-Memory gepinnt)
<i>In die System-Sphäre umgezogen (Abschnitt 3.4.1):</i>			
target_isa (System)	Vendor-Spec x86-64/AArch64/Power/RISC-V (+ SIMD)	—	Ziel-ISA als System-Komplex-Achse
Telemetrie (System/Mess)	Density-Tracker, Insert-Zähler, HdrHistogram, Pfad-Zähler	All-Node-Zähler (Cache-Line-Ping-Pong)	kohärenz-schonend Leaf-Only; zweigeteilt: Mess-Unter-Achse + Bau-Größe

Der systematische Kern ist die achsenweise *Sezierung*: Jeder Gesamt-Algorithmus wird in seine Organe zerlegt, und je Achse werden die beitragenden Organ-Algorithmen aufgeführt, mit kurzem Verweis auf das Herkunfts-Lebewesen. Das Framework führt *achtzehn Organ-Kompositions-Achsen* in fester Slot-Reihenfolge (T0–T17); daneben stehen die Größen der System-Sphäre — Ziel-ISA, SIMD- und Erweiterungs-Hardware sowie die zweigeteilte Telemetrie (die Dreiteilung der Achsen ordnet Abschnitt 3.4.1) — und die Prüfling-Build-Achse Seitentyp. Die frühere Neunzehn-Achsen-Zählung (T0–T18, Zwischenstand Juli 2026) ist durch diese Ordnung abgelöst. Viele Achsen sind in Sub-Achsen verfeinert (etwa SA1–SA4 oder AA1–AA7) und liefern anderen Achsen verteilte Interfaces für deren Detail-Operationen — genau diese *Achsen-Interfaces* trägt später das Mikro-Benchmarking (Abschnitt 3.4.5). Tabelle 3.2 gibt den Überblick, welche Veröffentlichungen je Achse ein Organ beitragen; die thematischen Herkunfts-Cluster der Vorarbeiten erscheinen dabei *aufgelöst* in ihren Organ-Beiträgen.

Die *Such-Achse* (T0) trägt die meisten Organe — als Original-Bindungen ART [LKN13], HOT [BZP⁺18], START [FJKN20], SuRF [ZLL⁺18] und Wormhole [WNJ19], dazu generische Such-Methoden wie k-äre Suche [SGL09], Interpolationssuche [PIA78], Eytzinger-Layout-Suche [KM17a], Skip-Liste [Pug90], B-Baum [BM72] und Open-Addressing-Hashing [Knu98]. Die Traversierungs-Achsen trennen Algorithmus- und Cache-Sicht: T1 reicht von der Cache-Line-Wanderung über Stride-Prefetch [CGM01] und HBM-Nähe [SKK⁺25] bis zur cache-oblivious-Rekursion [FLPR99], während T2 (Mapping) über lineare

Tabelle 3.2: Achsen-Sezierung der Suchverfahren: achtzehn Organ-Achsen in kanonischer Slot-Reihenfolge (T0–T17), die in die System-Sphäre umgezogenen Größen und die Prüfling-Build-Achse; je Achse die ein Organ beitragenden Quellen.

Achse	Sub-Achsen	Beitragende Quellen
T0 search_algo	SA1–SA4	P01,P02,P05,P07,P10; k-ary (TUD 2009), Interpol., Eytzinger, Skip-List, B-Baum, Hash
T1 cache_traversal	CT1–CT2	Cache-Line-Default; P17,P21,P32
T2 mapping	MP1–MP2	P03,P07; CE-Baseline
T3 path_compression	PC1–PC3	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM4	P06,P09,P11,P12,P32; Default
T6 allocator	AA1–AA7	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	CC1–CC2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	SR1–SR3	P01,P09,P10; LZ77
T10 value_handle	VH1–VH3	P01,P03,P07,P29
T11 index_organization	io1–io3	Bayer/McCreight,Comer,Oracle
T12 io_dispatch	IO1–IO3	Stonebraker,Crotty; Baselines
T13 migration_policy	MG1–MG3	Anti-Caching,NVM-Tiering,LeCaR; None
T14 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T15 queuing_q1	QS1–QS6	Disruptor, Bw-Tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T16 queuing_q2	FS1–FS4	Eager/Lazy/Watermark/Kafka/RocksDB
T17 persistence_target	—	CE-Baseline (In-Memory); Disk-Writeback als abgeschaltete Option
Sys. target_isa	IS1–IS3	x86-64/AArch64/Power/RISC-V (Vendor-Spec)
Sys. external_utils: SIMD	SE1–SE3	SSE2/AVX2/AVX-512/NEON/SVE2/RVV; GPU: CUDA
Sys. external_utils: Erw.-HW	HW1–HW4	HW-Strategie je Paper (Tab. 3.4)
Sys./Mess telemetry	TM1–TM3	P01,P02,P16,P17,P26,P28; HdrHistogram (Gil Tene)
Build (Prüfling) page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (2026)

1:1-Abbildung, Hash-Redirect [WNJ19] und Permutations-Index [MKM12] eine dünn besetzte Achse bleibt. Die *Pfadkompression* (T3) instanziiert die byte-weise Variante [LKN13], den Patricia-Split [Mor68, BZP⁺18], die Macro-Node [BFTV24] und das Multi-Byte-Rewiring [FJKN20], und die *Knotentyp-Achse* (T4) bündelt die adaptiven Layouts von Node4/16/48/256 [LKN13] über HOT-Compound [BZP⁺18], Internal/Border [MKM12], Decision/Span [SSL⁺21], LOUDS-Dense/Sparse [ZLL⁺18] und die CSS-/CSB⁺-/Hankins-Knoten bis zur PRT-ART-Familie.

Die *Speicher-Layout-Achse* (T5) reicht vom cache-line-ausgerichteten AoS über zeigerfreie Anordnung [RR99], Geschwister-Cluster [RR00] und eingebetteten Baum [SSL⁺21] bis zum bit-gepackten LOUDS [Jac89]. Die *Allokator-Achse* (T6) ist mit dem Korpus A01–A23 am dichtesten besetzt und in sieben Sub-Achsen verfeinert — von der Free-List-Topologie über Thread-Lokalität/NUMA-Affinität [UHL⁺17] und die Allokationsstrategie bis zur Reklamation (Epoch, RCU [McK01], Hazard-Pointer [Mic04a], QSBR); der Code stellt 25 Allokator-Implementierungen zum 23-Arbeiten-Korpus bereit (einzelne Arbeiten steuern mehrere Varianten bei), von denen Tabelle 3.3 die zehn lauffähigen Profile mit Lizenz und Workload-Affinität listet. Die *Wert-Achse* (T10) unterscheidet Inline-Wert [LKN13], externen Pool [WNJ19], versionierten Zeiger [MKM12] und immutable Shared-Ref [McK01], die *Serialisierungs-Achse* (T9) Raw-Binary, succinct LOUDS/FST und Varint [LKN13].

Tabelle 3.3: Allokator-Profile mit Lizenz und Workload-Affinität

Profil	Familie	Lizenz	expected_workload
hoard	A01	Apache-2.0	YCSB_A
michael_lockfree	A03	LGPL-2.1-or-later	YCSB_B
mimalloc	A04	MIT	YCSB_A
jemalloc	A05	BSD-2	YCSB_B
tcmalloc	A06	Apache-2.0	YCSB_C
snmalloc	A07	MIT	YCSB_A
scalloc	A08	BSD-3	YCSB_A
rpmalloc	A10	Public Domain	YCSB_C
lrmalloc	A11	MIT	YCSB_B
dlmalloc	A20	CC0	YCSB_C

expected_workload = YCSB-Workload-Klasse (A/B/C).

Auf der hardware-nahen Seite reicht die *Prefetch-Achse* (T7) von keinem Prefetch über software-festes [CGM01], adaptiv-distanziertes [Kha11] und fill-buffer-bewusstes [MWR25] bis zu pfadorientiertem [ZSZ⁺24] und hierarchischem Bündel-Prefetch [ZGH⁺25]; die explizite Hardware-Prefetch-Instruktion stammt aus Wormhole [WNJ19]. Die System-Achse *Ziel-ISA* und die SIMD-Erweiterung des Erweiterungs-Hardware-Hubs (beide System-Sphäre, Abschnitt 3.4.1) führen die Vendor-Spezifikationen (x86-64, AArch64, Power, RISC-V mit SSE2/AVX2/AVX-512, NEON/SVE2, RVV). Zwei übergreifende Strategie-Achsen halten fest, was ein Algorithmus aktiv *nutzt* bzw. wie er Arbeit *verteilt* — die *Hardware-Strategie* (SIMD-Familie, Cache-Level-Targeting, NUMA, Prefetch-Hardware, Atomic-Familie) und die *Scheduling-Strategie* (Worker-Pool-Layout, SIMD-Worker-Limit — hardwarebedingt typisch zwei von N Kernen —, heterogener P-/E-Kern-Dispatch, Co-Routinen und Batch-Granularität) —, die Tabelle 3.4 je Paper zuordnet. Die *Telemetrie* — in der Dreiteilung dieser Arbeit keine Organ-Achse mehr, sondern zweigeteilt in Mess-Tooling-Unter-Achse und Bau-Größe der System-Sphäre (Abschnitt 3.4.1) — instanziiert Density-Tracker [LKN13], Insert-Zähler [BZP⁺18], HdrHistogram, Pfad-Lese-Zähler [ZSZ⁺24] und die Probability-Hints (P16/P17 Bender [BDFC00, BDFC05]); die kohärenz-schonenden Leaf-Only-Varianten gehen auf Kuehn zurück (P28 [Kue23]; persönliche Kommunikation 2023, über den publizierten DaMoN-2023-Stand zum cache- und verteilungsbewussten B⁺-Layout hinaus), während der Zähler in allen (auch inneren) Knoten als *Anti-Pattern* (Cache-Line-Ping-Pong) gilt.

Die *Nebenläufigkeits-Achse* (T8) gliedert sich in die zwei Sub-Achsen Muster und Reklamation (Pattern $\perp$ Reclamation, Tabelle 3.1): Die Muster reichen vom seriellen Nullpunkt über grobes Sperren [Dij65], Leser-/Schreiber-Sperren [CHP71] und optimistisches Lock-Coupling [LABN16] zu lock-freien [MS96], wait-freien [Her91] sowie RCU- [McK01] und Hazard-Pointer-Verfahren [Mic04a]; der Sperr-Modus innerhalb eines Musters unterscheidet read-only, read-write, upgradeable und optimistische Validierung. Die *I/O-Achse* (T12) instanziiert gepufferte [Sto81], direkte und memory-mapped Ein-/Ausgabe [CLP22]. Die Achse setzt dabei auf dem Blocktransfer-Denken der External-Memory-Tradition auf (Abschnitt 2.2): Für das I/O-effiziente Datenstruktur-Engineering stellt TPIE eine Referenz-Bibliothek der External-Memory-Linie bereit (ESA 2002) [APV02]; die Lehrbuch-Grundlage liefern Mehlhorn und Sanders [MS08]. Die *Migrations-Achse* (T13) instanziiert Hot/Cold-Trennung und mehrstufige RAM $\rightarrow$ NVM $\rightarrow$ SSD-Migration, und die *Pufferungs-Achsen* (T15/T16) reichen vom Disruptor-Ringpuffer [TFB⁺11] über die Bw-Tree-Delta-Kette [LLS13], den Copy-on-Write-Puffer [DSST89] und den Tombstone-Puffer [OCGO96] bis zu lock-freien SPSC- [Lam77] und MPMC-Queues [MS96]. Die *Index-Organisations-Achse* (T11) unter-

Tabelle 3.4: In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profile je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; der Vollkatalog folgt im Anhang).

Paper	Hardware-Strategie	Scheduling-Strategie
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (Bit-Vektor), L1-aware, CAS	thread-per-core
P03 Masstree	scalar, L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (Multi-Byte-Span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (Instr.-Bundle), all-cache, CAS	thread-per-core + Co-Routine
P28 Kuehn	PREFETCH (scalar Counter), L1-aware, CAS	thread-per-core + Sampling
P31 Ungethuen	AVX-512 (HBM), HBM-aware, NUMA-aware, CAS	hybrid (P-/E-Kerne)
P32 Schmidt	AVX-512 (SIMD-Stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-Worker-Limit + hybrid + Co-Routine + micro-batch

scheidet geclusterte Anordnung [BM72], Heap-Ablage und nicht-geclusterten Sekundärindex [Com79], die *Filter-Achse* (T14) den Bloom- [Blo70], Cuckoo- [FAKM14], SuRF- [ZLL⁺18] und Xor-Filter [GL20]. Die *Persistenz-Ziel-Achse* (T17) schließlich macht das Ablage-Ziel explizit — In-Memory als Standard, ein Platten-Rückschreibe-Ziel als abgeschaltete Option —; im kanonischen Vermessungs-Katalog ist sie auf ihren In-Memory-Vertreter gepinnt, damit der Permutationsraum bei 2^{17} Identitäten bleibt (Abschnitt 3.4.5).

Auch die *Lasten* erscheinen als Instanzen. Die in der Literatur verwendeten Workloads entstammen *mehreren*, untereinander nicht vergleichbaren Benchmark-Frameworks; Tabelle 3.5 ordnet sie den nutzenden Veröffentlichungen zu. Am verbreitetsten ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; daneben treten der Index-Benchmark SOSD [KMR⁺19], die TPC-Familie [Tra24], die CPU-Suite SPEC CPU [Sta17], die Scale-out-Suite CloudSuite [FAK⁺12] sowie — für den Allokator-Korpus — mimalloc-bench [Lc21] und die Klassiker Larson, threadtest und shbench auf. Da diese Frameworks unvergleichbare Lasten erzeugen, abstrahiert die Arbeit sie zu einem gemeinsamen Satz von Lastprofilen; die noch nicht im eigenen Mess-Apparat realisierten Frameworks (TPC, SOSD, SPEC CPU, CloudSuite und die Allokator-Suiten) werden über einen *Datensatz-Lader* angebunden.

Aus dieser Analyse destilliert die Arbeit *vierzehn* distinkte Lastprofile (LP01–LP14, Tabelle 3.6): Sie geben den heterogenen Lasten ein gemeinsames, uint64-basiertes Operations-Modell — der Op-Mix nennt die Anteile von Einfügen, Lookup, Löschen, Scan und Read-Modify-Write über definierten Schlüssel-Verteilungen — und sind je aus konkreten Veröffentlichungen abgeleitet.

Der hier sezierte Stand der Technik beruht auf einem festen Korpus: Die Analyse stützt sich auf 33 tiefen-analyisierte Suchalgorithmus-Veröffentlichungen (P01–P33) und einen parallelen Allokator-Korpus aus 23 Arbeiten (A01–A23), deren Achsen-Belegungen als XML-Profile in der `comdare-cache-engine`

Tabelle 3.5: Benchmark-Frameworks und nutzende Veröffentlichungen

Framework	Typ	Nutzende Veröffentlichungen
YCSB [CST ⁺ 10]	KV-Workload	P02, P03, P07, P20, P26
SOSD [KMR ⁺ 19]	Index-Benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytisch	P19
Real-String-Korpora	String-Datensätze	P04, P06, P10
RocksDB-Integration	Real-System	P10 (SuRF)
SPEC CPU [Sta17]	CPU-Suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	Scale-out-Server	P24, P27
IBM DB2	Produktions-DBMS	P22
mimalloc-bench [Lc21]	Allokator-Suite	A04, A07, A09, A10, A18
Larson / threadtest / sh-bench	Allokator-Klassiker	A01, A03, P08, P30
LADDIS	Kernel-Allokator	A02
AggSum / Memory-Stream	Bandbreite	P32
Custom-Microbenchmarks	per Paper (Zipf/Uniform)	P01, P04, P06, P11–P14, P21, P28

Tabelle 3.6: Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)

LP	Kennung	Op-Mix	Verteilung	neg%	Abgeleitet aus
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01,P02,P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01,P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10,P20,P25,P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04,P06,P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04,P06,P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01,P10,P12,P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03,P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01,P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10,P19,P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07,P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07,P08,P25,P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19,P20

Op-Mix = Anteile von Einfügen/Lookup/Löschen/Scan/RMW; neg% = Anteil negativer Lookups.

persistiert sind (drei Arbeiten gehen nur seziert ein, siehe unten); die vollständigen Bausteine- und Allokator-Matrizen führt der Anhang. Die achsenweise Sezierung dieses Korpus folgt einer web-verifizierten Provenienz-Map über rund 110 Organ-Algorithmen; jeder Stand-der-Technik-Algorithmus erscheint so als ein Punkt im kartesischen Produkt der Achsen (Abbildung 3.1). Dass das Achsen-Gerüst auch nicht-baumartige Verfahren trägt, belegt eine reine Hash-Tabelle als Gegenprobe innerhalb der Map-Gattung — als zwei wählbare T0-Bausteine, von denen je Permutation genau einer gewählt wird und die beide permutiert werden: ein Open-Addressing-Hash nach Knuth (S14, implementiert) und die SwissTable [Goo18] (als eigener Baustein vorgesehen). Die Gegenprobe erfüllt das Punktanfrage-Subset des `std::map-Interface` (`find/insert/erase`); die ordnungsbasierten Operationen entfallen prinzipbedingt, da eine Hash-Tabelle ungeordnet ist. Über dieses Subset belegt sie ein limitiertes Achsen-Subset: Allokation, Prefetch, Nebenläufigkeit und Speicher-Layout bleiben geteilt — und auf der System-Seite trägt die SIMD-Erweiterung konkret das SwissTable-Gruppen-Lookup [Goo18] —,

während die baum- und trie-spezifischen Achsen (Pfadkompression, Knotentyp, Bereichs-Index) nicht wegfallen, sondern als Durchreich-Varianten (`None`) belegt werden; die Gegenprobe erfordert so keine eigene Gattung.

Die Veröffentlichungen entstammen sechs thematischen *Herkunfts-Clustern*, die hier nur der ganzheitlichen Einordnung jedes Lebewesens dienen, bevor es in seine Organe zerlegt wird. Die *Trie-Familie* umfasst ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20] sowie die succinct-Tries LOUDS (P09) [Jac89] und SuRF (P10) [ZLL⁺18]. Die *Hybrid- und B⁺-Familie* vereint den B²-Baum (P06) [SSL⁺21], Wormhole (P07) [WNJ19] und die cache-sensitiven CSS-, CSB⁺- und Hankins-Bäume (P11–P13) [RR99, RR00, HP03]. Die *Layout-Theorie* reicht von Samuel (P14) [SPB05] und Graefe/Larson (P15) [GL01] über die cache-oblivious Entwürfe von Bender et al. (P16/P17) [BDFC00, BDFC05, BDFC02, BCDFC02, BCD⁺02] bis zu den layout-invarianten Bäumen von Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. Das *Prefetching* umfasst *B-Trees Are Back* (P20) [MBL25], die Prefetching-B⁺-Bäume von Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25] sowie Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] und Kuehn (P28) [Kue23]. Die *Synchronisation* trägt das optimistische Lock-Coupling von ART (P08) [LABN16], RCU (P29) [McK01] und Hazard-Pointer (P30) [Mic04a]; die hardware-nahe *TU-Dresden-Linie* schließlich Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25] und VAMPIR (P33) [BH23]. Der parallele *Allokator-Korpus* (A01–A23) spannt von den Klassikern dmalloc [Lea96], Hoard [BMBW00], Slab samt Magazines-/Vmem-Erweiterung [Bon94, BA01] und Michaels lock-freier Allokation [Mic04b] über die mehrkern-skalierbaren mimalloc [LZM19], jemalloc [Eva06], TCMalloc [GM05], snmalloc [LDPB19], scallocc [AKLS15], rpmalloc [Jan17] und lrmalloc [LR18] bis zu den cache-aware/vorhersagbaren CAMA [HBHR11], NUMAalloc [YZZ⁺23], StarMalloc [RFP⁺24b] und PIM-malloc [VIA26]. Abbildung 3.2 verdichtet die wichtigsten rekonstruierten Suchstrukturen zu einer Galerie — je Verfahren seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet; alle erscheinen so als Punkte im gemeinsamen Entwurfsraum.

ART (P01) Trie adaptive Knoten 4/16/48/256, byte-weise Verzweigung, Pfadkompression Achsen T4, T3	HOT (P02) Trie höhenoptimierter dynamischer Span, kompakte SIMD-Knoten Achsen T4, T0	Masstree (P03) Trie + B ⁺ Trie aus B ⁺ -Bäumen, 8-Byte-Slices, B ⁺ -Blattknoten Achsen T4, T10	CoCo-Trie (P04) Succinct kompakt-kollabierter Trie, succinct kodierte Pfade Achsen T3, T4
START (P05) Trie selbstoptimierender adaptiver Radix-Span Achsen T0, T4	B²-Tree (P06) Hybrid B ⁺ mit binären In-Knoten, cache-bewusst Achsen T4, T5	Wormhole (P07) Hybrid Trie + Hash + B ⁺ -Leaf, $O(\log L)$ Punktsuche Achsen T0, T2	LOUDS (P09) Succinct unäre Grad-Sequenz, Bitmap-kodierter Trie Achsen T9, T5
SuRF (P10) Succinct succinct Range-Filter über LOUDS Achsen T14, T9	CSS-Tree (P11) cache-sensitiv zeigerloser impliziter Baum, arithmetische Navigation Achsen T4, T5	CSB⁺-Tree (P12) cache-sensitiv B ⁺ mit einem Kind-Gruppen-Zeiger Achsen T4, T5	SwissTable Hash (Gegenprobe) SIMD-Control-Bytes; nicht-baumartig, im selben Achsen-System Achse T0 + SIMD (System)

Abbildung 3.2: Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.

Für jeden SOTA-Algorithmus (*State of the Art* — der Stand der Technik aus Abschnitt 2.1) und jeden Allokator hält ein Profil-XML die Achsen-Belegung (page, node, traversal, value_handle, concurrency, allocator, prefetch, telemetry, isa, layout, reclamation), eine Schlüssel-Wert-Signatur und einen optionalen `<expected_workload>`-Tag fest, der die Mess-Heuristik überschreibt (Tabelle 3.7); alle 30 SOTA-Profil und alle 10 Allokator-Profil sind getaggt. Drei der 33 Korpus-Arbeiten besitzen bewusst kein eigenständiges Profil, sondern gehen nur seziert ein: das optimistische Lock-Coupling von ART [LABN16] als Nebenläufigkeits-Organ, die succinct LOUDS-Repräsentation [Jac89] als Speicher-Layout-Organ (u. a. in SuRF [ZLL⁺18]) und das Profiling-Werkzeug VAMPIR [BH23] als Mess-Technologie; die übrigen 30 tragen je ein eigenes Profil-XML.

Tabelle 3.7: SOTA-Algorithmus-Profil mit Workload-Affinität

Profil	Paper-Ref.	expected_workload
art	P01 (Leis 2013)	YCSB_C
hot	P02 (Binna 2018)	YCSB_C
masstree	P03 (Mao 2012)	YCSB_A
coco_trie	P04 (Boffa 2024)	YCSB_C
start	P05 (Fent 2020)	YCSB_C
b2tree	P06 (Schmeisser 2022)	YCSB_E
wormhole	P07 (Wu 2019)	YCSB_A
surf	P10 (Zhang 2018)	YCSB_A
css_tree	P11 (Rao/Ross 1999)	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	YCSB_E
hankins	P13 (Hankins 2003)	YCSB_C
samuel	P14 (Samuel 2005)	YCSB_C
graefe	P15 (Graefe 2001)	YCSB_C
bender_treelayout	P16 (Bender 2002)	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	YCSB_E
btreesareback	P20 (Mueller 2025)	YCSB_E
chen_prefetch	P21 (Chen 2001)	YCSB_A
chen_fractal	P22 (Chen 2002)	YCSB_A
khan_adaptive	P23 (Khan 2010)	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	YCSB_A
mahling	P25 (Mahling 2025)	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	YCSB_A
zhang_asplos	P27 (Zhang 2025)	YCSB_A
kuehn	P28 (Kuehn 2023)	YCSB_C
rcu	P29 (McKenney 2001)	YCSB_B
hazard_pointers	P30 (Michael 2004)	YCSB_B
ungethuem	P31 (Ungethuem 2017)	YCSB_C
to_stride	P32 (Schmidt 2025)	YCSB_C

expected_workload = YCSB-Workload-Klasse (A/B/C/E/F).

Über die Profile hinweg verteilen sich diese Affinitäten auf $13 \times$ YCSB_C (leselastig), $9 \times$ YCSB_A (Zipf-verteiltes Lesen und Aktualisieren), $6 \times$ YCSB_E (Bereichs-Scans) und $2 \times$ YCSB_B (95/5 Lesen/Aktualisieren); PRT-ART selbst ist mit YCSB_F (Read-Modify-Write, für die DensityTracker-Aktualisierungen) getaggt.

Der Kern-Befund dieser Lastprofil-Analyse ist ein systematischer *Bias*: Jede Veröffentlichung wählt das Lastprofil, unter dem ihr eigener Entwurf am besten abschneidet. Statische, read-only-orientierte Tries und

Filter (HOT [BZP⁺18], CoCo [BFTV24], B²-Baum [SSL⁺21], SuRF [ZLL⁺18], CSS [RR99]) rahmen „build-once, 100 % positive Lookups“ und meiden Aktualisierungen; komprimierte Tries und Filter (CoCo, B²-Baum, SuRF) machen den Negativ-Anfrage-Anteil zur Primärachse, weil sie bei Misses früh abbrechen; zipfian-zentrierte Arbeiten (B-Trees Are Back [MBL25], Mahling [MWR25], Kuehn [Kue23]) begünstigen cache-bewusste Reordering-Layouts; schreib- und nebenläufigkeits-lastige Arbeiten (ART [LKN13], Masstree [MKM12], Wormhole [WNJ19], OLC-ART [LABN16], RCU [McK01]) treten umgekehrt nur dort an, wo statische Strukturen versagen; und prefetch- bzw. hardware-nahe Arbeiten (CSB⁺ [RR00], Chen [CGM01], Schmidt [SKK⁺25]) vermessen über Knotengröße und Prefetch eher Cache und TLB als die Algorithmus-Logik. Vierzehn der Veröffentlichungen steuern kein distinktes Lastprofil zum Katalog bei — teils Survey-, Theorie- und Hardware-Arbeiten ohne eigenes Index-Lastprofil, teils Arbeiten, deren Custom-Microbenchmarks in bereits katalogisierten Profilen aufgehen. Erst wenn *alle* Lastprofile gegen *alle* Lebewesen-Binaries (und die Container-Hüllen, Abschnitt 3.3.2) laufen — der Workload als dynamische Achse (Kapitel 5) —, lässt sich der „Sieger nur im selbstgewählten Heimprofil“ sichtbar machen und bias-reduzierte Vergleichbarkeit gewinnen.

Schließlich erscheinen auch die Güte-Kriterien wissenschaftlichen Messens (vgl. die Definitions-Klassen in Abschnitt 3.4) als konkrete Technologien. *Hardware-Performance-Zähler* (PMC, über `perf`) [Mel10] sind nahezu universell und liefern Cache-Miss-Raten, dTLB-Misses, Branch-Misses und IPC/CPI — die Mehrzahl der Arbeiten P01–P28 berichtet solche Zähler —; *Profiler* wie OTF2/VAMPIR [BH23] und VTune [Rei05] erheben trace-basierte Laufzeitprofile, *Simulatoren* wie gem5 [BBB⁺11] liefern hardwaregenaue Schätzungen, wo reale Zähler fehlen (so P25 Mahling [MWR25], P27 Zhang [ZGH⁺25]), und das *HdrHistogram* [Ten12] erhebt die Latenz-Perzentile (p50/p95/p99). Auf der Erhebung baut die Auswertung auf: Statt arithmetischer Mittel werden Median und Perzentile berichtet, Unterschiede mit dem verteilungsfreien *Mann-Whitney-U-Test* [MW47] (mit Holm-Korrektur der Familienfehlerrate [Hol79]) auf Signifikanz geprüft und über das Effektmaß *Cliff's δ* [Cli93] quantifiziert, den Empfehlungen für wissenschaftliches Benchmarking [HB15] folgend. Eine eigene methodische Linie bildet die hardware-nahe Messmethodik der TU Dresden (Habich-Gruppe): Ungethüm et al. [UHL⁺17] systematisieren die Hardware-Optimierungen für Datenbank-Maschinen, Schmidt et al. [SKK⁺25] vermessen Stride-gegen Sequenz-Zugriffe auf HBM mit dTLB- und L2/L3-Zählern, und VAMPIR [BH23] profiliert nicht-funktionale Eigenschaften heterogener Speicher. Aus all diesen Technologien ergibt sich der Satz der Mess-Kategorien — Cache-Line-Auslastung, Cache-Misses (L1/L2/L3), dTLB-Misses, Speicher-Fußabdruck, Branch-Misses, IPC/CPI, Latenz (Mittel und Perzentile p50–p999), Durchsatz, Energie und Fill-Buffer-Auslastung —, deren zeit- und observer-basierte Teilmenge die eigene Apparatur bereits auflöst (Wall-Clock, HdrHistogram in der nachgelagerten Auswertung, Per-Achsen-Observer, die Statistik-Triade aus Median/Perzentilen, Signifikanztest und Effektmaß, die Zwei-Phasen-Operationsschleife und das Konformitäts-Gatter gegen `std::map`), während die zählerbasierten PMC-Kategorien an Hardware-Zähler-Zugriff gebunden bleiben; die vollständige Mess-Architektur entwickeln das Mess-System (Abschnitt 3.4) und Kapitel 5.

3.3 Lösungskonzept: das Anatomie-Modell

Aus der dialektischen Aneignung des Standes der Technik ergibt sich das Lösungskonzept dieser Arbeit; dieser Abschnitt entwickelt es als *Anatomie-Modell* und definiert dabei die Begrifflichkeiten und

konzeptionellen Spezialkonstrukte, die die folgenden Abschnitte tragen.

3.3.1 Achsen-Komposition und die Lebewesen-Metapher

Die zentrale Idee ist, einen Algorithmus nicht als Monolith zu behandeln, sondern als *Komposition orthogonaler Achsen* — jede Achse eine Teilentscheidung eines cache-bewussten Index (wie Seiten angeordnet werden, wie Knoten verzweigen, wie Speicher alloziert wird). Ein konkreter Algorithmus ist dann ein Punkt im kartesischen Produkt aller Achsen, ein Stand-der-Technik-Entwurf eine explizite Konfiguration. Entscheidend ist die *modulare Austauschbarkeit*: Eine neue Achsen-Variante lässt sich isoliert untersuchen — ihr Effekt je Achse *und* für den ganzen Algorithmus messen —, ohne die übrigen neu zu implementieren; genau das löst das Trennbarkeits-Problem, da der Beitrag jedes Bestandteils bei sonst fixierten Achsen isoliert messbar wird. Diese Zerlegung beruht auf einer Beobachtung und wird von einer Metapher getragen. Erstens teilen Suchalgorithmen im großen Bild dieselbe *Baum-Anatomie* in unterschiedlicher Ausprägung — von ausgewogenen Suchbäumen über Tries und Radix-Bäume bis zu eigenentwickelten Konstrukten; selbst eine sortierte Schlüssel-Wert-Liste ist streng genommen ein einzelnes Blatt eines Baumes, dessen Werte über Verweise weiter auf Daten-Blätter zeigen. Die Baum-Struktur ist damit universell über alle Suchalgorithmus-Architekturen hinweg und bildet die Leinwand, auf der die Achsen platziert werden; nicht-baumartige Suchverfahren — etwa die Hash-Gegenprobe aus Abschnitt 3.2 — bleiben dieselbe Gattung mit eingeschränktem Profil, denn eine eigene Gattung entstünde erst, wenn sich der Basis-Achsen-Satz grundlegend unterschiede. Zweitens fasst eine anschauliche Metapher einen konkreten Algorithmus als *Lebewesen* auf, dessen *Organe* die Achsen sind — und das Mess-System als seine medizinische Diagnostik (Abbildung 3.3). Diese Metapher ist dem Hintergrund des Autors in der Biomedizinischen Technik geschuldet und soll die Analyse durch Bildhaftigkeit vereinfachen.

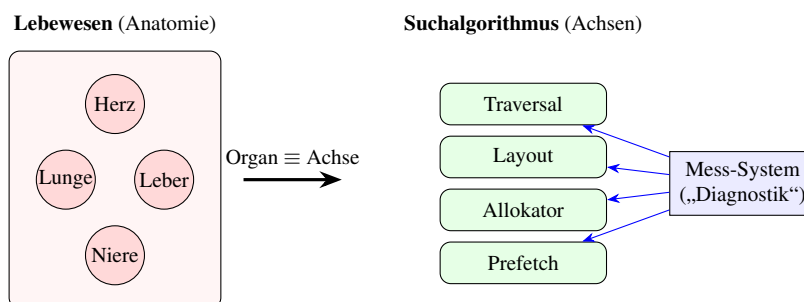


Abbildung 3.3: Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Mess-System übernimmt die Rolle der medizinischen Diagnostik und vermisst jedes Organ einzeln.

Die Permutation der Achsen gleicht dann einem genetischen Experiment: Der Austausch von Organen zwischen Lebewesen erzeugt neue Komposit-Algorithmen mit bisher unbekanntem Cache-Line-Verhalten. Die Organe selbst werden aus dem Stand der Technik gewonnen — die Entwurfsbausteine der bekannten Verfahren (Abschnitt 2.1) werden in orthogonale Achsen zerlegt und in einer gemeinsamen Achsen-Matrix gesammelt, sodass jede Achse die Beiträge des gesamten Korpus erntet (Abbildung 3.4).

Die so gewonnene *Anatomie* ist mehr als die Aufzählung der Organe: Sie ist ihre *Verdrahtung*. Jedes Organ exportiert ein generalisiertes Interface, über das die übrigen seine Dienste nutzen, ohne

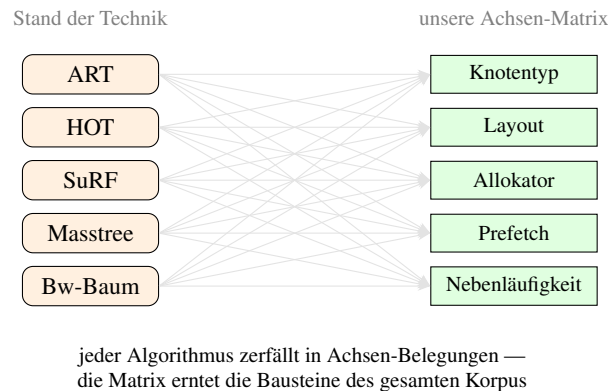


Abbildung 3.4: Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 3.1 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).

dessen konkrete Belegung zu kennen — so stellt das Allokations-Organ ein gemeinsames Allokations-Interface bereit, das Knoten-Layout, Wert-Ablage und Serialisierung für ihre Speicheranforderungen verwenden (Abbildung 3.5). Software-technisch ist dies die *Feature-Interaktion* feature-orientierter Software-Produktlinien [ABKS13, BO92]: eine Achse entspricht einem Feature, ein Lebewesen einer Feature-Komposition, ein Lebewesen-Typ (die Gattungs-Unterklasse, Abschnitt 3.3.2) einer statischen Feature-Set-Definition, eine Gattung einer (Such-)Algorithmus-Interface-Kategorie und die Anatomie der Verdrahtung zwischen den Features. Welche Achse welche nutzt, zeigt der Inter-Organ-Nutzungs-Graph (Abbildung 3.6).

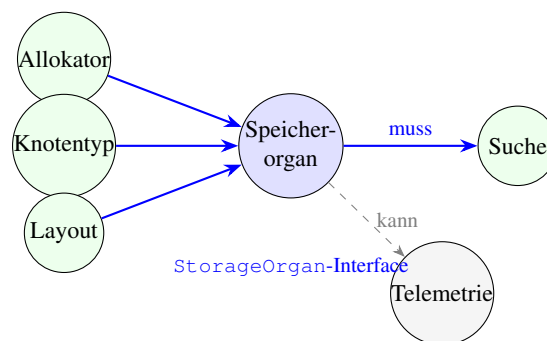


Abbildung 3.5: Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes `StorageOrgan-Interface`, über das die Suche läuft (durchgezogen, *verpflichtend*); weitere Organe wie die Telemetrie *können* es über optionale Beobachter-Hooks nutzen (gestrichelt, *optional*), müssen aber nicht.

Setzt man diese paarweisen Kopplungen über alle Organe zusammen, entsteht ein gerichteter Inter-Organ-Nutzungs-Graph.

3.3.2 Das Drei-Ebenen-Modell und die drei Gattungen

Diese Organe sind in *eine* Architektur eingebettet — eine einzige, in sich geschlossene Hierarchie ohne Parallel-Bäume. An ihrer Wurzel steht die `ExecutionEngine` als gemeinsame, ausmessbare

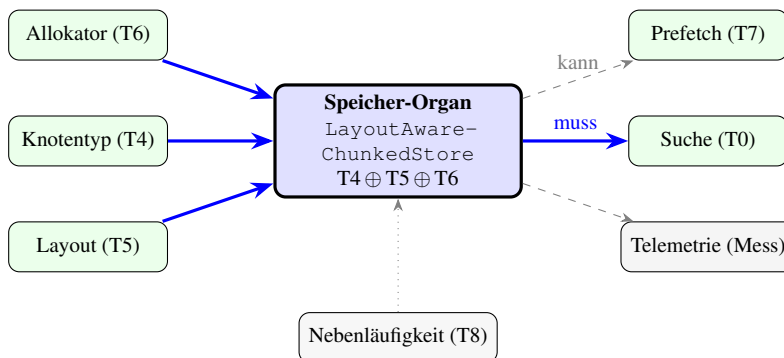


Abbildung 3.6: Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und die Telemetrie (Mess-Instrumentierung, Abschnitt 3.4.1) lesen den Store nur über *requires*-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist im Code (noch) nicht verdrahtet (gepunktet).

Abstraktion über allem Messbaren; darunter lassen sich drei Ebenen unterscheiden: die *Gattung* als nach außen sichtbares Algorithmus-Interface (das *Prüf-Dock*), darunter die *Gattungs-Unterklasse* (auch: *Tier-Unterklasse*) mit festem Achsensatz, darunter die *Organe* selbst (Abbildung 3.7). „Lebewesen“ und `SearchAlgorithm` bezeichnen denselben Gegenstand — Metapher und technischer Begriff —; die Anatomie ist kein zweites Modell, sondern der *Körper* dieses Lebewesens: die feste Komposition seiner achtzehn Achsen-Organen.

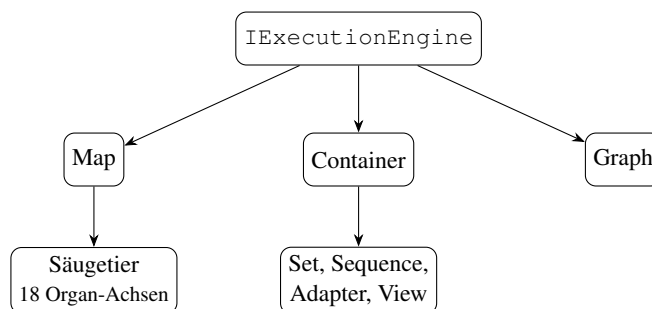


Abbildung 3.7: Das Drei-Ebenen-Modell: Gattung (Ebene 1, Außen-Interface) → Tier-Unterklasse (Ebene 2, fester Achsen-Satz) → Organe/Achsen (Ebene 3). Die Map-Gattung trägt mit ihrem einzigen Genus `SearchAlgorithm` das „Säugetier“ mit den achtzehn Organ-Achsen; die Container-Gattung die Unterklassen `Set/Sequence/Adapter/View`.

Auf der obersten Ebene stehen genau drei Gattungen. Die Map-Gattung — das schlüssel-wert-orientierte Prüf-Dock und der Fokus dieser Arbeit — trägt als einzige Unterklasse (Genus `SearchAlgorithm`) das volle Achtzehn-Achsen-„Säugetier“; die Container-Gattung (schlüsselloser Speicher) fächert in die Unterklassen `Set, Sequence, Adapter` und `View` auf; die Graph-Gattung trägt einen eigenen, andersartigen Organsatz (Abbildung 3.8). Das dort als Wurzel gezeigte Reich „Animalia“ ($\equiv$ `IAnatomyBase`) ist die anatomie-tragende Sicht unmittelbar unterhalb der messbaren Wurzel `IExecutionEngine` aus Abbildung 3.7; die Code-Trennung der beiden Interfaces vertieft Kapitel 4. Nicht-baumartige Verfahren

erfordern so keine eigene Gattung, sondern belegen ein eingeschränktes Achsen-Subset derselben Gattung.

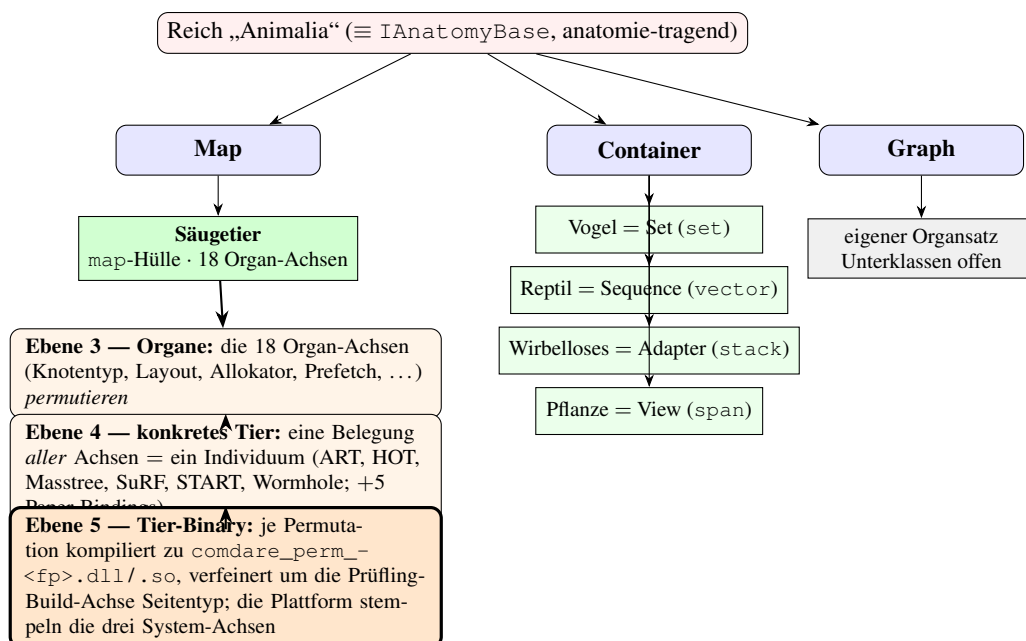


Abbildung 3.8: Die Gattungs-/Tier-Hierarchie über fünf Ebenen. Map, Container und Graph sind die drei *Gattungen* (Ebene 1, Außen-Interface). Jede Gattung beherbergt ihre *eigenen* Tier-Unterklassen (Ebene 2): Map ausschließlich das „Säugetier“ (Genus SearchAlgorithm, volle Achtzehn-Achsen-Anatomie), Container jede *nicht*-Säugetier-Unterklasse (Vogel/Reptil/Wirbelloses/Pflanze = Set/Sequence/Adapter/View), Graph noch offene. Der Säugetier-Ast ist hier bis zum Individuum ausgezogen: die achtzehn Achsen-Organen (Ebene 3) permutieren zu einer konkreten Achsen-Komposition = einem konkreten Tier (Ebene 4, z. B. ART, HOT, Masstree), das je Permutation als eigenständige Tier-Binary kompiliert wird (Ebene 5, zusätzlich verfeinert um die Prüfling-Build-Achse Seitentyp und plattform-gestempelt durch die drei System-Achsen, Abschnitt 3.4.1). *Jedes Tier-Binary ist damit ein eigenes Tier*; die Hierarchie reicht von der Gattung bis zur einzelnen kompilierten Permutation. Vgl. das verdichtete Modell in Abbildung 3.7.

3.3.3 Entwurfsmuster und Übersetzungszeit-Muster

Der eigentliche Beitrag liegt damit weniger in einer einzelnen Struktur als in einem *Werkzeug*: Statt das Vergleichsproblem direkt zu lösen, baut diese Arbeit zuerst die „Schaufel“ — den Permutations- und Mess-Apparat —, mit der sich beliebige Achsen-Kombinationen erzeugen und fair vermessen lassen. Wissenschaftlich ordnet sich dieser Apparat in das Software-Engineering der *Komposition* ein: die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03], die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92]. Das Achsen-Konzept selbst ist aus einer Vorarbeit des Autors (Marke Comdare / BEP Venture UG, ursprünglich zur Deduplikations-Optimierung der UltiHash-Datenbank) übernommen; der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine design-pattern-getriebene, mess-orientierte Realisierung. Software-technisch ist der Apparat eine Komposition benannter Entwurfsmuster und zweier neuer Übersetzungszeit-Muster (Abbildung 3.9).

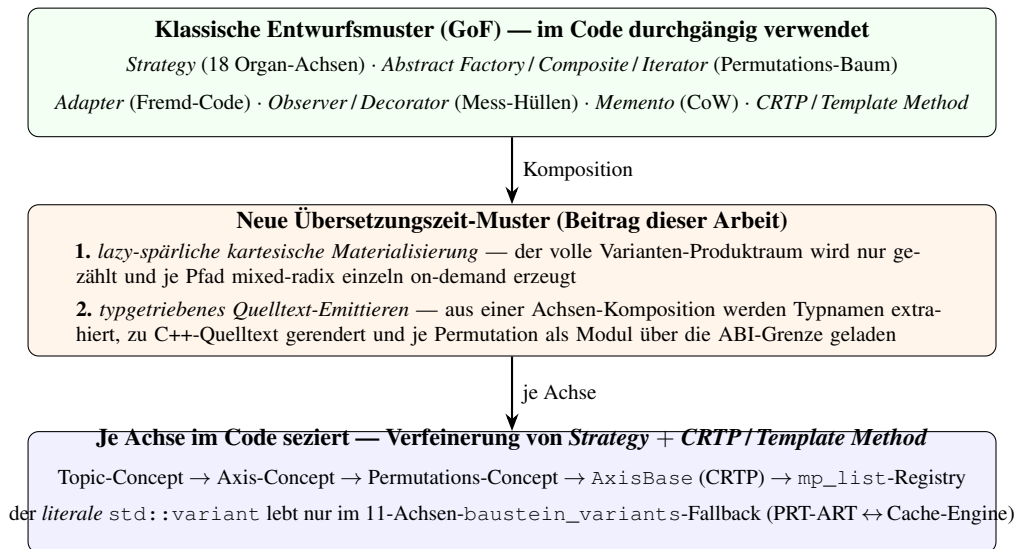


Abbildung 3.9: Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → mp_list-Registry) als Verfeinerung von *Strategy/CRTP* — wobei der *literale* `std::variant` nicht hier, sondern nur im elf-achsigen `baustein_variants`-Fallback-Modell auftritt. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Kapitel 4).

Die klassischen Muster sind dabei keine Zierde, sondern die Verkabelung des Apparats: *Strategy* kapselt jede der achtzehn Organ-Achsen als austauschbaren Algorithmus, *Abstract Factory*, *Composite* und *Iterator* bilden und durchlaufen den Permutations-Baum, *Adapter* bindet Fremd-Code hinter der stabilen Engine-Schnittstelle an, und *Observer* sowie *Decorator* (nicht-intrusive Mess-Hüllen) erheben die Messwerte, während ein *Memento* (Copy-on-Write) den Zustand zwischen Warmlauf und Messung sichert [GHJV94]; nicht verwendet werden hingegen *Singleton*, *Flyweight* und *Bridge*, während das *Command*-Muster allein als Befehls-Kanal des Hybrid-Modus auftritt (Abschnitt 3.6.2). Den statischen Polymorphismus liefert das *Curiously Recurring Template Pattern* (CRTP) samt *Template-Method-Concept*-Prüfungen; concept-geprüftes `if constexpr (resolve_baustein)` wählt je Achse zwischen Prüfling- und Cache-Engine-Standard, und *Policy-based Design* fasst die Komposition als Tupel von Achsen-Policies [Ale01]. C++23-Module strukturieren dabei die Übersetzungseinheiten, garantieren für sich genommen aber noch keine portable ABI-Stabilität [Xu24] — diese stellt erst die C-stabile Modulgrenze her. Über die etablierten Muster hinaus benennt die Arbeit zwei Konstruktionen, für die eine Web-Recherche keine etablierte Bezeichnung fand: die *lazy-spärliche kartesische Materialisierung*, die den kombinatorisch explodierenden Varianten-Produktraum aller Achsen-Kataloge nur arithmetisch zählt und je Pfad mixed-radix einzeln on-demand erzeugt, statt den Compiler mit dem vollen Typ-Baum zu sprengen; und das *typgetriebene Quelltext-Emittieren*, das aus einer compile-zeitlichen Achsen-Komposition voll-qualifizierte Typnamen extrahiert, zu C++-Quelltext rendert und je Permutation als eigenständiges Modul über die ABI-Grenze lädt — so wird der Prüfling PRT-ART per Metaprogrammierung in Matrix-Schichten gegen die Cache-Engine kompiliert.

Wie diese *eine* Architektur über eine ABI-stabile Hülle im Code sichtbar wird, vertieft Kapitel 4 (Ab-

schnitt 4.3.1); zuvor ordnet der folgende Unterabschnitt den Apparat als Ganzes: in welchen Schichten er von der Experiment-Definition bis zur Auswertung organisiert ist und wie er fertiggestellt wird.

3.3.4 Fünf Interface-Schichten: E4 bis E0

Das Anatomie-Modell ordnet das *Gemessene*: was ein Lebewesen ist und aus welchen Organen es besteht. Quer dazu braucht der Apparat, der solche Lebewesen erzeugt und vermisst, ein zweites Ordnungsprinzip — eine Sicht auf die *Experiment-Pipeline*, die eine deklarierte Achsen-Belegung in gemessene Ergebnisse überführt. Diese Pipeline gliedert sich in fünf Schichten E4 bis E0, ergänzt um den Querschnitt M (Abbildung 3.10). Die Zählung läuft von der abstraktesten zur konkretesten Ebene. *E4* ist die Definitions- und Auswertungs-Schicht — die Ebene, die ein Anwender des Frameworks in der Regel verwendet: Experimente werden deklarativ als XML-Profile beschrieben, und am Ende der Pipeline nimmt dieselbe Schicht die Messergebnisse als Rückkanal entgegen und bereitet sie zu Tabellen und Diagrammen auf. *E3* ist der Permutations-B⁺-Baum: Er entfaltet die deklarierte Belegung zum Experimentbaum, wobei die übersetzungsstatistischen Achsen die Identität der zu bauenden Binary bestimmen und die dynamischen Achsen die Laufzeit-Dimension aufspannen. *E2* sind die Tier-Binaries selbst: je Permutation eine eigenständige, über die ABI-stabile Schnittstelle geladene Binary (Ebene 5 der Gattungs-Hierarchie, Abbildung 3.8). *E1* ist die Laufzeit-Konfiguration — die präziseste Detail-Schicht, in der die dynamischen Sub-Achsen-Parameter an der bereits geladenen Binary variiert werden. *E0* schließlich ist die Infrastruktur (Bau-, Werkzeug- und Plattform-Unterbau); sie wird bewusst zuletzt fertiggestellt. Das Mess-System durchdringt als Querschnitt *M* alle Schichten, ohne selbst eine zu sein — diese Sonderrolle vertieft Abschnitt 3.4.

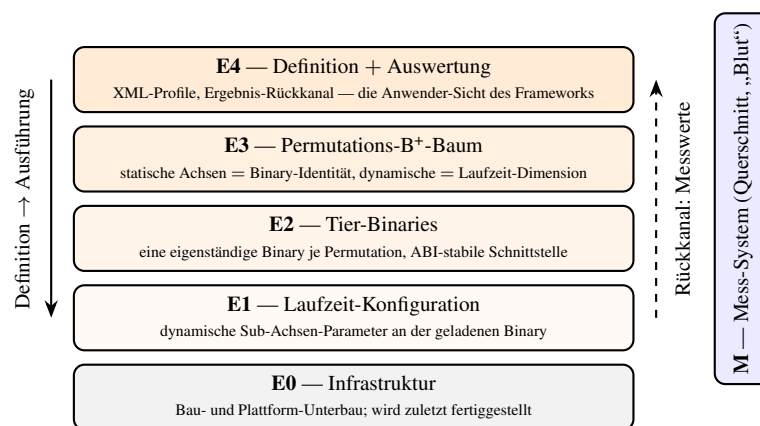


Abbildung 3.10: Die fünf Interface-Schichten der Experiment-Pipeline: von der deklarativen Definitions- und Auswertungs-Schicht E4 (Anwender-Sicht) über den Permutations-B⁺-Baum (E3) und die Tier-Binaries (E2) zur Laufzeit-Konfiguration (E1); die Infrastruktur (E0) trägt alles und wird zuletzt fertiggestellt. Der Rückkanal führt die Messwerte zur Auswertung nach E4 zurück; das Mess-System durchdringt als Querschnitt M alle Schichten („Blut“, Abschnitt 3.4). Jede Schicht wird einzeln fertiggestellt, mit eingefrorenem Interface-Vertrag zur Nachbar-Ebene.

Tragend ist das Fertigstellungs-Prinzip dieser Schichtung: Jede Schicht wird *einzel*n fertiggestellt, mit einem explizit *eingefrorenen Interface-Vertrag* zur jeweiligen Nachbar-Ebene — die Grenze E4→E3 etwa friert ein, wie eine XML-Definition in Achsen-Ebenen übersetzt wird, die Grenze E3→E2, wie aus einem

Baum-Pfad ein Bau-Auftrag wird, und die Grenze E2→E1, welche Laufzeit-Stellgrößen eine Binary über die ABI entgegennimmt. Die Prüffrage je Grenze lautet stets: Reicht die Schicht ihre Definition *verlustfrei* an die nächste weiter? Ein System, dessen Produktraum jede vollständige Materialisierung ausschließt (Abschnitt 3.4.5), ist für gleichzeitige Änderungen über mehrere Ebenen zu groß; erst die eingefrorenen Verträge machen jede Ebene einzeln abnehmbar, bevor die nächste beginnt. Dieses Prinzip korrespondiert mit der Matrix-Programmierung des Apparats (Abschnitt 3.3.3): So wie der Prüfling in Matrix-Schichten gegen die Cache-Engine kompiliert wird, wird der Apparat selbst in Interface-Schichten gegen eingefrorene Verträge entwickelt; wie jede Schicht dabei einzeln — gegen einen Ersatz ihrer Nachbar-Ebene — geprüft wird, zeigt Abschnitt 4.4.

Die E-Schichten sind ausdrücklich vom Drei-Ebenen-Modell (Abschnitt 3.3.2) zu unterscheiden, so ähnlich beide auf den ersten Blick wirken: Das Drei-Ebenen-Modell ist die *Anatomie-Sicht* — es ordnet *ein* Lebewesen von der Gattung über die Unterklasse zu den Organen und beschreibt damit die Struktur des Gemessenen. Die E-Schichten sind die *Pipeline-Sicht* — sie ordnen den Weg eines Experiments von der Definition über Baum, Binary und Laufzeit zurück zur Auswertung und beschreiben damit den Prozess des Messens. Beide Sichten stehen orthogonal zueinander: Dasselbe Lebewesen, das anatomisch in Gattung, Unterklasse und Organe zerfällt, durchläuft als Tier-Binary die Schichten E3 bis E1. Nachdem damit geklärt ist, was komponiert wird und in welchen Schichten der Apparat es verarbeitet, wendet sich der folgende Abschnitt der Frage zu, wie sich die so komponierten Lebewesen fair vermessen lassen.

3.4 Das Mess-System

Damit strukturell verschiedene Suchalgorithmen *fair* verglichen werden, bildet die Arbeit sie alle auf eine einzige, gut verstandene Schnittstelle ab: den C++-Vertrag der geordneten Map `std::map<Key, Value>` mit Punktanfrage (`find`), Einfügen/Aktualisieren (`insert`, `emplace`), Löschen (`erase`) und ordnungsbasierter Navigation (`lower_bound`/`upper_bound`, Bereichs-Iteration) [ISO24]. Die Achsen beschreiben das *innere* Verhalten hinter dieser festen Schnittstelle, sodass der Vergleich zweier konformer `std::map`-Implementierungen der zentrale Mess-Akt wird; parallel dient die Sequenz-Hülle `std::vector` (indizierter Zugriff, Größenänderung, zusammenhängender Speicher) als Container-Vergleichsfall, um zu prüfen, ob ein Achsen-Komposit auch die Container- und nicht nur die Suchsemantik trägt. Hinter der festen Außen-Schnittstelle unterscheidet das System mehrere *Ebenen der Dynamik*, die festlegen, *wann* eine Achsen-Entscheidung fällt: die oberste Trennung Übersetzungszeit gegen Laufzeit; der Lebewesen-Typ, der nur eine Teilmenge der Achsen belegt; die zur Übersetzungszeit in eine Binary einkompilierten Achsenalgorithmen (statische Rekombination); die dynamischen Sub-Achsen, deren Laufzeit-Parameter (Prefetch-Distanz, Stapelgröße, Inline-Schwelle, Pool-Budget) ohne Neuübersetzung variiert werden; und die statischen Sub-Achsen als übersetzungszeitliche Auswahl unter Teil-Algorithmen unterschiedlicher Komplexität. Die Idee, Datenstrukturen nicht einzeln, sondern als Entwurfsraum zu betrachten, prägt Idreos' *Periodic Table of Data Structures* [IZA⁺18] und den *Data Calculator* [IZH⁺18]: Strukturen entstehen aus der Kombination weniger Entwurfs-Primitive, deren Kosten sich synthetisieren lassen — das deckt sich unmittelbar mit der Achsen-Sezierung dieser Arbeit (Abschnitt 3.2), in der die klassischen Familien bloße Lese-Gruppierungen orthogonaler Achsen-Belegungen sind. Gegenüber der statischen Entwurfsraum-Aufzählung des Data Calculator und dem klassischen *Policy-based Design* [Ale01] führt

diese Taxonomie compile- und laufzeitvariable Achsen-Ebenen jedoch ausdrücklich gemeinsam; eine etablierte Benennung dieser Staffelung fand eine Web-Recherche nicht (vgl. Abschnitt 3.3.3).

Diese Bausteine sind nur so aussagekräftig wie ihre Vermessung. Wissenschaftliches Messen unterliegt etablierten Gütekriterien [HB15]: *Reproduzierbarkeit* (feste Seeds, deterministische Reihenfolge, dokumentierte Konfiguration, sodass Läufe methodisch wiederholbar und statistisch vergleichbar sind), *Validität* (intern: kausal isolierte Effekte; Konstrukt: die Metrik misst tatsächlich das gemeinte Konzept; extern: Übertragbarkeit auf andere Plattformen und Lasten), *Fairness* (identische Schlüssel, Workloads und Warmup; fehlende Fähigkeiten als *N/A* statt verdeckt emuliert) und die *Streuung* als Teil der Aussage (Perzentile statt arithmetischer Mittel, mit Signifikanzprüfung; die Verteilungssicht liefern HDR-Histogramme in der nachgelagerten Auswertung). Berichtete Perzentile folgen dem Nearest-Rank-Verfahren: der Rang $R(q, n) = \lceil q \cdot n \rceil$ ist einsbasiert, der zugehörige Feldindex $k = R - 1$ nullbasiert; der Median ist der Sonderfall $q = 0,5$ desselben Verfahrens und liefert bei geradem n die untere Mitte, nicht das Mittel der beiden mittleren Werte; p95, p99 und p999 sind dieselben Ränge mit $q = 0,95$, $q = 0,99$ und $q = 0,999$. Auf diesen Kriterien bauen die verwendeten Mess-Muster auf: eine *Zwei-Phasen-Operationsschleife* trennt einen verworfenen Warmlauf von der eigentlichen Messung; die Erhebung erfolgt *hybrid* — isolierte Achsen-Algorithmen direkt gegeneinander, ein zusammengesetztes Lebewesen zentral über eine ABI-stabile Per-Achsen-Beobachter-Schnittstelle —; und ein *Konformitäts-Gatter* prüft vor jeder Messung den `std::map`-Vertrag, damit nur Vergleichbares verglichen wird. Diese Konzepte bilden die *Definitions-Klassen* des Messens; ihre technischen *Instanzen* — Hardware-Performance-Zähler, OTF2-/VAMPIR-Profilung, die TU-Dresden-Hardware-Messmethodik — hat Abschnitt 3.2 bereits als konkrete Technologien eingeführt.

3.4.1 Die Dreiteilung: Organ-, System- und Mess-Achsen

Innerhalb dieser Definitions-Klassen beantwortet das Mess-System die in der Einleitung bewusst offen gehaltene Kategorien-Frage (Abschnitt 1.3): Die Achsen des Entwurfsraums trennen sich entlang der drei dort benannten Belange — das Gemessene, die Bau- und Ziel-Plattform, das Messen selbst — in *drei* Achsen-Kategorien, die im Folgenden als eigenständige *Realms* mit je eigener Klassen-Wurzel, je eigenem Angebots-Katalog und je eigener Stempel-Zeile (Abschnitt 3.4.4) entwickelt werden. Diese Dreiteilung ist die zweite Gestalt des Trennbarkeits-Problems: Nicht nur die Entwurfsentscheidungen untereinander, auch das Gemessene, die Plattform und das Mess-Instrument müssen getrennt bleiben, damit ein gemessener Effekt eindeutig zuzuordnen ist.

Die *Organ-Achsen* sind das *Gemessene*: Organe des Lebewesens, Teil der Gattungen, permutierbar — jede Belegung verändert die Identität der Tier-Binary. Kanonisch sind es *achtzehn* Organ-Haupt-Achsen in fester Slot-Reihenfolge T0–T17 (Tabelle 3.2); die T-Nummern bezeichnen die Slots der Komposition, keine Implementierungs-Nummerierung. Die frühere Neunzehn-Achsen-Zählung (Zwischenstand Juli 2026) ist eine historische Vorstufe dieser Ordnung: Aus ihr sind die Telemetrie und die ISA in die System-Sphäre umgezogen, und das Persistenz-Ziel ist als achtzehnte Organ-Achse hinzugekommen.

Die *System-Achsen* sind die Belange der Bau- und Ziel-Plattform. Kanonisch sind es *genau drei* Haupt-Achsen in fester Ordnung: die Ziel-ISA (`target_isa`), das Betriebssystem (`operating_system`) und die Erweiterungs-Hardware (`external_utils`); letztere ist ausschließlich der Sammel-Hub der Hardware-Meta-Achsen (SIMD-/Vektor-Erweiterungen, GPU/FPGA/NPU, externe Beschleuniger). In der Anatomie-Metapher sind die System-Achsen kein weiteres Organ, sondern das *Blut*: Sie durchdringen

den ganzen Körper, sind immer präsent, sobald gemessen wird, und gehören keiner Gattung an. Daraus folgen drei Invarianten: System-Achsen sind nicht permutierbar (sie sind nicht austauschbare, zentrale Systembestandteile); sie stehen nicht im Permutationsbaum; und sie berühren niemals die Binary-Identität — kein Mess-Instrument darf verändern, *was* gebaut wird. Umgekehrt dürfen die Effekte der Organ-Achsen nie mit den Effekten der Plattform und der Mess-Instrumente vermengt werden. Bau-Steuergrößen wie Optimierungsstufe und SIMD-Erweiterung multiplizieren deshalb nur die *Bau*-Matrix — sie erscheinen in der Bau-Versionskennung der Binary (Abschnitt 3.4.4) —, nie den Organ-Permutationsraum.

Die *Mess-Achsen* schließlich sind die Belange des Messens selbst — die Mess-Instrumente. Sie bilden den dritten, von beiden anderen getrennten Realm: Das Last-Rahmenwerk (`load_framework`), das die Workload-Konventionen aus Abschnitt 2.3 in den Apparat trägt, ist seine erste Haupt-Achse; das *Mess-Tooling* spannt die drei Granularitäten des Benchmarkings auf (Abschnitt 3.4.5); und die Mess-Kategorien (Abschnitt 3.2) — im Apparat zu sechzehn Kategorien ausdifferenziert — sind eine *Unter-Achse* dieses Toolings: Sie manifestieren sich als Spalten der Ergebnis-Tabellen, nicht als eigene Haupt-Achsen. Auch die Ablauf-Methodik eines Laufs (Debug-, Mess-, Release- oder Compare-Durchführung) bleibt Unter-Achse und wird nie Teil einer Binary-Identität; wie die System-Achsen sind die Mess-Achsen strikt binary-id-neutral. Innerhalb der sechzehn Kategorien besteht eine *Regime-Zweiteilung*: das zeit- und observer-basierte Regime (Laufzeit-Uhren und Beobachter-Hüllen; neun Kategorien, darunter Latenz, Durchsatz und Speicher-Fußabdruck) und das zählerbasierte PMC-Regime (Hardware-Performance-Zähler; sieben Kategorien, darunter Cache-, dTLB- und Branch-Misses sowie IPC/CPI). Das Regime ist ein Merkmal der Mess-Achse, nicht des gemessenen Organs: Dieselbe Organ-Achse kann von beiden Regimen beleuchtet werden. Die observer-basierte Teilmenge löst die Apparatur bereits auf, die PMC-Teilmenge bleibt an Hardware-Zähler-Zugriff gebunden (Abschnitt 3.2).

Konzeptionell folgt daraus ein doppelter Wurzel-Schnitt mit *zwei* Realm-Wurzeln — statt der *einen* gemeinsamen Wurzel aller Nicht-Organ-Achsen, die frühere Fassungen dieser Arbeit noch vorsahen: Der System-Realm wurzelt am `CacheEngineBuilder`, der die Plattform kennt und die Tier-Binaries baut; der Mess-Realm wurzelt am *Experiment-Planer*, der die Experimente aus der XML-Beschreibung auflöst (Abschnitt 3.4.5). Jede der drei Kategorien trägt damit eine eigene abstrakte Klassen-Wurzel und einen eigenen Angebots-Katalog; die Code-Gestalt dieser Wurzeln entwickelt Abschnitt 4.5. Verbindlich ist auch die kanonische Gesamt-Ordnung aller Haupt-Achsen: erst das Mess-Tooling, dann die drei System-Achsen in ihrer festen Reihenfolge, dann die Organ-Slots T0–T17.

Diese Ordnung ist das Ergebnis datierbarer *Umzüge*, keine bloße Umbenennung. Eine frühere Fünfer-Liste der System-Achsen führte neben den drei heutigen Haupt-Achsen noch den Compiler, das Scheduling und das Last-Rahmenwerk: Der Compiler ist als Unter-Achsen-Gruppe der Bau-Werkzeugkette in die äußere Komplex-Klammer gewandert (Abschnitt 3.4.2), das Scheduling als Unter-Achse an den Wrapper der Ziel-ISA, und das Last-Rahmenwerk hat die System-Welt *ersatzlos* in Richtung Mess-Realm verlassen — die System-Achsen sind damit auf ihre thematisch passenden Funktionen verschlankt. Aus der Organ-Zählung sind umgekehrt die ISA — heute die Komplex-Achse `target_isa` (Abschnitt 3.4.2) — und die Telemetrie in die System-Sphäre umgezogen; die Telemetrie *zweigeteilt*, denn Zähler-Instrumentierung muss zum Messen eingebaut sein: Als echte Sweep-Unter-Achse des Mess-Toolings im Planer erzeugt sie eine echte Übersetzungszeit-Haupt-Größe im Builder.

Dieses letzte Muster ist generisch und trägt einen eigenen Namen: die *zweigeteilte Haupt-Achse* — eine Achse, die über die Laufzeit der Vorstufe *und* die Übersetzungszeit der Folgestufe läuft, entlang der

Kette Planer → Builder → Tier-Binary; die Unter-Achsen-Freigabe der Vorstufe wird zur Haupt-Achsen-Annahme der Folgestufe. Das Last-Rahmenwerk ist die erste Instanz (der Planer wählt es zur Laufzeit, der Builder trägt es einkompiliert), die Telemetrie die zweite, und die Hardware-Erkennung (Abschnitt 3.4.3) die dritte.

Alle drei Achsen-Realms bindet ein methodisches Prinzip, die *honest-0-Doktrin* (ehrliche Null bzw. ehrliche Leere): Wo ein Wert nicht wirklich erhoben werden kann — eine Organ-Achse ohne echten Laufzeit-Parameter, ein Zähler ohne Hardware-Zugriff, eine Mess-Quelle, die auf der Plattform nicht existiert —, meldet das System *ehrlich ungültig oder leer* statt eines plausiblen Phantomwerts. Lieber eine als ungültig markierte Messung als ein erfundener Effekt: Ein Phantomwert würde die Heuristik-Ableitung (Abschnitt 3.6) systematisch vergiften, während eine ehrliche Lücke sichtbar bleibt und gezielt geschlossen werden kann. Das Prinzip gilt durchgängig — von der Achsen-Verdrahtung (nur Achsen mit real messbarem Laufzeit-Knopf melden Laufzeit-Effekte; die übrigen gelten ehrlich als übersetzungsstatisch) bis zur Verfügbarkeits-Meldung der Zähler-Quellen.

Was diese Mess-Muster konkret erheben, sind drei gegenläufige Komponenten — *Latenz*, *Durchsatz* und *Speicherverbrauch* —, gegen die sich jede Achsen-Permutation als Trade-off-Profil aufspannen lässt: dieselbe Permutation, die in Abbildung 3.1 ein Punkt im Entwurfsraum ist, erscheint hier als Punkt im Inneren des Trade-off-Dreiecks (Abbildung 3.11). Eine vierte, für cache-bewusste Suchstrukturen eigentlich zentrale Mess-Komponente — das cache-bewusste Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (die PMC-Erfassung ist standardmäßig abgeschaltet; selbst bei aktiviertem Schalter bleiben L2-Misses und Kohärenz-Invalidierungen mangels portablem Zähler strukturell ungültig, und der Last-Level-Zähler kann auf einzelnen AMD-Plattformen zusätzlich beim Öffnen scheitern) und bleibt daher als ausstehende vierte Mess-Komponente markiert.

3.4.2 Komplex-Achsen: die Ziel-ISA als feste Rekombination

Nach außen verhalten sich die drei System-Haupt-Achsen wie *eine*: Eine *Komplex-Achse* ist eine Klammer über einer festen Rekombination mehrerer Haupt-Achsen — alle Glieder bleiben Haupt-Achsen, treten aber gemeinsam als eine auf (als Befehls-Klammer im Sinne des *Command*-Musters gebündelt). Die *äußere* Komplex-Klammer spannt `target_isa × operating_system × external_utils` und trägt als Unter-Achsen-Gruppe die Bau-Werkzeugkette (Compiler, Optimierungsstufe, Atomik-Fähigkeit) — der Umzugsort der früheren Compiler-System-Achse; als Klammer ist sie selbst *kein* Glied der kanonischen Ordnung. Die Ziel-ISA ist *in sich* eine zweite, innere Komplex-Achse: eine feste Rekombination der Identitäts-Glieder RAM-Frequenz, CAS-Latenz und CPU-Fabrikation, rekursiv von der äußeren Klammer gewrappt; an ihrem Wrapper hängen die vereinbarten Unter-System-Achsen (Scheduling, NUMA-Knoten, Speicherseiten-Größe). Die Glieder werden je Maschine *deklariert* — in derselben Anwender-XML, die auch das Experiment anzeigt (Abschnitt 3.4.5) — und je Betriebssystem und ISA ermittelt; bei exaktem Eigenschafts-Match ist dieselbe Achse über formal verschiedene Maschinentypen wiederverwendbar, denn der Rechnernamen dient nur dem Instanz-Lookup — die Klassen-Identität ist das Eigenschafts-Tupel. Im Identitäts-Stempel (Abschnitt 3.4.4) belegt eine Komplex-Achse genau *ein* Feld ihrer Typ-Zeile, keine Klammer je Glied; die Betriebssystem-Distribution ist dort eine Stempel-Variable, keine vierte Ausprägung. Die CAS-Latenz ist im heutigen Stand ein rein *deklariertes* Glied (Provenienz-Stufe „deklariert, nicht gemessen“, Abschnitt 3.4.3); die Arbeit weist das aus, statt einen Messwert vorzutäuschen.

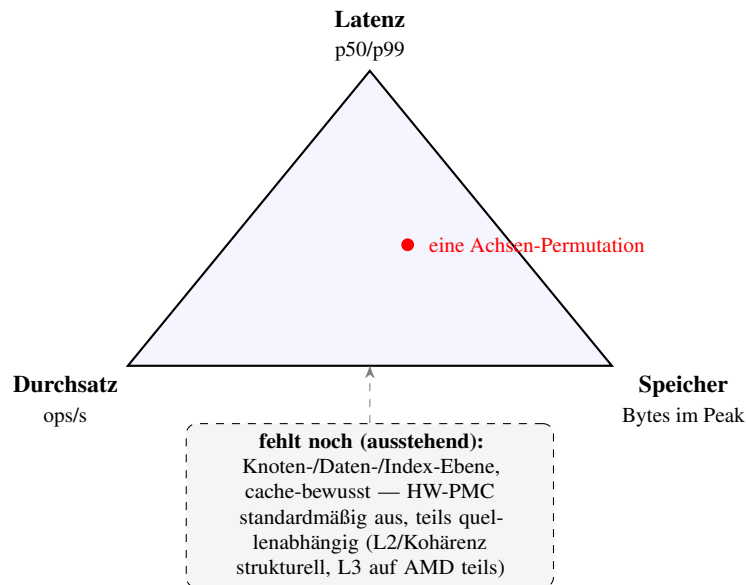


Abbildung 3.11: Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 3.1, wo dieselbe Permutation ein Punkt ist). Die für cache-bewusste Suchstrukturen eigentlich zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (gestrichelt).

3.4.3 Hardware-Erkennung als zweigeteilte Mess-Achse

Damit die deklarierten Plattform-Glieder keine still veraltenden Konstanten werden, führt das Konzept die Hardware-Erkennung als weitere zweigeteilte Haupt-Achse ein — und zwar als *Mess-Achse*: Sie misst Eigenschaften der Hardware aus, verändert aber weder System- noch Organ-Achsen. *Ebene 1* liegt beim Planer: Zu *seiner* Laufzeit erkennt er grob die Zelle aus ISA-Vendor-Familie und Betriebssystem und wählt per Metaprogrammierung aus, welche feingranulare Erkennungs-Software samt Betriebssystem-Zugriffen in den `CacheEngineBuilder` einkompiliert wird — eine Übersetzungszeit-Zellwahl, kein Laufzeit-Umschalter über Betriebssysteme. Die Auswahl-Matrix ist *total* über alle deklarierten ISA-Komplexe und alle dokumentierten Betriebssysteme: Nicht implementierte Zellen existieren als ehrliche Nur-Deklarations-Spezialisierung, und ein neuer Katalog-Eintrag ohne Zelle bricht die Übersetzung, statt eine stille Lücke zu lassen. *Ebene 2* liegt beim Builder: Mit der einkompilierten Erkennung erhebt er zu seiner Laufzeit die Ist-Eigenschaften der Maschine und setzt damit die Übersetzungszeit-Stempel der Tier-Binaries korrekt. Der Stempel dieser Mess-Achse selbst trägt die Vendor-Erkennungs-Familie und das Betriebssystem — als Geräte-Segment der Mess-Stempel-Zeile (Abschnitt 3.4.4) —, niemals einen Messwert.

Jedes Erhebungs-Ergebnis trägt eine *Provenienz*-Stufe samt Erhebungszeitpunkt, in absteigender Vertrauens-Ordnung: *konfiguriert-gemessen* (privilegierte Systemabfrage), *SPD-JEDEC-Basiswert* (unprivilegiert aus dem Modul-Speicher gelesen, bewusst nur die JEDEC-Nennrate; Übertaktungs-Profile erscheinen als separates Angebots-Feld, nie als Ist-Wert) und *deklariert-nicht-gemessen* (Anwender-XML). Eine niedrigere Stufe darf eine höhere nur mit mitwandernder Kennzeichnung verdrängen — sonst stünde eine deklarierte Nennrate in der Ergebnis-Tabelle, während die Maschine womöglich schneller fährt:

schlimmer als ehrlich deklariert. Nicht Erhobenes erhält ein ausdrückliches n/a-Zeichen, nie eine Null in der Wert-Spalte (honest-0-Doktrin, Abschnitt 3.4.1); vier Fehlerklassen trennen fehlende, unlesbare, korrupte und unbekannt formatierte Quellen, und eine korrupte Quelle degradiert nie still zur nächsten Stufe. Der Gewinn dieser Achse ist deshalb nicht ein besserer Einzelwert, sondern die *automatisierte Wiedervorlage*: Ein einmalig erhobenes und dann eingefrorenes Werkzeug-Ergebnis — so entstand die bisher hinterlegte RAM-Nennrate von 4800 MT/s aus einer manuellen `dmidecode`-Abfrage — wird durch eine Erhebung ersetzt, die bei jedem Lauf erneut stattfindet und ihre Vertrauensstufe mitführt. Im Implementierungsstand dieser Arbeit liegen von dieser Achse das Provenienz-Vokabular samt SPD-Parser und Fehler-Domäne sowie Erhebungs-Kette und Plattform-Auswahl-Matrix als Code vor (Kapitel 4).

3.4.4 Identitäts-Stempel: eine Zeile je Realm

Die Dreiteilung wird an der Tier-Binary selbst ablesbar. Ihre *Permutations*-Identität ist die `binary_id`: der Pfad aus achtzehn `achse=wert`-Segmenten in kanonischer Slot-Reihenfolge — ein Segment je Organ-Achse, und *nur* Organ-Achsen. Zusätzlich trägt jede Tier-Binary einkompilierte *Stempel-Zeilen*, genau eine je Achsen-Typ. Die *Organ-Zeile* hält für jede der achtzehn Haupt-Achsen den gewählten Algorithmus samt Algorithmus-Version fest. Die *System-Zeile* hält die drei System-Haupt-Achsen; als Version tragen sie ihre *Bau*-Version — die geordnete Suffix-Kennung aus Werkzeugkette, Optimierungsstufe, Erweiterung und weiteren Bau-Steuergrößen, in der ein leeres Glied schlicht kein Segment erzeugt —, denn eine Plattform-Achse hat keine Algorithmus-Version. Die *Mess-Zeile* führt das Last-Rahmenwerk als erstes Segment und dann die Menge der einkompilierten Mess-Toolings; eine *leere* Mess-Zeile ist dabei eine ehrliche Aussage — kein Mess-Tooling einkompiliert. Eine Komplex-Achse belegt in ihrer Zeile genau ein Feld (Abschnitt 3.4.2). Eine vierte *Realm-Zeile* für die Misch-Strategie eines Prüflings kann es dagegen nicht geben: Ein Prüfling mischt gegen die Organ-Achsen und ist damit bereits in der Organ-Zeile beschrieben. Kenntlich wird die Mischung dort am Algorithmus-Namen, den unterschiedliche Misch-Konstellationen hierarchisch erweitern (Muster `pvt-art.memory.abc@1.0.0.c`); die Versions-Bezifferung selbst markiert eine solche Mischung nicht mehr — ihr Suffix `e` steht seit der Flag-Grammatik v2 für einen Efficiency-Core-Algorithmus, nicht für eine Prüflings-Herkunft. Genau auf dieser Namens-Ebene bleiben die Hybrid- und Full-Join-Stände am Kompilat unterscheidbar; byte-verschiedene Mischbauten tragen deshalb nie namensgleiche Organ-Segmente. Abgeschlossen wird der Stempel folglich nicht von einer Misch-Zeile, sondern von der *Fingerprint-Zeile*: einem SHA512 über die drei Realm-Zeilen, der die gesamte Stempel-Identität zu *einem* vergleichbaren Wert verdichtet und an dem die Wiedererkennung einer bereits gebauten Permutation hängt. Gemessene *Werte* — etwa eine erhobene RAM-Rate — erscheinen niemals in Stempel oder `binary_id`: Gestempelt wird die Übersetzungszeit-Plattform-Klasse, nie ein Messwert (Abschnitt 3.4.3).

3.4.5 Subsysteme und die Kette von der XML zur Tier-Binary

Organisiert ist der Apparat als *ein* Hauptkanal: Eine Anwender-XML ist die einzige Quelle des gesamten Prozesses. Sie *zeigt an*, welche Experimente, Achsen, Einstellungen und Abläufe durchzuführen sind, während je Realm ein *Registry*-Katalog das *Angebot* des Existierenden hält — das deklarative Konfigurationsmodell aus Abschnitt 2.4, nach dem Vorbild des Java-POM: ein deklaratives Manifest, das gegen ein Repository aufgelöst wird; eine unregistrierte Referenz ist ein harter Validierungsfehler, keine stille

Auslassung. Der Experiment-*Planer* löst die XML gegen die drei Angebots-Kataloge auf und baut je Ziel-Plattform und Mess-Aufgabe *einen* CacheEngineBuilder, in den die gewählten Mess-Achsen einkompiliert werden; dieser Builder orchestriert mit ihnen den Bau der Tier-Binaries und stempelt sie mit seiner Plattform (Abschnitt 3.4.4); die Tier-Binary schließlich trägt die Organ-Permutation. Jede Stufe der Kette speist sich so aus ihrem art-eigenen Angebot — Mess-Katalog → Planer, System-Katalog → Builder, Organ-Katalog → Tier-Binary —, und die Tier-Binary bleibt in zwei strikt getrennten Schichten lesbar: dem Hardware- und Umgebungs-Layer der System-Achsen und dem Anwendungs-Logik-Layer der Organ-Achsen; so wenig, wie man ein Betriebssystem mit dem Anwendungs-Frontend gleichsetzt, so wenig verschmelzen diese beiden Layer in der Binary.

Auf diesen Mess-Mustern und dieser Kette aufbauend trennt das Mess-System vier Subsysteme und prüft in drei Stufen, die die drei Pflicht-Messreihen speisen; den Permutationsraum reduziert es gezielt. Vier formal getrennte Subsysteme mit unterschiedlichen Verantwortlichkeiten bilden das Modell: der `messung_driver` (der Auswertungs-Orchestrator, der je Pflicht-Messreihe A/B/C die Pipeline durchläuft), der CacheEngineBuilder (ein autonomes Plattform-Ausmess-System, das die Sieben-Phasen-Pipeline aus Abschnitt 4.3.2 über seinen `ExperimentDriver` ausführt), die CacheEngine (die Achsen-Bausteine-Bibliothek mit den Standard-Implementierungen je Achse) und der Prüfling PRT-ART (der einige Achsen-Organe selbst stellt und für die übrigen Cache-Engine-Dienste konsumiert); die Planer-Rolle der Kette und die Builder-Rolle teilen sich implementierungsseitig heute eine Binary in zwei Rollen (Kapitel 4). Die `ExecutionEngine` ist dabei kein fünftes Subsystem, sondern die gemeinsame messbare Wurzel (das `IExecutionEngine`-Interface), über die diese vier verbunden sind: Unter ihr registrieren sich der Prüfling und jede Engine als `IExecutionEngine`, und der Builder enumeriert den Permutationsraum je registrierter Engine. Definierend ist die *bidirektionale* Cache-Engine ↔ Prüfling-Beziehung: Die Cache-Engine instantiiert den Prüfling je Permutations-Konfiguration, und der Prüfling ruft zur Laufzeit Cache-Engine-Dienste (Telemetrie, Prefetch, Heuristik) während seiner eigenen Lookups auf. Diese Symmetrie trägt die *Multi-Prüfling*-Fähigkeit, mit der Messreihe A den Prüfling gegen viele SOTA-Adapter in einer einzigen Pipeline vergleicht (Abbildung 3.12).

Der Builder orchestriert eine *Drei-Stufen-Prüfung*: **Stufe 1** (nur Cache-Engine) nutzt die Standard-Variante je Achse, **Stufe 2** (nur Prüfling) ersetzt die überschriebenen Achsen durch die Prüfling-Varianten und behält sonst die Standards, und **Stufe 3** (Full Join, nicht-redundant) kombiniert Standard- und alle Prüfling-Varianten. Jede Stufe kann bis zu tausende nicht-redundante Permutations-Binaries erzeugen, die zur Laufzeit über die C-stabile Modulgrenze als C++23-Module geladen werden; der vorliegende Mess-Lauf materialisiert davon eine spärliche, gedeckelte Teilmenge (Abbildung 3.13).

Diese Stufen bilden *nicht* eins-zu-eins auf die drei Pflicht-Messreihen ab, sondern speisen sie: Die Standard-Binaries der Stufe 1 (samt der rekonstruierten SOTA-Profile) und die Prüfling-Binaries der Stufe 2 bilden *gemeinsam* die Vergleichs-Messreihe A; der nicht-redundante Full Join der Stufe 3 ist die Grundlage der Variations-Messreihe B; und die Merge-/Regressions-Messreihe C (alt gegen neu) ist an keine einzelne Stufe gebunden, sondern läuft build-übergreifend als alt-gegen-neu-Vergleich — über aufeinanderfolgende Stände derselben Konfiguration ebenso wie über konkrete Merge-Punkte (Tabelle 3.8).

Die drei Messreihen — alle vom `messung_driver` über die `ExperimentDriver`-Bibliothek ausgeführt, konfiguriert entweder als drei feste Reihen oder über eine externe `messreihen.xml` — sind damit: *Reihe A* (PRT-ART vs. Stand der Technik) mit den Modi `A_defined` (schnelle Verifikation gegen die 8 Rang-1-Profile) und `A_full` (alle 30 SOTA-Profile; der Standard, da erst der Voll-Lauf die

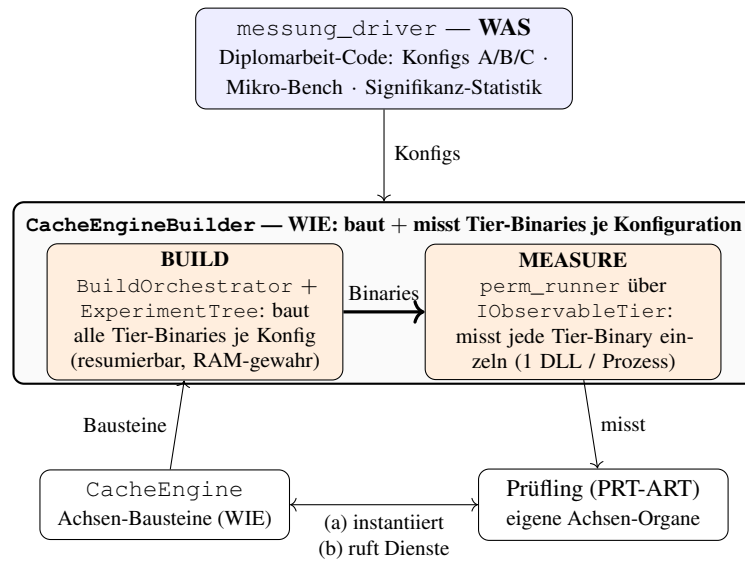


Abbildung 3.12: Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (WAS, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (WIE, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **BUILD** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **MEASURE** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) trägt die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel.

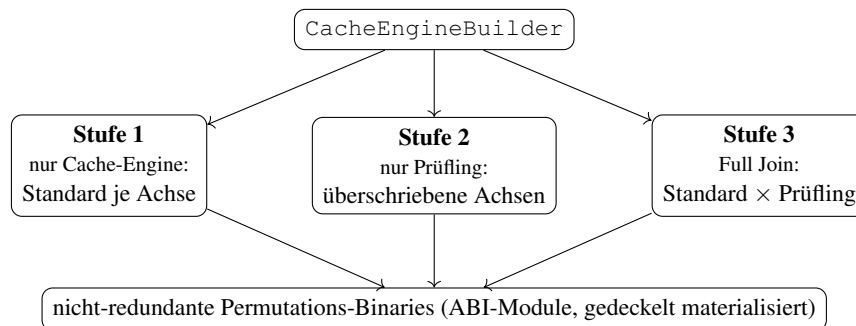


Abbildung 3.13: Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8.

Tabelle 3.8: Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen

Prüf-Stufe	Inhalt	speist Messreihe
Stufe 1 (nur Cache-Engine)	Standard-Bausteine + SOTA-Profil	A (Baseline auch für C)
Stufe 2 (nur Prüfling)	überschriebene Achsen = Prüfling-Varianten	A
Stufe 3 (Full Join)	Standard × Prüfling, nicht-redundant	B
build-übergreifend	alt gegen neu (Regression und Merge-Punkte)	C

Die drei Prüf-Stufen bilden nicht eins-zu-eins, sondern in dieser Zuordnung auf die drei Pflicht-Messreihen A/B/C ab; Reihe C ist build-/versionsübergreifend (nicht an eine einzelne Prüfstufe gebunden).

bias-reduzierte Vermessung aller Lastprofile gegen alle Verfahren einlöst, Abschnitt 3.1); *Reihe B* (systematische Variation der Achsen-Entscheidungen unter sonst gleichen Bedingungen, um den Beitrag jeder Achse zu isolieren); und *Reihe C* (Merge/Regression alt gegen neu: konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und Cache-Engine-Permutationen — etwa das PRT-ART-OLC mit `tcalloc` und ART-Page, der 4+1+2-Pool mit der HOT-Page, das Path-Prefetch mit der B²-Baum-Page oder das MultiLevel-Layout mit der Wormhole-Page —, build-übergreifend als alt-gegen-neu-Vergleich gefahren). Jede Reihe wird in drei Granularitäten erhoben. *Mikro-Benchmarking* vermisst jede Achse über ihr *Achsen-Interface* — die Funktionsmenge, die *alle* Algorithmen dieser Achse liefern müssen (jeder Allokator stellt Speicher bereit, gleich welcher): die achsen-spezifischen Haupteigenschaften, intern gemessen. Die Achse ist dabei *keine* Interface-Funktion des Suchalgorithmus; dessen Gattungs-Interface-Funktionen *verwenden* die Achsen-Interfaces — wie auch Achsen-Algorithmen fremde Achsen-Interfaces verwenden dürfen, denn Container etwa werden überall gebraucht. *Makro-Benchmarking* misst die einzelnen `std::map`-Interface-Operationen (Einfügen, Punkt- und Bereichssuche, Aktualisieren, Löschen) gegen Beispiel-Lasten, und *Gesamt-Benchmarking* fährt die YCSB-Lastprofile über jede Rekombination. Als Gegenprobe zur internen Mikro-Messung misst der Apparat zusätzlich drei *Wallclock-Ebenen*: die Wallclock der Achsen-Interfaces unter jedem Algorithmus, die Wallclock der Gattungs-Interface-Funktionen und die Wallclock einer Test-Last aus den Last-Rahmenwerken. Architektur-Auflage dieses Schnitts ist, dass alle Achsen-Algorithmen ausschließlich Achsen-Interfaces statt generischer Betriebssystem-Aufrufe verwenden — inline einkompiliert per Metaprogrammierung, ohne `std::variant` —, damit die Mikro-Messung einer Achse tatsächlich die Achse misst. Genau diese Methodik ist die Antwort auf das Trennbarkeits-Problem: Indem jeder Bestandteil zunächst isoliert über sein Achsen-Interface vermessen wird, bevor er im Operations- und Gesamt-Kontext erscheint, werden die Beiträge getrennt messbar, statt im Gesamtdurchsatz vermengt zu bleiben.

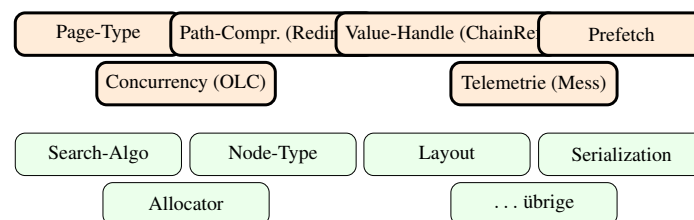
Den so aufgespannten Raum reduziert das System gezielt. Schon der kanonische Vermessungs-Katalog — je Organ-Achse zwei freigeschaltete Vertreter, die Persistenz-Achse auf ihren In-Memory-Vertreter gepinnt — zählt $2^{17} = 131072$ Binary-Identitäten je System-Permutation (je Kombination der Bau-Steuergrößen, Abschnitt 3.4.1), jede mit vollem Achtzehn-Segment-Pfad (Abschnitt 3.4.4); der rohe Varianten-Produktraum über alle Achsen-Kataloge samt ihrer Unter-Achsen liegt Größenordnungen darüber (bereits die historische Elf-Achsen-Vorstufe zählte Milliarden von Kombinationen). Drei Mechanismen dämpfen die Explosion: der `<expected_workload>`-Profil-Filter routet nur kompatible Workloads, der `Defined-Modus` (`MessreihenMode::Defined`) führt nur die deklarierten Tupel aus (rund 100–1000 je Reihe, die ins Manuskript einfließen), und der `Full-` bzw. `Full-Sampled-Modus` (etwa jede tausendste Permutation) läuft nur auf dem ZIH-Cluster unter Beachtung der vom Plattform-Probe gemeldeten Achsen-Kompatibilität.

3.5 PRT-ART als Demonstrator

PRT-ART ist der *Prüfling*, der zeigt, dass das System trägt: ein abstraktes Lebewesen, das einige Achsen-Organe selbst stellt und für die übrigen über einen Compile-Time-Fallback (`resolve_baustein.hpp`) auf die Cache-Engine-Standardbausteine zurückgreift, wo es eine Achse nicht überschreibt; die hybride `PrtArtSearchEngine<Ts...>`-API bleibt davon unberührt, und die Laufzeit-Anbindung erfolgt über den Execution-Engine-Adapter, nicht über eine eigene

Such-Engine-Schicht. Konkret überschreibt PRT-ART im Mess-Apparat die Build-Achse Page-Type, die vier Organ-Achsen Pfadkompression (Redirect), Wert-Handle (ChainRef-Slot), Prefetch und Concurrency-Pattern sowie die Laufzeit-Telemetrie (in der Dreiteilung eine Mess-Tooling-Unter-Achse, Abschnitt 3.4.1) — Path-Oriented-(Bundle-)Prefetch, optimistisches Lock-Coupling mit reservierten Wert-Blöcken sowie die Hypothesen-Metriken H1/H2/H3 —; seine weiteren eigenen Bausteine, der 4+1+2-Pool-Allokator (Free-List-Sub-Achse) und das MultiLevel-Layout, liegen im Prüfling-Repo und laufen im Mess-Pfad derzeit über den Cache-Engine-Standard (Anbindung als Folgeschritt E6 der internen Umsetzungsplanung; die Kennung ist von den Interface-Schichten E4–E0 unabhängig); für die übrigen Achsen verwendet er bestehende SOTA-Bausteine über die Permutations-Konfiguration wieder. So zeigt der Prüfling, dass das Mess-System trägt: Ein neuartiges Lebewesen entsteht aus wenigen eigenen Organen plus dem Standard-Baukasten (Abbildung 3.14) und ist über dasselbe Prüf-Dock messbar wie jedes rekonstruierte SOTA-Verfahren.

PRT-ART (Prüfling): ein Lebewesen aus 18 Achsen-Organen + Build-Achse Page-Type



orange = im Apparat überschriebener PRT-ART-Slot (inkl. Build-Achse Page-Type) **grün** = Cache-Engine-Standard (Compile-Time-Fallback)

Abbildung 3.14: PRT-ART als Demonstrator im M-Modell: Der Prüfling überschreibt im Mess-Apparat sechs Slots — die Build-Achse Page-Type, die vier Organ-Achsen Pfadkompression (Redirect), Wert-Handle (ChainRef), Prefetch und OLC-Nebenläufigkeit sowie die Laufzeit-Telemetrie (Mess-Tooling-Unter-Achse, Abschnitt 3.4.1) — und bezieht die übrigen vierzehn Organ-Achsen über den Compile-Time-Fallback aus der Cache-Engine; seine weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad derzeit über den Cache-Engine-Standard (Anbindung: Folgeschritt E6). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen.

3.6 Schärfung durch Heuristiken

Die Messreihen liefern mehr als Ranglisten: Aus ihnen lassen sich Heuristiken gewinnen, die das Mess-System schrittweise von einem Prüfstand zu einem sich selbst optimierenden Auslieferungs-System schärfen. Dieser Abschnitt entwickelt diese Schärfung als Konzept in drei Stufen — die Heuristik-Schleife über gelernte Lastprofile, die vier Betriebsmodi des Builders bis zum Hybrid-Modus als Zielbild der Arbeit und das Messkurven-Typsystem samt Filterkette und Messfehler-Erkennung, das diese Modi mit Daten versorgt. Vollzogenes und Geplantes werden dabei jeweils ausdrücklich getrennt.

3.6.1 Heuristik-Schleife und gelernte Lastprofile

Aus den Messreihen lässt sich je Last- und Datentyp-Profil die beste Achsen-Komposition ableiten und als wiederverwendbares Profil ablegen. *Die volle Schleife ist ein Ausblick-Mechanismus; im Rah-*

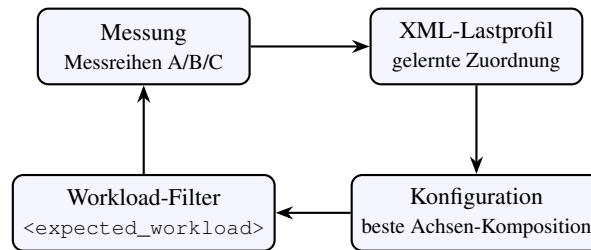


Abbildung 3.15: Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt.

men dieser Arbeit umgesetzt ist davon nur ein erstes Inkrement der automatischen Auswahl und des Binary-Versands (*best_binary_selector*, Abschnitt 1.4). Über die reine Rangbildung hinaus ist eine *Heuristik-Extraktion* als Folge-Phase nach **Persist** — der letzten Phase der Sieben-Phasen-Pipeline (Abschnitt 4.3.2) — vorgesehen, die aus den gesammelten Messungen automatisch ableitet, welche Achsen-Konfiguration je Workload und Datensatz die beste ist. Der angestrebte Mechanismus ist eine ML-Klassifikation in drei Schritten: Zunächst werden aus der Mess-CSV je Permutation Merkmale gewonnen — die Lastprofil- und Datensatz-Merkmale (Workload-Typ, Schlüssel-Verteilung, Schreib-/Lese-Anteil, Alphabet- und Präfix-Charakteristik sowie Schlüssel- und Wert-Datentyp) sowie die Achsen-Belegung —, mit der je Lastmuster gemessenen Bestleistung (etwa Durchsatz oder Tail-Latenz) als Zielgröße; darüber lernt ein *ML-Klassifikator* die Abbildung von Lastprofil-/Achsen-Merkmalen auf die beste Konfiguration; und dieser liefert je Workload-/Datensatz-Klasse die empfohlene Achsen-Komposition, ohne den gesamten Permutationsraum erneut zu fahren. Das Ergebnis wird als wiederverwendbares *XML-Lastprofil* persistiert — symmetrisch zu den Messreihen-XML-Konfigurationen — und kann ohne erneute Vollvermessung in den `<expected_workload>`-Filter zurückgespeist werden; damit schließt sich der Kreis zwischen Messung, gelernter Heuristik und konfigurationsleitender Auswahl (Abbildung 3.15). Dieser *measure* → *profile* → *config* → *filter*-Kreislauf ist die klassische, etablierte Form des *empirischen, messgetriebenen Autotunings*: Er steht in der Linie von Autotuning-Frameworks wie OpenTuner [AKV⁺14] und den messen-im-Kreis-Autotunern FFTW [FJ98] und ATLAS [WPD01]; die lastprofil-getriebene Konfigurationswahl mit Workload-Filter findet ihre Datenbank-Entsprechung in OtterTune [APGZ17] und im Database Tuning Advisor mit Workload-Kompression [ACK⁺04], die online-laufzeitnahe Variante in Active Harmony [TCH02]; die Bezeichnung „Software-in-the-Loop“ selbst entstammt der X-in-the-Loop-Validierungstaxonomie [TMS⁺16]. Die konkrete Umsetzung dieses ML-gestützten Auswahl-Mechanismus bleibt zukünftiger Arbeit vorbehalten (Kapitel 6); die Schleife selbst ist jedoch nur die erste Stufe eines größeren Betriebs-Zielbilds, das der folgende Unterabschnitt entwickelt.

3.6.2 Die vier Betriebsmodi des Builders

Die Heuristik-Schleife ist der Kern eines Betriebsregimes aus *vier* aufeinander aufbauenden Betriebsmodi des Builder-Interfaces. Von diesen vier Modi ist der erste der heutige Betrieb des Apparats; die Modi zwei bis vier sind das erklärte Ziel dieser Arbeit und werden hier als Konzept festgehalten.

Im *Mess-Modus* — dem Betrieb, den Abschnitt 3.4 beschreibt — fährt der Builder die Messreihen über die zugelassenen Permutationen. Nach Abschluss der Messung kennt das Framework die Eigenschaften aller

zugelassenen Achsen-Algorithmen gegen die Rekombination aller zugelassenen Lastprofil-Rahmenwerke und Workloads. Der *Auswertungs-Modus* verdichtet diese Messungen: Aus ihnen ergeben sich minimale Stränge der Permutationen aller Eigenschaften gegeneinander für bestimmte *Workload-Cluster* — für Gruppen ähnlicher Lasten bleibt der kleine Satz von Achsen-Kompositionen übrig, der sie dominiert. Wie Workloads zu Clustern aggregiert werden (welche Cluster-Verfahren tragen), ist bewusst offen gelassen und Gegenstand weiterer Untersuchung.

Der *Arbeits-Modus* ist das Produktions-Ziel: Das Builder-Interface hält alle für die aktuell stochastisch häufig auftretende Workload-Last und Interface-Funktion relevanten Tier-Binaries hot im Arbeitsspeicher und wechselt unter der Haube die Tier-Binary mit ihren speziell optimalen Achsen-Permutationen am ABI-stabilen Interface — derselbe Mechanismus des Binary-Wechsels, den der Mess-Modus nutzt, nur last-getrieben statt mess-getrieben. Ebenen-richtig bleibt dabei die Übersetzungszeit-Doktrin gewahrt: Es gibt keinen Laufzeit-Umschalter *im* Tier — die Binaries selbst bleiben übersetzungszeitlich permutiert; gewechselt wird *zwischen* vorgeladenen Binaries an der bewussten ABI-Grenze. Das Prüf-Dock der Gattung (Abschnitt 3.3.2) heißt in diesem Modus *Arbeits-Dock*: Dieselbe Schnittstelle, über die im Mess-Modus geprüft wird, liefert nun die produktiven Antworten.

Der *Hybrid-Modus* schließlich — das Ziel der Arbeit — verbindet Arbeitsmodus und Messung. Sein Kernstück ist, dass die Heuristik-Optimierung selbst eine *System-Achse* ist (Abschnitt 3.4): Sie übernimmt per Metaprogrammierung die Gattung ihrer Tier-Binaries, wird selbst zu einer Tier-Binary kompiliert und dockt am Prüf-Dock des Builders an wie jedes echte Tier; Befehle reicht sie über das *Command-Muster* [GHJV94] an die ihr statisch zugewiesenen echten Tier-Binaries weiter. Die Rekombination der zu verwendenden Heuristik-Tier-Binaries ist dabei selbst permutierbar gegen die existierenden echten Tier-Binaries; die Heuristik-Achse kann daher die Mess-Funktionskurven für Schätzungen verwenden und eine optimale mehrdimensionale Rekombination kompilieren, die durch ihr ABI-stabiles Gattungs-Interface eindeutig als Suchalgorithmus-Hülle verwendbar ist — nach außen ein *virtuelles ganzes Tier-Binary*, von einer einzelnen Binary nicht unterscheidbar. Zugleich misst der Hybrid-Modus mit: Über die optimierte Rekombination werden die kombinierte Wall-Clock-Zeit sowie Makro- und Mikro-Benchmarks per Mess-Command über alle Tier-Binaries und Operationen mitgeloggt.

Damit trägt der Hybrid-Modus den *heuristischen Gegenbeweis*: Nach Eintritt des Arbeitsmodus wird das Heuristik-Tier-Binary erneut vermessen — sein Wall-Clock-Verhalten im Vergleich gegen die besten Einzel-Binaries und gegen die rekonstruierten Paper-Algorithmen entscheidet, ob die heuristische Rekombination hält, was die Kurven versprechen. Das Ziel der Arbeit ist damit doppelt formuliert: die beste Rekombination beziehungsweise das beste einzelne Binary gegenüber allen Achsen-Eigenschaften — besonders der Cache-Line-Bewusstheit — zu bestimmen und am Ende beides produktionsfähig bereitzustellen, die beste einzelne Binary *und* das Heuristik-Tier-Binary. Alle vier Modi sollen dabei über die Auswertungs-Kette der Definitions-Schicht (Abschnitt 3.3.4) automatisch dokumentiert werden; die Messwerttabellen und Diagramme dieser Arbeit selbst sind das erste Erzeugnis dieser Kette, und die Bewertung des Gegenbeweises definiert Kapitel 5.

3.6.3 Messkurven-Typsystem, Filterkette und Messfehler-Erkennung

Damit der Auswertungs- und der Hybrid-Modus tragen, braucht die Auswertung mehr als flache Ergebnistabellen. Als Zielbild formuliert diese Arbeit dafür ein *Messkurven-Typsystem*; im heutigen Stand existiert davon erst der Keim — eine Kurvenanpassung über eine Achse und eine x-Dimension —, die

volle Struktur ist als Ziel formuliert. Jede Tier-Binary-Gesamtpermutation erhält je Mess-Achse eine Wall-Clock-Messkurve über das Working-Set — etwa die Lookup-Latenz als Funktion der Füllgröße —, und zwar getrennt nach Lastprofil-Rahmenwerk, Workload-Typ, Working-Set-Größe, Operationstyp und Beobachter-Typ; jede Achse wird im Experimentbaum übersetzungsstatistisch wie laufzeit-dynamisch separat durchgefahren, sodass jede Achse eine vollständige Übersicht ihrer Eigenschaften und Einzelmessungen erhält. Über alle Permutationen zusammengekommen entsteht so je Tier-Binary eine mehrdimensionale Mess-Datenbank, aus der die Heuristik-Kurven der Schleife (Abschnitt 3.6.1) abgelesen werden. Die Kardinalität jeder Dimension dieses Typsystems ist strategisch in Klassen und Hierarchien einzuteilen: design-feste Konstanten (etwa die achtzehn Organ-Achsen und die sechzehn Mess-Kategorien), profilgebundene Produkte (was ein Experiment-Profil an Belegungen zulässt) und laufzeit-freie Iterationsräume (die dynamischen Stützstellen je statischer Binary) — so bleiben die System-Achsen in der Lage, die Auswertung über alle Ebenen hinweg zu tragen.

Aus den Auswertungen ergibt sich — ebenfalls als Zielbild — automatisch eine *Filterkette*: Jedes Kettenglied prüft anhand der bisherigen Kurven, ob eine Permutations-Familie weiterverfolgt wird, und reicht die Entscheidung andernfalls an das nächste Glied weiter — das Entwurfsmuster *Chain of Responsibility* [GHJV94], strikt im Builder verortet, denn die Kette kontrolliert die *Generierung* der Tier-Binaries: Auswertung, Filter, dann die Entscheidung, welche Permutationen als Nächstes gebaut und welche verworfen werden. Diese Kette ist im heutigen Code nicht umgesetzt; sie wird als Konzept festgehalten und folgt nach den ersten vollständigen Messläufen.

Schließlich braucht der Hybrid-Modus eine Kontrolle des Messfehlers, den die Mess-Instanzen selbst eintragen — Beobachter kosten Zeit. Vorgesehen sind drei Bau-Varianten desselben Experiments: erstens Beobachter in allen Tier-Binaries *und* im Heuristik-Hybrid-Tier, zweitens Beobachter nur im Heuristik-Tier-Binary, drittens Beobachter in keinem — nur die Wall-Clock. Die Differenzen zwischen den drei Varianten quantifizieren den Messfehler durch die Messinstanzen. Komplementär folgt ein *vierter Schritt* der finalen heuristischen Zusammenstellung: Die dem Heuristik-Tier untergebenen Tier-Binaries (nicht die final optimierte Haupt-Binary) werden ohne Mikro- und Makro-Benchmarks und ohne Beobachter neu gebaut und rein an der Wall-Clock gegen alle bekannten Tier-Binary-Permutationen und gegen die bekannten Paper-Algorithmen verglichen — wobei der Paper-Vergleich Vorrang hat. Auch dies ist Zielbild: Weder die Bau-Varianten-Matrix noch der komplementäre vierte Schritt existieren im heutigen Code; gelegt ist das Fundament — die Messschalter je Kompilat und die ehrliche Gültigkeits-Markierung je Messwert im Sinne der honest-0-Doktrin (Abschnitt 3.4).

Damit ist klar getrennt, was vollzogen und was Ziel ist: Vollzogen sind der Mess-Apparat des Mess-Modus samt Drei-Stufen-Prüfung, die Auswertungs-Kette bis zu Tabellen und Diagrammen und der Keim der Kurvenanpassung; noch ausstehend sind die vollständigen Messläufe; Ziel sind Auswertungs-, Arbeits- und Hybrid-Modus, das volle Messkurven-Typsystem, die Filterkette und die Messfehler-Kontrolle. Die Bewertung des heuristischen Gegenbeweises nimmt Kapitel 5 auf; die Einordnung des Ausblicks über diese Arbeit hinaus gibt Kapitel 6. Zunächst vertieft Kapitel 4 die konkrete Implementierung der hier eingeführten Bausteine.

4 Implementierung und konkrete Algorithmusbestandteile

Dieses Kapitel steigt in das Detail: zuerst die Repository-Architektur, dann jede Schlüsselstelle von PRT-ART einzeln und bildhaft, anschließend die Achsen-Implementierung mit der *einen* Architektur im Code und der Code-Gestalt der Bau-Kette, schließlich die Qualitätssicherung über Contract-Tests je Schicht samt Zwei-Gate-Protokoll und die Code-Gestalt der Realm-Wurzeln — System- und Mess-Achsen einschließlich der Hardware-Erkennung.

4.1 Drei-Repository-Architektur

Die Implementierung verteilt sich auf drei Git-Repositories mit klar getrennten Verantwortlichkeiten:

- `comdare-cache-engine` — **wie** gemessen wird: die Bausteine-Bibliothek und der Builder (XML-Config-Parser, `generate_module_from_profile`-Codegenerierung, Permutations-Schleife, Modul-Loader, der sieben-phasige `ExperimentDriver` und der Experiment-Planer `ExperimentPlanDirector`, Abschnitt 4.3.2), die Bibliotheken `search_engine` und `execution_engine`, die ABI-Header unter `abi/` (`baustein_variants`, `resolve_baustein`, `module_abi_v1`), die 30 SOTA-Profilen unter `algorithm_profiles/sota` und die Test-Suites; der Quellbaum folgt einem Pitchfork-Layout (`apps/` für ausführbare Dateien, `libs/` für Bibliotheken, `libs/common` für querschnittlichen Code).
- `comdare-prt-art` — der **Prüfling**: die hybride `PrtArtSearchEngine` (Vector/Map/Tuple-API), ihre ABI-Adapter und ihre Bausteine (4+1+2-Allokator-Pools — sieben Pools: vier Größenklassen-Pools A–D, ein Rest-Pool R und zwei reservierte Wert-Pools —, OLC mit reservierten Blöcken, `MultiLevel-Layout`, pfadorientiertes Prefetch, `DensityTracker`, Hypothesen-Metriken H1/H2/H3) sowie die prüflingseitigen Vergleichs-Re-Implementierungen der Rang-2/3-Paper ohne öffentlichen Quellcode (`legacy_reimpl`, P11–P27), die im SOTA-Beitrittstest gegen die echten Adapter der Cache-Engine verglichen werden; Pool-, Prefetch- und Layout-Komponenten sind dabei derzeit Deskriptor-Skelette ohne echte Speicheranbindung — der Prefetch-Baustein überschreibt gleichwohl bereits den Prefetch-Slot des Mess-Apparats, während Pool und Layout dort noch über den Cache-Engine-Standard laufen (Abschnitt 4.2).
- `probst-Diplomarbeit-cache-engine` — **was** getestet wird: der `messung_driver` (Binary `comdare-messung-driver`), die Experiment-Konfigurationen, die Auswertungs-Pipeline und das Manuskript.

Die vier Subsysteme (Mess-Treiber, `CacheEngineBuilder`, `CacheEngine`, Prüfling) liegen dabei orthogonal zu dieser Drei-Repository-Aufteilung: Builder und `CacheEngine` teilen sich `comdare-cache-engine` — der Builder als ausführbare Datei (`apps/`) über seiner Builder-Bibliothek (`libs/cache_`

engine/builder), die CacheEngine als Bibliothek (libs/cache_engine). Quer dazu trennt die Terminologie zwei Engine-Begriffe, die nicht kurzgeschlossen werden dürfen. Die CacheEngine (libs/cache_engine) ist die *Bibliothek der Achsen-Bausteine*: Sie stellt je Achse die austauschbaren Entwurfsbestandteile als Concept-/CRTP-Komponenten bereit und enthält *keine* Mess-Logik. Die ExecutionEngine hingegen ist die quer zu allen Achsen liegende *Ausführungs- und Mess-Infrastruktur*: Ihre abstrakte messbare Wurzel ist IExecutionEngine (Abschnitt 3.4); der konkrete Laufzeit-Adapter SearchAlgorithmAbiAdapter<A> liefert über die SearchAlgorithmAnatomy die OS-Primitiven (cache-line-ausgerichtete Allokation, NUMA-konformes Read-Pinning, kohärenz-bewusstes Schreiben); die Mess- und Observer-Kopplungen des Adapters sind über COMDARE_MEASUREMENT_ON gated — daraus entstehen vier Auslieferungsvarianten derselben Permutation (Debug mit Observer, Messung mit Aggregator, Release ohne Mess-Kopplungen und das bau-gleich zu Release gehaltene Compare-Etikett) —, und den Mess-Aggregator führt der ExperimentDriver host-seitig; die CMake-Option COMDARE_EXPERIMENT_MODE markiert Experiment-Kompilate und kodiert übersetzungszeitlich die Invariante, dass ein Experiment-Bau stets ein Mess-Bau ist — sie selbst schaltet keine Hooks. Kurz: Die CacheEngine liefert die *Bausteine*, die ExecutionEngine die *Werkzeuge*; diese Trennung hält den Produktiv-Pfad mess-frei.

4.2 PRT-ART im Detail

Aufbauend auf dem bekannten Stand der Technik beschreiben wir jede Schlüsselstelle von PRT-ART einzeln und bildhaft. Der Prüfling steuert sechs eigene Schlüsselstellen bei, zwei davon auf der Telemetrie-Achse, die seit der Achsen-Neuordnung in der System-Sphäre liegt (Abschnitt 3.4.1). Auf der *Allokator-Achse* ersetzt ein *4+1+2-Pool-Allokator* (vier Größenklassen-Pools, ein Rest-Pool plus zwei reservierte Wert-Pools) die algorithmus-fest verdrahtete Allokation durch eine cache-line-ausgerichtete, fragmentierungsarme Free-List. Auf der *Nebenläufigkeits-Achse* — selbst gegliedert in Pattern (8.1) und Locking-Mode (8.2) — trägt PRT-ART *optimistisches Lock-Coupling (OLC) mit reservierten Wert-Blöcken* bei, koordiniert über einen Concurrency-Manager, der die Disziplinen OLC, HTM, STM, Lock-Free, RCU und Hazard Pointers führt; die Speicherfreigabe ist dabei eine Sub-Achse des Allokators (epoch-, RCU-, Hazard-Pointer- und QSBR-Strategien je Permutation wählbar). Auf der *Prefetch-Achse* liefert ein *pfadorientiertes Bundle-Prefetch* die heißen Suchpfade vorab, auf der *Layout-Achse* ein *MultiLevel-Layout*. Besonders cache-kritisch ist die *Telemetrie-Achse*: Per-Node-Zähler verursachen auf Mehr-Kern-Systemen Cache-Line-Ping-Pong und verfälschen genau jene Messungen, die sie erheben. Der Kuehn-Forschungslinie [Kue23] folgend bietet PRT-ART daher cache-kohärenz-bewusste Strategien — *Leaf-Only-Zähler* (Hauptvariante), *Sampling* (jeder N -te Blattzugriff) und *Offline-Recompute* (Bottom-up-Aggregation vor dem Reordering) —, während der klassische Per-Node-Zähler nur als gekennzeichnetes Anti-Pattern erhalten bleibt. Den DensityTracker und die Hypothesen-Metriken H1/H2/H3 ergänzt der Prüfling als eigene Telemetrie-Organe. Diese Schlüsselstellen sind *Bausteine des Prüfling-Repos*; im Mess-Apparat überschrieben sind davon derzeit Prefetch, Nebenläufigkeit und Telemetrie (zusammen mit der Build-Achse Seitentyp, Pfadkompressions-Redirect und ChainRef-Wert-Handle), während 4+1+2-Pool-Allokator und MultiLevel-Layout im Mess-Pfad über den Cache-Engine-Standard laufen, bis ihre Anbindung (Folgeschritt E6) landet (Abschnitt 3.5). Auf eine Galerie verdichtet zeigt Abbildung 4.1 die

sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse verortet. Die verbleibenden zwei der acht Bausteinschichten aus der Beitragsliste (Abschnitt 1.4) — die Inline-/External-/ChainRef-Wert-Handles und die Signaling-Bit-Serialisierung — treten nicht als eigene Galerie-Punkte auf: Erstere stellen den im Apparat überschriebenen ChainRef-Slot (T11), Letztere ist als `VarLenSerialization`-Baustein mit Signalisierungsbits in Anhang D katalogisiert.

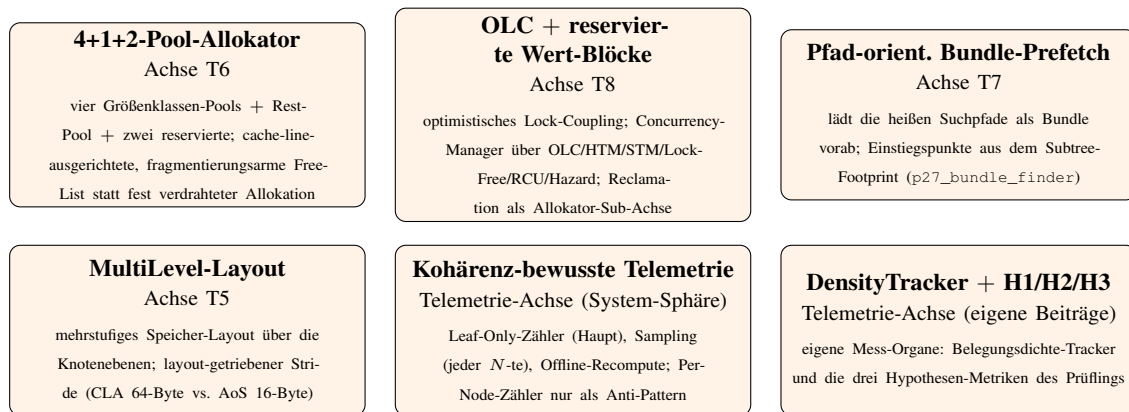


Abbildung 4.1: Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenz-bewusste Telemetrie samt DensityTracker und den Hypothesen-Metriken H1/H2/H3 — die Telemetrie-Achse liegt seit der Achsen-Neuordnung in der System-Sphäre (Abschnitt 3.4.1). Die Kacheln zeigen die *Bausteine des Prüfling-Repos*; im Apparat überschrieben sind derzeit T7/T8 und die Telemetrie, die Bausteine zu T5/T6 laufen dort noch über den Cache-Engine-Standard (E6). Zusätzlich überschreibt der Prüfling Pfadkompressions-Redirect (T3), ChainRef-Wert-Handle (T10) und die Build-Achse Seitentyp; die übrigen vierzehn Organ-Achsen (derzeit einschließlich T5/T6) bezieht er über den Compile-Time-Fallback aus der Cache-Engine.

4.3 Achsen-Implementierung und Mess-System-Ebenen

Jede Achse wird als generalisiertes Interface implementiert; daneben liegen die Ebenen des Mess-Systems von der Codegenerierung bis zur Sieben-Phasen-Pipeline. Jede Achse ist ein Topic-Verzeichnis, das ein zweischichtiges *Concept* (ein breites Topic-Concept und ein enges Achsen-Concept) sowie eine durch eine *requires*-Klausel abgesicherte *CRTP-Basisklasse* enthält. Die Prüfling-gegen-Standard-Auswahl wird für die vierzehn PRT-ART-Fallback-Achsen zur Übersetzungszeit über `resolve_baustein.hpp` (Concept-geprüftes `if constexpr` über Tag-Spezialisierungen; der literale Tag-Katalog der historischen Elf-Achsen-Vorstufe besteht dort nur als `baustein_variants`-Fallback fort) aufgelöst, sodass der Hot-Path weder `virtual`-Aufrufe noch Laufzeit-Switches enthält — die zero-cost abstraction aus Abschnitt 3.3. Jeder SOTA-Entwurf und jeder Allokator wird über einen dünnen *Adapter* integriert, der die Original-Implementierung hinter dem Engine-ABI bereitstellt: Allokator-Vendors werden über `per-Vendor-Include-Shims` (`#if COMDARE_AXIS_06_USE_<X>` mit Forward-Stubs), SOTA-Bausteine über `constexpr-Enable-Flags` samt `mp_filter` (`COMDARE_AXIS_*_USE_-/ENABLE_-Flag-Familie`) übersetzungszeitlich zu- oder abgeschaltet, jeweils mit sicherem, familienweise gewähltem Fallback — Allokator-Adapter ersetzen den fehlenden Vendor durch `std::malloc/std::free`, Datenstruktur-

Adapter durch einen Stub, der bei Benutzung kontrolliert eine `std::runtime_error`-Ausnahme wirft —, sodass der Build eigenständig lauffähig bleibt. Das frühere Detektions-Define `COMDARE_HAVE_<X>` besteht nur noch historisch. Die Integration des hierarchischen Prefetching [ZGH⁺25] liefert zusätzlich einen Call-Graph-Analyzer zur Übersetzungszeit (`p27_bundle_finder`, ein C++23-Port des Original-Werkzeugs `hp_soft`, der Funktionen mit Subtree-Footprint oberhalb eines Schwellwerts als Bundle-Einstiegspunkte identifiziert).

Die achtzehn Organ-Achsen (T0–T17, Abschnitt 3.4.1) sind dabei nicht abstrakt, sondern je ein konkretes *Strategie-Concept* mit mehreren austauschbaren Bausteinen, aus denen eine konkrete Composition je Slot genau einen wählt. Abbildung 4.2 fasst alle achtzehn als Galerie zusammen — je Achse der Kurzname, die Sub-Achsen-Familie und eine repräsentative Auswahl der Bausteine; die vollständige Bausteine-Matrix mit allen Herkunfts-Quellen führt Anhang D, die SOTA-Beiträge je Achse Tabelle 3.2. Die Neunzehner-Zählung früherer Fassungen dieses Kapitels (T0–T18, Zwischenstand Juli 2026) ist durch diese Ordnung abgelöst (Abschnitt 3.4.1). Dass keine Composition ein Organ auslässt, erzwingt das `IsComposition`-Concept: Es verlangt je Algorithmus die Belegung genau dieser achtzehn Organe — der fünfzehn Such-Achsen, der beiden Queuing-Achsen und des Persistenz-Ziels —, sodass keine Composition übersetzt, die eine Achse offenlässt; der vollständige Organ-Satz ist damit keine Konvention, sondern eine übersetzungszeitlich geprüfte Bedingung.

4.3.1 Die eine Architektur im Code

Die *eine* Architektur aus Abschnitt 3.3 wird auf Code-Ebene über eine ABI-stabile *Hülle* nach außen sichtbar. Das Lebewesen `SearchAlgorithm` und seine ABI-Laufzeit-Sicht — die „`SearchEngine`“ — sind nur zwei Sichten desselben Gegenstands über die Modulgrenze, kein paralleles Konstrukt: Die Schichtung `CacheEngine` → `ExecutionEngine` → `SearchEngine` trennt die Rollen (vgl. Abschnitt 4.1) — die `CacheEngine` als Achsen-Bausteine-Bibliothek, die `ExecutionEngine` als Wurzel der Ausführungs- und Mess-Werkzeuge (OS-Primitiven; im Experiment-Modus der Mess-Aggregator über genau eine Beobachter-Schnittstelle) und die `SearchEngine` als die über genau diese Schnittstelle vermessene Such-API-Sicht (Abbildung 4.3). Achsenlose *Viren* stehen als Geschwister unter derselben Wurzel, tragen aber keine Anatomie.

Konkret ist die Modulgrenze eine C-stabile Schnittstelle, die je Permutation genau ein Lebewesen exportiert; ihre variadische Hybrid-API spezialisiert sich nach Parameterzahl, und komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert (Abbildung 4.4). Wie diese und die übrigen Code-Interfaces im Detail zusammenhängen, fasst die UML-Landkarte in Abbildung 4.5 zusammen.

4.3.2 Vom Profil zur Tier-Binary: Planer, Builder und Mess-Pipeline

Die Kette von der Anwender-XML zur Tier-Binary (Abschnitt 3.4.5) hat im Code eine benannte Gestalt. Ihr Einstieg ist der Mess-Treiber `comdare-messung-driver`, der die Rollen der Kette als Subkommandos führt: `validate` prüft rein lesend das per XML-Parser aufgelöste Profil gegen die einkompilierten Registry-Kataloge (eine unregistrierte Referenz ist ein harter Validierungsfehler, Abschnitt 3.4.5), baut dabei keine Binary und misst nicht. Die Planer-Rolle `plan dump|ci|cmake` fährt den deterministischen Walk des `ExperimentPlanDirector` — nach den Entwurfsmustern *Director*

T0 Suchalgorithmus SA1-SA4 Array256, OriginalART/HOT, KAry, Interpol., Hash ... (21: 17 Basis + 4 per-K) <i>Std. Array256</i>	T1 Cache-Travers. CT1-CT2 Linear-Fanout, Hash- Lookup, Binär-Fanout <i>Std. Linear</i>	T2 Slot-Mapping MP1-MP2 Direkt-Platzierung, Pool-relativ <i>Std. Direkt</i>	T3 Pfadkompression PC1-PC3 Keine, Byte-weise (ART), Patricia <i>Std. Keine</i>
T4 Knotentyp NT1-NT3 Node4 / 16 / 48 / 256 <i>Std. Node256</i>	T5 Speicher-Layout HM1-HM4 AoS, Cache-Line-aligned, SoA, Bitmap (LOUDS), AoSoA <i>Std. Cache-Line</i>	T6 Allokator AA1-AA7 Hoard, Slab, Mimalloc, Jemalloc, TCMalloc, Buddy ... (25) <i>Std. Std-Malloc</i>	T7 Prefetch PF1-PF3 Keine, Distanz-Schätzer (ART), Hardware, Pfad-orient. <i>Std. Keine</i>
T8 Nebenläufigkeit CC1-CC2 Keine, Blocking, R/W, OLC, Lock-free, RCU, Hazard <i>Std. Keine</i>	T9 Serialisierung SR1-SR3 Roh-Binär, VarLen (LEB128), Succinct, LZ4/Delta <i>Std. Roh-Binär</i>	T10 Wert-Handle VH1-VH3 Inline, Extern-Pool, RCU- Shared, MVCC, Chain-Ref <i>Std. Inline</i>	T11 Index-Org. io1-io3 Clustered, Heap, IOT (em- bedded), Non-Clustered <i>Std. Heap</i>
T12 I/O-Dispatch IO1-IO3 In-Memory, Buffered, Di- rect (O_DIRECT), Mmap <i>Std. In-Memory</i>	T13 Migration MG1-MG3 Keine, Hot/Cold, Tier- basiert, Adaptiv (ML) <i>Std. Keine</i>	T14 Filter FT1-FT3 Bloom, Cuckoo, Range- SuRF (LOUDS), Xor <i>(kein None)</i>	T15 Op.-Puffer (Q1) QS1-QS6 Kein, Append, FIFO/LIFO, Ring, Priority ... (15) <i>Std. Kein-Puffer</i>
T16 Flush-Policy (Q2) FS1-FS4 Eager, Lazy, Watermark, Timed, Adaptiv-LSM <i>Std. Lazy</i>	T17 Persistenz-Ziel 18. Slot (ORG-18) In-Memory; Disk-Writeback (deklariert, per Option abgeschaltet) <i>Std. In-Memory</i>	Umzüge (Juli 2026) Telemetrie (TM1-TM3) → System-Sphäre, zweigeteilt; ISA (IS1-IS3) → System-Haupt-Achse target_isa (Abschnitt 3.4.1)	Lesart Jede Achse = ein Concept mit austauschbaren Bausteinen (Strategy); eine Composition wählt je Slot genau einen.

Abbildung 4.2: Galerie der achtzehn Organ-Achsen (T0–T17, kanonische Slot-Reihenfolge): je Achse das Implementierungs-Konzept — Kurzname, Sub-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein Concept-Interface mit mehreren Strategie-Bausteinen (Strategy-Muster); eine konkrete Composition belegt je Slot genau einen. Der *Standard* („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag; bei den großen Achsen T0/T6/T15 ist die Gesamt-Bausteinanzahl in Klammern angegeben. Die T-Nummern sind Slots der Composition, keine Verzeichnis-Nummern der Implementierung. Gegenüber der Neunzehner-Galerie früherer Fassungen (T0–T18, Zwischenstand Juli 2026) sind Telemetrie und ISA in die System-Sphäre umgezogen, und das Persistenz-Ziel ist als achtzehnter Slot hinzugekommen — im kanonischen Vermessungs-Katalog auf seinen In-Memory-Vertreter gepinnt (Abschnitt 3.4.1). Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D.

und *Builder* getrennt: Der Director besitzt den *einen* Enumerations-Walk über Mess-Kombinationen, System-Permutationen und Organ-Schritte, und je Ausgabe-Syntax (Text-Plan, CI-Pipeline-Beschreibung, CMake-Fragment) emittiert ein konkreter Builder über Hook-Methoden; zwei Läufe sind byte-gleich, und der Plan-Kopf annotiert das Registry-Trio der drei Angebots-Kataloge Organ/System/Mess. Die Builder-Rolle `tier ci|cmake` liegt schließlich bei der *gebauten* CEB: Sie emittiert selbst die Sicht der zweiten Ketten-Stufe — System-Permutationen und Tier-Bau-Aufträge ihres freigegebenen Bau-Raums —, denn der Planer steuert die CEB-Aufträge, die CEB steuert die Tier-Aufträge. Implementierungsseitig teilen sich Planer- und CEB-Rolle heute eine Binary; das ist ehrlich dokumentiert, kein Schein-Split. Die emittierten Bau- und Prüf-Testate halten die Layer-Trennung der Kette sichtbar: Sie tragen System- und Organ-Klammer stets getrennt, und nur Mess-Testate tragen alle drei Klammern.

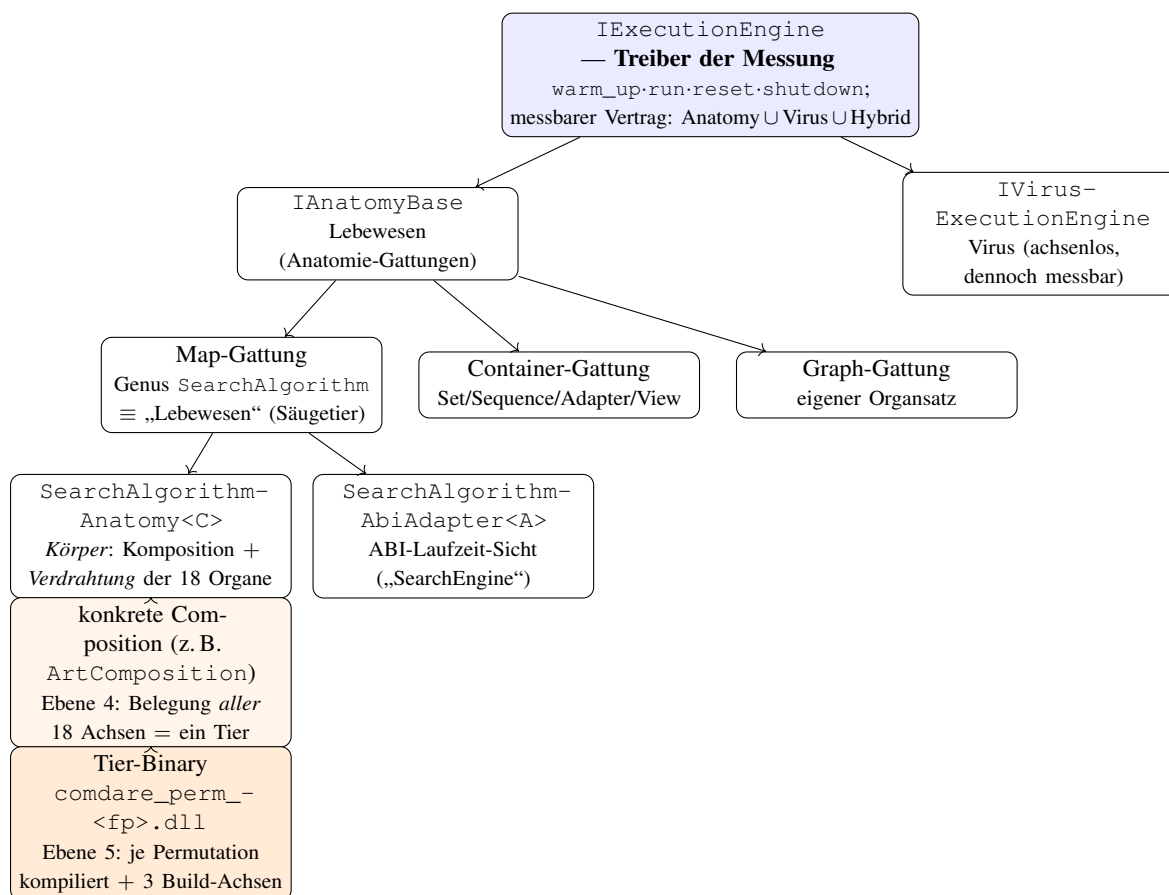


Abbildung 4.3: Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht IExecutionEngine als *Treiber der Messung* — der universelle messbare Vertrag (warm_up/run/reset/shutdown) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Gattungen* (Map, Container, Graph) und die achsenlosen *Viren*. SearchAlgorithm ist das fokale Lebewesen (Säugetier); seine Anatomie ist sein *Körper* — die feste Komposition und Verdrahtung der 18 Organe —, und „SearchEngine“ nur dessen ABI-Laufzeit-Sicht. Der Säugetier-Ast ist bis zum Individuum ausgezogen: eine konkrete Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5). Container (schlüsselloser Speicher) und Graph tragen eigene Organsätze.

```

// C-stabile Modulgrenze: exportiert genau EIN Lebewesen je Permutation
extern "C" IAnatomyBase* comdare_create_anatomy();

// Variadische Hybrid-API je Parameterzahl (Schlüssel K, Werte V):
// 1 Parameter -> std::vector<V>
// 2 Parameter -> std::map<K, V>
// N > 2       -> std::map<K, std::tuple<V1, ..., VN>>
template<class... Ts> class SearchEngine; // ABI-Laufzeit-Sicht des Lebewesens

```

Abbildung 4.4: Die ABI-stabile C-Modulgrenze und die variadische Hybrid-API. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert.

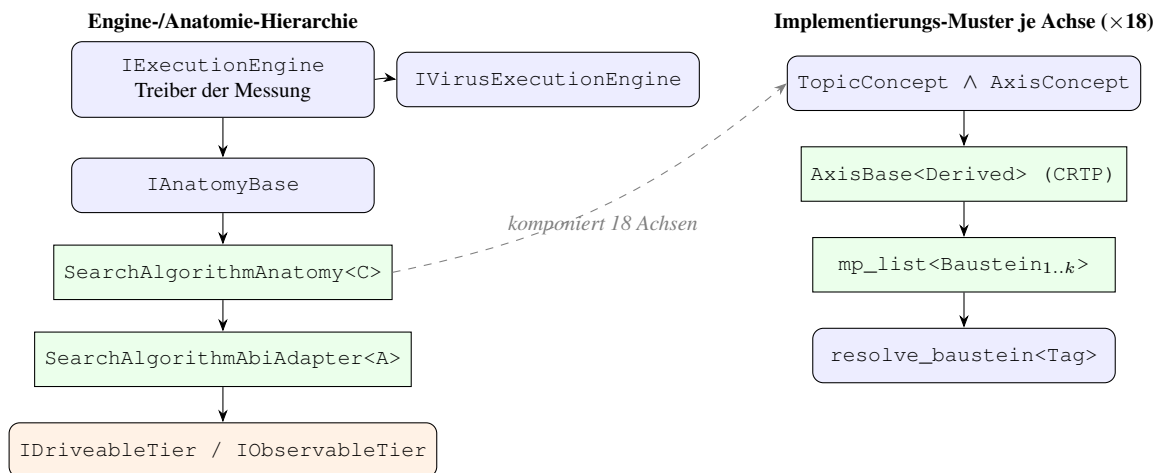


Abbildung 4.5: UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel `IExecutionEngine` als *Treiber der Messung* (Anatomy ∪ Virus ∪ Hybrid) über `IAnatomyBase` und der achsenlosen `IVirusExecutionEngine`, darunter der *Körper* `SearchAlgorithmAnatomy<C>` (Komposition und Verdrahtung der achtzehn Achsen) und seine ABI-Laufzeit-Sicht `SearchAlgorithmAbiAdapter<A>` („SearchEngine“), die über die Treib-/Beobachter-Verträge `IDriveableTier/IObservableTier` gemessen wird; rechts das je Achse wiederholte Implementierungs-Muster — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die `mp_list`-Baustein-Familie und die Übersetzungszeit-Auflösung `resolve_baustein`, die je Achse zwischen Prüfling-Slot und Cache-Engine-Standard wählt.

Zwei Eigenschaften sichern den *einen* Hauptkanal ab. Erstens weist der Text-Plan seit seiner Fassung v1.1 je Bau-Schritt den real gebauten Datei-Stamm aus — erzeugt von *derselben* Funktion, die auch der Bau-Orchestrator beim echten Bau ruft: Der Plan behauptet keinen Namen, den der Bau nicht produziert. Zweitens ist der frühere zweite Emissions-Kanal des Permutations-Codegens — ein CMake-seitiges Werkzeug-Trio, das Permutations-Quelltexte außerhalb dieser Kette erzeugte — im Juli 2026 zweistufig stillgelegt worden: erst in den konsumierenden Repositorien abgeschaltet, dann im Engine-Repository byte-neutral entfernt. Es bleibt *eine* XML-Quelle und *ein* Kanal, der sie baut; die Variantenbildung liegt so weit wie möglich in der C++23-Metaprogrammierung, und CMake bleibt auf die Rolle des Bauens beschränkt.

Innerhalb der CEB überführt der Builder jedes Profil in ein vorkompiliertes C++23-Modul; jede Permutation ist durch eine bit-gepackte *Permutations-Flag*-Struktur identifiziert, die mit der N-Phasen-Verfeinerung von neun Bänken (rund 50 Bit) auf einen breiteren *Permutations-Identifizier* mit vierzehn Bänken (82 Bit) und Sub-Bank-Bitfeldern wächst; die achtzehn Organ-Achsen identifiziert der aktive Mess-Pfad über den string-serialisierten Achtzehn-Segment-Pfad (die `binary_id`, Abschnitt 3.4.4) samt Fingerprint, und ein Constraint-Filter schließt ungültige Kombinationen vor jeder Modulerzeugung aus. Über diese Bausteine läuft die *Sieben-Phasen-Mess-Pipeline* (Abbildung 4.6) der `ExperimentDriver`-Bibliothek je Reihe: (1) **Enumerate** (XML-Configs parsen, Permutationsliste aufbauen), (2) **Codegen** (ein `comdare_perm_<fp>.cpp` je Permutation, plus `generate_module_from_profile` für die SOTA-Profile), (3) **Compile**, (4) **Load** (LoadLibrary/dlopen aller Module), (5) **Execute** (`create_instance` + profilbewusstes `run_workload`), (6) **Measure** (binäre Mess-Records

aggregieren) und (7) **Persist** (`measurements.csv/.json`); zwei *opt-in*-Phasen kompilieren fehlende Module *hot* und verifizieren den ABI-Vertrag je Modul. Die nachgelagerte Auswertung-Werkzeugkette (Abbildung 4.7) ist eine Folge kleiner Programme — `sample_data_generator` (deterministische Testdaten ohne reale Hardware) → `messung_driver` (steuert die Reihen) → `binary_to_csv` (deserialisiert die binären Mess-Records) → `csv_to_latex` (booktabs-Tabellen mit Algorithmus-Steckbriefen je Permutations-ID) → `diagram_generator` (A4-bewusste TikZ-Plots) → `latex_to_pdf` —; die Mess-Kopplungen sind über das `COMDARE_MEASUREMENT_ON`-Gating vom Produktiv-Pfad getrennt (Abschnitt 4.1), und Continuous Integration (GitLab CI plus gespiegelter GitHub-Workflow) läuft über alle drei Repositorien.

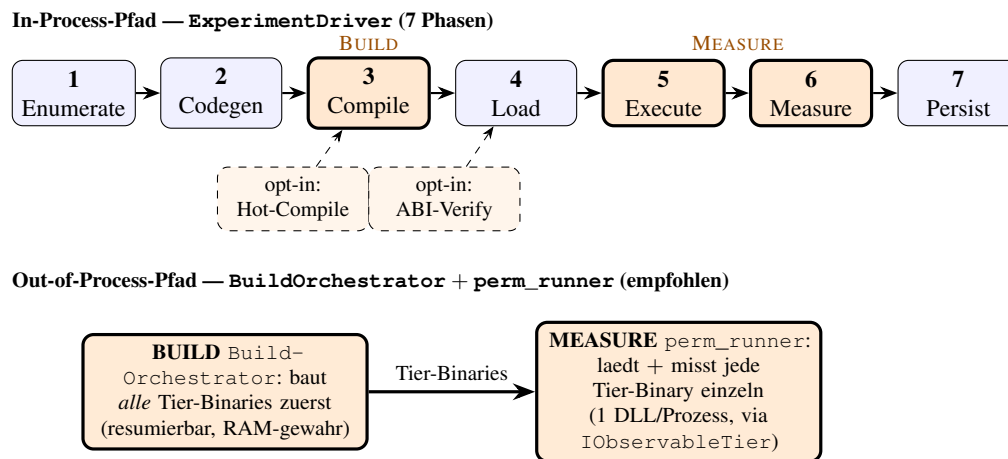


Abbildung 4.6: Die Mess-Pipeline in zwei Ausprägungen. Der **In-Process-Pfad** (oben) ist die Sieben-Phasen-Pipeline des `ExperimentDriver` je Messreihe (`Enumerate` → `Codegen` → `Compile` → `Load` → `Execute` → `Measure` → `Persist`; zwei *opt-in*-Phasen kompilieren fehlende Module *hot* und verifizieren den ABI-Vertrag). Hervorgehoben sind die eigentlich substanziellen Phasen: `COMPILE` (der *Build* der Tier-Binaries) und `EXECUTE/MEASURE` (die *Messung*); `Codegen/Load/Persist` sind reines Plumbing. Der **Out-of-Process-Pfad** (unten, der empfohlene Weg) trennt beide Aufgaben out-of-process: der `BuildOrchestrator` baut *zuerst alle* Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst `perm_runner` jede Binary einzeln (eine DLL je Prozess) über die Beobachter-Schnittstelle `IObservableTier`.

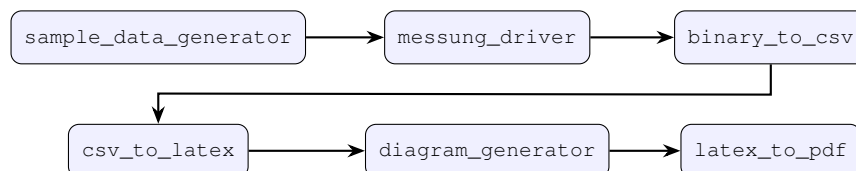


Abbildung 4.7: Die nachgelagerte Auswertung-Werkzeugkette als Sechs-Stufen-Kette: `sample_data_generator` erzeugt deterministische Testdaten, `messung_driver` steuert die Messreihen, `binary_to_csv` deserialisiert die binären Mess-Records, `csv_to_latex` rendert die booktabs-Steckbriefe je Permutations-ID, `diagram_generator` die A4-bewussten TikZ-Diagramme, und `latex_to_pdf` setzt das Manuskript.

4.4 Qualitätssicherung: Contract-Tests je Schicht

Das Schichtenmodell E4–E0 mit seinen eingefrorenen Interface-Verträgen hat Abschnitt 3.3.4 als Konzept entwickelt; dieser Abschnitt beschreibt seine Umsetzung als Prüf-Infrastruktur. Der Anlass ist eine konkrete Projekterfahrung: Zwei Implementierungs-Inkremente, die mehrere Schichten gleichzeitig veränderten, mussten vollständig zurückgenommen werden, weil ihre Fehler aus Wechselwirkungen zwischen den Ebenen stammten, die weder lokale Tests noch zeilenweises Review sichtbar machten. Für ein System, dessen Produktraum jede vollständige Materialisierung ausschließt (Abschnitt 3.4.5), sind Mehr-Ebenen-Änderungen damit praktisch nicht abnehmbar; die Konsequenz ist, jede Schicht einzeln fertigzustellen — mit explizit eingefrorenem Interface-Vertrag zur Nachbar-Ebene, eigenem Test-Harness und eigener Abnahme-Definition.

Das tragende Prüfprinzip lautet: *Jede Schicht wird gegen einen übersetzungszeitlichen Ersatz ihrer Nachbar-Ebene getestet.* Die Contract-Tests sind gewöhnliche Testziele der Build-Pipeline; die Nachbar-Ebene ersetzt dabei je ein *Fake* — eine Template- und Concept-basierte Attrappe im Test-Scope, kein Laufzeit-Umschalter im Produktiv-Code. So kann eine Schicht fertig und abgenommen sein, bevor die tiefere Ebene ihre Werte überhaupt konsumiert, und die Prüffrage jeder Grenze — reicht die Schicht ihre Definition *verlustfrei* an die nächste weiter? (Abschnitt 3.3.4) — wird vom Entwurfsgrundsatz zum ausführbaren Abnahme-Test.

Je Schicht konkretisiert sich das so: Für die Definitions- und Auswertungs-Schicht E4 prüft der Harness die Übersetzung der deklarativen XML-Profile in die Achsen-Ebenen-Struktur — Golden-Fixtures vergleichen die aus einem Profil emittierten Achsen-Ebenen strukturell gegen eine erwartete Referenz, wobei ein Fake-E3 den Permutationsbaum ersetzt; weder Baum-Aufbau noch Bau-Aufträge sind nötig, um die Grenze E4→E3 abzunehmen. Für den Permutations-B⁺-Baum E3 prüft der Harness die Bijektion der Binary-Identität: Aus Achsen-Ebenen-Fixtures wird die Permutations-Enumeration gefahren und je Pfad der mixed-radix-kodierte Binary-Identifizier hin- und zurückübersetzt (Zählung plus Signatur-Roundtrip) — gegen ein Fake-E2, das keine einzige Binary baut; dazu tritt ein Konformitäts-Gate im Voll-Lauf-Pfad, das jede importierte Binary vor der Messung gegen den eingefrorenen Vertrag prüft. Für die Tier-Binaries E2 ist der Vertrag die ABI selbst: Eine emittierte Referenz-DLL wird geladen und muss die eingefrorene Schnittstellen-Folge erfüllen — `IAAnatomyBase`, `IObservableTier` und `IResourceControllableTier` samt korrekter Fähigkeits-Auskunft —, und eine Stellgrößen-Anfrage oberhalb der gemeldeten Fähigkeiten muss deterministisch mit dem eingefrorenen Fehlerwert der `apply`-Antwort (`apply == 5`) quittiert werden, statt still angenommen zu werden; das zugehörige Fake-E1 legt die Laufzeit-Stellgrößen an und appliziert sie, verlangt aber keine Konsum-Wirkung — die gehört zur Abnahme von E1. Für die Laufzeit-Konfiguration E1 schließlich prüfen Edge-Tests die Randfälle des dynamischen Regimes: dass ein wiederverwendetes Tier zwischen zwei Läufen vollständig zurückgesetzt wird, dass ein Pool-Budget unterhalb des lebenden Datenbestands ehrlich behandelt statt still verletzt wird und dass jede applizierte Stellgröße als benannte Spalte in der persistierten Ergebnis-Datei sichtbar ist — geprüft an der CSV-Ausgabe, nicht am In-Process-Zustand —, dazu der Erhalt der Zwei-Phasen-Operationsschleife (verworfenen Warmlauf, Messlauf) als Invariante.

Oberhalb der Schicht-Harnesse sichert ein *Zwei-Gate-Protokoll* jedes Implementierungs-Paket ab. *Gate 1* ist das scharfe Gate des Engine-Repositories: Die volle Test-Suite — im Stand dieser Arbeit 320 registrierte Tests — läuft ungekürzt *zweimal*, und abgenommen wird ausschließlich „0 failed“, ohne Ausnahmeliste.

Bestandteil dieses Doppellaufs ist die Golden-Verankerung des Permutationsraums: Die 2^{17} Binary-Identitäten des kanonischen Vermessungs-Katalogs (Abschnitt 3.4.5) werden bedarfsweise regeneriert und gegen eine eingefrorene CRC-64-Prüfsumme bestätigt (0x56F1B721C72DC10E, CRC-64/ECMA-182 über die 131072 Identitäten in kanonischer Reihenfolge) — der Anker ist nicht gerechnet, sondern literaler Werkzeug-Output, und seine Änderung ist nur im koordinierten Schnittstellen-Fenster zulässig —, während ein kleiner, materialisierbarer Katalog von 320 Identitäten als byte-genaue Wache der Serialisierungsform dient. *Gate 2* ist das Integrations-Gate des übergeordneten Repositories: Der CI-Zwilling des Pakets baut grün, wobei eine *namentlich eingefrorene* Ausnahmemenge von fünf bekannten roten Tests mit je eingefrorener Fehlerklasse gilt — ein sechster roter Test oder ein Klassenwechsel innerhalb der fünf stoppt das Paket, statt die Menge aufzuweichen.

Dieses Protokoll ist Methodik, kein Systembestandteil: Es beschreibt, *wie* Änderungen am Apparat abgenommen werden, nicht, was die Binaries tragen. Die Verzeichnis- und Suite-Struktur, auf der es läuft, dokumentiert Anhang B.

Die Fertigstellung folgt der Schichtung top-down — E4, E3, E2, dann der Mess-Querschnitt M unmittelbar vor E1 —, und die Infrastruktur E0 wird bewusst zuletzt konsolidiert (Abschnitt 3.3.4); die Grenze E3→E2 ist im heutigen Stand bereits golden-getestet, die übrigen Harnesse sind als Abnahme-Definition der laufenden Schicht-Fertigstellung festgelegt. Der Querschnitt M besitzt dabei einen eigenen Harness, der ohne jede Tier-Binary auskommt: Eine synthetische Mess-Quelle wird über die System-Achsen-Wurzel getrieben und ihre Werte bis in die Ergebnis-Spalten verfolgt; seine Code-Gestalt beschreibt der folgende Abschnitt.

4.5 Realm-Wurzeln im Code: System- und Mess-Achsen

Die Dreiteilung der Achsen-Realms (Abschnitt 3.4.1) ist im Code als Typ-Trennung ausgeführt. Alle Achsen teilen ein gemeinsames Familiendach — eine leere, nicht-polymorphe CRTP-Basis, deren Concept-Wache jede eingeschleppte vtable schon beim Übersetzen zurückweist — und deklarieren ihre Familie über einen Diskriminator mit fünf Werten: die Organ-Achsen, die Mess-System-Achsen des Blut-Querschnitts, die Bau- und Steuer-Achsen der CEB-Konfiguration, die System-Meta-Achsen unter dem Erweiterungs-Hub und die Mess-Realm-Achsen des Planers. Je Familie existiert eine eigene CRTP-Wurzel; eine Achse *ist* ihre Familie durch ihre Basisklasse, nicht durch ein umhängbares Etikett — die CEB-Konfigurations-Wurzel etwa liefert ihre Art als harten Rückgabewert, weshalb der Mess-Realm eine eigene Wurzel trägt statt einer wiederverwendeten. Frühere Fassungen dieses Kapitels beschrieben an dieser Stelle *eine* gemeinsame Wurzel aller Nicht-Organ-Achsen (Zwischenstand Juli 2026); der doppelte Wurzel-Schnitt — der System-Realm wurzelt am Builder, der Mess-Realm am Planer (Abschnitt 3.4.1) — ist seit der Achsen-Neuordnung auch im Code vollzogen.

Auf der System-Seite ist die kanonische Ordnung selbst Code: Ein konstantes Drei-Element-Array hält die System-Haupt-Achsen `target_isa`, `operating_system` und `external_utils` in ihrer verbindlichen Reihenfolge, und jeder Konsument, der eine Ordnung braucht — die Stempel-Zeile, die Generator-Blockfolge, der XML-Kopf, die Suffix-Segmente —, muss sie dort lesen, statt sie nachzubilden (Ordnungs-Single-Source). Drei *Abgangs-Wachen* halten die Umzüge der früheren Fünfer-Liste (Abschnitt 3.4.1) übersetzungszeitlich fest: `static_assert`-Prüfungen weisen Compiler, Scheduling und Last-Rahmenwerk als System-Haupt-Achsen zurück — wer einen der drei wieder einträgt, bricht den Bau

und muss den Umzug ausdrücklich widerrufen, statt ihn still zu revidieren. Auch die Bau-Versionskennung der System-Zeile (Abschnitt 3.4.4) speist sich aus einer einzigen deklarativen Quelle: einer geordneten Acht-Segment-Folge der Bau-Steuergrößen, in der ein leeres Glied kein Segment erzeugt — sie löste drei zuvor divergierende Verkettungs-Ketten ab.

Auf der Mess-Seite trägt der Planer seine eigene Meta-Meta-Wurzel; deren erste und derzeit einzige Haupt-Achse ist das Last-Rahmenwerk, das die System-Welt ersatzlos verlassen hat (Abschnitt 3.4.1), und wie alle Mess-Realm-Achsen ist sie strikt binary-id-neutral. Ihr Angebot halten vier Mess-Registries als `constexpr`-Kataloge: die Ablauf-Methodik eines Laufs (Debug-, Mess-, Release- und ein bereits wählbares Compare-Verfahren, dessen Bau-Zeile bewusst release-gleich ist und dessen Replay-Vergleichs-Vollzug noch aussteht), das Last-Rahmenwerk (derzeit genau ein Eintrag, YCSB in Version 1.0.0), das Rückschreibe-Verfahren (CSV, LaTeX-Tabelle, Vergleichs-Metriken) und das Mess-Tooling mit genau drei versionierten Einträgen — Wallclock, Makro, Mikro —, dessen Wache ein viertes, unabgestimmtes Tooling zum Übersetzungsfehler macht. Ein eigenes PMC-Tooling existiert nicht: Die Zähler-Instrumentierung steckt im Mikro-Tooling. Dieser Registry-Schnitt ist Implementierungs-*Stand*, nicht Soll: Das Ziel-Schnittbild ist das Achsen-Interface-Modell aus Abschnitt 3.4.5 — Mikro-Benchmarking vermisst eine Achse über ihr Achsen-Interface, und die drei Wallclock-Ebenen liefern die Gegenmessung —, und die Registry zieht diesem Modell nach.

Der host-seitige Mess-Querschnitt des System-Realms — das *Blut* des Apparats, das alle Schichten durchdringt, ohne je die Binary-Identität zu berühren (Abschnitt 3.4.1) — wurzelt in der CRTP-Basis `SystemAxis` (Datei `system_axis.hpp`). Sie ist als zero-cost abstraction gebaut — eine leere, nicht-polymorphe Basis ohne virtuelle Aufrufe im Hot-Path, deren Konformität ein eigenes Concept erzwingt — und übersetzt die Konzept-Invarianten in übersetzungszeitliche Vollständigkeits-Gates: Ein `constexpr` Gate prüft, dass jede der sechzehn Mess-Kategorien genau einem der beiden Regimes zugeordnet ist (neun zeit- und observer-basierte, sieben zählerbasierte PMC-Kategorien, Abschnitt 3.4.1), und ein zweites weist jede dieser Achsen zurück, die Kategorien beider Regimes mischt — das Regime-Misch-Verbot bricht so die Übersetzung statt erst die Auswertung.

Die sechzehn Mess-Kategorien führt eine einzige `constexpr`-Registry (`measurement_axis_registry.hpp`): je Kategorie der Name und das aus der einen Quelle abgeleitete Regime. Ein statisches Gate erzwingt, dass der Tabellen-Index mit dem Kategorie-Wert übereinstimmt, kein Name leer ist und kein Regime von der Wurzel-Zuordnung abweicht — eine siebzehnte Kategorie oder eine stille Index-Drift wird damit zum Übersetzungsfehler. Konzeptionell sind die Kategorien eine *Unter*-Achse des Mess-Toolings und manifestieren sich als Spalten der Ergebnis-Tabellen (Abschnitt 3.4.1); die Registry-Namen sind zugleich das Berichts-Vokabular der Auswertungs-Schicht (die Spalten-Namen der E4-Berichte), und eine compile-time-Iteration über alle Einträge stellt die Registry der Metaprogrammierung zur Verfügung.

Die Erhebung abstrahiert eine vendor-neutrale Mess-Quellen-Schnittstelle (`i_measurement_source.hpp`). `IMeasurementSource` beschreibt eine Quelle über ihre *strukturellen* Fähigkeiten — welche der zehn Ereignis-Semantiken (Zyklen, Instruktionen, L1-/L2-/L3-Cache-, dTLB- und Branch-Misses, Speicher-Stalls, Kohärenz-Invalidierungen, Energie) sie überhaupt ausdrücken kann — und liefert je Messintervall ein `MeasuredDelta` mit *per-Ereignis-Gültigkeit*: Der Wert Null mit gesetztem Gültigkeits-Flag ist eine echte Null-Messung; ein nicht gesetztes Flag heißt *nicht gemessen*. Die Fähigkeiten-Auskunft verspricht dabei nie eine Mess-Garantie — die Wahrheit je Messung tragen allein die Gültigkeits-Flags. Als garantierter, ausdrücklich *degradiert*er Fallback existiert die Wall-Clock-Quelle

(WallClockSource): Sie liefert nur einen Nanosekunden-Proxy für die Zyklen und lässt alle Hardware-Ereignisse ehrlich ungültig; die Bestands-PMC-Familie bindet ein Adapter an, der nur die sieben strukturell ausdrückbaren Kanäle bewirbt.

Die honest-0-Doktrin (Abschnitt 3.4.1) ist in diesen Bausteinen einprogrammiert statt dokumentiert. Die Wall-Clock-System-Achse verweigert es, ihren Mittelwert als Perzentil zu etikettieren: Anfragen nach p50 bis p999 beantwortet sie ungültig, denn die durch die Operationszahl geteilte Gesamtzeit ist ein Mittelwert, und ein als Perzentil ausgegebener Mittelwert wäre ein Phantomwert — echte Perzentile entstehen erst nachgelagert in der Auswertung: nearest-rank über den erhobenen Rohlatenzen, ergänzt um das HDR-Histogramm, das bei drei signifikanten Stellen mit einer relativen Bucket-Toleranz von $1/1024 \approx 0,0977\%$ auflöst; im Messpfad selbst steht bewusst kein Histogramm-Bucket, weil er den Rohwert schon bei der Aufnahme unumkehrbar quantisierte. Die Cache-Line-Auslastung wird als echte Auslastungs-Ratio erhoben — genutzte Nutzdaten-Bytes gegen belegte Cache-Lines mal Zeilenbreite —, nicht als roher Zeilen-Zähler, der gerade die *inverse* Metrik wäre. Speicher-Fußabdruck und Fill-Buffer-Auslastung bleiben ungültig, bis eine echte Peak-Spalte beziehungsweise eine Hardware-Quelle existiert (der Software-Operations-Puffer der Pufferungs-Achse ist kein Proxy für den Hardware-Line-Fill-Buffer), und IPC/CPI bleibt ungültig, solange das Zähler-POD keine Instruktions- und Zyklen-Spalten trägt.

Die Hardware-Erkennung — konzeptionell die dritte zweigeteilte Haupt-Achse und eine *Mess*-Achse (Abschnitt 3.4.3) — liegt im Stand dieser Arbeit in beiden Ausbaustufen als Code vor. Die erste Stufe umfasst das Provenienz-Vokabular (die drei Vertrauensstufen als eigener Ergebnis-Typ; „nicht erhoben“ ist bewusst keine vierte Stufe, sondern ein eigener Zustand), einen DDR5-SPD-Parser als reine, `constexpr`-fähige Byte-Funktionen über einem übergebenen Speicherbereich — unter zweihundert Zeilen, ohne Ein-/Ausgabe und ohne Fremdcode-Vendoring; Übertaktungs-Profile erkennt er als separates Angebot, nie als Ist-Wert — sowie eine eigene Fehler-Domäne mit vier Klassen (Quelle fehlt, unlesbar, korrupt, unbekannt formatiert), damit eine korrupte Quelle nie still zur nächsten Stufe degradiert. Die zweite Stufe verdrahtet die Erhebung: Die Laufzeit-Kette Boot-Cache → SPD → Deklaration ist eine Chain of Responsibility ohne virtuelle Aufrufe — ihre Glieder sind heterogene Typen, ihre Verdrahtung ist eine Typliste, sodass die Reihenfolge (das Wesentliche einer solchen Kette) im Typ steht statt in einer Laufzeit-Liste; die Kette bricht bei der höchsten erreichbaren Vertrauensstufe ab, bezieht ihre Quell-Pfade per Injektion (jede Stufe ist ohne die Ziellplattform testbar) und läuft ausschließlich, einmalig gepuffert, an der Freigabe-Naht des Builders. Welche Kette gilt, entscheidet eine Übersetzungszeit-Factory über der Zelle aus ISA-Komplex und Betriebssystem: Ihr primäres Template ist deklariert, aber nicht definiert — eine Zelle ohne bewusste Entscheidung ist ein unvollständiger Typ und bricht den Bau —, eine Totalitäts-Wache prüft die volle Matrix aller deklarierten ISA-Komplexe und dokumentierten Betriebssysteme, und nicht implementierte Zellen (Windows, macOS) sind ehrliche Nur-Deklarations-Zellen mit ausdrücklich besetzten oberen Stufen, damit der Degradations-Pfad „gefragt, gibt es hier nicht“ meldet statt „übersprungen“.

Den Abgleich zwischen Deklaration und Erhebung fällt ein benanntes Laufzeit-Verdikt mit genau sieben Ausgängen — Übereinstimmung, Abweichung, nur deklariert, nicht erhoben, keine Deklaration, unvergleichbare Stufen und Etikett-Widerspruch —, wobei nur provenienz-kompatible Größen überhaupt verglichen werden; kein Verdikt und kein Messwert erreicht je die Stempel- oder Identitäts-APIs (Abschnitt 3.4.4). Damit ist die in Abschnitt 3.4.3 geforderte automatisierte Wiedervorlage code-seitig eingelöst: Die bisher hinterlegte RAM-Nennrate von 4800 MT/s — ein einmalig erhobenes, dann eingefro-

renes `dmidecode`-Ergebnis — wird durch eine Erhebung ersetzt, die bei jedem Lauf erneut stattfindet und ihre Vertrauensstufe mitführt.

Am Auswertungs-Ende des Querschnitts liegt der Keim der Kurvenanpassung aus Abschnitt 3.6.3 als Skeleton vor (`curve_fit.hpp`): Eine Mess-Achsen-Kurve über die Working-Set-Größe wird per kleinsten Quadrate an das Modell $y = a \log_2 x + b$ angepasst — die für Cache-Hierarchie-Kurven erwartete Form. Auch hier gilt ehrliche Leere statt Phantom-Fit: Der Fit-Status unterscheidet *keine Daten*, *unterbestimmt* (weniger als zwei Punkte oder keine Streuung der x-Werte) und *ungültige Daten* (undefinierter Logarithmus, nicht-endliche Werte) ausdrücklich vom Erfolgsfall — nie ein *Ok* mit NaN-Koeffizienten —, und der Ergebnis-Leser übernimmt nur streng numerische Zellen und zählt übersprungene Zeilen als Diagnose mit, statt sie still zu Phantom-Punkten zu machen.

Vollzogen sind damit die Realm-Wurzeln samt Ordnung und Wachen, die Registries, die Quellen-Abstraktion mit degradiertem Fallback, die zweistufige Hardware-Erkennung und der Fit-Keim; die Verdrahtung dieser Querschnitts-Bausteine in Runner und Prüf-Dock sowie die Abbildung der Registry-Namen auf die Bestands-Spalten der Ergebnis-Dateien sind die nächsten Schicht-Inkremente (Abschnitt 4.4). Die drei Konzept-Invarianten — System-Achsen sind nicht permutierbar, stehen nicht im Permutationsbaum und berühren nie die Binary-Identität — wahrt dabei die Bauform selbst: Die Wurzel dockt nur lesend an die host-seitigen Ergebnis-Strukturen an und bleibt der ABI-Grenze gegenüber unsichtbar.

Damit ist der Apparat vollständig beschrieben — von den Repositorien über den Prüfling und die achtzehn Organ-Achsen bis zur Kette vom Plan zur Tier-Binary, ihrer schichtweisen Qualitätssicherung und den Realm-Wurzeln im Code. Kapitel 5 legt die Methodik offen, mit der dieser Apparat bewertet wird.

5 Evaluation

Die Evaluation legt zuerst die Methodik offen und wertet darauf aufbauend die Messreihen aus.

5.1 Methodik

Drei *Hypothesen* leiten die Auswertung:

H1 (Page-Type-Kosten) Die mittleren Zugriffskosten je Page-Type-Variante unterscheiden sich systematisch nach Workload: dichte Page-Typen gewinnen erwartungsgemäß unter Zipf-Verteilung (YCSB-A), bereichsoptimierte Pages unter Bereichs-Scans (YCSB-E).

H2 (Code-Qualität pro Quelle) Der vom Original-Quell-Repository geerbte Code-Qualitäts-Score — je Quell-Repository als dokumentiertes Qualitäts-Audit über sieben Bewertungs-Achsen (Stil, Tests, Dokumentation, Wartbarkeit, Pflege-Status, Lizenz, Build-System; Skala 1–5, docs/quality_audit) erhoben und ergänzt um die dokumentierte is_original-Markierung je Funktion (Anhang D) — korreliert messbar mit dem erreichten Durchsatz.

H3 (Inline- vs. External- vs. Chain-Verteilung) Die beobachtete Verteilung der drei ValueHandle-Varianten korreliert mit der Page-Dichte: dichte Pages bevorzugen Inline, spärliche Pages External und PRT-ART-Heuristiken ChainRef.

Diese *Evaluations-Hypothesen* sind nicht zu verwechseln mit den gleichnamigen PRT-ART-Telemetrie-Metriken H1–H3 (Abschnitt 3.5): Erstere sind prüfbare Aussagen über den Entwurfsraum, Letztere Mess-Organen des Prüflings, die unter anderem die Daten für diese Prüfung liefern.

Phase 5 der Mess-Pipeline (Execute, Abschnitt 4.3.2) leitet den YCSB-Workload je Permutation als Fallback-Heuristik aus dem Traversal-Tag des Profils ab (Tabelle 5.1), sofern kein `<expected_workload>`-Tag des Profils die Heuristik überschreibt (Abschnitt 3.2); Module ohne Profil verwenden den globalen Default-Workload. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik (Tabelle 5.2), jeder mit einer Reproduzierbarkeits-Akte aus Quelle, Prüfsumme, Zeilenzahl, Vorverarbeitung und Seed-Regel für die synthetischen Miss-, Update- und Delete-Schlüssel.

Tabelle 5.1: Profil-bewusstes Workload-Routing (Fallback-Heuristik)

Traversal-Tag	YCSB-Workload	Charakteristik
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	Bereichsabfragen
HOT_PATH / PRTART_HOT_PATH	YCSB-A	50/50 Lesen/Aktualisieren, Zipf
STANDARD (Default)	YCSB-C	nur Lesen

Die Messungen zielen auf eine lokale Workstation zur Entwicklungs-Verifikation und den TU-Dresden-ZIH-Cluster (Barnard-CPU-Knoten mit Intel Sapphire Rapids, Capella GPU) für die vollständigen Sweeps,

Tabelle 5.2: Reale Datensätze und ihre Charakteristik

Datensatz	Charakteristik
url	sehr lange Schlüssel, lange gemeinsame Präfixe, mittleres Alphabet
dna	kleines Alphabet, hohe Wiederholung, tiefe Präfix-Entropie
protein	biologische Sequenzen, kleines bis mittleres Alphabet
xml	strukturierte Strings mit wiederkehrenden Präfixen
tpcds-id	datenbanknahe IDs, sortiert und unsortiert
trec-terms	Wörterbuch-/Suchindex-Terme mit definierten Miss-Schlüsseln

plus die eigenen Produktions-Zielmaschinen (Ryzen 9 9950X3D, Intel Core i9-12900K) sowie eine HBM-Plattform. Ein `IPlatformProbe` (Phase 1) meldet je Plattform die lauffähige Teilmenge der Achsen (etwa AVX-512 nur dort, wo verfügbar, HBM-aware nur auf HBM-Hardware), wobei der Ausschluss einer nicht verfügbaren Erweiterung stets *maschinenseitig* fällt — mit Warnung und benannter Fehlerklasse, nie als stiller Profil-Filter, denn das Experiment-Profil bleibt für alle Maschinen gleich; je Plattform werden ein Scalar-, ein AVX2- und — auf Barnard — ein AVX-512-Build vorbereitet, und auf Hybrid-CPUs werden P- und E-Cores getrennt vermessen (über die eigenen Zähler-Domänen `cpu_core` und `cpu_atom`) statt zusammenaggregiert. Der produktive Mess-Betrieb selbst ist ein *Zwei-Maschinen-Betrieb* — eine AMD-Maschine (der genannte Ryzen 9 9950X3D) neben einer Intel-Maschine (Core i9-12900K, Alder Lake) —, und die zählerbasierte Erhebung führt dafür zwei getrennte Hersteller-Lanes (AMD und Intel), die hart gegatet sind: Erst wenn beide Lanes ihre Zähler-Voraussetzungen nachgewiesen haben, arbeitet eine Maschinen-Lane ihren Mess-Strang ab — eine stille Teil-Messung nur eines Herstellers gibt es nicht. Auf den Produktions-Zielmaschinen soll zudem jede Messung unter *zwei Betriebssystem-Regimes* erhoben werden, um Produktions-Treue von Mess-Tiefe zu trennen: Ein *immutable* Betriebssystem (Talos) spiegelt den Produktiv-Einsatz und erzwingt boot-/übersetzungsstatische Cache-Einstellungen (Abschnitt 2.2); ein *root-Linux mit vollem Hardware-Zähler-Zugriff* (`perf/MSR`) eröffnet hingegen überhaupt erst den Zugang zu den zählerbasierten Mess-Kategorien (Abschnitt 3.2) und zu den laufzeit-konfigurierbaren cache-nahen Achsen-Parametern (Prefetch-Distanz, Hardware-Prefetcher). Zum Stand dieser Fassung ist das zweite Regime *entworfen, aber nicht vollzogen*: Die berichteten Messungen stammen sämtlich vom root-Linux-Regime der beiden Produktionsmaschinen; der Talos-Spiegel bleibt als Anforderung bestehen und wird nach der Abgabe erhoben.

Davon zu trennen ist, welche Zähler dieser Zugang in der vorliegenden Fassung tatsächlich füllt: Die Mess-Kette öffnet über `perf_event_open` vier generische Zähler — L1D-Misses, Last-Level-Misses, dTLB-Misses sowie, seit M-3a, Branch-Misses (`PERF_COUNT_HW_BRANCH_MISSES`, generisch auf Intel wie AMD). Für L2-Misses und Kohärenz-Invalidierungen existiert kein portabler generischer Zähler; ein hersteller-spezifischer RAW-Zähler wird bewusst *nicht* geraten. IPC/CPI ist dagegen nicht verdrahtet; die Energie-Aufnahme ist über einen best-effort-Schnappschuss der RAPL-Sysfs-Zone verdrahtet, bleibt aber ohne Root- bzw. Zonen-Leserecht — seit Linux 5.10 der Regelfall — leer. Die Spalten der nicht verdrahteten Zähler tragen deshalb keine *gemessene* Null: Sie sind als *nicht implementiert* zu lesen und werden mit der nächsten Ausbaustufe nachgezogen. Diese Trennung ist Teil der Ehrlichkeits-Doktrin dieser Arbeit — eine Null nur dort, wo tatsächlich null gemessen wurde.

Die zeit- und observer-basierten Kategorien laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit; die PMC-Kategorien werden nur unter dem privilegierten Regime erhoben und je

Plattform mit Betriebssystem, Kernel und Zähler-Domäne protokolliert.

Der Umfang des vollständigen Baus ist als *Soll* festgelegt: Je System-Permutation — je im Profil deklarerter Kombination der Bau-Steuergrößen aus Optimierungsstufe und SIMD-Erweiterung (Abschnitt 3.4.1) — werden die $2^{17} = 131072$ Organ-Permutationen des kanonischen Vermessungs-Katalogs gebaut (Abschnitt 3.4.5). Wie viele solcher Bau-Varianten anstehen, entscheidet allein das Profil; im Stand dieser Arbeit deklariert es vier (zwei Optimierungsstufen mal zwei SIMD-Stufen), wobei die aggressivste Optimierungsstufe bewusst fehlt, weil sie den IEEE-754-Gleitkomma-Determinismus des Apparats bräche. Ergebnisse dieser Voll-Bau-Läufe stehen aus (Abschnitt 5.2).

Faire Vergleichbarkeit ist über feste Regeln gesichert: Jeder Vergleich trennt einen gemeinsamen Minimalmodus (Common-Denominator: externe Werte-Handles, keine PRT-ART-Spezialpfade) vom PRT-ART-Native-Modus (Inline-Umschaltung und adaptive Seitentyp-Wahl je Verzweigungspunkt); alle Verfahren erhalten identische, normalisierte Byte-Schlüssel sowie identische Miss-, Update- und Delete-Mengen; Warmup, Randomisierung und Thread-Pinning werden je Plattform und Workload protokolliert. Fehlende Fähigkeiten einer Baseline — etwa Bereichs-Scans einer Hash-Tabelle — werden als N/A ausgewiesen statt verdeckt emuliert; für jede Fremdbibliothek werden Compiler, Flags, ISA-Pfad, Allokator und Commit-Hash mitgeloggt, und jede Konfiguration wird in mehreren getrennten Läufen vermessen; die Latenz-Perzentile bestimmt die nachgelagerte Auswertung nearest-rank aus den Rohwerten und ergänzt sie um die HDR-Verteilungssicht (Abschnitt 4.5).

Auch der Mess-*Betrieb* ist Teil dieser Fairness-Regeln. Die Messung bleibt strikt vom Bau getrennt: Mess-Läufe starten nie automatisch mit einem grünen Bau, sondern werden ausdrücklich ausgelöst — ein grüner Bau beweist Bau-Fähigkeit, kein Mess-Ergebnis. Während der Debug-Modus zur Verifikation parallel über alle Threads einer Maschine fahren darf, misst der Mess-Modus mit genau *einem* Mess-Thread; das Threading der Gattung selbst bleibt dabei der Tier-Binary überlassen, statt vom Harness überschrieben zu werden. Ein im Lager vorgefundenes Binary wird nur dann übersprungen, wenn sein grünes Prüf-Testat am Binary inventarisiert ist, und vor jeder Messung steht die Konformitätsprüfung am Prüf-Dock (Abschnitt 4.4); Bau- wie Mess-Läufe werden je Maschine in Meilenstein-Batches von je 4096 Binaries am Stück abgearbeitet, mit gleichverteilter Zuteilung der gemeinsam baubaren Binaries über die Maschinen und inventarisierter Wiederaufnahme nach Unterbrechung.

Über die drei Hypothesen hinaus erhält die Auswertung eine vierte, eigenständige *Evaluations-Dimension*: den heuristischen Gegenbeweis. Sein Konzept ist in Kapitel 3 entwickelt — der Hybrid-Modus kompiliert die heuristisch geschätzte Rekombination der Tier-Binaries selbst zu einer Tier-Binary und dockt sie am Prüf-Dock an (Abschnitt 3.6.2); das Messkurven-Typsystem liefert die Schätzgrundlage, und seine Messfehler-Erkennung formuliert Abschnitt 3.6.3. Die Bewertungsfrage dieser Dimension lautet: „Ist die heuristisch geschätzte Rekombination der Tier-Binaries schneller als die existierenden Paper-Algorithmen?“ Anders als H1–H3, die Aussagen über den Entwurfsraum prüfen, prüft der Gegenbeweis den Apparat der Arbeit selbst: ob die aus den Messkurven geschätzte Rekombination im realen Wall-Clock-Verhalten hält, was die Kurven versprechen.

Der Prüfplan hierfür ist zweistufig festgelegt; da die zugehörigen Messläufe noch ausstehen, ist er — wie das Zielbild in Abschnitt 3.6.3 — im Futur formuliert. Die erste Stufe wird den Messfehler quantifizieren, den die Mess-Instanzen selbst eintragen, denn die Schätzgrundlage der Heuristik sind *beobachtete* Kurven, ihr Anspruch aber gilt dem unbeobachteten Produktions-Verhalten. Dazu wird dasselbe Experiment in drei Bau-Varianten aufgelegt:

1. Beobachter in *allen* Tier-Binaries *und* im Heuristik-Hybrid-Tier,
2. Beobachter *nur* im Heuristik-Tier-Binary,
3. Beobachter in *keinem* Kompilat — gemessen wird nur die Wall-Clock.

Die Differenzen zwischen den drei Varianten beziffern den Beobachter-Overhead getrennt nach seinem Ursprung — Mess-Organen der untergebenen Tiere (Differenz von Variante 1 zu 2) gegen Mess-Kommandos des Hybrid-Tiers (Differenz von Variante 2 zu 3) — und ergeben je Workload und Plattform ein *Unsicherheitsband*, unterhalb dessen zwei Konfigurationen als nicht unterscheidbar gelten.

Die zweite Stufe wird der komplementäre *vierte Schritt* der finalen heuristischen Zusammenstellung sein: Die dem Heuristik-Tier untergebenen Tier-Binaries — nicht die final optimierte Haupt-Binary — werden ohne Mikro- und Makro-Benchmarks und ohne Beobachter neu gebaut, sodass kein Mess-Artefakt im Kompilat verbleibt. Verglichen wird dann rein an der Wall-Clock, in zwei Vergleichsmengen mit festgelegtem Vorrang: primär gegen die rekonstruierten Paper-Algorithmen — der Paper-Vergleich geht vor, denn er beantwortet die Bewertungsfrage wörtlich —, sekundär gegen alle bekannten Tier-Binary-Permutationen, insbesondere das beste Einzel-Binary; dieser zweite Vergleich ordnet ein, ob ein etwaiger Vorsprung der Heuristik als Ganzer oder schlicht einer einzeln überlegenen Permutation zuzuschreiben ist. Daraus folgt die Entscheidungsregel: Der Gegenbeweis gilt je Workload-Klasse als *erbracht*, wenn die beobachterfrei neu gebaute Rekombination die schnellste Paper-Rekonstruktion in der Wall-Clock schlägt und der Abstand das Unsicherheitsband der Bau-Varianten übersteigt; er gilt als *verfehlt*, wenn ein Paper-Algorithmus oder das beste Einzel-Binary vorn liegt — auch dieses Ergebnis wäre ein Resultat, denn es bezifferte erstmals, wie viel Schätzfehler zwischen Kurven-Prognose und realem Verhalten liegt. Die Messdaten dieser Dimension stehen aus; sie werden nach den ersten vollständigen Messläufen zusammen mit den Reihen A–C in Abschnitt 5.2 eingespeist.

5.2 Ergebnisse und Analyse

Zum Zeitpunkt der Erstellung sind Methodik und Auswertungs-Pipeline vorhanden und auf Beispieldaten verifiziert; eine erste reale Smoke-Messreihe (43 Permutationen, echte DLL-Messung auf der Entwicklungsmaschine) dokumentiert Anhang A samt ihren Limitierungen (Abschnitt A.10), die vollständigen Läufe der Pflicht-Messreihen auf den Zielplattformen stehen aus (Abschnitt 6.2). Pro Reihen-Lauf schreibt der `messung_driver` unter `Code/_runs/<date>/<spec_id>/` die Dateien `measurements.csv`, `measurements.json` und optionale binäre Records. `measurements.csv` trägt dabei eine Zeile je Permutation mit den Spalten des kanonischen Mess-Records: `op_count`, `total_cycles`, L1-/L2-/L3-Cache-Misses, dTLB-Misses, Kohärenz-Invalidierungen, Energie sowie die Speicher-Felder (`bytes_allocated`, `bytes_in_use_peak`, Fragmentierung); Branch-Misses sind seit M-3a real erhoben und liegen als additive Schema-Erweiterung des Records vor (`pmc_branch_misses`); der Durchsatz wird als Schema-Erweiterung noch ergänzt und liegt bislang im Kommando-Pfad vor. Die Auswertung reichert diese Dateien über `binary_to_csv` um Profil-ID, Paper-Referenz und Achsen-Belegung an, rendert `csv_to_latex`-Steckbriefe je Permutations-ID (in der Reihe A_{defined} entspricht das je einem SOTA-Profil) und `diagram_generator`-Diagramme (A4-bewusste TikZ-Balkendiagramme, Scatter-Plots, Heatmaps), die das Manuskript über `\input{tikz/<spec_id>/...}` und

`\input{tabellen/<spec_id>...}` aus den sprachneutralen Verzeichnissen einbindet. Die CSV-Codierung ist dabei der implementierte Stand; als künftiger Default der Auswertung ist die xlsx-Codierung beschlossen: Ein eigener Writer wird — als Factory über den Ausgabe-Encodierungen — je Lauf *eine* Datei mit einem Blatt je gewählter Unter-Achsen-Permutation und einem Info-Blatt (System-Informationen der messenden Maschine, verwendete Haupt-Achsen) schreiben; die CSV-Ausgabe bleibt als einstellbarer Fallback derselben Factory erhalten (je Blatt dann eine eigene Datei), und Spalten, deren Wert sich innerhalb eines Laufs nie ändert, wandern in die Metadaten, statt in jeder Zeile wiederholt zu werden. Die drei Messreihen werden dann ausgewertet: Reihe A (PRT-ART vs. Stand der Technik, mit den Evaluations-Hypothesen H1–H3 aus Abschnitt 5.1), Reihe B (systematische Variation der Cache-Engine-Permutationen, gespeist aus dem Full Join der Stufe 3) und Reihe C (Merge/Regression alt gegen neu, build-übergreifend; Tabelle 3.8). Quer zu dieser Zuordnung liegt das in der Anwender-XML deklarierte *Mess-Schema des Prüflings* mit seinen drei Modi: Im *Ersetzungs*-Modus ersetzen die Prüflings-Achsen die Standard-Belegung der Cache-Engine vollständig (der Default der Stufe 2), im *Hybrid*-Modus wird der gemeinsame Cache-Engine-Prüfling-Hybrid je Prüfling vermessen, und der *Full-Join*-Modus der Stufe 3 kombiniert alle Achsen-Varianten, ohne etwas zu verwerfen. Reihe A arbeitet in den ersten beiden Modi, Reihe B im dritten; die Hybrid- und Full-Join-Stände bleiben an der Binary selbst unterscheidbar, denn die Misch-Strategie schlägt sich im Stempel als *e*-Suffix der Versions-Bezifferung und in hierarchisch erweiterten Algorithmus-Namen der Organ-Zeile nieder (Abschnitt 3.4.4); eine eigene Stempel-Zeile trägt sie nicht, denn der Prüfling mischt gegen die Organ-Achsen und steht damit bereits in deren Zeile. Bei reinen Ersetzungs- wie reinen Cache-Engine-Bauten bleiben Suffix und Namens-Erweiterung folgerichtig aus.

Den Kern der Auswertung bildet die *Achsen-Sensitivitätsanalyse*, die unmittelbar das Trennbarkeits-Problem der Aufgabenstellung beantwortet: Im Stand der Technik sind die Gewichtungen der Verbesserungen *einzelner* Algorithmus-Bestandteile bislang nicht trennbar, weil jede Veröffentlichung ihren Nutzen in einer eigenen, inkompatiblen Messumgebung belegt. Indem das Achsen-Framework jeden Bestandteil als austauschbare Achse über einer sonst konstanten Struktur isoliert, wird der Beitrag jeder Entwurfsentscheidung erstmals unter sonst gleichen Bedingungen separierbar und messbar zurechenbar. Anhand der Records je Permutation quantifiziert die Analyse auf der Mikro-Granularität — der Vermessung jeder Achse über ihr *Achsen-Interface*, die Funktionsmenge, die alle Algorithmen dieser Achse liefern müssen (Abschnitt 3.4.5) —, welche Achse am meisten zur Performance-Varianz beiträgt; Makro- und Gesamt-Benchmarking ergänzen die Ebene der Gattungs-Interface-Operationen und die YCSB-Lastprofil-Ebene, die Wallclock-Gegenmessung der Achsen-Interfaces prüft die interne Mikro-Messung von außen, und aus der Rangbildung über die drei Granularitäten folgt die Empfehlung von Standard-Konfigurationen je Workload-Klasse. Die Ergebnisse werden schließlich vor dem Leitmotiv der Arbeit diskutiert — dem Übergang vom heuristischen zum informierten Cache-Management, also der Frage, ob telemetrie-getriebene, automatisch adaptierte Konfigurationen statisch gewählte übertreffen. Kapitel 6 zieht daraus das Fazit.

6 Fazit und Ausblick

Dieses Kapitel zieht Bilanz: Es beantwortet die Forschungsfragen FF0–FF4 aus Abschnitt 1.3 samt ihren Teilfragen, benennt die Grenzen des erreichten Standes und skizziert die nächsten Schritte.

6.1 Beantwortung der Forschungsfragen

Quer zu allen fünf Fragen hatte die Einleitung eine Kategorien-Frage bewusst offen gehalten: ob sich die Achsen des Entwurfsraums sauber in Organ-, System- und Mess-Achsen trennen lassen — und wie eine solche Trennung zu bewerkstelligen ist, wo sie noch nicht vorhanden ist. Beides ist beantwortet (Abschnitt 3.4.1): Die Achsen trennen sich entlang der drei Belange in drei *Realms* mit je eigener Klassen-Wurzel, je eigenem Angebots-Katalog und je eigener Stempel-Zeile an der Binary (Abschnitt 3.4.4). Bewerkstelligt wurde die Trennung nicht durch Umbenennung, sondern durch datierbare *Umzüge*: Telemetrie und ISA verließen die Organ-Zählung in die System-Sphäre, Compiler und Scheduling wanderten aus der früheren Fünfer-Liste der System-Achsen in die Komplex-Klammer beziehungsweise an den ISA-Wrapper, und das Last-Rahmenwerk verließ die System-Welt ersatzlos in Richtung Mess-Realm; im Code halten Übersetzungszeit-Abgangs-Wachen diese Umzüge fest (Abschnitt 4.5). Diese Dreiteilung ist die zweite Gestalt des Trennbarkeits-Problems und trägt die Antworten auf die folgenden Teilfragen.

FF0 (Hauptfrage). Wie stark aktive, messgetriebene cache-bewusste Entscheidungen ihre passiven, statisch ausgelegten Gegenstücke übertreffen, lässt sich abschließend erst nach den End-to-End-Läufen je Plattform beziffern (Abschnitt 6.3). Vorbereitet ist die Apparatur, die genau diese Differenz isoliert: Dieselbe achsen-konstante Struktur wird einmal aktiv (messgetriebene Seitentyp-, Layout-, Prefetch- und Value-Ablage-Entscheidung) und einmal passiv (statisch ausgelegt) permutiert und unter identischer Compiler- und Flag-Basis auf Hybrid-CPU's und Sapphire-Rapids gegenübergestellt. Die Erweiterungs-Teilfrage — ob sich ein gemessener Gewinn überhaupt eindeutig zuordnen lässt — ist strukturell beantwortet, und die dafür nötige Trennung ist genau die Dreiteilung: Ein Organ-Effekt permutiert die `binary_id`, ein Plattform-Belang steht als System-Stempel und Bau-Versionskennung an der Binary, ohne sie zu permutieren, und ein Mess-Belang bleibt strikt binary-id-neutral — kein Mess-Instrument darf verändern, *was* gebaut wird.

FF1 (Komposition). Die kanonische Zerlegung trägt: *Achtzehn* Organ-Haupt-Achsen in fester Slot-Reihenfolge T0–T17 spannen die Permutationsmatrix auf (Abschnitt 3.4.1); die frühere Neunzehn-Achsen-Zählung (Zwischenstand Juli 2026) ist eine historische Vorstufe dieser Ordnung. Der kanonische Vermessungs-Katalog — je Organ-Achse zwei freigeschaltete Vertreter, die Persistenz-Achse auf ihren In-Memory-Vertreter gepinnt — zählt $2^{17} = 131072$ Binary-Identitäten je System-Permutation, jede mit vollem Achtzehn-Segment-Pfad; der rohe Varianten-Produktraum über alle Achsen-Kataloge samt ihrer Unter-Achsen liegt Größenordnungen darüber (Abschnitt 3.4.5). Stand-der-Technik-Entwürfe werden über

Algorithmus-Profil-XMLs eindeutig rekonstruiert: 30 SOTA-Profilen (8 Rang-1 + 22 Rang-2/3) zeigen, dass die Achsen-Zerlegung P01–P33 trägt, und die variadische API-Spezialisierung ($1/2/N$ Parameter auf `std::vector/std::map/Tuple`) macht algorithmus- wie container-orientierte Muster auf einer Engine ausdrückbar. Die Teilfrage nach dem Kategorien-Schnitt ist bejaht: Die Zerlegung bleibt nur orthogonal, weil Plattform- und Mess-Belange aus der Permutationsmatrix herausgehalten sind — der Schnitt wurde durch die datierten Umzüge bewerkstelligt und wird durch die Abgangs-Wachen im Code gehalten (Abschnitt 4.5). Die nicht voll-orthogonalen Organ-Kopplungen ($\text{Knotentyp} \times \text{SIMD} \times \text{Pfadkompression}$) werden als solche markiert und bei der Permutation explizit eingeschränkt. Die Teilfrage nach dem Cache-Update- und Cache-Line-Füllverhalten unter allein variiertter ISA-/SIMD-Achse ist methodisch beantwortet — gerade *weil* die SIMD-Erweiterung keine Organ-Achse ist: Als Bau-Steuergröße multipliziert sie nur die Bau-Matrix, sodass ein und dieselbe `binary_id` unter mehreren System-Permutationen gebaut und bei sonst unveränderter Struktur verglichen werden kann (Abschnitt 3.4.1); die empirische Antwort folgt mit den Voll-Bau-Läufen (Abschnitt 6.3).

FF2 (Mess-Methodik). Die Messung ist in einen Produktiv-Modus (`COMDARE_MEASUREMENT_ON` abgeschaltet, kein Overhead) und einen Experiment-Modus geteilt: Der `ResultAggregator` wird host-seitig im `ExperimentDriver` geführt (Phasen 5–7), und der Experiment-Modus aktiviert die Mess-Hooks und die Observer-Kopplung im ABI-Adapter. Profil-bewusstes Workload-Routing wählt den Workload je Permutation aus dem Traversal-Tag und sichert so faire Vergleiche zwischen bereichsabfrage- und punktzugriffsorientierten Entwürfen; dieses Routing betrifft nur die Default-Zuweisung der Reihe A, während die Antwort auf den Heimspiel-Bias die bias-freie Vollmatrix bleibt, die im Gesamt-Benchmarking alle Lastprofile als dynamische Achse gegen alle Lebewesen-Binaries fährt (Kapitel 3). Jede Reihe wird in drei Granularitäten erhoben: Das Mikro-Benchmarking vermisst jede Achse über ihr *Achsen-Interface*, Makro- und Gesamt-Benchmarking ergänzen die Gattungs-Interface-Operationen und die Lastprofil-Ebene, und drei Wallclock-Ebenen prüfen die interne Mikro-Messung von außen (Abschnitt 3.4.5). Die Teilfrage nach den Mess-Achsen ist bejaht: Die Mess-Instrumente sind ein eigener Realm im Planer — das Last-Rahmenwerk als erste Haupt-Achse, das Mess-Tooling mit den sechzehn Kategorien als Unter-Achse —, stets präsent, sobald gemessen wird, und strikt `binary-id-neutral`; bewerkstelligt wurde diese Trennung durch den doppelten Wurzel-Schnitt — der System-Realm wurzelt am Builder, der Mess-Realm am Planer (Abschnitt 3.4.1). Widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine gegenüber bis zu sechzehn Cache-Lines) werden so nicht aus inkompatiblen Original-Setups übernommen, sondern achsen-isoliert und unter sonst gleichen Bedingungen nachgemessen. Die Erweiterbarkeits-Teilfrage ist im Konzept beantwortet und im Ausbau ehrlich unfertig: Ein neuer Prüfling gliedert sich als Lebewesen seiner Gattung ein — er dockt am Prüf-Dock an, durchläuft die Drei-Stufen-Prüfung, und sein Mess-Schema (Ersetzungs-, Hybrid- oder Full-Join-Modus) bleibt über das `e`-Suffix der Versions-Beifferung und die hierarchisch erweiterten Algorithmus-Namen der Organ-Zeile an der Binary unterscheidbar — eine eigene Stempel-Zeile braucht es dafür nicht, denn der Prüfling mischt gegen die Organ-Achsen (Abschnitt 5.2); ob er eine bestehende Kategorie trifft, eine neue begründet oder eine Mischform darstellt, entscheidet der Gattungs-Schnitt — eine eigene Gattung entstünde erst bei grundlegend anderem Basis-Achsen-Satz (Abschnitt 3.3.2). Der dort erfragte Betriebsmodus, der die Entwurfsbestandteile genau *einer* Veröffentlichung als geschlossenes Achsen-Set isoliert anbietet, ist dagegen nicht gebaut und bleibt Ausblick (Abschnitt 6.3). Als erfolgreich verifiziert

gilt eine Plattform erst, wenn das Konformitäts-Gatter den `std::map`-Vertrag je Modul bestätigt und die Plattform-Probe die gemeldete Achsen-Teilmenge deckt; die plattform-kalibrierten Schwellwerte selbst werden mit den realen Läufen festgelegt.

FF3 (Prüfling-Vergleich). Der konkrete Vergleich von PRT-ART gegen die 30 SOTA-Profile steht nach den ersten Mess-Läufen aus (Abschnitt 6.3). Die Architektur ist vollständig vorbereitet: das PRT-ART-Profil, die drei Pflicht-Messreihen-Templates, der `ExperimentDriver` mit Auto-Pickup der SOTA-Profile und der ABI-konforme Adapter mit dem `std::map`-Kernvertrag (Konformitäts-Teilmenge, Anhang F); der Vollausbau auf alle siebzehn Vertragsoperationen samt Notify-Hooks ist als Folgeschritt ausgewiesen. Auch die günstigste lokale Seitendarstellung je Verzweigungspunkt (Punkt- gegenüber Kompaktseite) und ihre plattform-kalibrierten Umschaltregeln sind Teil des PRT-ART-Profils. Die Compile-Time-Teilfrage — ob latenz-kritische Messung an sich laufzeit-wählbare Entscheidungen in die Übersetzungszeit zwingt — ist bejaht und zur Bauform geworden: Jede Achse gliedert sich in eine übersetzungszeit-fixierte *Haupt*-Entscheidung und laufzeit-variable *Unter*-Entscheidungen; die Haupt-Achsen sind statisch einkompiliert — bei Organ-Achsen als `binary_id`-Segment, bei System-Achsen als Stempel samt Bau-Versionskennung —, während dynamische Unter-Achsen ihre Laufzeit-Parameter ohne Neuübersetzung variieren (Abschnitt 3.4). Genau die von der Teilfrage vermutete Gliederung ist damit die tragende Doktrin des Apparats; FF4 führt sie fort.

FF4 (Compile-Time vs. Laufzeit). Die Grenze zwischen Übersetzungszeit und Laufzeit verläuft entlang der Achsen-Kategorien, und die System-Erweiterungs-Teilfrage ist damit bejaht: Die Plattform-Belange — die drei System-Haupt-Achsen samt ihrer Komplex-Klammer (Abschnitt 3.4.2) — gehören zur Übersetzungszeit-Identität der Binary; sie erscheinen als System-Stempel-Zeile und Bau-Versionskennung, nie als `binary_id`-Segment, und erzeugen provenienz-nachweisbare Binaries je Permutation (SHA256-validierte Original-Bodies — deren Voll-Implementierung Abschnitt 6.2 als offen ausweist —, `get_compiler()`-Kennung). Mess-Wahlen bleiben laufzeit-variabel, und Layout- wie Seiten-Selektion bleiben über einen bit-codierten Permutations-Identifizier zur Laufzeit wählbar, ohne die gemeinsame Compiler- und Flag-Basis zu verletzen. Wo eine Achse beide Seiten der Grenze braucht, ist sie *zweigeteilt*: Last-Rahmenwerk, Telemetrie und Hardware-Erkennung laufen über die Laufzeit des Planers *und* die Übersetzungszeit des Builders — die Unter-Achsen-Freigabe der Vorstufe wird zur Haupt-Achsen-Annahme der Folgestufe (Abschnitt 3.4.1) —, und die zweigeteilte Hardware-Erkennung setzt mit ihrer Ebene 2 die Übersetzungszeit-Stempel der Tier-Binaries aus den zur Builder-Laufzeit erhobenen Ist-Eigenschaften korrekt (Abschnitt 3.4.3). Gestempelt wird dabei stets die Übersetzungszeit-Plattform-Klasse, nie ein Messwert; als *Entwurf* schließt daran das Konzept eines *Passungs-Stempels* an, der die gestempelte Plattform-Klasse bei jedem Mess-Lauf gegen das benannte Laufzeit-Verdikt der Hardware-Erkennung (Abschnitt 4.5) prüft — die Passung zwischen Maschine und Binary würde damit selbst zur geprüften Größe; dieses Konzept ist im Stand dieser Arbeit Entwurf, nicht gebaut (Abschnitt 6.3). Die vorwärtsgerichtete Teilfrage, ob die Messung selbst ISA- und betriebssystem-optimierte Binaries für eine Rekombination zuschneiden kann, greift der Ausblick als Heuristik-Automation auf (Abschnitt 6.3); die Betriebssystem-Dimension ist über die zwei Mess-Regimes aus Kapitel 5 bereits im Messplan verankert.

6.2 Limitierungen

- **Modul-Bodies:** der adhoc-Codegen des aktiven Mess-Pfads emittiert je Permutation reale Achsen-Library-Bodies (im Stand dieser Arbeit die achtzehn voll qualifizierten Achsen-Typen samt Umbrella-Include); nur der Legacy-Profil-Codegen erzeugt weiterhin ABI-konforme Skelett-Module, und als Lücken bleiben der PRT-ART-Prüfling-Stub sowie die ehrlich auf null gesetzten Storage-/Prefetch-Achsen der Pool-Familien offen.
- **Voll-Bau und Abgabe-Messung ausstehend:** das deklarierte Bau-Soll (Abschnitt 5.1) — je System-Permutation die 2^{17} Organ-Permutationen des kanonischen Katalogs — ist noch nicht gefahren, ebenso wenig die Abgabe-Messung über den kleinen materialisierbaren 320er-Katalog (Abschnitt 4.4); die End-to-End-messung_driver-Läufe über die Zielplattformen (AVX2/AVX-512, ARM NEON/SVE) stehen aus, während die apparat-seitige Abnahme je Paket bereits über das Zwei-Gate-Protokoll läuft.
- **Ergebnis-Kapitel:** aktuell methodisch; konkrete Messungen folgen. Eine erste reale Smoke-Messreihe (43 Permutationen) liegt in Anhang A vor, ersetzt die Pflicht-Messreihen aber nicht.
- **Hardware-Zähler standardmäßig aus und teils quellenabhängig:** die PMC-Erfassung (L1D-, Last-Level-, dTLB- und seit M-3a Branch-Misses über `perf_event_open`) ist default-abgeschaltet; L2-Misses und Kohärenz-Invalidierungen bleiben unabhängig vom Schalter mangels portablen Zähler strukturell auf null, der Last-Level-Zähler kann auf einzelnen AMD-Plattformen zusätzlich beim Öffnen scheitern, und die Energie-Erfassung über RAPL ist best-effort verdrahtet, bleibt aber ohne Root- bzw. Zonen-Leserecht leer; IPC ist als Erweiterung des Zähler-Satzes geplant, wird derzeit nicht erfasst. In der bisherigen Smoke-Reihe stehen die Hardware-Spalten auf null — die cache-bewusste Knoten-/Daten-/Index-Messkomponente steht damit noch aus (Abbildung 3.11).
- **Hardware-Erkennung im Teilausbau:** die ersten beiden Ausbaustufen — Provenienz-Vokabular, SPD-Parser und Fehler-Domäne sowie Erhebungs-Kette, Übersetzungszeit-Factory und Laufzeit-Verdikt — liegen als Code vor (Abschnitt 4.5); ausstehend sind die Verdrahtung in Runner und Prüf-Dock, der additive Ergebnis-Rückschrieb samt seiner Konsumenten, der Provenienz-Roundtrip der Maschinen-Deklarationen und der Passungs-Stempel-Abgleich (Abschnitte 3.4.3 und 6.1).
- **Auswertungs-Codierung:** CSV ist der implementierte Stand; der als künftiger Default beschlossene xlsx-Factory-Writer (ein Blatt je gewählter Unter-Achsen-Permutation plus Info-Blatt, Abschnitt 5.2) ist noch nicht gebaut.
- **Smoke-Reihe mit Apparat-Vorbehalten:** die bisher realen Messungen variieren sieben Achsen der damaligen Neunzehner-Zählung (Zwischenstand Juli 2026) in zwei Teilblöcken (27 SIMD×Layout×Allokator-Kombinationen plus 16 Kombinationen über Node-Breite, Pfadkompression, interne Suche und Telemetrie) und approximieren Achsen-Effekte teils über Wall-Clock-Proxys; einzelne Effekte sind Apparat- statt Organ-Artefakte („Beschattung“). Die vollständige Limitierungs-Tabelle führt Abschnitt A.10.

6.3 Ausblick

Kurzfristig: der Voll-Bau nach dem deklarierten Soll (Abschnitt 5.1), die Abgabe-Messung über den 320er-Katalog, der End-to-End-Lauf der drei Pflicht-Messreihen auf realer Hardware samt Generierung der Mess-Diagramme, der Ausbau der Hardware-Erkennung um Verdrahtung, Ergebnis-Rückschrieb und Passungs-Stempel, der beschlossene `xlsx-Factory-Writer` und die Schließung der verbleibenden Modul-Body-Lücken (PRT-ART-Stub, honest-0-Storage-/Prefetch-Achsen der Pool-Familien). Dazu tritt der in FF2 als Ausblick ausgewiesene Paper-Isolations-Modus: ein Betriebsmodus, der die Entwurfsbestandteile genau einer Veröffentlichung als geschlossenes Achsen-Set isoliert anbietet, um sie — bei weiterhin freistehenden Mess- und System-Achsen — originalgetreu durchzumessen.

Heuristik-Automation und Binary-Delivery. Der in der Aufgabenstellung (Teilaufgabe 7) geforderte Mechanismus, der aus den Messergebnissen automatisch die beste Konfiguration ableitet, ist in seiner heuristik-gesteuerten *Voll*automatisierung bewusst noch nicht implementiert und bleibt einer Folge-Phase vorbehalten; ein erstes Inkrement der automatischen Auswahl und des Binary-Versands liegt jedoch bereits als eigenständiges Werkzeug (`best_binary_selector`) vor, das je Metrik die beste Permutation aus den Mess-CSVs rangiert und ausliefert. Der Vollausbau setzt drei Komponenten voraus; die zweite liegt bereits vor, und zur ersten wie zur dritten stellt dieses Werkzeug ein erstes, rein messwert-getriebenes Inkrement: (i) einen *Heuristik-/ML-Klassifikator*, der aus den Lastprofil- und Achsen-Features (Operationsmix, Schlüssel-Charakteristik, Hardware-Kennung) auf die empfohlene Achsen-Konfiguration abbildet; (ii) den bereits implementierten *XML-Parser* (Import und Export samt Extraktor) für die als XML-Lastprofil-Ergebnis abgelegten, je Architekturfokus wiederverwendbaren Heuristik-Richtlinien; und (iii) einen *Binary-Packager*, der den so ausgewählten Gesamtalgorithmus als eigenständige Binary hinter dem einheitlichen Suchalgorithmus-Interface ausliefert. Voraussetzung für alle drei ist, dass die End-to-End-Messreihen zuvor vollständig vorliegen — erst die belastbare Datenbasis über alle Permutationen und Lastprofile macht eine messgetriebene, automatisch gesteuerte Auswahl sinnvoll; bis dahin werden die Konfigurations-Empfehlungen des Auswertungs-Schemas aus Abschnitt 5.2 nach Vorliegen der Daten zunächst manuell abgeleitet.

Mittel- bis langfristig: ein GPU-Adapter (Grace-Hopper-Cluster, ZIH), die PIM-Allokator-Integration (A16) und die formale Verifikation ausgewählter Permutations-Klassen (StarMalloc-Stil, A13). Die Drei-Repository-Architektur erlaubt es künftigen Studierenden, weitere Prüfling-Algorithmen analog zu PRT-ART einzubringen, ohne die Cache-Engine-Bausteine-Bibliothek zu modifizieren — ein zentrales Designprinzip für die Skalierbarkeit der Forschungsinfrastruktur. Über die hier behandelten baumartigen Suchstrukturen hinaus ist das Anatomie-Prinzip über höherwertige Abstraktionen auf die weiteren Strukturklassen aus Abschnitt 2.1 erweiterbar — hash-basierte, räumliche und flache Strukturen —, sodass derselbe Mess- und Permutationsapparat ohne Neubau auf ganz neue Klassen angewendet werden kann; darin liegt ein wesentlicher Hebel für künftigen experimentellen Fortschritt.

Literaturverzeichnis

- [ABKS13] APEL, Sven ; BATORY, Don ; KÄSTNER, Christian ; SAAKE, Gunter: *Feature-Oriented Software Product Lines: Concepts and Implementation*. 1. Springer, 2013
- [ACK⁺04] AGRAWAL, Sanjay ; CHAUDHURI, Surajit ; KOLLÁR, Lubor ; MARATHE, Arunprasad P. ; NARASAYYA, Vivek R. ; SYAMALA, Manoj: Database Tuning Advisor for Microsoft SQL Server 2005. In: *30th International Conference on Very Large Data Bases (VLDB)*, 2004, S. 1110–1121
- [ADHW99] AILAMAKI, Anastassia ; DEWITT, David J. ; HILL, Mark D. ; WOOD, David A.: DBMSs on a Modern Processor: Where Does Time Go? In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 266–277
- [Adv23] ADVANCED MICRO DEVICES. *Software Optimization Guide for the AMD Zen Microarchitecture (Family 19h)*. Advanced Micro Devices, Santa Clara, CA, USA. 2023
- [AKLS15] AIGNER, Christoph ; KIRSCH, Christoph ; LIPPAUTZ, Michael ; SOKOLOVA, Ana: Fast, Multicore-Scalable, Low-Fragmentation Memory Allocation. In: *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2015, S. 513–531
- [AKV⁺14] ANSEL, Jason ; KAMIL, Shoaib ; VEERAMACHANENI, Kalyan ; RAGAN-KELLEY, Jonathan ; BOSBOOM, Jeffrey ; O'REILLY, Una-May ; AMARASINGHE, Saman: OpenTuner: An Extensible Framework for Program Autotuning. In: *23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2014, S. 303–316
- [Ale01] ALEXANDRESCU, Andrei: *Modern C++ Design: Generic Programming and Design Patterns Applied*. 1. Addison-Wesley, 2001
- [AMH08] ABADI, Daniel J. ; MADDEN, Samuel R. ; HACHEM, Nabil: Column-Stores vs. Row-Stores: How Different Are They Really? In: *ACM SIGMOD International Conference on Management of Data*, 2008, S. 967–980
- [Apa26] APACHE SOFTWARE FOUNDATION. *Maven — POM Reference*. <https://maven.apache.org/pom.html>, abgerufen am 01.08.2026. 2026
- [APGZ17] AKEN, Dana V. ; PAVLO, Andrew ; GORDON, Geoffrey J. ; ZHANG, Bohan: Automatic Database Management System Tuning Through Large-scale Machine Learning. In: *ACM SIGMOD International Conference on Management of Data*, 2017, S. 1009–1024
- [App24] APPLE INC. *Apple Silicon CPU Optimization Guide*. Apple Inc., Cupertino, CA, USA, Version 4. 2024

- [APV02] ARGE, Lars ; PROCOPIUC, Octavian ; VITTER, Jeffrey S.: Implementing I/O-efficient Data Structures Using TPIE. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 88–100
- [Arm24] ARM LIMITED. *Arm Architecture Reference Manual for A-profile Architecture (DDI 0487)*. Arm Limited, Cambridge, UK. 2024
- [Aßm03] ASSMANN, Uwe: *Invasive Software Composition*. 1. Springer, 2003
- [AV88] AGGARWAL, Alok ; VITTER, Jeffrey S.: The Input/Output Complexity of Sorting and Related Problems. In: *Communications of the ACM* 31 (1988), Nr. 9, S. 1116–1127
- [AVL62] ADELSON-VELSKY, Georgy ; LANDIS, Evgenii: An Algorithm for the Organization of Information. In: *Doklady Akademii Nauk SSSR* 146 (1962), S. 263–266
- [BA01] BONWICK, Jeff ; ADAMS, Jonathan: Magazines and Vmem: Extending the Slab Allocator to Many CPUs and Arbitrary Resources. In: *Proceedings of the USENIX Annual Technical Conference (ATC)* USENIX, 2001
- [Bay72] BAYER, Rudolf: Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. In: *Acta Informatica* 1 (1972), Nr. 4, S. 290–306
- [BBB⁺11] BINKERT, Nathan ; BECKMANN, Bradford ; BLACK, Gabriel ; REINHARDT, Steven K. ; SAIDI, Ali ; BASU, Arkaprava ; HESTNESS, Joel ; HOWER, Derek R. ; KRISHNA, Tushar ; SARDASHTI, Somayeh ; SEN, Rathijit ; SEWELL, Korey ; SHOAIB, Muhammad ; VAISH, Nilay ; HILL, Mark D. ; WOOD, David A.: The gem5 Simulator. In: *ACM SIGARCH Computer Architecture News* 39 (2011), Nr. 2, S. 1–7
- [BBF09] BLAIR, Gordon ; BENCOMO, Nelly ; FRANCE, Robert B.: Models@run.time. In: *Computer* 42 (2009), Nr. 10, S. 22–27
- [BCD⁺02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin ; ZITO, Jack: Two Simplified Algorithms for Maintaining Order in a List. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 152–164
- [BCDFC02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Scanning and Traversing: Maintaining Data for Traversals in a Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 139–151
- [BDFC00] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2000, S. 399–409
- [BDFC02] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Efficient Tree Layout in a Multilevel Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 165–173

- [BDFC05] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *SIAM Journal on Computing* 35 (2005), Nr. 2, S. 341–358
- [Ben75] BENTLEY, Jon L.: Multidimensional Binary Search Trees Used for Associative Searching. In: *Communications of the ACM* 18 (1975), Nr. 9, S. 509–517
- [BFTV24] BOFFA, Antonio ; FERRAGINA, Paolo ; TOSONI, Francesco ; VINCIGUERRA, Giorgio: CoCo-trie: Data-aware Compression and Indexing of Strings. In: *Information Systems* 120 (2024), S. 102316
- [BH23] BERTHOLD, Johannes ; HABICH, Dirk: VAMPIR/OTF2 Profiling and Performance Analysis for Database Workloads. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. – Poster presentation
- [Blo70] BLOOM, Burton H.: Space/Time Trade-offs in Hash Coding with Allowable Errors. In: *Communications of the ACM* 13 (1970), Nr. 7, S. 422–426
- [BM72] BAYER, Rudolf ; MCCREIGHT, Edward M.: Organization and Maintenance of Large Ordered Indexes. In: *Acta Informatica* 1 (1972), Nr. 3, S. 173–189
- [BMBW00] BERGER, Emery D. ; MCKINLEY, Kathryn S. ; BLUMOF, Robert D. ; WILSON, Paul R.: Hoard: A Scalable Memory Allocator for Multithreaded Applications. In: *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, ACM, 2000, S. 117–128
- [BO92] BATORY, Don ; O’MALLEY, Sean: The Design and Implementation of Hierarchical Software Systems with Reusable Components. In: *ACM Transactions on Software Engineering and Methodology* 1 (1992), Nr. 4, S. 355–398
- [Bon94] BONWICK, Jeff: The Slab Allocator: An Object-Caching Kernel Memory Allocator. In: *Proceedings of the USENIX Summer 1994 Technical Conference* USENIX, 1994, S. 87–98
- [Bri59] DE LA BRIANDAIS, René: File Searching Using Variable Length Keys. In: *Western Joint Computer Conference*, 1959, S. 295–298
- [BZP⁺18] BINNA, Robert ; ZANGERLE, Eva ; PICHL, Martin ; SPECHT, Günther ; LEIS, Viktor: HOT: A Height Optimized Trie Index for Main-Memory Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 521–534
- [CE00] CZARNECKI, Krzysztof ; EISENECKER, Ulrich W.: *Generative Programming: Methods, Tools, and Applications*. 1. Addison-Wesley, 2000
- [CGM01] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Improving Index Performance through Prefetching. In: *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ACM, 2001, S. 235–246
- [CGM02] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Fractal Prefetching B+-Trees. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ACM, 2002, S. 157–168

- [CHL99] CHILIMBI, Trishul M. ; HILL, Mark D. ; LARUS, James R.: Cache-Conscious Structure Layout. In: *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, 1999, S. 1–12
- [CHP71] COURTOIS, Pierre-Jacques ; HEYMANS, Frans ; PARNAS, David L.: Concurrent Control with “Readers” and “Writers”. In: *Communications of the ACM* 14 (1971), Nr. 10, S. 667–668
- [Cli93] CLIFF, Norman: Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. In: *Psychological Bulletin* 114 (1993), Nr. 3, S. 494–509
- [CLP22] CROTTY, Andrew ; LEIS, Viktor ; PAVLO, Andrew: Are You Sure You Want to Use MMAP in Your Database Management System? In: *12th Conference on Innovative Data Systems Research (CIDR 2022)*, Chaminade, CA, USA, January 9-12, 2022, www.cidrdb.org, 2022
- [CLP23] CHOCKCHOWWAT, Supawit ; LIU, Wenjie ; PARK, Yongjoo: AirIndex: Versatile Index Tuning Through Data and Storage. In: *Proceedings of the ACM on Management of Data* 1 (2023), Nr. 3, S. 204:1–204:26. – Artikel-Nr. 204; präsentiert auf der ACM-SIGMOD-Konferenz 2024
- [Com79] COMER, Douglas: The Ubiquitous B-Tree. In: *ACM Computing Surveys* 11 (1979), Nr. 2, S. 121–137
- [CST⁺10] COOPER, Brian F. ; SILBERSTEIN, Adam ; TAM, Erwin ; RAMAKRISHNAN, Raghu ; SEARS, Russell: Benchmarking Cloud Serving Systems with YCSB. In: *1st ACM Symposium on Cloud Computing (SoCC)*, 2010, S. 143–154
- [Des14] DESROCHERS, Cameron. *moodycamel::ConcurrentQueue: A Fast Multi-Producer, Multi-Consumer Lock-Free Queue for C++*. <https://github.com/cameron314/concurrentqueue>. 2014
- [Dij65] DIJKSTRA, Edsger W.: Solution of a Problem in Concurrent Programming Control. In: *Communications of the ACM* 8 (1965), Nr. 9, S. 569
- [DPT⁺13] DEBRABANT, Justin ; PAVLO, Andrew ; TU, Stephen ; STONEBRAKER, Michael ; ZDONIK, Stan: Anti-Caching: A New Approach to Database Management System Architecture. In: *Proceedings of the VLDB Endowment* 6 (2013), Nr. 14, S. 1942–1953
- [Dre07] DREPPER, Ulrich: What Every Programmer Should Know About Memory / Red Hat, Inc. 2007. – Forschungsbericht. White paper
- [DSST89] DRISCOLL, James R. ; SARNAK, Neil ; SLEATOR, Daniel D. ; TARJAN, Robert E.: Making Data Structures Persistent. In: *Journal of Computer and System Sciences* 38 (1989), Nr. 1, S. 86–124
- [EB75] VAN EMDE BOAS, Peter: Preserving Order in a Forest in Less Than Logarithmic Time. In: *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, S. 75–84

- [Eva06] EVANS, Jason: A Scalable Concurrent malloc(3) Implementation for FreeBSD. In: *Proceedings of BSDCan 2006* BSDCan, 2006
- [FAK⁺12] FERDMAN, Michael ; ADILEH, Almutaz ; KOCBERBER, Onur ; VOLOS, Stavros ; ALISAFEE, Mohammad ; JEVDJIC, Djordje ; KAYNAK, Cansu ; POPESCU, Adrian D. ; AILAMAKI, Anastasia ; FALSAFI, Babak: Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. In: *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2012, S. 37–48
- [FAKM14] FAN, Bin ; ANDERSEN, David G. ; KAMINSKY, Michael ; MITZENMACHER, Michael: Cuckoo Filter: Practically Better Than Bloom. In: *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*, ACM, 2014, S. 75–88
- [FB74] FINKEL, Raphael A. ; BENTLEY, Jon L.: Quad Trees: A Data Structure for Retrieval on Composite Keys. In: *Acta Informatica* 4 (1974), Nr. 1, S. 1–9
- [FJ98] FRIGO, Matteo ; JOHNSON, Steven G.: FFTW: An Adaptive Software Architecture for the FFT. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* Bd. 3, 1998, S. 1381–1384
- [FJKN20] FENT, Philipp ; JUNGMAIR, Michael ; KIPF, Andreas ; NEUMANN, Thomas: START: Self-Tuning Adaptive Radix Tree. In: *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2020, S. 147–153
- [FLPR99] FRIGO, Matteo ; LEISERSON, Charles E. ; PROKOP, Harald ; RAMACHANDRAN, Sridhar: Cache-Oblivious Algorithms. In: *40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999, S. 285–297
- [Fre60] FREDKIN, Edward: Trie Memory. In: *Communications of the ACM* 3 (1960), Nr. 9, S. 490–499
- [FW93] FREDMAN, Michael L. ; WILLARD, Dan E.: Surpassing the Information Theoretic Bound with Fusion Trees. In: *Journal of Computer and System Sciences* 47 (1993), Nr. 3, S. 424–436
- [GHJV94] GAMMA, Erich ; HELM, Richard ; JOHNSON, Ralph ; VLISSIDES, John: *Design Patterns: Elements of Reusable Object-Oriented Software*. 1. Addison-Wesley, 1994
- [GL01] GRAEFE, Goetz ; LARSON, Per-Åke: B-Tree Indexes and CPU Caches. In: *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, IEEE, 2001, S. 349–358
- [GL20] GRAF, Thomas M. ; LEMIRE, Daniel: Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. In: *ACM Journal of Experimental Algorithmics* 25 (2020), Nr. 1, S. 1.5:1–1.5:16

- [GM05] GHEMAWAT, Sanjay ; MENAGE, Paul: TCMalloc: Thread-Caching Malloc / Google. 2005. – Forschungsbericht. Design documentation
- [GMUW09] GARCIA-MOLINA, Hector ; ULLMAN, Jeffrey D. ; WIDOM, Jennifer: *Database Systems: The Complete Book*. 2. Upper Saddle River, NJ, USA : Pearson Prentice Hall, 2009
- [Goo18] GOOGLE. *Abseil Swiss Tables (flat_hash_map)*. <https://abseil.io/about/design/swisstable>. 2018
- [Göt13] GÖTZ, Sebastian: *Multi-Quality Auto-Tuning by Contract Negotiation*, Technische Universität Dresden, Diss., 2013. – URN: urn:nbn:de:bsz:14-qucosa-119938
- [GS78] GUIBAS, Leo J. ; SEDGEWICK, Robert: A Dichromatic Framework for Balanced Trees. In: *19th Annual Symposium on Foundations of Computer Science (FOCS)*, 1978, S. 8–21
- [Gut84] GUTTMAN, Antonin: R-Trees: A Dynamic Index Structure for Spatial Searching. In: *ACM SIGMOD International Conference on Management of Data*, 1984, S. 47–57
- [HB15] HOEFLER, Torsten ; BELL, Roberto: Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In: *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015
- [HBHR11] HERTER, Jörg ; BACKES, Peter ; HAUPENTHAL, Florian ; REINEKE, Jan: CAMA: A Predictable Cache-Aware Memory Allocator. In: *23rd Euromicro Conference on Real-Time Systems (ECRTS)*, 2011, S. 23–32
- [Her91] HERLIHY, Maurice: Wait-Free Synchronization. In: *ACM Transactions on Programming Languages and Systems* 13 (1991), Nr. 1, S. 124–149
- [HIKY12] HALIM, Felix ; IDREOS, Stratos ; KARRAS, Panagiotis ; YAP, Roland H. C.: Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. In: *Proceedings of the VLDB Endowment* 5 (2012), Nr. 6, S. 502–513
- [HKT⁺21] HUNTER, A. H. ; KENNELLY, Chris ; TURNER, Paul ; GOVE, Darryl ; MOSELEY, Tipp ; RANGANATHAN, Parthasarathy: Beyond malloc Efficiency to Fleet Efficiency: A Hugepage-Aware Memory Allocator. In: *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021
- [Hol79] HOLM, Sture: A Simple Sequentially Rejective Multiple Test Procedure. In: *Scandinavian Journal of Statistics* 6 (1979), Nr. 2, S. 65–70
- [HP03] HANKINS, Richard A. ; PATEL, Jignesh M.: Effect of Node Size on the Performance of Cache-Conscious B+-Trees. In: *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ACM, 2003, S. 120–131
- [HP19] HENNESSY, John L. ; PATTERSON, David A.: *Computer Architecture: A Quantitative Approach*. 6th. Morgan Kaufmann, 2019

- [HS89] HILL, Mark D. ; SMITH, Alan J.: Evaluating Associativity in CPU Caches. In: *IEEE Transactions on Computers* 38 (1989), Nr. 12, S. 1612–1630
- [IKM07] IDREOS, Stratos ; KERSTEN, Martin L. ; MANEGOLD, Stefan: Database Cracking. In: *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, S. 68–78
- [Int24a] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, Santa Clara, CA, USA. 2024
- [Int24b] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide*. Intel Corporation, Santa Clara, CA, USA. 2024
- [ISO24] ISO/IEC. *ISO/IEC 14882:2024 — Programming Languages — C++*. International Organization for Standardization, Geneva, Switzerland. 2024
- [IZA⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; ATHANASSOULIS, Manos ; DAYAN, Niv ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi ; MAAS, Lukas M. ; QIN, Wilson ; WASAY, Abdul ; SUN, Yiyu: The Periodic Table of Data Structures. In: *IEEE Data Engineering Bulletin* 41 (2018), Nr. 3, S. 64–75
- [IZH⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi: The Data Calculator: Data Structure Design and Cost Synthesis from First Principles, and Learned Cost Models. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 535–550
- [Jac89] JACOBSON, Guy J.: Space-Efficient Static Trees and Graphs. In: *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1989, S. 549–554
- [Jan17] JANSSON, Mattias. *rpmalloc: A Public Domain License Per-thread Memory Allocator*. GitHub Repository. 2017
- [JED20] JEDEC SOLID STATE TECHNOLOGY ASSOCIATION: DDR5 SDRAM Standard (JESD79-5) / JEDEC Solid State Technology Association, Arlington, VA, USA. 2020. – Forschungsbericht. Norm
- [JTJE10] JALEEL, Aamer ; THEOBALD, Kevin B. ; JR., Simon C. S. ; EMER, Joel: High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In: *37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, S. 60–71
- [KAS21] KOYLE, Neal ; AYYAPUREDDI, Sujeet ; SEIBERT, Mike: Micron DDR5: Client Module Features / Micron Technology, Inc. 2021 (CCM004676576390-11566, Rev. A). – White Paper. Online-Ressource
- [KBC⁺18] KRASKA, Tim ; BEUTEL, Alex ; CHI, Ed H. ; DEAN, Jeffrey ; POLYZOTIS, Neoklis: The Case for Learned Index Structures. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 489–504

- [KCH⁺90] KANG, Kyo C. ; COHEN, Sholom G. ; HESS, James A. ; NOVAK, William E. ; PETERSON, A. S.: Feature-Oriented Domain Analysis (FODA) Feasibility Study / Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA. 1990 (CMU/SEI-90-TR-021). – Forschungsbericht. Technical Report
- [KCS⁺10] KIM, Changkyu ; CHHUGANI, Jatin ; SATISH, Nadathur ; SEDLAR, Eric ; NGUYEN, Anthony D. ; KALDEWEY, Tim ; LEE, Victor W. ; BRANDT, Scott A. ; DUBEY, Pradeep: FAST: Fast Architecture Sensitive Tree Search on Modern CPUs and GPUs. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, S. 339–350
- [Kha11] KHAN, Minhaj A.: Data Cache Prefetching With Dynamic Adaptation. In: *The Computer Journal* 54 (2011), Nr. 5, S. 815–823
- [Kit26] KITWARE, INC. *CMake Documentation: cmake(1)*. <https://cmake.org/cmake/help/latest/manual/cmake.1.html>, abgerufen am 01.08.2026. 2026
- [KM17a] KHUONG, Paul-Virak ; MORIN, Pat: Array Layouts for Comparison-Based Searching. In: *ACM Journal of Experimental Algorithmics* 22 (2017), S. 1.3:1–1.3:39
- [KM17b] KHUONG, Paul-Virak ; MORIN, Pat: Array Layouts for Comparison-Based Searching. In: *ACM Journal of Experimental Algorithmics* 22 (2017)
- [KMR⁺19] KIPF, Andreas ; MARCUS, Ryan ; VAN RENEN, Alexander ; STOIAN, Mihail ; KEMPER, Alfons ; KRASKA, Tim ; NEUMANN, Thomas: SOSD: A Benchmark for Learned Indexes. In: *NeurIPS Workshop on Machine Learning for Systems*, 2019
- [Kno65] KNOWLTON, Kenneth C.: A Fast Storage Allocator. In: *Communications of the ACM* 8 (1965), Nr. 10, S. 623–625
- [KNR11] KREPS, Jay ; NARKHEDE, Neha ; RAO, Jun: Kafka: A Distributed Messaging System for Log Processing. In: *6th International Workshop on Networking Meets Databases (NetDB)*, 2011, S. 1–7
- [Knu98] KNUTH, Donald E.: *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd. Addison-Wesley, 1998
- [Kro81] KROFT, David: Lockup-Free Instruction Fetch/Prefetch Cache Organization. In: *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, 1981, S. 81–87
- [Kue23] KUEHN, Lena: Towards Data-Based Cache Optimization of B+-Trees. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2023, S. 8:1–8:8
- [LABN16] LEIS, Viktor ; ANDERSEN, Michael ; BIRKEDAL, Erik ; NEUMANN, Thomas: The ART of Practical Synchronization. In: *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2016, S. 8:1–8:8
- [Lam77] LAMPORT, Leslie: Concurrent Reading and Writing. In: *Communications of the ACM* 20 (1977), Nr. 11, S. 806–811

- [Lam13] LAMETER, Christoph: NUMA (Non-Uniform Memory Access): An Overview. In: *ACM Queue* 11 (2013), Nr. 7, S. 40–51
- [LC86] LEHMAN, Tobin J. ; CAREY, Michael J.: A Study of Index Structures for Main Memory Database Management Systems. In: *12th International Conference on Very Large Data Bases (VLDB)*, 1986, S. 294–303
- [Lc21] LEIJEN, Daan ; CONTRIBUTORS. *mimalloc-bench: Suite for Benchmarking Malloc Implementations*. <https://github.com/daanx/mimalloc-bench>. 2021
- [LCL⁺25] LI, Meng ; CHAI, Huayi ; LUO, Siqiang ; DAI, Haipeng ; GU, Rong ; ZHENG, Jiaqi ; CHEN, Guihai: VEGA: An Active-tuning Learned Index with Group-Wise Learning Granularity. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1, S. 86:1–86:26. – SIGMOD 2025; Artikel-Nr. 86
- [LDPB19] LIETAR, Paul ; DREYER, Derek ; PARKINSON, Adam ; BORNAT, Richard: snmalloc: A Message Passing Allocator. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, ACM, 2019, S. 122–135
- [Lea96] LEA, Doug. *A Memory Allocator*. Design article and C implementation. 1996
- [LJY25] LI, Ruihao ; JOHN, Lizy K. ; YADWADKAR, Neeraja J.: Old is Gold: Optimizing Single-Threaded Applications with Exgen-Malloc. In: *IEEE Computer Architecture Letters* 24 (2025), Nr. 2
- [LKN13] LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas: The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In: *29th IEEE International Conference on Data Engineering (ICDE)*, 2013, S. 38–49
- [LLS13] LEVANDOSKI, Justin J. ; LOMET, David B. ; SENGUPTA, Sudipta: The Bw-Tree: A B-tree for New Hardware Platforms. In: *29th IEEE International Conference on Data Engineering (ICDE 2013), Brisbane, Australia, April 8-12, 2013*, IEEE Computer Society, 2013, S. 302–313
- [LNOM08] LINDHOLM, Erik ; NICKOLLS, John ; OBERMAN, Stuart ; MONTRYM, John: NVIDIA Tesla: A Unified Graphics and Computing Architecture. In: *IEEE Micro* 28 (2008), Nr. 2, S. 39–55
- [LR18] LEITE, Ricardo ; ROCHA, Rui: LRMalloc: A Modern and Competitive Lock-Free Dynamic Memory Allocator. In: *Proceedings of the 13th International Conference on Parallel Processing and Applied Mathematics (VECPAR)*, Springer, 2018, S. 188–200
- [LYTL19] LI, Tianlin ; YAO, Yiping ; TANG, Wenjie ; LIN, Zhongwei: HMalloc: A Hybrid, Scalable, and Lock-Free Memory Allocator for Multi-Threaded Applications. In: *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, 2019, S. 406–409
- [LZM19] LEIJEN, Daan ; ZORN, Benjamin ; DE MOURA, Leonardo: Mimalloc: Free List Sharding in Action. In: *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, Springer, 2019, S. 172–192

- [LZS⁺23] LUO, Xuchuan ; ZUO, Pengfei ; SHEN, Jiacheng ; GU, Jiazhen ; WANG, Xin ; LYU, Michael R. ; ZHOU, Yangfan: SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023
- [MBL25] MÜLLER, Marcus ; BENSON, Lawrence ; LEIS, Viktor: B-Trees Are Back: Engineering Fast and Pageable Node Layouts. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1
- [McK01] MCKENNEY, Paul E.: Read-Copy Update. In: *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, 2001, S. 338–367
- [Me110] DE MELO, Arnaldo C.: The New Linux perf Tools. In: *Linux Kongress*, 2010, S. 1–42
- [MF02] MANZINI, Giovanni ; FERRAGINA, Paolo: Engineering a Lightweight Suffix Array Construction Algorithm. In: *Algorithms — ESA 2002, 10th Annual European Symposium Bd.* 2461, Springer, 2002, S. 698–710
- [Mic04a] MICHAEL, Maged M.: Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. In: *IEEE Transactions on Parallel and Distributed Systems* 15 (2004), Nr. 6, S. 491–504
- [Mic04b] MICHAEL, Maged M.: Scalable Lock-Free Dynamic Memory Allocation. In: *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2004, S. 35–46
- [Mit16] MITTAL, Sparsh: A Survey of Recent Prefetching Techniques for Processor Caches. In: *ACM Computing Surveys* 49 (2016), Nr. 2, S. 35:1–35:35
- [MKM12] MAO, Yandong ; KOHLER, Eddie ; MORRIS, Robert T.: Cache Craftiness for Fast Multicore Key-Value Storage. In: *7th ACM European Conference on Computer Systems (EuroSys)*, 2012, S. 183–196
- [MMP18] MOKHOV, Andrey ; MITCHELL, Neil ; PEYTON JONES, Simon: Build Systems à la Carte. In: *Proceedings of the ACM on Programming Languages* 2 (2018), Nr. ICFP, S. 79:1–79:29
- [Mor68] MORRISON, Donald R.: PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. In: *Journal of the ACM* 15 (1968), Nr. 4, S. 514–534
- [MPU⁺23] MINERVINI, Francesco ; PALOMAR, Oscar ; UNSAL, Osman ; REGGIANI, Enrico ; QUIROGA, Josue ; MARIMON, Joan [u. a.]: Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications. In: *ACM Transactions on Architecture and Code Optimization* 20 (2023), Nr. 2
- [MS96] MICHAEL, Maged M. ; SCOTT, Michael L.: Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In: *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, ACM, 1996, S. 267–275
- [MS08] MEHLHORN, Kurt ; SANDERS, Peter: *Algorithms and Data Structures: The Basic Toolbox*. 1. Springer, 2008. – ISBN 978–3–540–77977–3

- [MW47] MANN, Henry B. ; WHITNEY, Donald R.: On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. In: *The Annals of Mathematical Statistics* 18 (1947), Nr. 1, S. 50–60
- [MWR25] MAHLING, Fabian ; WEISGUT, Marcel ; RABL, Tilmann: Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In: *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2025
- [NR21] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: Brief Announcement: Crystalline: Fast and Memory Efficient Wait-Free Reclamation. In: *35th International Symposium on Distributed Computing (DISC)* Bd. 209, 2021, S. 60:1–60:4
- [NR24] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: A Family of Fast and Memory Efficient Lock- and Wait-Free Reclamation. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. PLDI
- [NTSA16] NADERAN-TAHAN, Mahmood ; SARBAZI-AZAD, Hamid: Why Does Data Prefetching Not Work for Modern Workloads? In: *The Computer Journal* 59 (2016), Nr. 2, S. 244–259
- [OCGO96] O’NEIL, Patrick E. ; CHENG, Edward ; GAWLICK, Dieter ; O’NEIL, Elizabeth J.: The Log-Structured Merge-Tree (LSM-Tree). In: *Acta Informatica* 33 (1996), Nr. 4, S. 351–385
- [PIA78] PERL, Yehoshua ; ITAI, Alon ; AVNI, Haim: Interpolation Search - A Log Log N Search. In: *Communications of the ACM* 21 (1978), Nr. 7, S. 550–553
- [PR04] PAGH, Rasmus ; RODLER, Flemming F.: Cuckoo Hashing. In: *Journal of Algorithms* 51 (2004), Nr. 2, S. 122–144
- [Pug90] PUGH, William W.: Skip Lists: A Probabilistic Alternative to Balanced Trees. In: *Communications of the ACM* 33 (1990), Nr. 6, S. 668–676
- [QJP⁺07] QURESHI, Moinuddin K. ; JALEEL, Aamer ; PATT, Yale N. ; JR., Simon C. S. ; EMER, Joel: Adaptive Insertion Policies for High Performance Caching. In: *34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, S. 381–391
- [Rei05] REINDERS, James: *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*. 1. Hillsboro, OR, USA : Intel Press, 2005. – ISBN 978-0-9743649-5-7
- [RFP24a] REITZ, Antonin ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. OOPSLA2
- [RFP⁺24b] REITZ, Chris ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan ; CHEN, Boniface ; FIEDLER, Talia ; BICHHAWAT, Ananya ; ALHANAHNAH, Dalia ; OU, Elaine ; BHARGAVAN, Karthik-eya: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the 2024 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2024. – Also available as arXiv:2403.09435

- [RIS21] RISC-V INTERNATIONAL. *RISC-V V Vector Extension, Version 1.0*. https://docs.riscv.org/reference/isa/extensions/vector/_attachments/riscv-v-spec.pdf. 2021
- [RLK⁺18] VAN RENEN, Alexander ; LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas ; HASHIDA, Takushi ; OE, Kazuichi ; DOI, Yoshiyasu ; HARADA, Lilian ; SATO, Mitsuru: Managing Non-Volatile Memory in Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 1541–1555
- [RR99] RAO, Jun ; ROSS, Kenneth A.: Cache Conscious Indexing for Decision-Support in Main Memory. In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 78–89
- [RR00] RAO, Jun ; ROSS, Kenneth A.: Making B+-Trees Cache Conscious in Main Memory. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ACM, 2000, S. 475–486
- [SBB⁺17] STEPHENS, Nigel ; BILES, Stuart ; BOETTCHER, Matthias ; EYOLE, Mbou ; GABRIELLI, Giacomo ; HORSNELL, Matt ; MAGKLIS, Grigorios ; MARTINEZ, Alejandro ; PREMILLIEU, Nathanael ; REID, Alastair ; RICO, Alejandro ; WALKER, Paul: The ARM Scalable Vector Extension. In: *IEEE Micro* 37 (2017), Nr. 2, S. 26–39
- [SDF⁺00] SRINIVASAN, Jagannathan ; DAS, Souripriya ; FREIWALD, Chuck ; CHONG, Eugene I. ; JAGANNATH, Mahesh ; YALAMANCHI, Aravind ; KRISHNAN, Ramkumar ; TRAN, Anh-Tuan ; DEFAZIO, Samuel ; BANERJEE, Jayanta: Oracle8i Index-Organized Table and its Application to New Domains. In: *26th International Conference on Very Large Data Bases (VLDB)*, 2000, S. 285–296
- [SGL09] SCHLEGEL, Benjamin ; GEMULLA, Rainer ; LEHNER, Wolfgang: k-Ary Search on Modern Processors. In: *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN 2009)*, ACM, 2009, S. 52–60
- [SKK⁺25] SCHMIDT, Lennart ; KÜHN, Roland ; KRAUSE, Matti ; TEUBNER, Jens ; LEHNER, Wolfgang ; HABICH, Dirk: To Stride or Not to Stride the Memory Access? In: *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, ACM, 2025, S. 19–26
- [SPB05] SAMUEL, Curt ; PEDERSEN, Torben B. ; BONNET, Philippe: Processor-Conscious Indexing. In: *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, ACM, 2005, S. 235–246
- [SSL⁺21] SCHMEISSER, Josef ; SCHÜLE, Maximilian E. ; LEIS, Viktor ; NEUMANN, Thomas ; KEMPER, Alfons: The B²-Tree: Cache-Friendly String Indexing within B-Trees. In: *Datenbanksysteme für Business, Technologie und Web (BTW)*, 2021, S. 39–58
- [SSS08] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Binary Trees. In: *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)* Bd. 273, Springer, 2008

- [SSS16] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Dynamic Binary Trees. In: *The Computer Journal* 59 (2016), Nr. 5, S. 630–649
- [Sta17] STANDARD PERFORMANCE EVALUATION CORPORATION. *SPEC CPU 2017 Benchmark Suite*. <https://www.spec.org/cpu2017/>. 2017
- [Sto81] STONEBRAKER, Michael: Operating System Support for Database Management. In: *Communications of the ACM* 24 (1981), Nr. 7, S. 412–418
- [ŢCH02] ŢAPUŞ, Cristian ; CHUNG, I-Hsin ; HOLLINGSWORTH, Jeffrey K.: Active Harmony: Towards Automated Performance Tuning. In: *ACM/IEEE Conference on Supercomputing (SC)*, 2002, S. 1–11
- [Ten12] TENE, Gil. *HdrHistogram: A High Dynamic Range (HDR) Histogram*. <https://github.com/HdrHistogram/HdrHistogram>. 2012
- [TFB⁺11] THOMPSON, Martin ; FARLEY, Dave ; BARKER, Michael ; GEE, Patricia ; STEWART, Andrew: Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads / LMAX. 2011. – Technical Paper. Version 1.0
- [TMS⁺16] TIBBA, Ghizlane ; MALZ, Christoph ; STOERMER, Christoph ; NAGARAJAN, Natarajan ; ZHANG, Licong ; CHAKRABORTY, Samarjit: Testing Automotive Embedded Systems under X-in-the-Loop Setups. In: *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, S. 1–8
- [Tra24] TRANSACTION PROCESSING PERFORMANCE COUNCIL. *TPC Benchmark C and TPC-DS Specifications*. Transaction Processing Performance Council, San Francisco, CA, USA. 2024
- [UHL⁺17] UNGETHUEM, Sigmund ; HABICH, Dirk ; LEHNER, Wolfgang [u. a.]: Hardware Acceleration for Database Systems: A Comprehensive Survey. In: *ACM Computing Surveys* 50 (2017), Nr. 1, S. 3:1–3:35
- [VIA26] VIA RESEARCH: PIM-malloc: A Fast and Scalable Dynamic Memory Allocator for Processing-In-Memory Systems. In: *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2026. – Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025)
- [VRM⁺18] VIETRI, Giuseppe ; RODRIGUEZ, Liana V. ; MARTINEZ, Wendy A. ; LYONS, Steven ; LIU, Jason ; RANGASWAMI, Raju ; ZHAO, Ming ; NARASIMHAN, Giri: Driving Cache Replacement with ML-based LeCaR. In: *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018
- [Vyu11] VYUKOV, Dmitry. *Bounded MPMC Queue*. <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>. 2011
- [Wil64] WILLIAMS, J. W. J.: Algorithm 232: Heapsort. In: *Communications of the ACM* 7 (1964), Nr. 6, S. 347–348

- [Wil83] WILLARD, Dan E.: Log-Logarithmic Worst-Case Range Queries are Possible in Space $\Theta(N)$. In: *Information Processing Letters* 17 (1983), Nr. 2, S. 81–84
- [WM08] WEAVER, Vincent M. ; MCKEE, Sally A.: Can Hardware Performance Counters be Trusted? In: *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2008, S. 141–150
- [WNJ19] WU, Xingbo ; NI, Fan ; JIANG, Song: Wormhole: A Fast Ordered Index for In-Memory Data Management. In: *14th ACM European Conference on Computer Systems (EuroSys)*, 2019
- [WPD01] WHALEY, R. C. ; PETITET, Antoine ; DONGARRA, Jack J.: Automated Empirical Optimization of Software and the ATLAS Project. In: *Parallel Computing* 27 (2001), Nr. 1–2, S. 3–35
- [Xu24] XU, Chuanqi: Modules ABI Requirement / ISO/IEC JTC1/SC22/WG21. 2024 (P3092R0). – Forschungsbericht. Online-Ressource
- [YZZ⁺23] YANG, Hanmei ; ZHAO, Xin ; ZHOU, Jin ; WANG, Wei ; KUNDU, Sandip ; WU, Bo ; GUAN, Hui ; LIU, Tongping: NUMAlloc: A Faster NUMA Memory Allocator. In: *2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2023
- [ZDL⁺13] ZAHARIA, Matei ; DAS, Tathagata ; LI, Haoyuan ; HUNTER, Timothy ; SHENKER, Scott ; STOICA, Ion: Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In: *24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, S. 423–438
- [ZGH⁺25] ZHANG, Tingji ; GROT, Boris ; HE, Wenjian ; LV, Yashuai ; QU, Peng ; SU, Fang ; WANG, Wenxin ; ZHANG, Guowei ; ZHANG, Xuefeng ; ZHANG, Youhui: Hierarchical Prefetching: A Software-Hardware Instruction Prefetcher for Server Applications. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, ACM, 2025, S. 529–544
- [ZGV⁺24] ZHOU, Zhuangzhuang ; GOGTE, Vaibhav ; VAISH, Nilay ; KENNELLY, Chris ; XIA, Patrick ; KANEV, Svilen ; MOSELEY, Tipp ; DELIMITROU, Christina ; RANGANATHAN, Parthasarathy: Characterizing a Memory Allocator at Warehouse Scale. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, S. 192–206
- [ZL77] ZIV, Jacob ; LEMPEL, Abraham: A Universal Algorithm for Sequential Data Compression. In: *IEEE Transactions on Information Theory* 23 (1977), Nr. 3, S. 337–343
- [ZLL⁺18] ZHANG, Huanchen ; LIM, Hyeontaek ; LEIS, Viktor ; ANDERSEN, David G. ; KAMINSKY, Michael ; KEETON, Kimberly ; PAVLO, Andrew: SuRF: Practical Range Query Filtering with Fast Succinct Tries. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 323–336

-
- [ZSZ⁺24] ZHANG, Qian ; SONG, Haoyun ; ZHOU, Kaiyan ; WEI, Jianhao ; XIAO, Chuqiao: A Prefetching Indexing Scheme for In-Memory Database Systems. In: *Future Generation Computer Systems* 156 (2024), S. 179–190

Abbildungsverzeichnis

1.1	Die „Cache-Wand“: die Zugriffslatenz wächst über die Speicher-Hierarchie um rund zwei Größenordnungen.	5
1.2	Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar. .	6
2.1	Die Landschaft der Suchstrukturen: fünf Klassen mit typischen Vertretern.	11
2.2	Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.	13
3.1	Der Entwurfsraum: die Achsen spannen ein kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der achtzehn Organ-Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat achtzehn Organ-Dimensionen.	22
3.2	Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.	29
3.3	Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Mess-System übernimmt die Rolle der medizinischen Diagnostik und vermisst jedes Organ einzeln. . . .	32
3.4	Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 3.1 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).	33
3.5	Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes <code>StorageOrgan</code> -Interface, über das die Suche läuft (durchgezogen, <i>verpflichtend</i>); weitere Organe wie die Telemetrie <i>können</i> es über optionale Beobachter-Hooks nutzen (gestrichelt, <i>optional</i>), müssen aber nicht.	33

- 3.6 Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und die Telemetrie (Mess-Instrumentierung, Abschnitt 3.4.1) lesen den Store nur über `requires`-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist im Code (noch) nicht verdrahtet (gepunktet). 34
- 3.7 Das Drei-Ebenen-Modell: Gattung (Ebene 1, Außen-Interface) → Tier-Unterklasse (Ebene 2, fester Achsen-Satz) → Organe/Achsen (Ebene 3). Die Map-Gattung trägt mit ihrem einzigen Genus `SearchAlgorithm` das „Säugetier“ mit den achtzehn Organ-Achsen; die Container-Gattung die Unterklassen `Set/Sequence/Adapter/View`. 34
- 3.8 Die drei Gattungen und die Tier-Hierarchie bis zum kompilierten Binary 35
- 3.9 Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → `mp_list`-Registry) als Verfeinerung von *Strategy/CRTP* — wobei der *literale* `std::variant` nicht hier, sondern nur im elf-achsigen `baustein_variants`-Fallback-Modell auftritt. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Kapitel 4). 36
- 3.10 Die fünf Interface-Schichten der Experiment-Pipeline: von der deklarativen Definitions- und Auswertungs-Schicht E4 (Anwender-Sicht) über den Permutations-B⁺-Baum (E3) und die Tier-Binaries (E2) zur Laufzeit-Konfiguration (E1); die Infrastruktur (E0) trägt alles und wird zuletzt fertiggestellt. Der Rückkanal führt die Messwerte zur Auswertung nach E4 zurück; das Mess-System durchdringt als Querschnitt M alle Schichten („Blut“, Abschnitt 3.4). Jede Schicht wird einzeln fertiggestellt, mit eingefrorenem Interface-Vertrag zur Nachbar-Ebene. 37
- 3.11 Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 3.1, wo dieselbe Permutation ein Punkt ist). Die für cache-bewusste Suchstrukturen eigentlich zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (gestrichelt). 42

- 3.12 Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (WAS, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (WIE, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **BUILD** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **MEASURE** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) trägt die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel. 45
- 3.13 Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8. 45
- 3.14 PRT-ART als Demonstrator im M-Modell: Der Prüfling überschreibt im Mess-Apparat sechs Slots — die Build-Achse Page-Type, die vier Organ-Achsen Pfadkompression (Redirect), Wert-Handle (ChainRef), Prefetch und OLC-Nebenläufigkeit sowie die Laufzeit-Telemetrie (Mess-Tooling-Unter-Achse, Abschnitt 3.4.1) — und bezieht die übrigen vierzehn Organ-Achsen über den Compile-Time-Fallback aus der Cache-Engine; seine weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad derzeit über den Cache-Engine-Standard (Anbindung: Folgeschritt E6). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen. 47
- 3.15 Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt. 48
- 4.1 Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenzbewusste Telemetrie samt DensityTracker und den Hypothesen-Metriken H1/H2/H3 — die Telemetrie-Achse liegt seit der Achsen-Neuordnung in der System-Sphäre (Abschnitt 3.4.1). Die Kacheln zeigen die *Bausteine des Prüfling-Repos*; im Apparat überschrieben sind derzeit T7/T8 und die Telemetrie, die Bausteine zu T5/T6 laufen dort noch über den Cache-Engine-Standard (E6). Zusätzlich überschreibt der Prüfling Pfadkompressions-Redirect (T3), ChainRef-Wert-Handle (T10) und die Build-Achse Seitentyp; die übrigen vierzehn Organ-Achsen (derzeit einschließlich T5/T6) bezieht er über den Compile-Time-Fallback aus der Cache-Engine. 53

- 4.2 Galerie der achtzehn Organ-Achsen (T0–T17, kanonische Slot-Reihenfolge): je Achse das Implementierungs-Konzept — Kurzname, Sub-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein `Concept-Interface` mit mehreren Strategie-Bausteinen (*Strategy-Muster*); eine konkrete Composition belegt je Slot genau einen. Der *Standard* („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag; bei den großen Achsen T0/T6/T15 ist die Gesamt-Bausteinanzahl in Klammern angegeben. Die T-Nummern sind Slots der Composition, keine Verzeichnis-Nummern der Implementierung. Gegenüber der Neunzehner-Galerie früherer Fassungen (T0–T18, Zwischenstand Juli 2026) sind Telemetrie und ISA in die System-Sphäre umgezogen, und das Persistenz-Ziel ist als achtzehnter Slot hinzugekommen — im kanonischen Vermessungs-Katalog auf seinen In-Memory-Vertreter gepinnt (Abschnitt 3.4.1). Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D. 55
- 4.3 Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht `IExecutionEngine` als *Treiber der Messung* — der universelle messbare Vertrag (`warm_up/run/reset/shutdown`) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Gattungen* (Map, Container, Graph) und die achsenlosen *Viren*. `SearchAlgorithm` ist das fokale Lebewesen (Säugetier); seine Anatomie ist sein *Körper* — die feste Composition und *Verdrahtung* der 18 Organe —, und „`SearchEngine`“ nur dessen ABI-Laufzeit-Sicht. Der Säugetier-Ast ist bis zum Individuum ausgezogen: eine konkrete Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5). Container (schlüsselloser Speicher) und Graph tragen eigene Organsätze. 56
- 4.4 Die ABI-stabile C-Modulgrenze und die variadische Hybrid-API. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert. 56
- 4.5 UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel `IExecutionEngine` als *Treiber der Messung* (`Anatomy ∪ Virus ∪ Hybrid`) über `IAnatomyBase` und der achsenlosen `IVirus-ExecutionEngine`, darunter der *Körper* `SearchAlgorithmAnatomy<C>` (Composition und *Verdrahtung* der achtzehn Achsen) und seine ABI-Laufzeit-Sicht `SearchAlgorithmAbiAdapter<A>` („`SearchEngine`“), die über die Treiber-/Beobachter-Verträge `IDriveableTier/IObservableTier` gemessen wird; rechts das je Achse wiederholte Implementierungs-Muster — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die `mp_list`-Baustein-Familie und die Übersetzungszeit-Auflösung `resolve_baustein`, die je Achse zwischen Prüfling-Slot und Cache-Engine-Standard wählt. 57

4.6	Die Mess-Pipeline in zwei Ausprägungen. Der In-Process-Pfad (oben) ist die Sieben-Phasen-Pipeline des <code>ExperimentDriver</code> je Messreihe (<code>Enumerate</code> → <code>Codegen</code> → <code>Compile</code> → <code>Load</code> → <code>Execute</code> → <code>Measure</code> → <code>Persist</code> ; zwei opt-in-Phasen kompilieren fehlende Module <code>hot</code> und verifizieren den ABI-Vertrag). Hervorgehoben sind die eigentlich substanziellen Phasen: <code>COMPILE</code> (der <i>Build</i> der Tier-Binaries) und <code>EXECUTE/MEASURE</code> (die <i>Messung</i>); <code>Codegen/Load/Persist</code> sind reines Plumbing. Der Out-of-Process-Pfad (unten, der empfohlene Weg) trennt beide Aufgaben out-of-process: der <code>BuildOrchestrator</code> baut <i>zuerst alle</i> Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst <code>perm_runner</code> jede Binary einzeln (eine DLL je Prozess) über die Beobachter-Schnittstelle <code>IObservableTier</code>	58
4.7	Die nachgelagerte Auswertungs-Werkzeugkette	58
A.1	Zyklen je Lebewesen-Permutation (<code>v5_pipeline_demo</code>)	103
A.2	Zyklen je Lebewesen-Permutation (<code>cartesian_smoke43</code>)	104
A.3	Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)	104
A.4	Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	104
A.5	Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	105
A.6	Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	105
A.7	Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	105
A.8	Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	105
A.9	3D-Rohdaten-Ansicht: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)	106
A.10	3D-Rohdaten-Ansicht: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	106
A.11	3D-Rohdaten-Ansicht: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	109
A.12	3D-Rohdaten-Ansicht: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	109
A.13	3D-Rohdaten-Ansicht: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	109
A.14	3D-Rohdaten-Ansicht: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	109
A.15	Working-Set-Sweep: Gesamt-Latenz (ns/op)	110
A.16	Working-Set-Sweep: Insert-Latenz p50 (ns/op)	111
A.17	Working-Set-Sweep: Lookup-Latenz p50 (ns/op)	112
A.18	Gestapelte Per-Achsen-Latenz-Attribution	113
A.19	Latenz-Spanne p50-p99 je Suchalgorithmus x Operation; Punkt=p50, Whisker=p99.	114
A.20	ECDF der Gesamt-Latenz - Verteilung ueber Konfigurationen	117
A.21	Pareto-Streuung: Median- gegen Tail-Latenz (ein Punkt je Konfiguration und Op-Art; Diagonale = kein Tail-Aufschlag)	118
A.22	Verhaeltnis zur Referenz <code>linear_scan</code> : Gesamt-Latenz (ns/op) (1 = wie die Referenz)	118
A.23	Verhaeltnis zur Referenz <code>linear_scan</code> : Insert-Latenz p50 (ns/op) (1 = wie die Referenz)	118
A.24	Verhaeltnis zur Referenz <code>linear_scan</code> : Lookup-Latenz p50 (ns/op) (1 = wie die Referenz)	118
A.25	Verhaeltnis zur Referenz <code>linear_scan</code> : Erase-Latenz p50 (ns/op) (1 = wie die Referenz)	118
A.26	Verhaeltnis zur Referenz <code>linear_scan</code> : Scan-Latenz p50 (ns/op) (1 = wie die Referenz)	119
A.27	Verhaeltnis zur Referenz <code>linear_scan</code> : RMW-Latenz p50 (ns/op) (1 = wie die Referenz)	119

Tabellenverzeichnis

2.1	Cache-nahe Struktur-Entwürfe und die Hardware-Parameter, die sie explizit in den Entwurf binden.	18
3.1	Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in kanonischer Slot-Reihenfolge T0–T17; die frühere Zählung T0–T18 führte Telemetrie und ISA noch als Kompositions-Achsen — beide sind in die System-Sphäre umgezogen, Abschnitt 3.4.1. Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware/Messung/Engineering)	23
3.2	Achsen-Sezierung der Suchverfahren: achtzehn Organ-Achsen in kanonischer Slot-Reihenfolge (T0–T17), die in die System-Sphäre umgezogenen Größen und die Prüfling-Build-Achse; je Achse die ein Organ beitragenden Quellen.	25
3.3	Allokator-Profil mit Lizenz und Workload-Affinität	26
3.4	In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profil je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; der Vollkatalog folgt im Anhang).	27
3.5	Benchmark-Frameworks und nutzende Veröffentlichungen	28
3.6	Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)	28
3.7	SOTA-Algorithmus-Profil mit Workload-Affinität	30
3.8	Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen	45
5.1	Profil-bewusstes Workload-Routing (Fallback-Heuristik)	65
5.2	Reale Datensätze und ihre Charakteristik	66
A.1	Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, run_workload)	101
A.2	Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, run_workload)	102
A.3	Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)	103
A.4	Per-Achsen-Observer-Detailwerte je Tier-Binary (nur echt gemessene Zaehler)	106
A.5	Achsen-Austauschbarkeit: search_algo (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	111
A.6	Achsen-Austauschbarkeit: node_type (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: Vorbehalt)	111
A.7	Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: Vorbehalt)	112
A.8	Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	112

A.9	Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)	114
F.1	Vollständige <code>std::map</code> -Schnittstelle (C++23)	179
F.2	Vollständige <code>std::vector</code> -Schnittstelle (C++23)	182
F.3	Zusammengesetzte Funktionen („unter der Haube“)	184

A Vollständige Messreihen

Dieser Anhang enthält die vollständigen Messreihen der vorkonfigurierten Lebewesen-Permutationen. Tabelle A.1 listet die je Permutation tabellierten Kennzahlen (Operationen; Taktzyklen approximiert; Cache-Misses L1–L3 in dieser PMC-losen Messreihe 0 – die Mess-Kette erhebt L1D-, Last-Level-, dTLB- und seit M-3a Branch-Misses real, sofern mit `COMDARE_ENABLE_PMC` gebaut wird, vgl. Limitierung 1 in Abschnitt A.10); Abbildung A.1 stellt die Taktzyklen grafisch gegenüber. Die in diesem Anhang dokumentierten Reihen sind Smoke-Nachweise des vollständigen Ketten-Durchlaufs mit Achsen-Variation; sie ersetzen die Pflicht-Messreihen A–C (Abschnitt 5.1) nicht, deren Voll-Bau-Ergebnisse ausstehen.

Permutation	Ops	Zyklen	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Tabelle A.1: Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, `run_workload`)

A.1 Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)

Die folgende Messreihe entstand aus dem vollständigen Ketten-Durchlauf: die gebauten Permutations-DLLs (43 Kombinationen in zwei Teilblöcken: 27 kartesische $\text{SIMD} \times \text{Layout} \times \text{Allokator}$ -Kombinationen plus 16 weitere über Node-Breite, Pfadkompression, interne Suche und Telemetrie (²⁴) — in Summe sieben variierte Achsen der damaligen Neunzehner-Zählung, Zwischenstand Juli 2026) wurden am Prüf-Dock gemessen, als binäre Mess-Records persistiert, via Pipeline-Stufe 03 in das 16-Spalten-Schema konvertiert und automatisch hier eingebunden. Tabelle A.2 und Abbildung A.2 zeigen die Ergebnisse.

A.2 Bias-Bruch-Matrix

Tabelle A.3 fasst die je Suchverfahren und Lastprofil gemessene Median-Latenz (ns/op, nearest-rank) über alle dynamischen Einstellungen und Wiederholungen zusammen; berücksichtigt sind ausschließlich die als `two_phase_valid` markierten Läufe. Die Matrix macht sichtbar, bei welchen Lastprofilen das jeweilige Verfahren seinen systematischen Bias bricht.

Permutation	Ops	Zyklen	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Tabelle A.2: Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, run_workload)

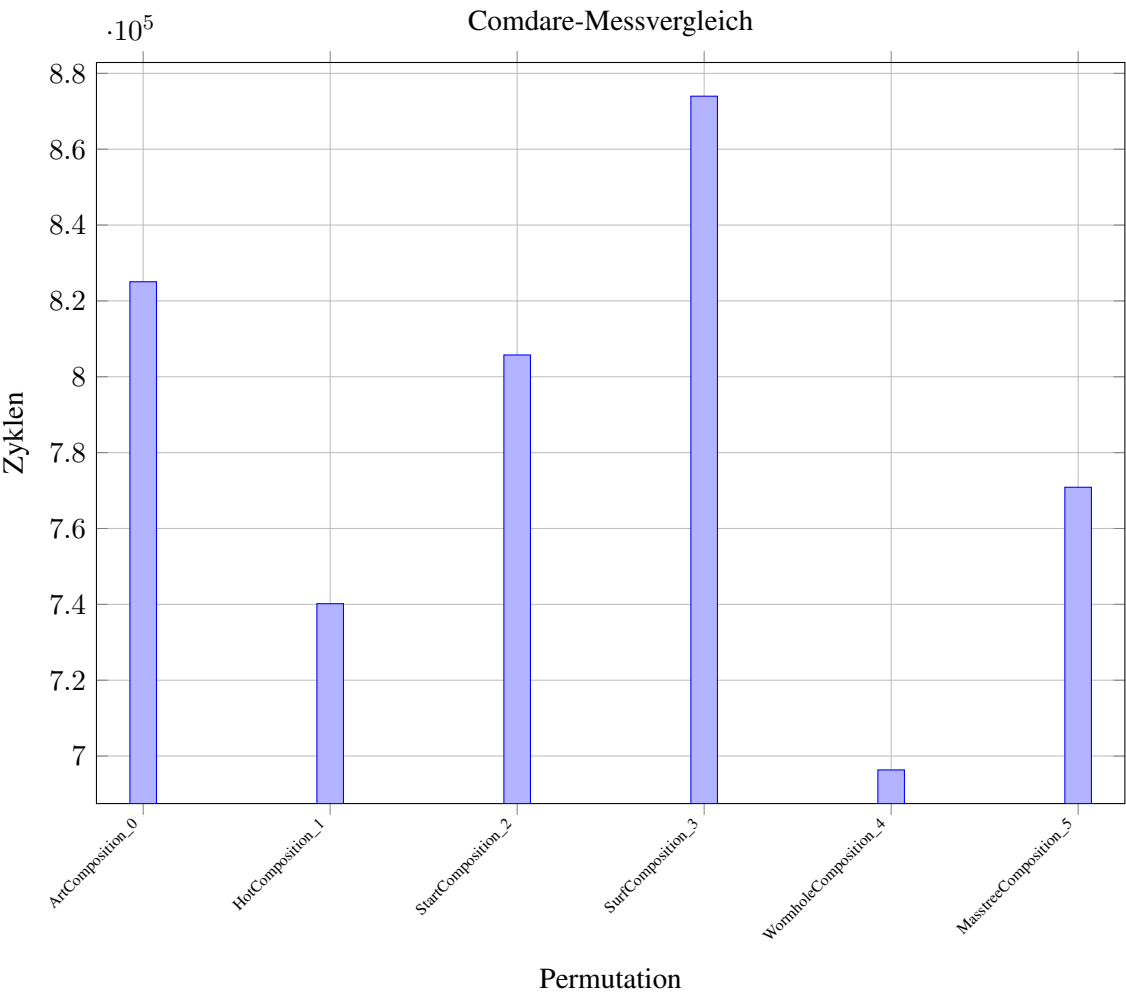


Abbildung A.1: Zyklen je Lebewesen-Permutation (v5_pipeline_demo)

Suchverfahren

y_{csb}-c

k_{ary}

672

Tabelle A.3: Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)

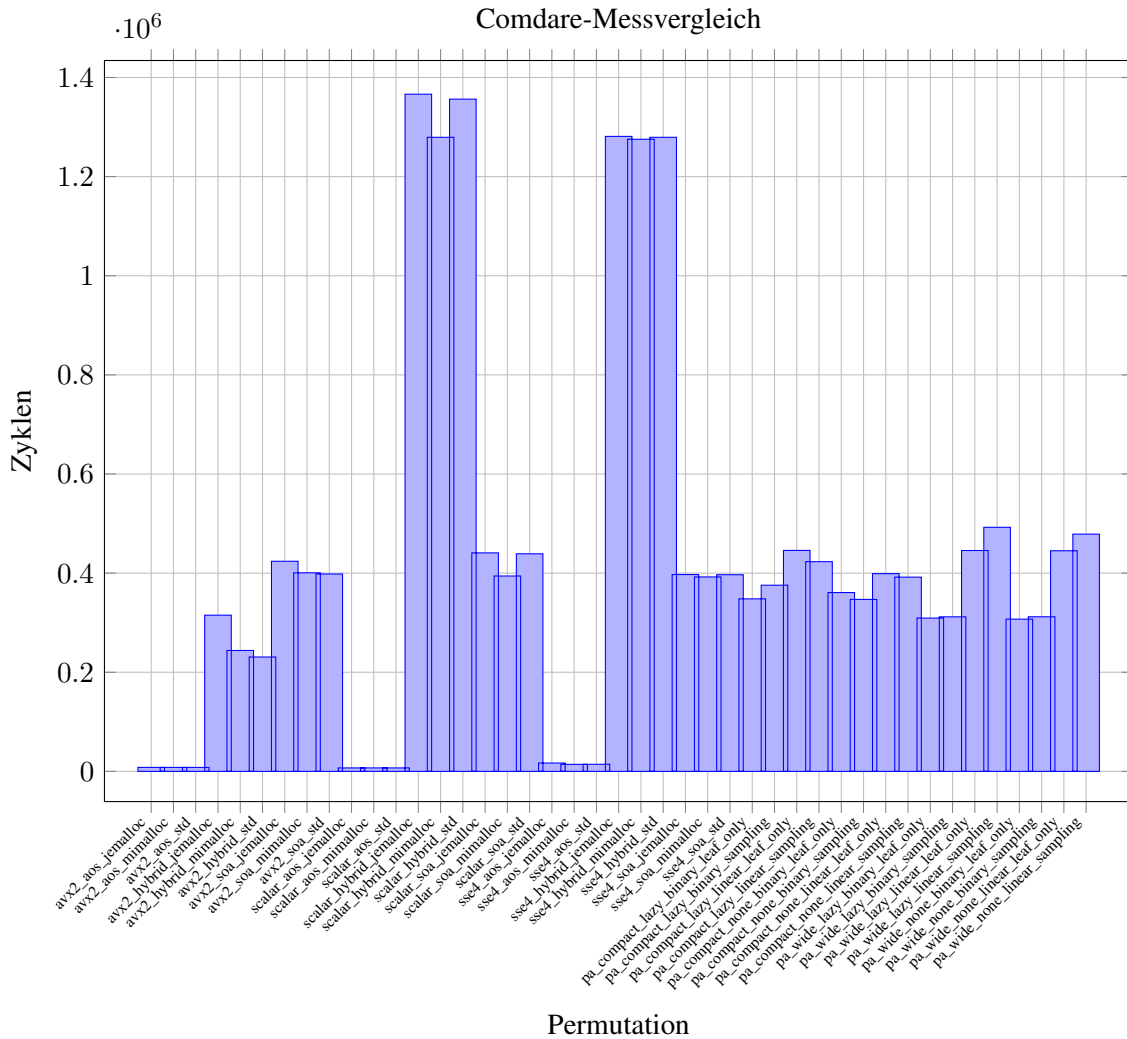


Abbildung A.2: Zyklen je Lebewesen-Permutation (cartesian_smoke43)

A.3 Latenz-Surfaces je Schnittstellen-Funktion

Die folgenden sechs Heatmaps stellen die Latenz-Oberfläche (Suchalgorithmus $\times$ Lastprofil) je Schnittstellen-Funktion dar: die Gesamt-Latenz (ns_per_op) sowie die p50-Latenzen der Einzeloperationen insert, lookup, erase, scan und rmw.

(Gemessen, aber nicht als Fläche darstellbar: Gesamt-Latenz (ns/op) liegt im vorliegenden Korpus nur als 1x1-Matrix vor (Suchalgorithmen $\times$ Workloads); die pgfplots-Flächenform verlangt mindestens 2x2. Der Messwert selbst ist im Korpus vorhanden; die Fläche wird ehrlich ausgelassen, statt den Bau zu brechen.)

Abbildung A.3: Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus $\times$ Workload)

(Keine Messwerte: Insert-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen, statt eine 0-ns-Messung zu behaupten.)

Abbildung A.4: Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus $\times$ Workload)

(Gemessen, aber nicht als Fläche darstellbar: Lookup-Latenz p50 (ns/op) liegt im vorliegenden Korpus nur als 1x1-Matrix vor (Suchalgorithmen x Workloads); die pgfplots-Flächenform verlangt mindestens 2x2. Der Messwert selbst ist im Korpus vorhanden; die Fläche wird ehrlich ausgelassen, statt den Bau zu brechen.)

Abbildung A.5: Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: Erase-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen, statt eine 0-ns-Messung zu behaupten.)

Abbildung A.6: Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: Scan-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen, statt eine 0-ns-Messung zu behaupten.)

Abbildung A.7: Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: RMW-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen, statt eine 0-ns-Messung zu behaupten.)

Abbildung A.8: Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

A.4 Dreidimensionale Rohdaten-Ansicht

Dieselben sechs Oberflächen sind zusätzlich als echte 3D-Fläche über der Ebene (Suchalgorithmus × Lastprofil) aufgetragen. Beide Flächen-Achsen sind *nominal*: die aufgespannte Fläche trägt bewusst *keine* Interpolations- oder Trendaussage und ist ausschließlich als Rohdaten- und Prüfansicht neben der Heatmap zu lesen.

A.5 Arbeitsmengen-Sweep

Die folgenden Kurven tragen die Latenz über der Arbeitsmengen-Größe n (Schlüssel) auf — je eine Kurve je gesweepter Achsen-Ausprägung, und zwar für die Gesamt-Latenz (`ns_per_op`) sowie die p50-Latenzen von `insert` und `lookup`. Messreihen ohne `working_set_n`-Spalte erzeugen bewusst *keine* Kurve; es wird kein Stützpunkt erfunden.

A.6 Per-Achsen-Latenz-Attribution

Die folgende Abbildung zerlegt die Segment-Laufzeit je Suchverfahren in die Beiträge der einzelnen Organ-Achsen (gestapelte Balken). Das 100 %-Ganze je Balken ist die Wall-Clock des separaten Segment-Laufs (`seg_run_total_ns`), *nicht* die Real-Workload-Latenz (`total_ns`); die Segmente — im Datenstand der damaligen Neunzehner-Zählung (Zwischenstand Juli 2026) zwanzig: neunzehn Organ-Achsen plus Framework-Rest — summieren sich exakt zu dieser Basis (Coverage $\approx 1,0$).

A.6.1 Per-Achsen-Observer-Detail

Die folgende Langtabelle listet je Tier-Binary die echt gemessenen Observer-Zähler (`stat_*`-Spalten aus dem DLL-seitigen Achsen-Schema) auf — je Achse und Observer-Feld den gemessenen uint64-Zählwert;

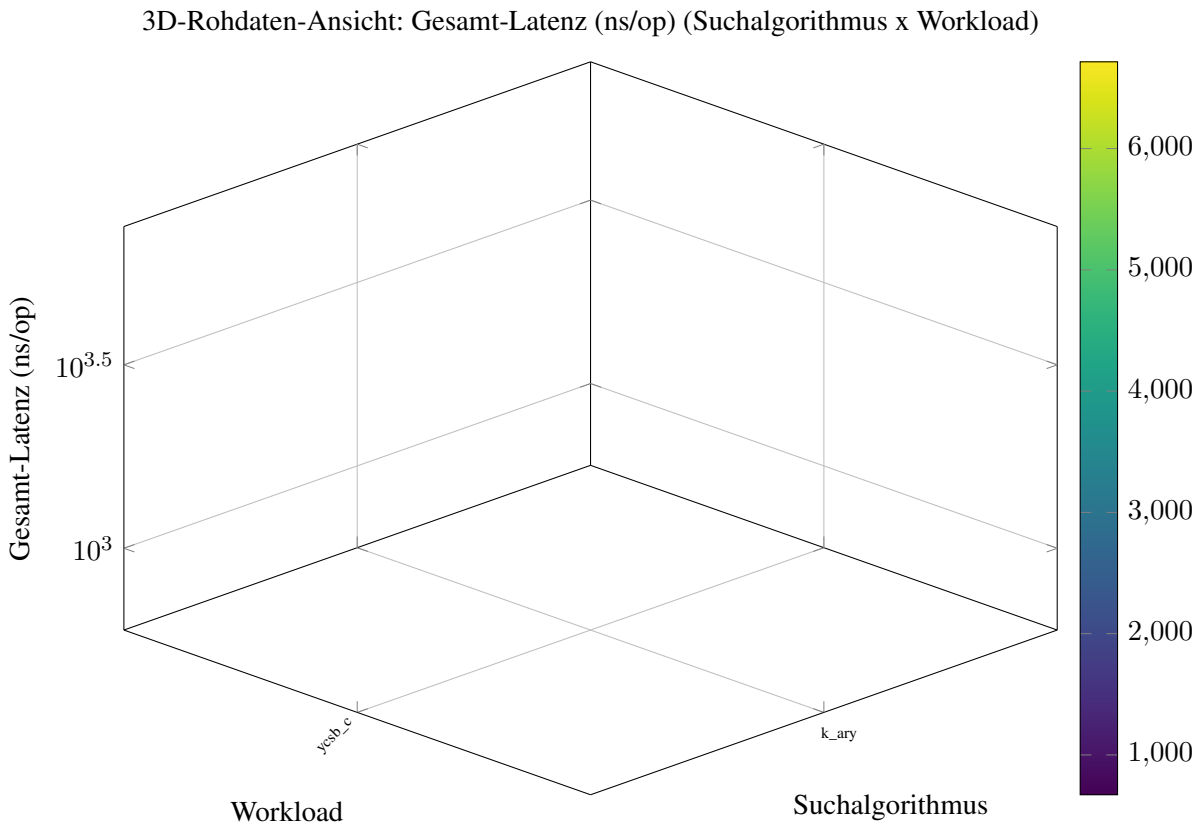


Abbildung A.9: 3D-Rohdaten-Ansicht: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: Insert-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen.)

Abbildung A.10: 3D-Rohdaten-Ansicht: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

nicht erhobene Felder (n/a) werden weggelassen (kein erfundener 0-Wert).

Achse	Observer-Feld
search_algo=k_ary/cache_traversal=linear_fanout/mapping=direct_placement/path_compression=path_compression_none/node_type=node4/memory_layout=	allocator
	alloc_cnt
	budget_reject
	bytes_alloc
	bytes_in_use
	dealloc_cnt
cache_traversal	fail
	batch_size
	batch_visited
	peak_tracked
	register
	resolve
concurrency	resolve_hit
	resolve_miss
	unregister
	acquire
	contention

Achse	Observer-Feld
filter	pattern_id
	release
	validation_fail
	checksum
	hash_probes
	neg
index_organization	pos
	probe
	checksum
	indirect_lookups
	predicate_evals
io_dispatch	records
	scan
	align_adjusts
	bytes
	checksum
mapping	dispatch_cnt
	rounds
	indirect_steps
	peak_mapped
	register
	resolve
memory_layout	resolve_hit
	resolve_miss
	reverse_lookup
	cache_lines
	checksum
	field_bytes
migration_policy	line_bytes
	records
	scan
	cold_votes
	decisions
	hot_votes
node_type	migrations
	tier_moves
	checksum
	find
path_compression	keys_stored
	queries
	bytes_saved
	checksum
	compress
persistence_target	cuts
	prefix_len
	bytes_staged
	checksum
	device_flushes
prefetch	records_staged
	rounds
	addrs_enqueued
	hot_path_hints
	max_queue_depth
queuing_q1	suggestions
	trigger
	get

Achse	Observer-Feld
queuing_q2	overflow
	peak_size
	put
	underflow
	decisions
	flush_complete
	full_flush
search_algo	no_flush
	partial_flush
	erase
	hit
	insert
	lookup
	miss
serialization	peak
	bytes
	checksum
	records
value_handle	serialize
	access
	indirect_deref
	peak_chain_depth
	version_strips

A.7 Latenz-Verteilung

Die Range-Balken zeigen je Suchverfahren und Operation die p50–p99-Spanne (Punkt bei p50, Whisker zu p99); da nur diese beiden Perzentile gemessen werden, wird bewusst *kein* Box-Plot mit interpolierten Quartilen gezeichnet. Die ECDF stellt die Verteilung der Gesamt-Latenz *über die Konfigurationen* dar (Anteil der Permutationen mit $ns_per_op \leq x$) — keine Per-Operation-CDF, da die Rohsamples je Operation nicht vorliegen.

Die Pareto-Streuung trägt schließlich die Median- gegen die Tail-Latenz auf (ein Punkt je Konfiguration und Operationsart). Sichtbar wird dadurch die untere linke Front: jene Konfigurationen, die in beiden Kennzahlen zugleich günstig sind und damit von keiner anderen Konfiguration in beiden Kennzahlen zugleich übertroffen werden.

A.8 Referenzbezogene Auswertung

Die folgenden Darstellungen setzen alle Messwerte zu *einer* Referenz ins Verhältnis. Diese Referenz ist eine gemessene Achsen-Ausprägung des Korpus selbst (Voreinstellung `search_algo/linear_scan`) und ausdrücklich *keine* externe Bibliotheks-Baseline: eine gemessene `std::map`-Leistungsserie existiert nicht, denn jedes `std::map` im Korpus ist das Konformitäts-Orakel des Prüf-Docks und damit ein Korrektheits-, kein Leistungs-Datenpunkt. Kommt die Referenz in einer Messreihe nicht vor, entfallen die Darstellungen dieses Abschnitts ersatzlos.

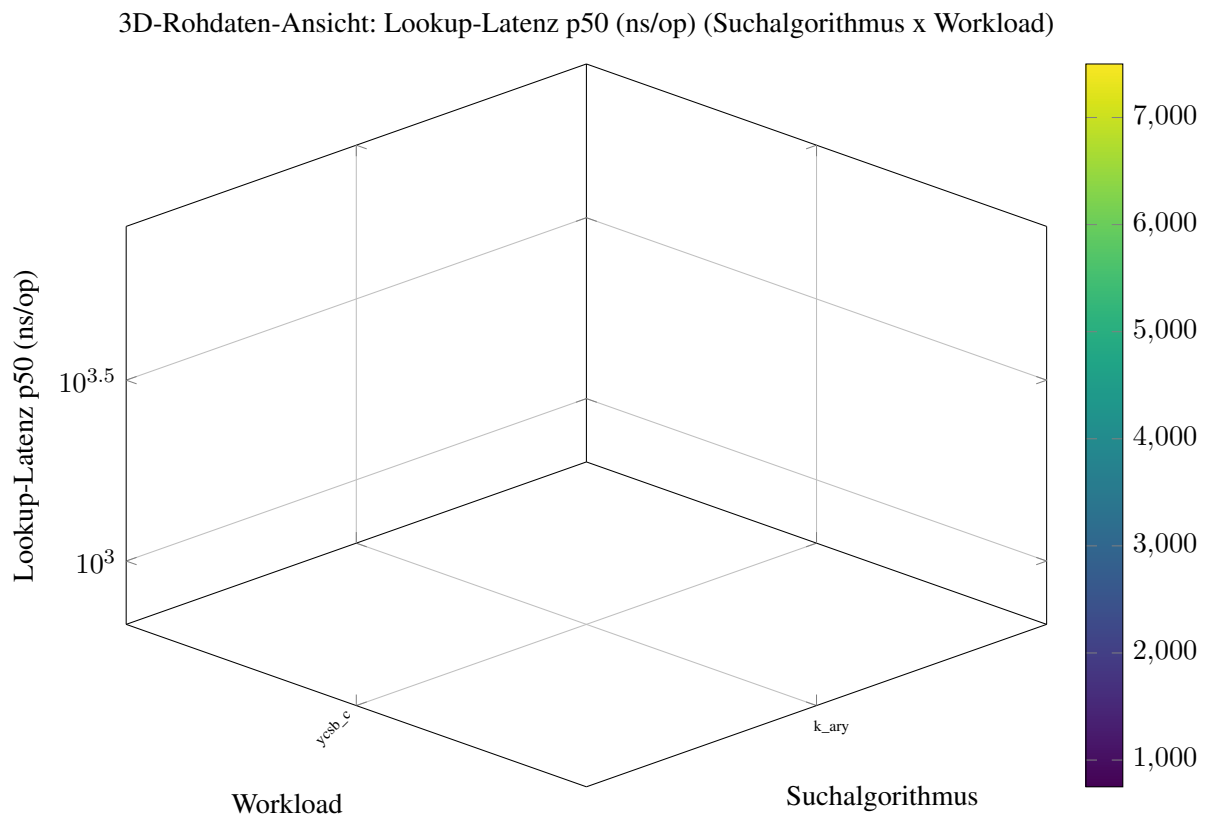


Abbildung A.11: 3D-Rohdaten-Ansicht: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: Erase-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen.)

Abbildung A.12: 3D-Rohdaten-Ansicht: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: Scan-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen.)

Abbildung A.13: 3D-Rohdaten-Ansicht: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

(Keine Messwerte: RMW-Latenz p50 (ns/op) wurde im vorliegenden Korpus nie ausgeführt. Diese Fläche wird ehrlich ausgelassen.)

Abbildung A.14: 3D-Rohdaten-Ansicht: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

A.8.1 Verhältnis-Matrizen

Je Schnittstellen-Funktion zeigt eine Matrix den Quotienten aus gemessenem Median und Referenz-Median. Die divergente Farbskala ist um die Gleichheit (Verhältnis 1) zentriert, sodass Vorteil und Nachteil gegenüber der Referenz ohne Zahlenvergleich unterscheidbar sind. In der Analyse-Rolle löst diese Matrix die Heatmap aus Abschnitt A.3 ab; die rohe Heatmap bleibt als Rohwert-Ansicht daneben bestehen.

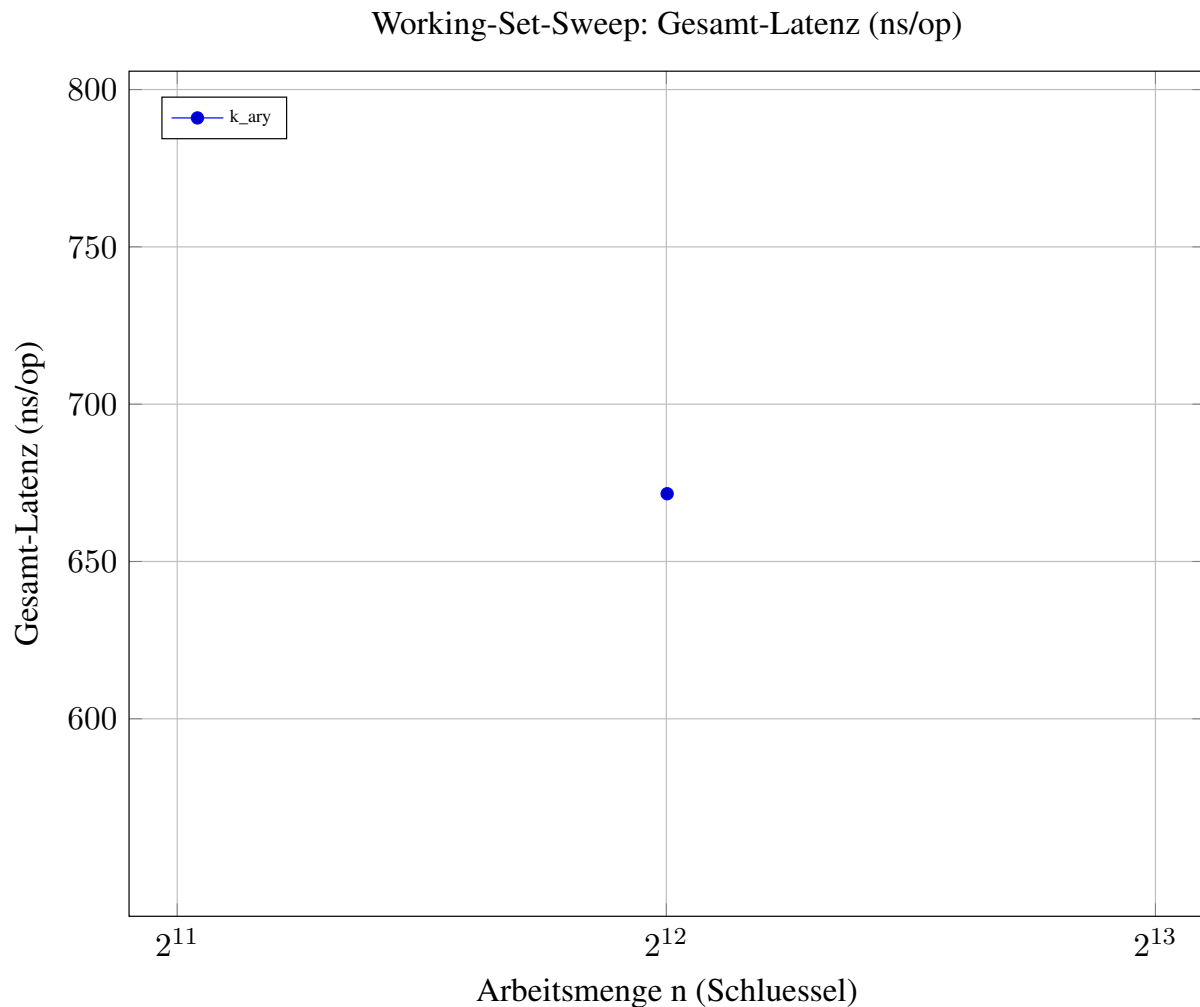


Abbildung A.15: Working-Set-Sweep: Gesamt-Latenz (ns/op)

A.8.2 Normalisierte Balken

Dieselbe Aussage über die Lastprofile hinweg verdichtet: ein Balken je Suchalgorithmus, die Referenzlinie liegt bei 1. Wo die Verhältnis-Matrix das Lastprofil-Muster zeigt, gibt der Balken die Gesamtaussage je Suchalgorithmus.

*(Hier stünden die normalisierten Balken der **Gesamt-Latenz** (`ns_per_op`) — ein Balken je Suchalgorithmus, Referenzlinie bei 1. Sie liegen nicht vor: sie teilen den Nenner der Verhältnis-Matrizen, und die Referenz-Ausprägung ist in den bisher vorliegenden Messreihen nicht mitgemessen.) (Hier stünden dieselben Balken für die **p50-Latenz von insert**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von lookup**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von erase**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von scan**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von rmw**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.)*

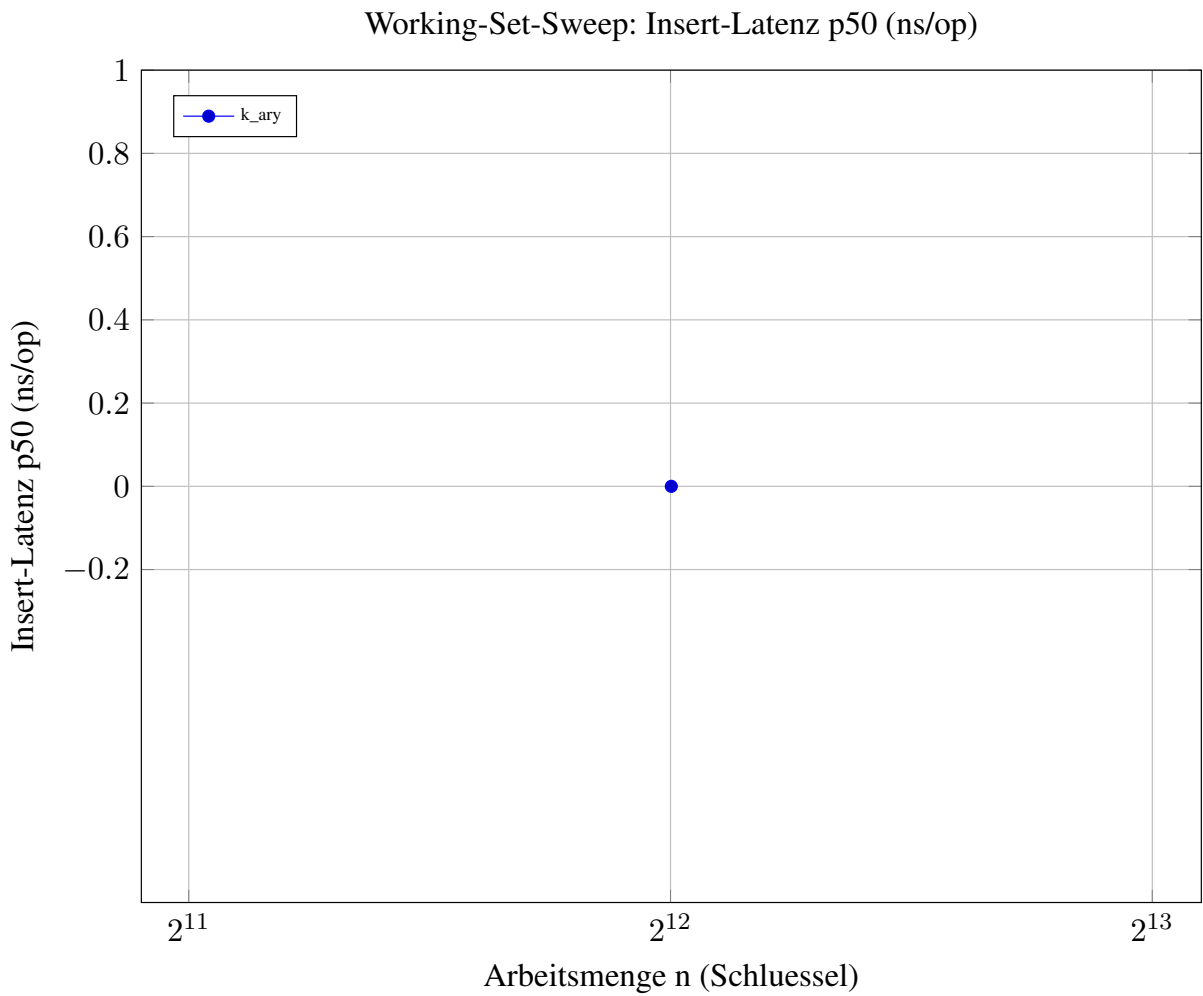


Abbildung A.16: Working-Set-Sweep: Insert-Latenz p50 (ns/op)

A.9 Achsen-Austauschbarkeit

Die vier folgenden Langtabellen belegen die Austauschbarkeit je variabler Achse anhand der Geschwister-Paar-Differenzen (Median rel. Δ ns/op bezüglich v): `search_algo` (Tabelle A.5), `node_type` (Tabelle A.6), `memory_layout` (Tabelle A.7) und `prefetch` (Tabelle A.8). Am wenigsten konfundiert sind `search_algo` und `prefetch`.

Tabelle A.5: Achsen-Austauschbarkeit: `search_algo` (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
-----	------	----------	---------------------------	----------------------	----------	-----	----------

Tabelle A.6: Achsen-Austauschbarkeit: `node_type` (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
-----	------	----------	---------------------------	----------------------	----------	-----	----------

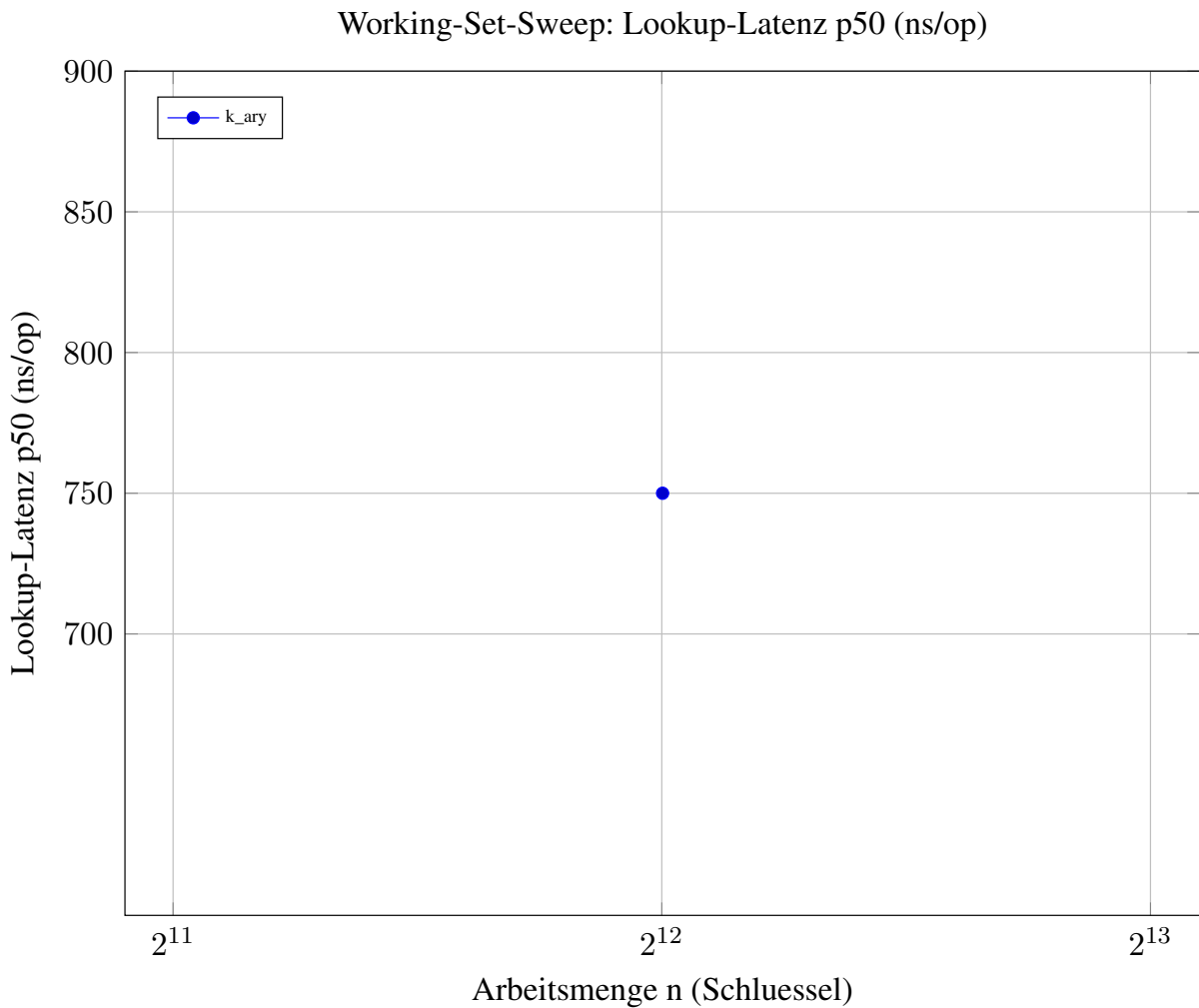


Abbildung A.17: Working-Set-Sweep: Lookup-Latenz p50 (ns/op)

Tabelle A.7: Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
-----	------	----------	---------------------------	----------------------	----------	-----	----------

Tabelle A.8: Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 0 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
-----	------	----------	---------------------------	----------------------	----------	-----	----------

Der folgende Forest-Plot fasst die Austauschbarkeit der `ns_per_op`-Kennzahl zusammen (Punkt = Median rel. Δ , Whisker = IQR, 0-Referenzlinie; kleine- n -Zeilen ausgegraut). Der Whisker liegt symmetrisch um den Median (über die IQR-Breite), da das Aggregat nur die IQR-Breite, nicht die Quartilsgrenzen trägt.

(Der Forest-Plot erscheint nach dem nächsten Messlauf.)

Der zweite Forest-Plot zeigt dieselben Aggregate, dreht aber alle Paare auf *eine* feste Referenz-Achsenausprägung im Sinne von Abschnitt A.8, sodass der Maßstab des Vergleichs eindeutig benannt ist statt

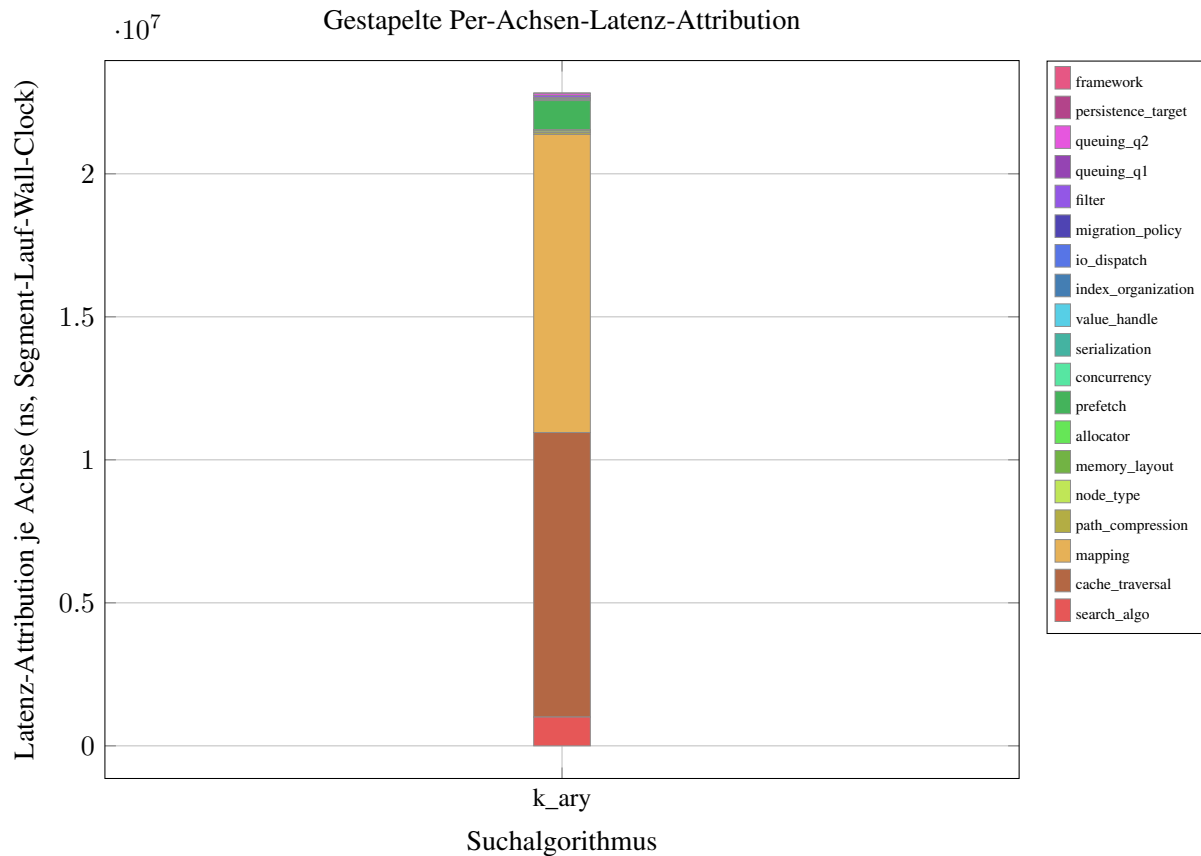


Abbildung A.18: Gestapelte Per-Achsen-Latenz-Attribution

offen zu bleiben. Die Drehung rechnet das relative Delta exakt um — aus d wird $d' = -d/(1 + d)$, denn mit dem Bezugswert wechselt auch der Nenner; eine bloße Vorzeichen-Umkehr wäre hier ein Rechenfehler. Punkte, die sich so nicht abbilden lassen, werden ausgelassen und nicht ersetzt.

(Der referenzbezogene Forest-Plot erscheint nach dem nächsten Messlauf, sofern die Referenz-Ausprägung gemessen wurde.)

A.10 Limitierungen

Tabelle A.9 führt alle nicht-gefixten Vorbehalte der Messreihen auf, damit kein Befund still verfällt; Zeile 1 benennt die verbleibenden PMC-Lücken der Kernmetrik (L2 und Coherence ohne portablen Zähler; Energie best-effort).

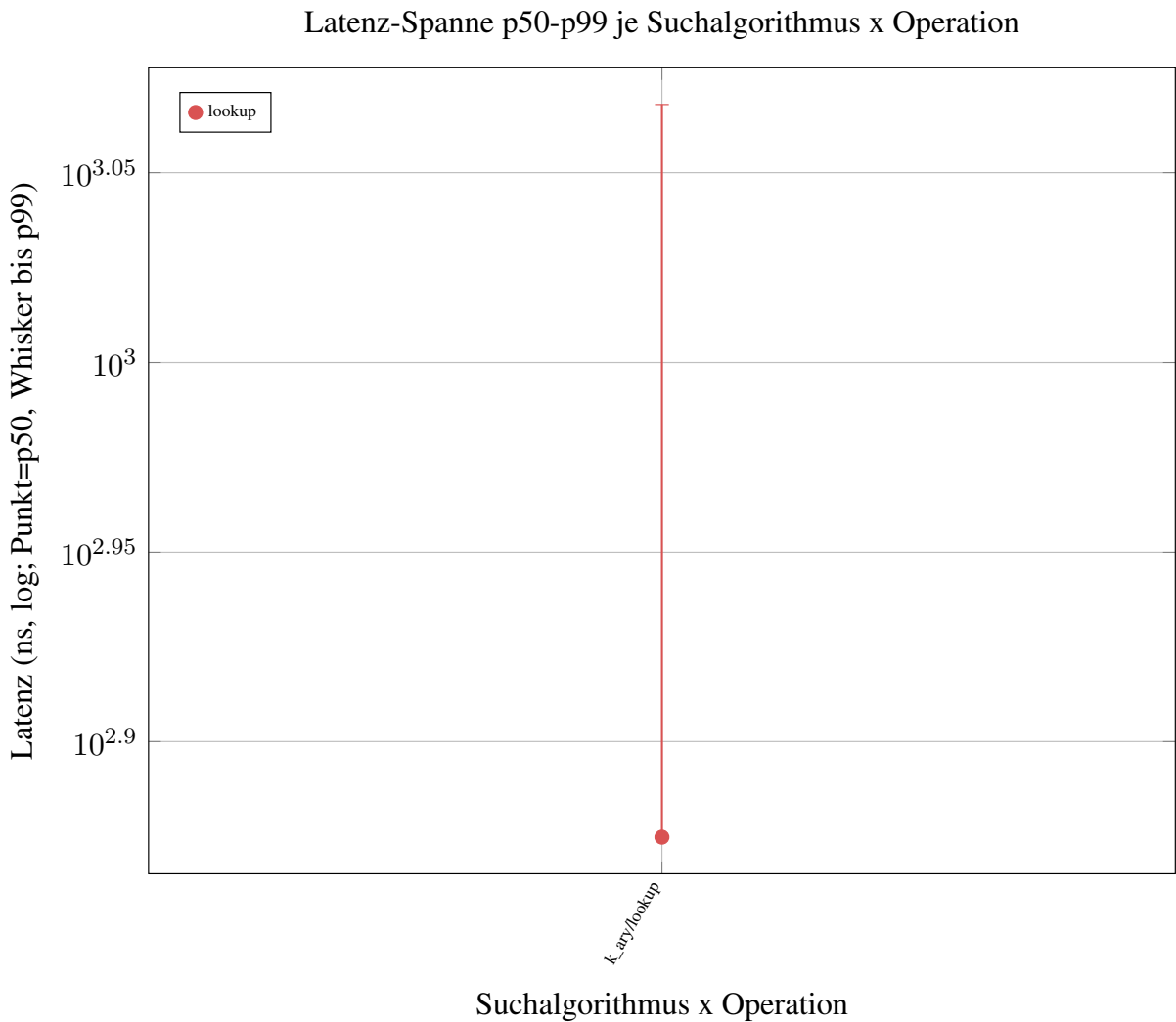


Abbildung A.19: Latenz-Spanne p50-p99 je Suchalgorithmus x Operation; Punkt=p50, Whisker=p99.

Tabelle A.9: Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)		
#	Vorbehalt	Status / Konsequenz
1	Cache-Misses (Kernmetrik): L2 + Coherence strukturell 0; L1/L3/dTLB nur mit gesetztem PMC-Schalter erhoben; Energy best-effort	Ohne COMDARE_ENABLE_PMC (Build-Default OFF, damit <code>test:unit</code> ohne HW-Zugriff baut) liefert <code>NullPmcSource available=false</code> , und alle HW-Spalten bleiben 0 – so entstanden die tabellierten Nullen. Mit dem Schalter erhebt <code>LinuxPerfPmcSource</code> L1-, L3- (Last-Level) und dTLB-Misses real über <code>perf_event_open(2)</code> (Kernel-Syscall, keine Vendor-Lib, kein PAPI nötig – PAPI bleibt optionaler Sekundärpfad). L2 und Coherence-Invalidierungen bleiben mangels portablem generischem Zähler auch dann strukturell 0; L3 kann auf einzelnen AMD-Plattformen zusätzlich beim Öffnen des Zählers scheitern; Energie läuft best-effort über RAPL-sysfs und bleibt ohne Zonen-Leserecht leer. Der Intel-PCM-Windows-Treiber steht für die Windows-Lane weiterhin aus (#26/P4). Die verbleibende L2/Coherence-Lücke ist nur indirekt über den Wall-Clock-Proxy (<code>seg_memory_layout_ns/ns_per_op</code>) beobachtbar.

Fortsetzung nächste Seite

Tabelle A.9 – Fortsetzung

#	Vorbehalt	Status / Konsequenz
2	14 gepinnte Achsen = 0 Austauschbarkeits-Belege	Nur 4 der 18 Kompositions-Achsen variieren (search_algo, node_type, memory_layout, prefetch). Gepinnt (je 1 Wert): cache_traversal, mapping, path_compression, allocator, concurrency, serialization, value_handle, index_organization, io_dispatch, migration_policy, filter, queuing_q1, queuing_q2, persistence_target. (isa ist seit INC-2d die System-Achse target_isa und telemetry seit INC-2c keine binary_id-Kompositions-Achse mehr – beide zählen daher nicht mehr als gepinnte Organ-Achsen.)
3	Observer-Proben-Zähler (*_find/*_probe/*_get) by-design ≈ 1	Erwartetes Verhalten, kein Phantom: je Lookup eine Probe. Keine Aussage über interne Iterationen.
4	Honest-0 inaktiver Sub-Features gepinnter Strategien	concurrency/migration/io/value_handle melden 0 (Feature inaktiv), nicht Mess-Fehler.
5	Wall-Clock nicht bit-reproduzierbar	Seed steuert Keys (deterministisch), NICHT das CPU-Timing; Wall-Clock variiert lauf-zu-lauf.
6	RC-Dimension nur nominal	K1: $\times 18 \rightarrow \times 3$ degeneriert; RC-Achse liefert keine volle Ausprägungs-Breite.
7	2/21 Scan-Profile = No-Op (ausgeschlossen)	ycsb_e und lp_range_scan führen keinen echten Scan aus; bei scan-Diffs ausgeschlossen (19 valide Lastprofile).
8	Insert-Profile messen Upserts; stat_*-Spalten enthalten Load-Phase	Insert-Pfad ist faktisch Upsert; die stat_*-Zähler akkumulieren die Lade-Phase mit.
9	prefetch misst Key-Werte als Pseudo-Adressen (K9)	Prefetch-Achse interpretiert Schlüssel-Werte als Adressen; kein echter Speicher-Prefetch.
10	node_type + memory_layout: Q2-Schritt-4-Beschattung	Apparat-Artefakt möglich (search_organ_-Beschattung); search_algo + prefetch sind am wenigsten konfundiert.
11	container_-Spiegel (Weg-B): O(n)-Flatten+Rebuild als Zusatz-Apparat	Weg-B-Lebewesen (SOTA-Bäume/Tries/Hash) halten neben dem echten Such-Organ einen sortierten container_-Spiegel; dessen O(n)-Flatten+Rebuild (bei NEU-Inserts + erfolgreichen Erases; #211, bis-M3 ungefixt) ist ZUSÄTZLICHER Apparat-Aufwand, den die echte Baumstruktur nicht hat $\rightarrow$ die gemessene ns_per_op der Weg-B-Tiere ist um diesen Spiegel-Anteil überhöht. (Bei der Weg-A-Array-Familie ist derselbe O(n)-Rebuild der ECHTE Struktur-Kostenanteil, kein Zusatz-Artefakt.) Behebung mit #188/#211 + Nachfolge-Neubau (#215).
12	memory_layout-Effekt teils sub-noise	Teil der Layout-Differenzen liegt unter dem Mess-Rauschen $\rightarrow$ PMC extern-gated (#26).
13	Sekundäre Indirektions-Organen (cache_traversal/mapping): uint16-Schlüssel-Sättigung	Der uint64-Primärschlüssel wird beim Auto-Koppeln in die schmalere slot_index/key_type-Felder der GEPINNTEN cache_traversal-/mapping-Organen verengt (low-bits); bei records > Typbreite kollidieren deren Observer-Einträge. Betrifft NICHT den Primär-Lookup (uint64); reine Observer-Limitierung gepinnter Achsen (vgl. Zeile 2/4).
14	Kapsel-internes (verschlucktes) OOM in einer Mess-Op	Nach aussen geworfenes OOM entwertet die Messung jetzt hart (two_phase_valid=false). Ein vom Tier-Code INTERN per catch(...) gefangenes OOM (z.B. defensiver Memento-Verzicht) ist von aussen nicht beobachtbar $\rightarrow$ konservativ als Restvorbehalt dokumentiert, nicht erzwungen.
15	Timer-Overhead (clock::now()) nicht kalibriert	Die Wall-Clock-Segmentzeiten enthalten den nicht subtrahierten Eigen-Overhead der std::chrono::steady_clock::now()-Aufrufe (eine additive, lauf-stabile ns-Komponente je gemessenem Segment). Es findet KEINE Leer-Schleifen-Kalibrierung statt $\rightarrow$ dokumentierter Vorbehalt; betrifft den absoluten ns-Nullpunkt, nicht die relativen Achsen-Vergleiche (G5-Audit, MESS-MINOR-7).

Fortsetzung nächste Seite

Tabelle A.9 – Fortsetzung

#	Vorbehalt	Status / Konsequenz
16	Entwurfs-Etiketten = Namenskonvention, kein Mess-Einfluss	Einzelne Pattern-Etiketten im Code/Doku sind Benennungs-/Entwurfsvorbehalte ohne Mess-Wirkung: „hybrider Visitor“(real ein Memento, kein <code>accept()</code> /Host-Besuch), sowie Interpreter-/Template-/Decorator-Etiketten und <code>reserve-if</code> <code>constexpr</code> -Hygiene. Reine Naming-Konvention → als Vorbehalt dokumentiert, kein Einfluss auf Messwerte (G5-Audit, PATTERN-MINOR-1..7).

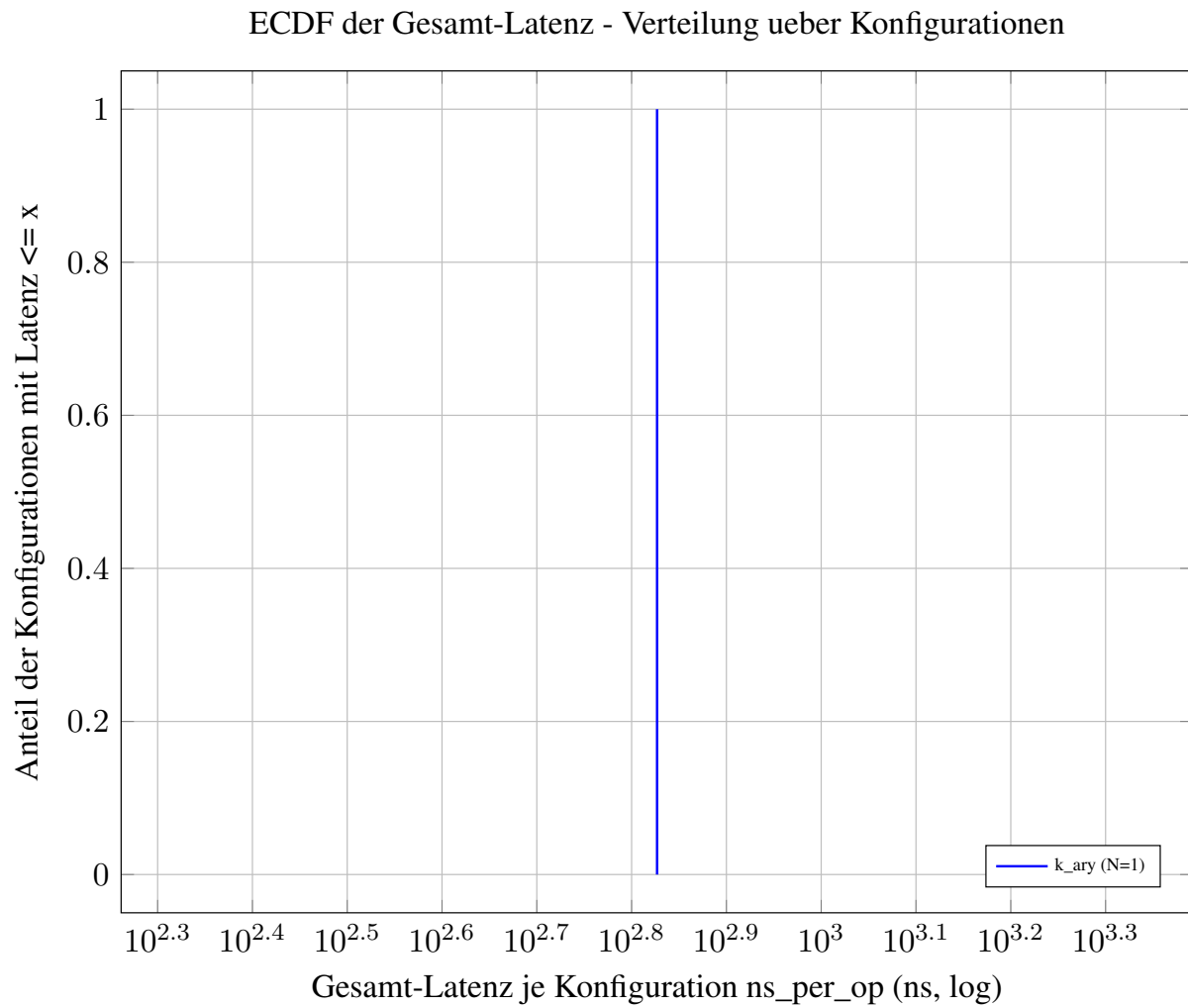


Abbildung A.20: ECDF der Gesamt-Latenz - Verteilung ueber Konfigurationen

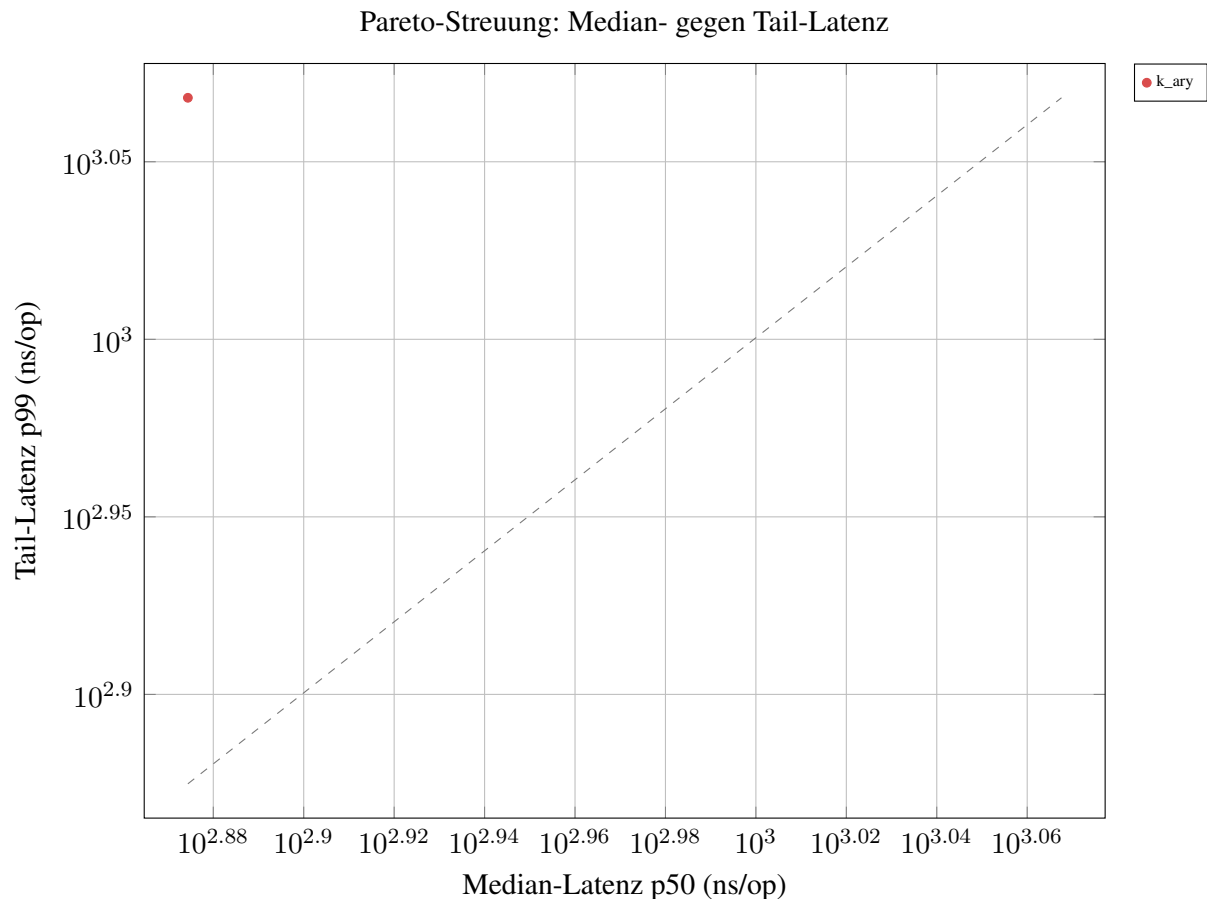


Abbildung A.21: Pareto-Streuung: Median- gegen Tail-Latenz (ein Punkt je Konfiguration und Op-Art; Diagonale = kein Tail-Aufschlag)

(Keine vergleichbaren Messwerte: Gesamt-Latenz (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.22: Verhaeltnis zur Referenz linear_scan: Gesamt-Latenz (ns/op) (1 = wie die Referenz)

(Keine vergleichbaren Messwerte: Insert-Latenz p50 (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.23: Verhaeltnis zur Referenz linear_scan: Insert-Latenz p50 (ns/op) (1 = wie die Referenz)

(Keine vergleichbaren Messwerte: Lookup-Latenz p50 (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.24: Verhaeltnis zur Referenz linear_scan: Lookup-Latenz p50 (ns/op) (1 = wie die Referenz)

(Keine vergleichbaren Messwerte: Erase-Latenz p50 (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.25: Verhaeltnis zur Referenz linear_scan: Erase-Latenz p50 (ns/op) (1 = wie die Referenz)

(Keine vergleichbaren Messwerte: Scan-Latenz p50 (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.26: Verhaeltnis zur Referenz linear_scan: Scan-Latenz p50 (ns/op) (1 = wie die Referenz)

(Keine vergleichbaren Messwerte: RMW-Latenz p50 (ns/op) liegt im vorliegenden Korpus fuer linear_scan nicht als Referenz vor (oder wurde nie ausgefuehrt). Diese Flaeche wird ehrlich ausgelassen, statt ein Verhaeltnis gegen eine fehlende Referenz zu behaupten.)

Abbildung A.27: Verhaeltnis zur Referenz linear_scan: RMW-Latenz p50 (ns/op) (1 = wie die Referenz)

B Code-Struktur

Dieser Anhang dokumentiert die reale Quellcode-Struktur der drei Repositorien aus Abschnitt 4.1 nachprüfbar auf Verzeichnis-Ebene. Erhoben wurde der Code-Stand am Stichtag 2. August 2026 per Verzeichnis-Listing (zwei bis drei Ebenen tief); maßgeblich sind die Commits 2b0a9bd8 (comdare-cache-engine), 32e8ffa (comdare-prt-art) und daee6c85 (probst-Diplomarbeit-cache-engine, Super-Repository). Build-Artefakte (build*/), IDE-Verzeichnisse und der vendorte Fremdcode unter ext/ sind ausgeblendet beziehungsweise zu einem Sammel-Eintrag zusammengefasst; alle übrigen genannten Pfade existieren im erhobenen Stand. Pfadangaben sind relativ zur jeweiligen Repository-Wurzel.

comdare-cache-engine — Bausteine-Bibliothek und Builder

Das Repository folgt dem Pitchfork-Layout: apps/ enthält die ausführbaren Programme, libs/ die Bibliotheken, libs/common den querschnittlichen Code. Der vendorte Fremdcode der A-/P-Serie liegt gesammelt unter ext/ und wird hier nicht aufgefächert.

```
comdare-cache-engine/                               Stand: Commit 2b0a9bd8
|-- adapters/                                       ABI-Adapter der A-/P-Serie
|                                                    (A01-hoard ... A20-dlmalloc, P01-ART ... P30-HazardPointers)
|-- apps/                                           Executables (Pitchfork)
|   |-- cache_engine_builder/   Builder-Hauptprogramm (main.cpp)
|   |-- perm_runner/           Runner fuer generierte Permutations-Module
|   |-- anatomy_codegen_tool/, catalog_codegen_tool/, adhoc_emitter/,
|       fl5_compare/, h2_score_akte_tool/, is_original_validator/
|                                   Codegen- und Pruef-Werkzeuge
|-- libs/                                           Bibliotheken (Pitchfork)
|   |-- cache_engine/           CacheEngine: Achsen-Bausteine + Builder-Bibliothek
|   |   |-- axes/               ein Unterverzeichnis je Achse (alloc, layout,
|   |   |                       prefetch_axis, telemetry_axis, concurrency_axis,
|   |   |                       persistence_target, ...)
|   |   |-- topics/, anatomy/, compositions/, profile_facade/
|   |   |                       Achsen-Dach (AxisKind), Anatomie-Kern,
|   |   |                       Kompositions- und Katalog-Schicht
|   |   |-- builder/           codegen/, permutation_loop/, module_loader/,
|   |   |                       experiment_driver/ (ExperimentDriver),
|   |   |                       experiment_tree/ (ExperimentPlanDirector),
|   |   |                       build_orchestrator/, pruef_dock/, bestandslog/,
|   |   |                       curve_fit/, ...
|   |   |-- include/cache_engine/abi/   ABI-Header (baustein_variants.hpp,
|   |   |                               resolve_baustein.hpp, module_abi_v1.hpp,
|   |   |                               anatomy_version_stamp.hpp, ...)
|   |   |-- include/cache_engine/measurement/   Realm-Wurzeln + Registries
|   |   |                                       (system_axis.hpp, measurement_*_registry.hpp,
```

```

|   |   |               ram_frequency_reading.hpp, spd_ddr5_parser.hpp,
|   |   |               ram_probe_chain.hpp, hardware_probe_factory.hpp)
|   |   |-- algorithm_profiles/ sota/ (33 Profil-Dateien *.profile.xml +
|   |   |               sota_h2_scores.xml), allocators/, load_profiles/,
|   |   |               thesis_profiles/, permutation_axes.xml
|   |-- common/         Querschnitt: config/, measurement/, platform/,
|   |               serialization/ (xml_config_parser/), succinct/
|   |-- execution_engine/ ExecutionEngine-Bibliothek (result_aggregator)
|   |-- search_engine/   SearchEngine-Bausteine (node/, page/, traversal/,
|   |               operations/, memory_layout/, value_handle/, ...)
|   |-- test_infra/      benchmark_suite/, test_data/, workload_generator/
|-- ext/                vendorte Fremd-Repositorien der A-/P-Serie (Sammel-Eintrag)
|-- tests/              unit/, integration/, generic_module_tests/,
|               module_specific_tests/, fixtures/
|-- tools/              Hilfswerkzeuge (abi_stability_test/, gen_golden_fullpilot/,
|               best_binary_selector/, p27_bundle_finder/, ycsb_cli/,
|               axis_registry_gen/ u.a.; der fruehere permutation_codegen-
|               Zweitkanal ist entfernt, vgl. Kapitel 4)
|-- docs/               Architektur-, Status- und Session-Dokumentation

```

Komponente	Pfad	Zweck
CacheEngineBuilder	apps/cache_engine_builder/	Builder-Executable (Konfiguration zu Permutations-Modulen)
ExperimentDriver	libs/cache_engine/builder/ experiment_driver/	sieben-phasiger Experiment-Treiber (host-seitig)
Codegenerierung	libs/cache_engine/builder/codegen/	generate_module_ from_profile samt Templates
Permutations- Schleife	libs/cache_engine/builder/ permutation_loop/	Aufzählung der Achsen-Permutationen
Modul-Loader	libs/cache_engine/builder/ module_loader/	Laden der erzeugten Module
ABI-Header	libs/cache_engine/include/ cache_engine/abi/	u. a. baustein_ variants.hpp, resolve_baustein.hpp
XML-Config-Parser	libs/common/serialization/ xml_config_parser/	Parser der Profil- und Experiment-XMLs
Achsen-Bausteine	libs/cache_engine/axes/	je Achse ein Unterverzeichnis
SOTA-Profil	libs/cache_engine/ algorithm_profiles/sota/	33 Profil-Dateien (*profile.xml): die 30 eigenständigen SOTA-Algorithmus-Profile (Abschnitt 3.2) plus drei Dateien für die Nebenläufigkeits-Disziplinen olc, rcu und hazard_ pointers
Adapter A-/P-Serie	adapters/	ABI-Adapter über dem ext/-Fremdcode (10 A-, 12 P-Einträge)

comdare-prt-art — Prüfling

Der Prüfling ist als Header-Bibliothek unter `prt_art/include/prt_art/` organisiert; jeder Baustein liegt in einem eigenen thematischen Unterverzeichnis.

```
comdare-prt-art/                                Stand: Commit 32e8ffa
|-- CMakeLists.txt, comdare_pruefling.cmake    Build-Einstieg (CMake)
|-- prt_art/
|   |-- include/prt_art/    Bausteine des Prueflings (Header)
|   |   |-- identity/      prt_art_search_engine.hpp (PrtArtSearchEngine),
|   |   |                  prt_art_execution_engine_adapter.hpp,
|   |   |                  prt_art_pruefling_factory.hpp
|   |   |-- allocator/     pool_descriptor.hpp, pool_router.hpp, pool_set.hpp
|   |   |-- concurrency/   olc_with_reserved_blocks.hpp
|   |   |-- prefetch/      path_oriented_prefetch.hpp, redirect_prefetch.hpp,
```

```

| | | distance_estimator.hpp
| | |-- memory_layout/ multi_level_layout.hpp u.a.
| | |-- measurement/ density_tracker.hpp, hypothesis_metrics.hpp
| | |-- telemetry/ leaf_only_counter.hpp
| | '-- ... nodes/, slots/, default_lookup/, value_handle/,
| | value_buffer/, internal_search/,
| | serialization/, ...
| |-- legacy_reimpl/ 14 Re-Implementierungen im Nummernband P11-P27
| | (P11-CSS-tree ... P27-Zhang-ASPLoS-Hierarchical)
| |-- algorithm_profiles/ prtart_pruefling.profile.xml,
| | permutation_axes_extension.xml
| |-- codegen/ + permutations_codegen/ Codegen-Vorlagen des Prueflings
| |-- registry_gen/, src/ Registry-Generator und Uebersetzungseinheiten
| '-- tests/unit/ modulnahe Tests
|-- tests/unit/ repositoriumsweite Unit-Tests
|-- cmake/ Build-Hilfsdateien
'-- docs/ sessions/ (Entwicklungs-Dokumentation)

```

Komponente	Pfad	Zweck
PrtArtSearchEngine	prt_art/include/prt_art/identity/ prt_art_search_engine.hpp	hybride Engine des Prüflings (Vector/Map/Tuple-API)
ExecutionEngine- Adapter	prt_art/include/prt_art/identity/ prt_art_execution_engine_ adapter.hpp	ABI-Anbindung an die Mess-Infrastruktur
Pool-Allokator OLC	prt_art/include/prt_art/allocator/ prt_art/include/prt_art/ concurrency/	Deskriptoren der 4+1+2-Pools OLC mit reservierten Blöcken
Prefetch	prt_art/include/prt_art/prefetch/	pfadorientiertes Prefetch, Redirect-Prefetch
MultiLevel-Layout	prt_art/include/prt_art/ memory_layout/	multi_level_ layout.hpp u. a.
DensityTracker, H1-H3	prt_art/include/prt_art/ measurement/	density_tracker.hpp, hypothesis_ metrics.hpp
legacy_reimpl	prt_art/legacy_reimpl/	14 Vergleichs-Re- Implementierungen (P11-P27)
Prüflings-Profil	prt_art/algorithm_profiles/	Profil- und Achsen-Erweiterungs-XML

probst-Diplomarbeit-cache-engine — Super-Repository

Das Super-Repository bündelt Mess-Treiber, Experiment-Konfigurationen, Auswertungs-Pipeline und Manuskript; die beiden Code-Repositories sind unter `Code/external/` als Submodule eingebunden.

```

probst-Diplomarbeit-cache-engine/          Stand: Commit daee6c85
|-- Code/
| |-- 01_sample_data_generator/  Erzeugung von Beispieldaten
| |-- 02_messung_driver/         Mess-Treiber (main.cpp, v32_orchestrator.hpp,
| |                             plugin_loader.hpp, measurement_writer.hpp, ...)

```

```

| |-- 03_binary_to_csv/      Auswertungs-Stufe 1: binaere Messdaten -> CSV
| |-- 04_csv_to_latex/      Auswertungs-Stufe 2: CSV -> LaTeX-Tabellen
| |-- 05_diagram_generator/  Auswertungs-Stufe 3: Diagramm-Erzeugung
| |-- 06_latex_to_pdf/      Manuskript-Build (build_thesis.sh/.bat)
| |-- 07_tier_binary_report/ Bericht ueber die Tier-Mess-Binaries
| |-- 08_appendix_generator/ In-Process-Generator des Mess-Anhangs
| |                          (ersetzt generate_wide_appendix.ps1)
| |-- 09_tex_formatter/     satz-neutraler LaTeX-Quelltext-Formatierer
| |-- experiment_config/    config_a_prt_art_vs_sota.xml,
| |                          config_b_cache_engine_perms.xml,
| |                          config_c_merge_alt_neu.xml, messreihen.xml,
| |                          sample_data/
| |-- external/             Submodule: comdare-cache-engine, comdare-prt-art
| '-- tests/, cmake/, tools/ repoweite Tests, Build- und Hilfsskripte
|-- thesis/diplomarbeit/    Manuskript: diplomarbeit.tex, Makefile,
|                            kapitel/{de,en}/, anhang/{de,en}/,
|                            tabellen/, tikz/
|-- docs/                  Ledger, Architektur- und Session-Dokumentation
|-- Forschungsarbeiten/    Literatur-Sammlung je Achse (Sammel-Eintrag)
|-- measurement/           Mess-Ergebnis-Rueckschrieb (persistierte
|                            Messlaeufer der CI)
'-- Messdaten-Backup/      additive Sicherung der Roh-Messdaten

```

Komponente	Pfad	Zweck
messung_driver	Code/02_messung_driver/	Mess-Treiber: lädt Permutations-Module, orchestriert die Messreihen
Experiment-Konfigurationen	Code/experiment_config/	XML-Konfigurationen A/B/C und messreihen.xml
binary_to_csv	Code/03_binary_to_csv/	Auswertungs-Pipeline, Stufe 1
csv_to_latex	Code/04_csv_to_latex/	Auswertungs-Pipeline, Stufe 2
diagram_generator	Code/05_diagram_generator/	Diagramm-Erzeugung aus CSV-Daten
latex_to_pdf	Code/06_latex_to_pdf/	Manuskript-Build-Skripte
Submodule	Code/external/	Einbindung der beiden Code-Repositorien
Manuskript	thesis/diplomarbeit/	LaTeX-Quellen der Arbeit (DE/EN)
Messdaten-Backup	Messdaten-Backup/	additive Sicherung der Roh-Messdaten

C Glossar

Dieses Glossar fasst die in der Arbeit verwendeten Abkürzungen und zentralen Begriffe zusammen.

ABI Application Binary Interface — die binär-stabile Schnittstelle, über die der Builder die Permutations-Module (Lebewesen) lädt.

Achse Eine orthogonale Teilentscheidung („Organ“) eines Suchalgorithmus; das Framework führt achtzehn Organ-Kompositions-Achsen (T0–T17) neben den drei System-Haupt-Achsen und dem Mess-Realm (vgl. Abschnitt 3.4.1).

Adaptiv vs. dynamisch vs. messgetrieben Begriffsabgrenzung für die Anpassung des Algorithmus-Verhaltens. *Adaptiv*: Anpassung anhand laufzeit-beobachteter Datenmerkmale (z. B. adaptive Knotengrößen in ART). *Dynamisch*: die Laufzeit-Auswahl des fertig kompilierten Permutations-Moduls über den Permutations-Identifizier (Modul-Ladung) — alle achtzehn Organ-Achsen sind je Binary compile-time-gebacken; hinzu kommen laufzeit-variierbare Parameter dynamischer Sub-Achsen (Prefetch-Distanz, Stapelgröße, Inline-Schwelle, Pool-Budget; vgl. Abschnitt 3.4), keine Laufzeit-Wahl einzelner Achsen-Strategien. *Messgetrieben* (*measurement-driven*): Auswahl der Konfiguration auf Basis empirisch erhobener Messreihen — das Leitmotiv dieser Arbeit; der zugehörige Auswahl-Mechanismus ist in einem ersten Inkrement implementiert (`best_binary_selector`: Ranking und Auslieferung der besten Permutation je Metrik aus den Mess-CSVs), die vollautomatische, lastprofilübergreifende Auswahl bleibt Ausblick.

Anatomie Der *Körper* eines Lebewesens: die feste Komposition seiner achtzehn Achsen-Organen und vor allem deren *Verdrahtung* — die generalisierten Interfaces, über die ein Organ die Dienste der übrigen nutzt (etwa das gemeinsame Allokations-Interface). Kein zweites Modell neben dem Suchalgorithmus, sondern dessen Innenstruktur (vgl. Abschnitt 3.3.1).

ART Adaptive Radix Tree [LKN13]; Referenzentwurf mit adaptiven Knotengrößen (Node4/16/48/256).

Autonome Heuristik-Profil-Auswertung (*autonomous profile-driven heuristic evaluation*)
Der angestrebte Mechanismus, der aus den Messreihen je Lastprofil automatisch die beste Achsen-Konfiguration ableitet und als wiederverwendbare Heuristik-Richtlinie ablegt (Ziel-Mechanismus: ML-Klassifikation von Lastprofil-/Achsen-Features auf Konfigurationen). Erste Bausteine sind implementiert — die automatische Best-Binary-Auswahl aus Messreihen (`best_binary_selector`) und die Extraktion/Ablage als XML-Lastprofil-Ergebnis —; die ML-Klassifikation selbst ist weiterhin Ausblick (vgl. Abschnitt 6.3).

AVX, AVX-512 x86-SIMD-Erweiterungen (256- bzw. 512-Bit).

built_stem Feld der Plan-Textform (Fassung v1.1), das je Bau-Schritt den real gebauten Datei-Stamm ausweist — erzeugt von derselben Funktion, die auch der Bau-Orchestrator beim echten Bau ruft; Beleg des *einen* Hauptkanals (vgl. Abschnitt 4.3.2).

CacheEngineBuilder Das autonome Plattform-Ausmess-System der Kette: Wurzel des System-Realms; baut und stempelt die Tier-Binaries je Plattform und führt die Sieben-Phasen-Pipeline über den `ExperimentDriver` aus (vgl. Abschnitte 3.4.5 und 4.3.2).

CLU Cache-Line-Utilization — Anteil genutzter Bytes je geladener Cache-Line.

CPI, IPC Cycles per Instruction bzw. Instructions per Cycle.

CRTP Curiously Recurring Template Pattern — C++-Idiom für statischen Polymorphismus.

Drei-Ebenen-Modell Die Schichtung der einen Architektur: *Gattung* (Ebene 1, das äußere Algorithmus-Interface — Map, Container, Graph), *Gattungs-Unterklasse* (Ebene 2, fester Achsen-Satz) und *Organe/Achsen* (Ebene 3). Siehe Abschnitt 3.3.2.

dTLB Daten-Translation-Lookaside-Buffer; cacht Daten-Adressübersetzungen.

Gattung Ein nach außen sichtbares Interface (Map, Container, Graph) — die oberste Ebene (Ebene 1) des Frameworks; siehe *Drei-Ebenen-Modell*.

Gattungs-Unterklasse Die zweite Ebene unter einer Gattung mit festem Achsen-Satz (Code-Begriff: *Tier-Unterklasse*, `AnatomyGenus`). In der biologischen Lesart das „Säugetier“ = `SearchAlgorithm` (volle Achtzehn-Achsen-Anatomie); unter `Container` die Unterklassen `Set`, `Sequence`, `Adapter` und `View` („Vogel“, „Reptil“, „Wirbelloses“, „Pflanze“).

GoF „Gang of Four“ — die kanonischen Entwurfsmuster nach Gamma et al. [GHJV94].

HBM High-Bandwidth Memory.

HOT Height Optimized Trie [BZP⁺18].

Komplex-Achse Klammer über einer festen Rekombination mehrerer Haupt-Achsen, die gemeinsam wie *eine* Achse auftreten: die äußere Klammer über Ziel-ISA, Betriebssystem und Erweiterungs-Hardware sowie die innere Komplex-Achse `target_isa` (RAM-Frequenz, CAS-Latenz, CPU-Fabrikation); im Identitäts-Stempel belegt eine Komplex-Achse genau ein Feld (vgl. Abschnitt 3.4.2).

Lebewesen Ein vollständiger Suchalgorithmus als Komposition aller Achsen (metaphorischer Code-Begriff; technischer Identifier: `SearchAlgorithm-Instanz`).

LOUDS Level-Ordered Unary Degree Sequence — succinct Baum-Kodierung (Jacobson 1989).

MESI, MOESI, MESIF Cache-Kohärenzprotokolle (Grund-Protokoll bzw. AMD-/Intel-Varianten).

Mess-Realm Der dritte Achsen-Realm neben Organ- und System-Achsen: die Belange des Messens selbst, verwurzelt am *Experiment-Planer*; erste Haupt-Achse ist das Last-Rahmenwerk, und alle Mess-Realm-Achsen sind strikt binary-id-neutral (vgl. Abschnitt 3.4.1).

NINE Non-Inclusive-Non-Exclusive — Inklusions-Politik des L3 in Intel-Server-Prozessoren.

NUMA Non-Uniform Memory Access.

OLC Optimistic Lock Coupling — optimistische Nebenläufigkeits-Technik (ARTSync, P08) [LABN16].

Organ Metaphorischer Begriff für eine *Achse* (Ebene 3 des Drei-Ebenen-Modells); ein Suchalgorithmus besitzt achtzehn Organ-Achsen.

Passungs-Stempel *Entwurf* eines Abgleichs, der die einkompilierte Plattform-Klasse einer Tier-Binary bei jedem Mess-Lauf gegen das Laufzeit-Verdikt der Hardware-Erkennung prüft; im Stand dieser Arbeit nicht gebaut (vgl. Abschnitt 6.1).

Planer Der Experiment-Planer (`ExperimentPlanDirector`): löst die Anwender-XML gegen die drei Angebots-Kataloge auf, wählt die Mess-Achsen und steuert die Builder-Aufträge der Kette; Wurzel des Mess-Realms (vgl. Abschnitt 3.4.5).

PMC (Hardware) Performance Monitoring Counter.

Provenienz-Stufe Vertrauensstufe eines erhobenen Hardware-Werts — konfiguriert-gemessen, SPD-JEDEC-Basiswert oder deklariert-nicht-gemessen —, die jedes Ergebnis samt Erhebungszeitpunkt mitführt; eine niedrigere Stufe verdrängt eine höhere nur mit mitwandernder Kennzeichnung (vgl. Abschnitt 3.4.3).

Prüf-Dock Das nach außen sichtbare Mess-Interface einer Gattung, über das der Builder ein Lebewesen lädt, durch die Interface-Operationen treibt, vermisst und persistiert (vgl. Abschnitt 3.3.2).

Prüfling Der testweise einzugliedernde neue Suchalgorithmus (hier PRT-ART); wird vom Builder gegen die Kernbestandteile der Cache-Engine kompiliert.

PRT-ART Probst-Redirect-Tree-/ART-Hybrid — der Prüfling dieser Arbeit.

Rang Zentralitäts-Einordnung der Stand-der-Technik-Veröffentlichungen (Rang-1/2/3); erklärt in Abschnitt 1.3.

RCU Read-Copy-Update — Reklamations-Schema (McKenney, P29) [McK01].

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — Cache-Ersetzungs-Heuristik [JTJE10].

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — Index-Benchmark [KMR⁺19].

SOTA State of the Art — der Stand der Technik.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM- (SVE, NEON) bzw. RISC-V- (RVV) SIMD-Erweiterungen.

TLB Translation Lookaside Buffer.

XML-Lastprofil-Ergebnis (*XML load-profile result*) Das in der Aufgabenstellung (Teilaufgabe 7) geforderte Persistenz-Format, in dem die für einen Gesamtalgorithmus extrahierte, je Architekturfokus wiederverwendbare Heuristik-Richtlinie unter Referenz der angewendeten Lastprofile abgelegt wird; dient als Eingabe der autonomen Heuristik-Profil-Auswertung. Import-Parser und Export-Writer

samt Extraktor sind implementiert (round-trip-fähiges Schema `comdare_load_profile`); als Ausblick verbleibt die darauf aufsetzende autonome Heuristik-Auswertung (ML-Klassifikation).

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

Zwei-Gate-Protokoll Das Abnahme-Protokoll je Implementierungs-Paket: Gate 1 ist das scharfe Doppellauf-Gate des Engine-Repositories (ausschließlich „0 failed“, ohne Ausnahmeliste), Gate 2 das Integrations-Gate des Super-Repositories mit namentlich eingefrorener Ausnahmemenge (vgl. Abschnitt 4.4).

D Bausteine-Matrix

Dieser Anhang enthält die vollständige Bausteine-Matrix, auf die der Stand der Technik (Abschnitt 3.2) verweist. Jede Achse ist dabei ein *Organ* des Lebewesens: Ein konkreter Suchalgorithmus (technisch `SearchAlgorithm`, metaphorisch ein Lebewesen) ist die feste Komposition seiner achtzehn Organ-Achsen (T0–T17), von denen jede einem Organ entspricht; die je Achse aufgeführten Baustein-Varianten sind die austauschbaren Ausprägungen dieses Organs, und ihre Permutation gleicht einem genetischen Experiment am Lebewesen. Die *Anatomie* dieses Lebewesens ist dabei die Verdrahtung *zwischen* den Organen: Achsen-Algorithmen nutzen die Interfaces anderer Achsen (so stellt die Allokations-Achse ein gemeinsames Allokations-Interface für die übrigen Organe bereit). Je Achse sind der Zweck, die *statischen* Sub-Achsen (übersetzungszeitlich wählbare Varianten-Familien) und die *dynamischen* Sub-Achsen (Laufzeit-Parameter) sowie die konkreten Baustein-Varianten mit ihrer Quelle (Paper-Kennung oder Code/Baseline) aufgeführt. Die Angaben sind aus dem Code der `comdare-cache-engine` und gegen die Paper-Provenienz-Map (Doc 18) verifiziert.

Über die achtzehn Organ-Slots hinaus dokumentiert dieser Anhang die beiden übrigen Achsen-Kategorien der Dreiteilung (vgl. Abschnitt 3.4.1). Der *System-Realm* umfasst genau drei Haupt-Achsen in kanonischer, code-gewachter Ordnung — Ziel-ISA (`target_isa`), Betriebssystem (`operating_system`) und Erweiterungs-Hardware (`external_utils`) — zuzüglich der äußeren Komplex-Klammer `build_target_complex`, die ausdrücklich keine vierte Haupt-Achse ist; die Reihenfolge ist dabei kein redaktioneller Entschluss, sondern folgt der einen Ordnungs-Quelle des Codes. Der *Mess-Realm* umfasst die Mess-Kategorien, das Mess-Tooling und das Last-Rahmenwerk als Meta-Meta-Achse am Realm-Ende. In der Anatomie-Metapher sind System- und Mess-Achsen kein weiteres Organ, sondern das *Blut*: nicht permutierbar, nicht Teil des Permutationsbaums und strikt neutral gegenüber der Binary-Identität. Die Angaben beider Sphären sind zusätzlich gegen die drei Achsen-Registries des Codes verifiziert (`cache_engine_axis_registry.xml` für den Organ-Realm, `system_axis_registry.xml` und `measurement_axis_registry.xml`). Historisch konservierte Alt-Sektionen — der SIMD-Angebots-Katalog unter der Erweiterungs-Hardware sowie der Seitentyp-Katalog Build-PG und die Plattform-Hardware-Strategie Build-HW — sind als solche markiert und stehen am Ende des jeweiligen Blocks.

D.1 T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)

Diese Achse ist das *Such-Organ* (T0) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Systematische Permutation von 21 konkreten Suchalgorithmen und Such-Strukturen (S01–S21: 17 Basis-Bausteine — Array-basiert, sortierte Vektoren, Original-Paper-Wrapper, Such-Methoden wie `k-ary/Interpolation`, sowie geordnete Strukturen wie `Skip-Liste/B-Baum` — plus 4 opt-in per-K-k-ary-Wrapper `K2/K4/K8/K16`, standardmäßig abgeschaltet) zur messären Quantifizierung des Einflusses der

Such-Paradigma-Wahl auf Performance-Charakteristiken im einheitlichen `std::map`-Interface. Sub-Achsen klassifizieren nach Zugriffsdichte (SA1 dense, SA2 sparse, SA3/SA4 multilevel), ermöglichen Compliance-Nachweise über Paper-Mixins (Habich-Anforderung: `is_original`-Markierung pro Funktion/Wrapper), und identifizieren SIMD-fähige sowie store-traversierbare Varianten.

Statische Sub-Achsen

ID	Name	Beschreibung
SA1	Direkt-adressiert hohe Dichte (Dense Access)	Array-Familie: direkt-adressiert ($O(1)$ lookup), hohe Füllungsdichte $\geq 50\%$. Umfasst <code>Array256SearchAlgo</code> (256-Slot compact array, ART Node256 Re-Impl) und <code>Array65535SearchAlgo</code> (uint16-basiert, mid-density 25-50 % direkt-adressiert).
SA2	Sparse/Geordnet (Sparse Access)	Sortierte Vektoren, geordnete Such-Methoden und tree/hash-Strukturen: niedrige Füllungsdichte, Range-Scan-Support, variable $O(\log n)$ - $O(1)$ Lookup. Umfasst <code>VectorU8U8SearchAlgo</code> , <code>Original-Paper-Wrapper</code> (ART/HOT/START/Wormhole/SuRF), Such-Methoden (k-ary, Interpolation, Eytzinger, Hash), sowie Strukturen (SkipList, BST, B-Baum).
SA3	Multi-Byte-Diskriminatoren (Multilevel Access)	Mehrbyteige Such-Strukturen mit indizierten Sub-Diskriminatoren (z.B. START mit Cost-DP-Mehrebenen). <code>VectorU16U16SearchAlgo</code> (u16-Keys, sorted lower_bound) und <code>OriginalStartSearchAlgo</code> (Paper P05) gehören hier.
SA4	Direkt-adressiert mittlere Dichte (Direct Multibyte)	Teilt die Direkt-Adressierungs-Zugriffsmuster (wie SA1) mit Multi-Byte-Granularität (uint16-Größe) und mittlerer Füllungsdichte 25–50 % (im Gegensatz zu SA1 $\geq 50\%$). Beispiel: prt-art <code>Array65535</code> (REV 6 §5.17). Diese Subachse wurde hinzugefügt, um die SA1-Abstraktion nicht zu brechen, aber die Density-Variante zu erfassen.

Bausteine

Variante	Beschreibung	Quelle
<code>Array256SearchAlgo</code> (S01)	ART Node256 direkt-adressierte Re-Implementierung: <code>std::array<std::optional<u64>, 256></code> mit $O(1)$ Lookup/Insert/Erase. Cache-Line-aligned, SIMD-fähig, kein Heap-Allocation. Baseline für dichte Zugriffe.	Code/Baseline (Leis ICDE 2013 ART Paper P01)
<code>VectorU8U8SearchAlgo</code> (S02)	Sparse sortiertes Key-Value-Vektor-Paar: <code>std::vector<u8> keys + std::vector<u64> values</code> mit <code>std::lower_bound</code> . $O(\log n)$ Lookup, $O(n)$ Insert/Erase. Konzept-äquivalent zu HOT-k-Constrained. SIMD-fähig (Bitmask-Scan).	Code/Baseline (Binna SIGMOD 2018 HOT Paper P02)
<code>VectorU16U16SearchAlgo</code> (S03)	Sparse sortiertes u16-Vektor-Paar: <code>std::vector<u16> keys + std::vector<u64> values</code> . Vereinfachtes START-Konzept (Multi-Byte-Diskriminatoren ohne Cost-DP). $O(\log n)$ Lookup, $O(n)$ Insert/Erase.	Code/Baseline (Fent ICDEW 2020 START Paper P05)
<code>Array65535SearchAlgo</code> (S09)	Mid-Density direkt-adressiert uint16-basiert (25–50 % Füllungsdichte). Aus prt-art Legacy-Code (REV 6 §5.17) migriert. $O(1)$ Lookup aber mit Speicher-Overhead vs. SA1 densen Arrays.	Code/Baseline (prt-art F.6-Migration)
<code>OriginalArtSearchAlgo</code> (S04)	ART Paper-Bindung (<code>unodb::db::ist_original_module()=true</code> (4/4 Funktionen original). Paper-Mixin via SHA256-validierte Tool-Codegenerierung. Habich-Compliance: <code>get_compiler()= 'gcc-9.5'</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)

Variante	Beschreibung	Quelle
OriginalHotSearchAlgo (S05)	HOT Paper-Bindung (HOTRowex): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, aber erase/clear Lücken (HotRowex ist append-only; Re-Impl fallback). Paper P02.	P02 (Binna et al. SIGMOD 2018, ISC ISC)
OriginalStartSearchAlgo (S06)	START Paper-Bindung (sosd-Adapter): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear Lücken (kein remove im sosd-Adapter). Paper P05, Multi-Byte Diskriminatoren.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)
OriginalWormholeSearchAlgo (S07)	Wormhole Paper-Bindung: <code>is_original_module()=false</code> (3/4 original). put/get/del original, clear Lücke. Hybrid Hash+Trie+B+ Index. GPL-3.0-lizenziert (blockiert Original-Code-Linking für Thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF Paper-Bindung (FST mit LOUDS-Bitmap): <code>is_original_module()=false</code> (1/4 original, nur lookup). Succinct Range Filter, Multi-Header-Bibliothek. Paper P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)
KArySearchAlgo (S10)	k-ary Search Such-METHODE (Schlegel/Gemulla/Lehner DaMoN 2009): K-Wege-Partition statt Halbierung bei der Binärsuche. K ist <code>iterable_aspect</code> ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ Iterationen, parallele ILP-Vergleiche. Array-Familie aber kein treues dediziertes Traversal-Organ (SA2-Klassifikation). C++23-Re-Impl, <code>is_original=false</code> .	Schlegel et al. DaMoN 2009 [SGL09]
InterpolationSearchAlgo (S11)	Interpolationssuche Such-METHODE (Perl/Itai/Avni CACM 1978): verteilungsbewusste Positionsschätzung statt blinder Halbierung. $O(\log \log N)$ avg bei Gleichverteilung, $O(N)$ worst-case. Nicht SIMD-vektorisierbar (datenabhängiger Index). C++23-Re-Impl.	Perl/Itai/Avni CACM 1978 [PIA78]
EytzingerSearchAlgo (S12)	Eytzinger/BFS-Layout branch-free Suche (Khuong/Morin JEA 2017): Cache-Layout-Such-METHODE mit branch-free Kern. Keys in BFS-Ordnung des impliziten Suchbaums. Fuer große Arrays laut Paper schnellste Allzweck-Layout. C++23-Re-Impl (Experiment-Harness ohne Standard-Lizenz).	Khuong/Morin JEA 2017 [KM17b]
SkipListSearchAlgo (S13)	Skip-Liste probabilistische geordnete STRUKTUR (Pugh CACM 1990): $O(\log n)$ erwartet search/insert/erase ohne explizites Rebalancing. Jeder Knoten hat 1..kMaxLevel Forward-Zeiger (Münzwurf Probability 0.5). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, value-semantisch). Lehrbuch-Algorithmus, C++23-Re-Impl.	Pugh CACM 1990 [Pug90]
HashSearchAlgo (S14)	Open-Addressing-Hashtabelle UNGEORDNET (Knuth TAOCP 3 §6.4): Fibonacci-Hash, Linear Probing, Tombstone-Erase bei Resize. $O(1)$ erwartet lookup/insert bei <code>load_factor < 0.7</code> . Keine Range-Scan-Unterstützung (distinkte Klassifikation). Korrekte Tombstone-Markierung zur Probe-Ketten-Integrität. C++23-Re-Impl.	Knuth TAOCP 3 §6.4 [Knu98]
LinearScanSearchAlgo (S15)	Unsortierter linearer Scan Baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup auf unsortiertem KV-Array. Einfachste Vergleichs-Nullpunkt-Struktur: insert $O(1)$ -amortisiert, erase $O(1)$ via swap-and-pop. Store-traversierbar (Array-Familie). Keine Range-Scan-Unterstützung.	Code/Baseline (Leis ICDE 2013 ART Paper P01 Node4)

Variante	Beschreibung	Quelle
BinarySearchTreeSearchAlgo (S16)	Unbalancierter Binärer Suchbaum (Knuth TAOCP 3 §6.2.2 Hibbard-Deletion): deterministische Vergleichs-Baum-Baseline (vs. SkipList probabilistisch). Index-basiert (<code>std::vector<Node></code> mit <code>uint32 indices</code> , Free-List). Hibbard-erase (3 Fälle: Blatt/1 Kind/2 Kinder via In-Order-Nachfolger). $O(n)$ worst-case bei sortiertem Input.	Knuth TAOCP 3 §6.2.2 [Knu98]
BTreeSearchAlgo (S17)	Balancierter Block-orientierter Mehrwege-B-Baum (Bayer/McCreight Acta Inf. 1972 / CLRS Kap. 18): $t=4$, daher [3..7] Keys pro Knoten, bis zu 8 Kinder. IMMER balanciert ($O(\log n)$ guaranteed). Index-basiert, <code>alignas(64)</code> -Knoten (Cache-Line-Alignment). Top-Down Splitten beim Insert, Borrow/Merge beim Erase. Das DB-Index-Arbeitspferd.	Bayer/McCreight Acta Inf. 1972 / CLRS [BM72]

Beitragende Quellen: P01, P02, P05, P07, P10, k-ary (Schlegel et al. 2009), Interpolation (Perl et al. 1978), Eytzinger (Khuong/Morin 2017), Skip-List (Pugh 1990), Hashing/BST (Knuth TAOCP 3), B-Baum (Bayer/McCreight 1972) [LKN13, BZP⁺18, FJKN20, WNJ19, ZLL⁺18, SGL09, PIA78, KM17b, Pug90, Knu98, BM72].

D.2 T1 Cache-Speicher-Traversierung (Fanout-Lookup)

Diese Achse ist das *Cache-Traversierungs-Organ* (T1) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Die Achse `axis_03b` definiert drei distinkte Strategien zur Schlüssel-Auflösung in kleinen Fanout-Arrays (typisch: B+-Tree oder Trie Inner Nodes). Sie spannt ein Spektrum von linearer Suche ($O(N)$, cache-freundlich, klein), über multiplikatives Hashing mit Open-Addressing ($O(1)$ amortisiert, Knuth-Fibonacci), bis zur sortierten Binärsuche ($O(\log N)$, geordnete Traversierung, B+-Tree-Standard). Alle drei sind Lehrbuch-Baselines ohne Original-Code-Linking, implementiert als reine C++23-Re-Implementierungen.

Statische Sub-Achsen

ID	Name	Beschreibung
CT1	Lineare Zugriffe	Linear-scan-basierte Lookup-Methoden auf kleinen, unsortierten Fanout-Arrays. Charakterisiert durch $O(N)$ -Auflösung mit einfachen Vergleichen, typisch für Nodes mit Fanout < 32 in B+-Trees. Sub-Achse für <code>LinearFanout</code> und <code>BinarySearchFanout</code> (beide: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-basierte Zugriffe	Hash-Lookup mit Open-Addressing und linearem Probing. $O(1)$ amortisierte Auflösung, charakterisiert durch Fibonacci-Multiplikator $(11400714819323198485 \approx 2^{64}/\varphi)$, Power-of-2 Bucket-Count, dynamisches Rehashing bei Load-Factor > 0.7 . Nur Hash Lookup-Wrapper. Sub-Achse: <code>axis_tag = hash_access_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
LinearFanout	Unsortierte Fanout-Traversal via <code>std::find_if</code> und linearem Scan. $O(N)$ Lookup, kein Hash-Overhead, cache-freundlich für kleine N (< 32). Klassische B-Tree-Baseline (Bayer/McCreight 1972). Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
HashLookup	Fibonacci-Hash open-addressing mit linearem Probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortisiert, dynamisches Rehashing (verdoppelt Capacity bei $\text{load} \geq 0.7$), iterable Laufzeit-Kapazität (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23-Re-Impl, keine Original-Code-Bindung. Gehört zu Sub-Achse CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
BinarySearch Fanout	Sortierte Fanout-Traversal via <code>std::lower_bound</code> . $O(\log N)$ Lookup bei sortierter Ordnung, bewahrt Schlüssel-Sequenzierung für Range-fähige Routing-Layer. Klassische B+-Tree-Inner-Node-Suche (Bayer/McCreight 1972), dritte distinkte Strategie der Achse. C++23-Re-Impl (R7.2, 2026-05-29), <code>is_original=false</code> . Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Beitragende Quellen: Bayer/McCreight 1972, Knuth TAOCP Vol. 3 [BM72, Knu98].

Verwandte Literatur: P17, P21, P32 [BDFC05, CGM01, SKK⁺25].

D.3 T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)

Diese Achse ist das *Mapping-Organ* (T2) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Vermittlung zwischen logischen Slot-Indizes und physischen Speicher-Offsets in Trie-Inner-Nodes und Speicher-Pools. Die Achse bietet zwei grundsätzlich verschiedene Zuordnungsstrategien: direkte absolute Offsets (lineare Packing) oder pool-relative Offsets mit automatischem Rebase-Mechanismus zur Unterstützung von Speicher-Umallokation und Persistent-Data-Structures.

Statische Sub-Achsen

ID	Name	Beschreibung
MP1	Direct-Access-Mapping (MP1)	Direkter Slot-zu-Absolute-Offset-Mapping ohne Indirection. Wrapper-Familie für klassisches Array-of-Pointers-Layout wie in B-Tree Inner-Nodes (Bayer/McCreight 1972). Einfache Cache-Locality, $O(N)$ Lookup per <code>std::find_if</code> .
MP2	Pool-Relative-Mapping (MP2)	Slot-zu-Pool-Relativem-Offset-Mapping mit Based-Pointer-Arena-Pattern. Alle Offsets relativ zu dynamischem <code>pool_base_address</code> gespeichert. Ermöglicht $O(1)$ Rebase bei Speicher-Umallokation — zentrale Eigenschaft für Persistent-Data-Structures (Driscoll et al. 1989) und CustomAligned Structure-Pattern in prt-art.

Bausteine

Variante	Beschreibung	Quelle
DirectPlacement	Wrapper für direktes Slot-zu-Absolute-Offset-Mapping (lineares Packing). <code>std::vector<pair<slot_index_type, offset_type>;</code> <code>resolve_offset/register_slot</code> via <code>linear std::find_if</code> . Verwendungskontext: prt-art <code>Inode::placement_page_ +</code> Mapping-Table. Speicher: 16-bit Slot-Index + <code>size_t</code> Offset pro Entry. Keine Pool-Basis-Abhängigkeit.	CE-Baseline
PoolRelative	Wrapper für Pool-Relatives-Offset-Mapping mit $O(1)$ -Rebase-Fähigkeit. <code>std::vector<pair<slot_index_type, relative_offset></code> + <code>mutable pool_base_address</code> . Bei Pool-Umlokation: <code>rebase(new_pool_base)</code> setzt nur <code>pool_base_neu</code> — alle relativen Offsets bleiben gültig. Verwendet bei prt-art CustomAligned Structure + Versioning-Szenarien (Persistent Data Structures nach Driscoll/ Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Beitragende Quellen: — (CE-Baseline, eigene Wrapper).

Verwandte Literatur: Bayer/McCreight 1972, Driscoll et al. 1989, P03, P07 [BM72, DSST89, MKM12, WNJ19].

D.4 T3 Pfadkompression

Diese Achse ist das *Pfadkompressions-Organ* (T3) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Definiert die Kompressions-Granularität und -Strategie für Schlüsselpfade in Trie-Strukturen. Steuert, wie innere Knoten redundante Byte-Sequenzen oder signifikante Bits gemeinsamer Präfixe speichern und navigieren, um Speicher- und Cache-Effizienz zu optimieren.

Statische Sub-Achsen

ID	Name	Beschreibung
pc1	Kompressions-Granularität (PC1)	Definiert die minimale Einheit der Pfad-Kompression: Bit-weise (Patricia), Byte-weise (ART) oder keine Kompression (raw path). Bestimmt die Trie-Höhe und navigationsoptimale Codeblöcke im Traversal.
pc2	Sprung-Strategie (PC2)	Definiert das Navigations-Pattern bei inneren Knoten: Linear über 8 Bytes (ByteWise), Single-Bit-Vergleich (Patricia) oder direkter Sprung ohne Zwischenschritte. Beeinflusst Branch-Dichte und Prefetch-Verhalten.
pc3	Decode-Komplexität (PC3)	Klassifiziert die Laufzeit-Komplexität beim Extrahieren von Präfix-Informationen: $O(1)$ für direkte Extraktion, $O(\log n)$ für struktur-basierte Suche, $O(n)$ für sequenzielle Prüfung. Wichtig für Operationen wie <code>common_prefix_len()</code> und Schlüssel-Vergleiche.

Bausteine

Variante	Beschreibung	Quelle
PathCompression None	Keine Pfad-Kompression; rohe Schlüsselpfade werden vollständig und unkomprimiert in Knoten gespeichert. Dient als Mess-Baseline mit <code>compression_ratio=1.0</code> . Verwendet als Kontrollfall zur Quantifizierung der Speicher- und Höhen-Einsparungen komprimierender Strategien.	Code/Baseline

Variante	Beschreibung	Quelle
ByteWisePath Compression	Byte-für-Byte-Kompression (ART-Stil); speichert bis zu 7 Präfix-Bytes pro Knoten in einem kompakten u64-Wort mit Längenetikett im High-Byte. Reduktion der Trie-Höhe um typischerweise Faktor 2 (<code>compression_ratio=0.5</code>). Ursprung: Leis et al., ART ICDE 2013 (P01). Verwendet Goldstandard-Operationen <code>common_prefix_len()</code> und <code>cut()</code> für Navigations-Optimierung.	P01
PatriciaPath Compression	Single-Bit-Split-Pfadkompression (Patricia-Trie); navigiert bitweise statt byteweise, reduziert Trie-Höhe um typischerweise Faktor 3 (<code>compression_ratio=0.3</code>). Ursprung: Morrison PATRICIA JACM 1968; modern implementiert in HOT (Binna SIGMOD 2018, P02) und Wormhole (Wu EuroSys 2019, P07). F15-operative Primitiven: <code>key_split_bit()</code> für MSB-first Bit-Extraktion, <code>path_descend_scan()</code> für goldstandard-konforme <code>_scan</code> -Signatur mit schlüssel-abhängiger variabler Tiefe.	P02

Beitragende Quellen: P01, P02, P07, Morrison 1968 [LKN13, BZP⁺18, WNJ19, Mor68].

Verwandte Literatur: P04, P05, P09 [BFTV24, FJKN20, Jac89].

D.5 T4 Knotentypen (Node-Type-Achse)

Diese Achse ist das *Knotentyp-Organ* (T4) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Die Achse `axis_04_node_type` definiert die vier Knotenformate des Adaptive Radix Tree (ART) als austauschbare Varianten: Node4, Node16, Node48 und Node256. Jeder Knotentyp bietet eine andere Balance zwischen Speicher, Cache-Effizienz und Suchgeschwindigkeit. Diese Achse bildet die Basis für adaptive Baum-Reorganisationen und ist zentral für das Design der Suchalgorithmen.

Statische Sub-Achsen

ID	Name	Beschreibung
NT1	Kapazitätsklasse (NT1)	Klassifizierung nach Fanout-Größe und Slotanzahl: klein (Node4 mit 4 Slots, linear search), mittel (Node16 mit 16 Slots, SIMD-binary-search) bis maximal (Node256 mit 256 Slots, direct-addressed).
NT2	Zugriffsmuster (NT2)	Art der Schlüsselsuche innerhalb eines Knotens: linearer Scan (Node4), binäre Suche mit SIMD (Node16), indirekte Indizierung (Node48) oder direkte Adressierung (Node256).
NT3	Kompaktheit (NT3)	Speicherlayout und Dichte: spärlich und cache-aligned (Node4), dicht mit Indirection (Node48) oder maximal direkt (Node256), mit Auswirkungen auf Speicherverbrauch und Cache-Line-Misses.

Bausteine

Variante	Beschreibung	Quelle
Node4NodeType	Kleiner ART-Knoten mit 4 Slots und linearer Suche. Ideal für spärlich besetzte obere Baumebenen, passt komplett in eine Cache-Line (64 Byte). Implementiert als CE-Reimplementierung der ART-Spezifikation mit <code>node_find_scan()</code> als linear scan über sortierte Schlüssel.	P01

Variante	Beschreibung	Quelle
Node16NodeType	Mittlerer ART-Knoten mit 16 Slots, optimiert für SIMD-basierte binäre Suche (SSE2 PCMPESTRM / AVX2 VPCMPESTRM). Balance zwischen Speicher und Suchgeschwindigkeit; größte Kapazität ohne Indirection. CE-Re-Impl der Leis-ICDE-2013-Spezifikation.	P01
Node48NodeType	Großer ART-Knoten mit 48 Slots, nutzt Indirection: 256-Byte ChildIndex (Byte→Slot-Mapping, 0=leer) plus 48er ChildArray. Trade-off zwischen Speicher (vs Node256 direct) und Cache-Misses (vs Node4 linear). Doppelter Indirektions-Zugriffsmuster (Index + Child).	P01
Node256NodeType	Maximaler ART-Knoten mit 256 Slots, direkt adressiert ohne Index. O(1) Lookup (kein Scan, kein Index-Touch). Hoher Speicherverbrauch (256 Pointer), aber optimale Suchgeschwindigkeit für dicht besetzte Knoten. Default für untere Baumebenen.	P01
Internal/Border	Knotentyp-Paar des Masstree (Trie aus B ⁺ -Bäumen): Internal-Knoten navigieren die 8-Byte-Schlüssel-Slices der B ⁺ -Ebene, Border-Knoten bilden die Blatt-Ebene mit den Werten bzw. dem Übergang zur nächsten Trie-Schicht. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P03 (Mao et al. 2012) [MKM12]
Decision/Span	Knotentyp-Paar des B ² -Baums (B ⁺ mit eingebetteten binären In-Knoten, cache-bewusst): Decision-Knoten tragen die binäre Verzweigungs-Entscheidung innerhalb der Seite, Span-Knoten fassen zusammenhängende Schlüsselbereiche zusammen. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P06 (Schmeisser et al. 2022) [SSL ⁺ 21]
LOUDS-Dense/Sparse	Knotentyp-Paar des succinct Range-Filters SuRF: LOUDS-Dense kodiert die oberen, dicht besetzten Trie-Ebenen bitmap-basiert mit direkter Adressierung, LOUDS-Sparse die unteren, spärlich besetzten Ebenen byte-sequenziell succinct. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P10 (Zhang et al. 2018) [ZLL ⁺ 18]
NodeTypeSlotStore	Storage-Organ: node_type-getriebener boundierter Slot-Speicher. Erstreckt sich über std::array mit Kapazität N: :max_capacity() über einen gemeinsamen uint64-Key. Erfüllt das StorageOrgan-Concept und ist Drop-in-Substrat für ComposedSearch. Baseline-Variante ohne Layout/Allocator-Integration.	Code/Baseline
ComposedStore	3-Achsen-Storage-Organ: node_type(N) ⊕ layout(L) ⊕ allocator(A). Unboundierter vector über Allocator A; layout fließt als statisches cache_line_size und via optionale organ_scan_field_sum() ein. Real messbar (F15-Brücke).	Code/Baseline
NodeChunkedStore	3-Achsen-Storage-Organ mit Laufzeitwirkung der node_type-Achse: Slots liegen in Chunks der Kapazität N: :max_capacity(). Chunk-Count = Zahl der node-großen Allokationen; damit ist die node_type-Achse REAL messbar (Node4 viele Chunks, Node256 wenige). Beweis für Delegation.	Code/Baseline
LayoutAware ChunkedStore	Layout-ehrende Variante von NodeChunkedStore: speichert Records am layout-getriebenen Stride (eff_stride). Jeder Record belegt eff_stride Bytes (Key 0..8, Value 8..16, Rest Padding). Macht memory_layout-Achse echt messbar, OOB-sicher für scan-Methoden.	Code/Baseline
ObservableNodeType	ObservableAxis-Hülle um eine node_type-Strategie. Reicht die statische API durch (max_capacity/name/topic_tag) + bietet observe_node_find() als Instanz-Driver. Trackt find_count, keys_stored, queries_run, last_checksum. Format-divergente Lookup-Latenz messbar (Out-of-Process-Pfad).	Code/Baseline

Beitragende Quellen: P01 [LKN13], P03 [MKM12], P06 [SSL⁺21], P10 [ZLL⁺18].

D.6 T5 Speicher-Layout (Memory Layout)

Diese Achse ist das *Speicher-Layout-Organ* (T5) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Diese Achse klassifiziert die räumliche Organisation von Datensätzen im Speicher und beeinflusst damit Cache-Effizienz, SIMD-Vectorisierbarkeit und False-Sharing-Charakteristiken. Sie erfasst orthogonal vier Design-Dimensionen: (1) Alignment-Strategien (Cache-Line-Padding vs. Strict-Packing), (2) Datenorganisation (zeilenorientiert AoS vs. spaltenorientiert SoA vs. Hybrid AoSoA), (3) Packing-Dichte (dense vs. succinct bit-packed), (4) Stride-Muster (sequenziell vs. interleaved). Die Layout-Wahl ist ein primärer Treiber für Cache-Lokalität und SIMD-Fähigkeit bei Index-Traversal.

Statische Sub-Achsen

ID	Name	Beschreibung
HM1	Alignment-Strategie	Entscheidung zwischen Cache-Line-gepaddetem Layout (False-Sharing-Vermeidung) und Strict-Packing (Speicherkompaktheit). Bestimmt Stride-Overhead bei sequenzieller Traversal.
HM2	Daten-Organisation	Grundlegende Speicher-Ordnung: Array-of-Structs (AoS, zeilenorientiert), Struct-of-Arrays (SoA, spaltenorientiert), oder Hybrid AoSoA (Block-basierte Mischung). Kritisch für Cache-Line-Ausnutzung bei Feld-selektiven Scans.
HM3	Packing-Dichte	Dichtegrad der Speichernutzung: von Standard-Records (8–64 Byte) bis zu kompaktem Bit-Packing (z.B. LOUDS-Bitmaps mit ~1 Bit pro Slot). Beeinflusst Decode-Latenz und L1/L2-Auslastung.
HM4	Stride-Muster	Muster der Speicheradressierung bei Traversal: Sequenzielle Strides (aufeinanderfolgende Records), Interleaved (z.B. um Hot-Cold-Daten zu trennen) oder Cache-Aware-Layouts (z.B. Eytzinger BFS). Optimiert Prefetch-Effizienz.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
CacheLineSize	Cache-Line-Größe (Unterachse KF-5)	Runtime-Parameter für die Zielplattform-Cache-Zeile: 64 Byte (Standard x86-64/ARM64), 128 Byte (Azure, manche Power ISAs) oder 32 Byte (embedded). Wird zur Compile-Time über <code>CacheLineConfig-NTTP</code> gewählt und beeinflusst Alignment/Padding-Overhead aller alignierten Layouts.

Bausteine

Variante	Beschreibung	Quelle
<code>AoSStrictMemoryLayout</code>	Array-of-Structs, kompakt gepackt ohne Cache-Line-Alignment. Vorteil: minimale Speichernutzung (<code>record_size</code> -Strides, kompakt). Nachteil: False-Sharing bei concurrent Writes, schlechte Cache-Hit-Quote bei schmalen Feldscans. Typisch für Wormhole-Index.	Code/Baseline
<code>CacheLineAlignedMemoryLayout</code>	AoS mit 64-Byte-Alignment per Record. Vermeidet False-Sharing durch Padding (<code>Stride = round_up(record_size, 64)</code>). Standard für ART/HOT/Masstree/START. Operativ: Bei <code>record_size=48</code> (Mess-Build) entsteht 16-Byte-Overhead pro Record → messbar höhere Latenz als <code>aos_strict</code> , echter Cache-Effekt.	Code/Baseline

Variante	Beschreibung	Quelle
SoAMemoryLayout	Struct-of-Arrays: Alle Felder spaltenweise hintereinander. Vorteil: Kontiguierliche Feldscans (z.B. nur Keys ohne Values) nutzen maximal Cache-Lines (16× weniger Cache-Lines als AoS bei Einfeld-Scan). Ideal für SIMD-Vectorization und OLAP-Indizes. Vorteil beschrieben in Abadi et al. SIGMOD 2008.	Abadi et al. 2008 [AMH08]
PackedBitmap MemoryLayout	Bit-packed succinct Layout (LOUDS / SuRF FST). Speichert n Einträge mit $\sim n \lg(\sigma)$ Bits statt bytes. Extreme Kompaktheit (32× weniger Cache-Lines als AoS), aber höherer Decode-Overhead per Lookup. Fundament: Jacobson LOUDS FOCS 1989 (P09). Moderne Impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: Hybrid-Layout mit Block-Breite W=8 (AVX2-Lanes). Innerhalb eines Blocks: SoA (Felder kontiguierlich). Über Blocks: AoS (Blocks sind gestrided). Vereint SIMD-Vectorisierbarkeit (SoA innerhalb mit räumlicher Lokalität (AoS über). Standard in HPC/SIMD-Engines. Konvention; kein einzelnes Ursprungspaper, aber ISPC-Idiom.	Code/Baseline (HPC/SIMD-Konvention)

Beitragende Quellen: P09, Abadi et al. 2008 [Jac89, AMH08].

Verwandte Literatur: P06, P11, P12, P32 [SSL⁺21, RR99, RR00, SKK⁺25].

D.7 T6 Allokator-Achse (T6)

Diese Achse ist das *Allokations-Organ* (T6) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Diese Achse stellt die Speicherverwaltungsstrategie für die Cache-Engine bereit. Sie definiert 25 konkrete Allocator-Wrapper für verschiedene Algorithmen und Techniken (von klassischen Baselines wie Buddy und Slab über moderne Research-Allokatoren wie `mimalloc`, `jemalloc`, `snmalloc` bis zu spezialisierten Varianten für NUMA, PIM und formal verifizierte Systeme). Alle Wrapper erfüllen standardisierte Concepts und sind orthogonal nach 7 Compile-Time-Varianten-Familien (Freelist-Topologie, Size-Class-Schema, Thread-Lokalität, Synchronisation, Allokationspolicy, Reklamation, Fragmentierungsstrategie) strukturierbar.

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Wie sind freie Speicherblöcke organisiert? Beispiele: Per-Page-Sharding (<code>mimalloc</code> A04), Per-Thread-Caching (<code>snmalloc</code> A07), Buddy-Tree (A19), Single-Global-Free-List (<code>dlmalloc</code> A20), Spans (<code>scallop</code> A08), hybride Strategien (<code>hmalloc</code> A15).
AA2	Size-Class-Schema	Wie werden Block-Größen klassifiziert? Beispiele: Slab pro Objektgröße (A02), <code>dlmalloc</code> Bins (A20), <code>jemalloc</code> Größenklassen (A05), <code>scallop</code> Spans (A08), Buddy Power-of-2 (A19), formal verifizierte Bins (StarMalloc A13), <code>std::pmr</code> pools (A22).
AA3	Thread-Lokalität	Wie lokal pro Thread/Core ist die Allokation? Beispiele: <code>tcmmalloc</code> thread-local-caches (A06), <code>rpmalloc</code> thread-heaps (A10), <code>snmalloc</code> message-passing (A07), <code>tcmmalloc</code> -warehouse hyperscale (A14), Hoard per-heap-locks (A01).

ID	Name	Beschreibung
AA4	Synchronisation	Welcher Concurrency-Mechanismus? Beispiele: Lock-Free CAS (Michael A03, LRMalloc A11), Per-Heap-Locks (Hoard A01), Single-Threaded (Exgen A18), Wait-Free (Crystalline A17).
AA5	Allokationspolicy	Wie werden Allokationsanfragen geroutet? Beispiele: NUMA-Origin-Awareness (NUMAlloc A09), Cache-Set-Awareness (CAMA A12), NUCA-Awareness (TCMalloc-Warehouse A14), PIM-Awareness (PIM-malloc A16), polymorphe Memory-Resource (std::pmr A22).
AA6	Reklamation	Wie werden gelöschte Blöcke zurückgegeben? Beispiele: Magazine-Cache (Slab A02), Page-Heap-Release (TCMalloc A06), Lock-Free-Reclamation (Crystalline A17), Deferred-Free (mimalloc A04), Vmem-Magazine-Extension (A23).
AA7	Fragmentierungsstrategie	Wie wird Fragmentierung vermieden? Beispiele: dlmalloc Coalescing (A20), Object-Coloring (Slab A02), Buddy-Splitting (A19), Page-Local-Sharding (mimalloc A04), Span-Pooling (scalloc A08).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierbarkeit. Berger et al. (PPoPP 2000). Synchronisierungskonzept via Per-Heap-Locks + lokale Heap-Caches pro Thread.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-Caching-Kernel-Allokator mit Magazine-Cache-Strategie. Bonwick (USENIX 1994, 2001). Klassisches Lehrbuch-Paradigma ohne eigenständigen OSS-Code.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allokator mit Hazard-Pointers. Maged Michael (PLDI 2004). Re-Implementierung mit IBM-Patent-Hintergrund.	P— (LGPL Re-Impl)
MimallocAllocator (A04)	Free-List-Sharding mit deferred-free und page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
JemallocAllocator (A05)	Arena-basierte Größenklassen mit per-CPU Arenas und tcache Layer. Evans (BSDCan 2006). Standard-Allokator von FreeBSD und Facebook. Original Code-Linking (is_original=2/2).	P— (OSS, BSD-2-Clause, is_original)
TCMallocAllocator (A06)	Thread-Caching Malloc mit zentralem Page-Heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google-Produktions-Standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-Passing-Allokator mit asymmetrischen Heap-Operationen. Paul Liétar et al. (ISMM 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
ScallocAllocator (A08)	Span-basierte Free-List mit Virtual Spans für Multi-Core-Skalierbarkeit. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low-Fragmentierung via Span-Pooling.	P— (OSS, BSD-3-Clause)
NUMAllocAllocator (A09)	NUMA-Origin-Awareness mit schnellerer Speicherallokation. UTSASRG (ISMM 2023). Optimiert für Multi-Socket-Architekturen.	P— (OSS, vermutlich Apache-2.0)
RPMallocAllocator (A10)	Per-Thread Span-Caching ohne Dependency zu anderen Allokator-Bibliotheken. Mattias Jansson (2017). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/MIT, is_original)
LRMallocAllocator (A11)	Lock-Free Allokator mit Hazard-Pointers für Multi-Core-Skalierbarkeit. Leite/Rocha (VECPAR 2018, LNCS 11333). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)

Variante	Beschreibung	Quelle
CAMAllocator (A12)	Cache-Aware Memory Allocator mit vorhersehbarem Verhalten. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). Keine eigenständige OSS-Implementierung.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formal verifizierte härtete Memory-Allocator mit F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematisch verifizierter Allocator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc-Hyperscale-Erweiterung mit Huge Page-Unterstützung für Warehouse-Scale-Computing. TEMERAIRE (Hunter OSDI 2021) + Warehouse-Charakterisierung (Zhou ASPLOS 2024). NUCA-aware TCMalloc-Variante.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid Lock-Free Allocator mit skalierbare Speicherverwaltung. Li/Yao/Tang/Lin (IEEE ICPADS 2019). Keine eigenständige OSS-Implementierung.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-In-Memory Allocator für heterogene HW mit schnellem lokalen Speicher. VIA-Research (HPCA 2026, arXiv:2505.13002). Spezialisiert auf PIM-Hardware.	P— (OSS, MIT)
CrystallineReclamation (A17)	Wait-free Safe-Memory-Reclamation-Familie (Nikolaev/Ravindran DISC 2021 / PLDI 2024). Reclamation-Policy, kein eigenständiger Allokator.	P— (Research)
ExgenAllocator (A18)	Single-Threaded Specialized Allocator mit Optimierungen für Single-Threaded Workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). „Old is Gold“ Prinzip.	P— (Research, Konfidenz medium)
BuddyAllocator (A19)	Buddy-System mit Power-of-2-Splitting für einfache Fragmentierungskontrolle. Knuth (TAoCP Vol.1, 1968). Klassisches Lehrbuch-Paradigma.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea Malloc mit Bins-basierter Klassifizierung. Lea (1987–2012). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) mit expliziter Wrapper-Kontrolle. Wolfram Gloger / Ulrich Drepper (1990er-2000er). Lizenz-blockiert (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blockiert Linking)
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis) — portable Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphe Memory-Ressource). Halpern (N3916, C++17). Abstrakte Schnittstelle ohne verhaltens-distinkte Default-Implementierung.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource mit eigenem Size-Class-Pool. Halpern (N3916, C++17). Verhaltens-distinkt ohne Vendor-Linking (F15-operativ).	P— (Standard-Library, C++17, F15-operativ)
VmemMagazinesAllocator (A23)	Vmem + Magazines als Erweiterung des Slab-Allocators für Multi-Core-Skalierung. Bonwick (USENIX ATC 2001). Keine eigenständige OSS-Implementierung.	P— (Baseline/Textbook)

Beitragende Reclamation-Quellen: P29, P30 [McK01, Mic04a]; vollständiger Allokator-Korpus A01–A23 siehe Abschnitt „A-Korpus“.

D.8 T7 Prefetch-Strategie

Diese Achse ist das *Prefetch-Organ* (T7) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2). Die Prefetch-Achse bestimmt die Strategie zum vorausschauenden Laden von Knoten und Adressen entlang des erwarteten Such-Pfads in einer Trie-Datenstruktur. Sie unterstützt vier konkrete Strategi-

en mit unterschiedlichen Triggermechanismen und Heuristiken: keine Prefetch (Mess-Baseline), distanzgeschätzte Software-Prefetch auf Basis von Knoten-Dichte und Cache-Latenz (ART-Standard), explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard) und pfadorientierte Prefetch mit Bundle-Granularität (PRT-ART Diplomarbeit).

Statische Sub-Achsen

ID	Name	Beschreibung
PF1	Triggermechanismus	Art der Kontrolle über den Prefetch-Moment: ob kein Mechanismus (Baseline), durch Compiler-Hinweise via Distance-Estimator, durch explizite Hardware-Prefetch-Instruktionen oder durch pfadorientierte Vorhersage
PF2	Distance-Heuristik	Berechnungsweise für die Prefetch-Distanz (wie weit voraus geladen wird): linear, adaptiv basierend auf Knoten-Dichte und Tier-Latenz, oder keine Heuristik
PF3	Granularität	Prefetch-Größe pro Trigger: Cache-Line (64 Byte, einzeln), Page (4K, Betriebssystem-Fenster) oder Bundle (mehrere Cache-Lines, Trie-pfadorientiert)

Bausteine

Variante	Beschreibung	Quelle
NonePrefetch	Keine Prefetch-Strategie (Baseline). Dient als Vergleichs-Referenz für Mess-Reihen. Trigger: keine; Heuristik: keine; Granularität: N/A. <code>is_active()</code> gibt false zurück.	Code/Baseline
DistanceEstimator Prefetch	Distanzgeschätzte Software-Prefetch (ART-Standard nach Leis ICDE 2013). Schätzt Cache-Distance zur nächsten Node basierend auf Knoten-Dichte [%] und Tier-Latenz [cycles] mit Heuristik: <code>estimate = clamp(1 + sparseness*8 + latency_factor, [1,16])</code> . Trigger: Compiler-Hinweise via <code>__builtin_prefetch</code> ; Heuristik: adaptiv (Dichte + Latenz); Granularität: Cache-Line. Setzt auf ART-Pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard nach Wu EuroSys 2019). Nutzt PREFETCHT0/T1/T2/NTA auf Hash-Anchor-Chain ohne Software-Heuristik; CPU schätzt Distance autonom. Trigger: explizite Hardware-Instruktion; Heuristik: Hardware-managed; Granularität: Hardware-bestimmt. Lizenz GPL-3.0 blockiert <code>is_original-Linking</code> .	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Pfadorientierte Prefetch-Heuristik (PRT-ART Diplomarbeit 2026). Pre-loadet alle Knoten entlang vorhergesagtem Trie-Pfad bevor Suche startet. Tracker verfolgt Suchpfad und empfiehlt nächste Adresse via linearer Schritt-Extrapolation. V11.1 Hot-Path-Hints: interpretiert erste 8 Byte eines Schlüssels als uint64-Adresse. Trigger: pfadorientierte Vorhersage; Heuristik: linear (Schritt-Extrapolation); Granularität: Bundle (mehrere Cache-Lines). Verwandte Literatur: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (verwandt: Chen/Gibbons/Mowry SIGMOD 2001); Diplomarbeit 2026

Beitragende Quellen: P01, P07, P21 [LKN13, WNJ19, CGM01].

Verwandte Literatur: P23, P25, P26, P27 [Kha11, MWR25, ZSZ⁺24, ZGH⁺25].

D.9 T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)

Diese Achse ist das *Nebenläufigkeits-Organ* (T8) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Diese Achse regelt Synchronisationsmuster und sichere Speicherreclamation für nebenläufige Zugriffe auf Baum-/Indexstrukturen. Die Achse stellt compile-time wählbare Synchronisations-Patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) bereit sowie orthogonale Reclamation-Schemata (RCU/Hazard-Pointer) für sichere Speicherfreigabe in lock-freien Strukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
CC1	Synchronisations-Pattern (CC1)	Compile-time Varianten-Familie der Synchronisationsmuster: None (single-threaded Baseline), Blocking (pessimistic Mutex), ReaderWriter (shared_mutex), Optimistic (lock-free Versionsvalidierung), LockFree (CAS-Loops), WaitFree (gebundene Schritte). Jede Familie hat distinkte Laufzeit-Charakteristiken (0-Overhead bis Mutex-Overhead).
CC2	Reclamation-Schema (CC2)	Orthogonale Compile-time Varianten-Familie für sichere Speicherfreigabe in lock-freien Strukturen: RCU (deferred grace-period, via userspace-rcu), Hazard-Pointer (per-thread in-use-Marken). Kombinierbar mit lock-freien Patterns, optional bei Blocking/Optimistic.

Bausteine

Variante	Beschreibung	Quelle
NoneConcurrency	Keine Synchronisation, single-threaded Baseline. Beide <code>acquire()</code> und <code>release()</code> sind no-ops (volatile Dummy). Vergleichs-Nullpunkt für Concurrency-Overhead-Messung (F15).	Code/Baseline
Blocking Concurrency	Pessimistisches Mutex-Locking mit <code>std::mutex</code> . Globaler coarse-grained Lock. Serialisiert alle Zugriffe. Obergrenze des Locking-Overheads. Paper: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-Writer/Multi-Reader via <code>std::shared_mutex</code> . Lock-Sharing bei read-lastigen Workloads. Mittelgewicht zwischen pessimistischen und optimistischen Mustern. Paper: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): Versions-Counter pro Knoten, lock-freie Reader mit Retry-on-Conflict. Billig bei niedrigen Konflikten. Paper: Leis DaMoN 2016 / DEBULL 2019 (P08, <code>is_original=true</code> , Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (<code>std::atomic</code> CAS-Loops). System-weiter Fortschritt. Generisches CAS-Pattern ohne Blockierung. Paper: Michael/Scott PODC 1996 (Anker-Paper).	Code/Baseline
WaitFree Concurrency	Wait-free: Jeder Thread schließt in beschränkter Schrittzahl ab (stärkste Garantie). OHNE Retry-Loop wie LockFree. Realisiert via Sequence-Lock / Fetch-Add. Paper: Herlihy TOPLAS 1991 (Theorie-Anker).	Code/Baseline
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): Lock-freie Reader via per-Thread Nesting-Zähler (RELAXED-Order, KEIN Memory-Barrier auf Read-Side). Reclamation-Schema orthogonal zu Pattern. Lokale Re-Impl (P29 userspace-rcu LGPL-2.1, <code>is_original=false</code>).	P29

Variante	Beschreibung	Quelle
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): Pro-Thread markierte in-use-Zeiger verhindern vorzeitige Speicherfreigabe (ABA-safe). Reclamation-Schema. Lokale Re-Impl (P30 NO LICENSE, is_original=false). SEQ_CST-Store auf acquire().	P30
OlcReservedBlocks Concurrency	OLC + Cache-Line-Reservierte Value-Blocks (Multi-Writer Cache-Coherence-Storm-Vermeidung). Spezialisierung des Optimistic-Patterns mit zusätzlicher atomarer Block-ID-Reservierung (fetch_add). F.6 Doku 19 (prt-art-Optional, nicht ist-original).	Code/Baseline

Beitragende Quellen: Dijkstra 1965, Courtois et al. 1971, P08, Michael/Scott 1996, Herlihy 1991, P29, P30 [Dij65, CHP71, LABN16, MS96, Her91, McK01, Mic04a].

D.10 T9 Serialisierung (Kodierung und Kompression)

Diese Achse ist das *Serialisierungs-Organ* (T9) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Achse 10 bestimmt, wie Daten innerhalb des Cache-Engine-Systems kodiert und komprimiert werden: vom Raw-Binary-Speicherlayout über variable Längenkodierung bis zur Succinct-Bit-Packing. Die Wahl der Serialisierungsstrategie beeinflusst direkt CPU-Kosten (Delta-Encoding, Bit-Manipulation, LEB128-Varint), Speicher-Overhead und IO-Bandbreitennutzung — kritisch für LSM-Trees und Read-Heavy-Workloads mit Approx-Filter (SuRF/LOUDS).

Statische Sub-Achsen

ID	Name	Beschreibung
SR1	Byte-Order und Wortformat	Wie werden Bytes innerhalb eines Datensatzes angeordnet? Raw (memcpy Roh-Layout), Varlen (LEB128-Varint mit Signalisierungsbits), oder Packed (Bit-für-Bit-Packing)?
SR2	Dichte (Speicherauslastung)	Speicher-Overhead pro Datensatz: Sparse (vollständige Records), Dense (mit Padding), oder Succinct (Information-theoretisch dicht, $n \cdot H + o(n)$ bits)?
SR3	Kompression und Vorverarbeitung	Block-Kompression (LZ4/Snappy, LZ77-basiert) oder Delta-Encoding (Zigzag-Transform für negative Deltas)? Oder keine Kompression?

Bausteine

Variante	Beschreibung	Quelle
RawBinary Serialization	Roh-Byte-Speicherlayout (memcpy). Baseline ohne Transformation. Minimaler CPU-Aufwand. Typ. für In-Memory-Workloads ohne Kompression. Variante: <code>serialisierte_scan()</code> liest 32-Bit Felder + Order-sensitive Checksum.	Code/Baseline
VarLen Serialization	Variable-Längen-Kodierung mit Signalisierungsbits (LEB128-VarInt). Standard für ART (Leis ICDE 2013 P01). Kleine Werte = wenige Bytes. Signalisierungsbits markieren Byte-Länge. Datenabhängige Schleife (1-5 Bytes). FNV-1a-Mix der Varint-Bytes.	P01

Variante	Beschreibung	Quelle
Succinct Serialization	Bit-für-Bit-Packing mit minimaler Bitbreite (LOUDS/SuRF). Erreicht Information-theoretisches Minimum $n \cdot H + o(n)$ bits. Read-Only nach Build. Optimal für Approx-Filter + Read-Heavy. Höchster Per-Element-CPU-Aufwand (echte Bit-Manipulation). Variante: <code>serialize_scan()</code> mit 64-Bit Akkumulator.	P10,P09
Compressed Serialization	Block-Kompression (LZ4/Snappy) auf Roh-Binary aufgesetzt. Trade-off: CPU-Kosten vs IO/Memory-Bandbreite. Variante: Delta-Encoding gegen Vorgänger + Zigzag-Transform (negatives Delta $\rightarrow$ kleine unsigned). Typisch für LSM-Trees. Mittlerer CPU-Aufwand.	Code/LZ77 (Ziv/Lempel 1977) [ZL77]

Beitragende Quellen: P01, P09, P10, Ziv/Lempel 1977 [LKN13, Jac89, ZLL⁺18, ZL77].

D.11 T10 Wert-Handle (Value-Handle)

Diese Achse ist das *Wert-Handle-Organ* (T10) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Die Wert-Handle-Achse definiert, wie Datenwerte (Values) im Cache-Index gespeichert und zugegriffen werden: inline im Node-Slot, extern in einem Pool, mittels immutable-shared References (RCU-Style), versioned Pointer (MVCC), oder verkettete externe Referenzen (Multi-Value). Diese Achse charakterisiert die Speicherlayout-Strategie für Values und bestimmt damit die Zugriffslatenz (direkt vs. indirekt) und Concurrency-Semantik (Snapshot vs. Versioning).

Statische Sub-Achsen

ID	Name	Beschreibung
VH1	Speicherort (Storage-Location)	Compile-Time Varianten für die physische Platzierung des Value: inline im Node-Slot (kombiniert Key/Value in einem Slot ohne Indirektion) oder extern in einem separaten Pool (Node hält nur Offset/Pointer).
VH2	Eigentümer-Semantik (Ownership)	Compile-Time Varianten für Ownership und Sharing-Strategie: owned (exclusive per Key), shared (immutable reference-counted via RCU-style), weak (borrowing ohne Increment).
VH3	Versionierung (Versioning)	Compile-Time Varianten für Concurrency-Kontrolle: keine Versionierung (simple Ownership), Version-Tags in Pointer (MVCC für Reader-Snapshot-Isolierung).

Bausteine

Variante	Beschreibung	Quelle
InlineValueHandle	Value wird direkt im Node-Slot eingebettet (keine Indirektion). Standard für kleine Values wie <code>uint64_t</code> . Minimale Zugriffslatenz (1 direkter Read), aber Node-Speicher wird durch größere Values beansprucht. Quelle: ART (Leis ICDE 2013, P01).	P01
ExternalPoolValue Handle	Value extern in Pool gespeichert; Node hält nur Pool-Offset. Ermöglicht variable Value-Größen und kompaktere Nodes, kostet aber 1 zusätzlichen Cache-Miss pro Lookup (Pointer-Chasing). Quelle: Wormhole (Wu EuroSys 2019, P07).	P07

Variante	Beschreibung	Quelle
ImmutableSharedRef ValueHandle	Value via reference-counted Pointer (RCU-Style Snapshot-Sharing). Optimal für Reader-Concurrency: Leser sehen stabile Snapshots, Writer erstellen neue immutable Snapshots. Requires RefCount-Acquire/Release pro Zugriff. Quelle: RCU (McKenney OLS 2001, P29).	P29
VersionedPointer ValueHandle	Pointer mit Version-Tag im oberen Byte (MVCC). Ermöglicht Multi-Version-Concurrency-Control: Reader bekommt Snapshot-Version zur Sichtbarkeitskontrolle, Writer erstellt neue Version. Tombstone-Erkennung via Versions-Bit. Quelle: Masstree (Mao EuroSys 2012, P03) und SMART (OSDI 2023).	P03
ChainRefValue Handle	Multi-Value verkettete externe Referenz (Linked-List). Node hält Offset zum Chain-Head im Pool; jeder Chain-Knoten hält (value_offset, next_offset). Teuerste Variante mit 2 abhängigen Derefs pro Value-Zugriff. Quelle: prt-art (lokale Re-Impl, F.6 Migration).	Code/Baseline

Beitragende Quellen: P01, P03, P07, P29, SMART (OSDI 2023) [LKN13, MKM12, WNJ19, McK01, LZS⁺23].

D.12 T11 Index-Organisation

Diese Achse ist das *Index-Organisations-Organ* (T11) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Klassifizierung der Datenspeicherung und Indexstrukturierung nach drei orthogonalen Dimensionen: Storage-Order (Heap/Clustered/Ungeordnet), Index-Count (keine/einzeln/mehrfach) und Data-Embedding (separate Speicherung vs. in Leaf-Pages eingebettet). Diese Achse modelliert fundamentale DBMS-Entwurfsentscheidungen, die Cache-Verhalten, Zugriffsmuster und I/O-Charakteristiken auf Basis von Index-Order vs. Storage-Order-Alignment entscheidend beeinflussen.

Statische Sub-Achsen

ID	Name	Beschreibung
io1_ storage_ order	Storage-Order	Anordnung der Datensätze im Speicher: ungeordnet (Insert-Order im Heap), sortiert nach Primary-Key (Clustered), oder mit Pointer-Indirektion (Non Clustered). Definiert die sequenzielle oder zufällige Zugriffsmuster bei Range-Scans.
io2_index_ count	Index-Multiplizität	Anzahl unterstützter Indizes pro Tabelle: keine (Heap-Baseline), Single-Index (Clustered, MySQL InnoDB Primärschlüssel), oder Multiple Secondary-Indizes (NonClustered, SQL Server NONCLUSTERED INDEX).
io3_data_ embedding	Daten-Einbettung	Speicherplatzierung der Daten relativ zum Index: getrennt im Row-Store (Clustered/NonClustered mit Pointer-Hop), oder direkt in Index-Leaf-Pages eingebettet (IOT/Trie-Style wie ART/HOT).

Bausteine

Variante	Beschreibung	Quelle
ClusteredIndex Organization	Speicher-Reihenfolge entspricht Index-Reihenfolge (1 Primary-Key). Optimiert für Range-Scans entlang des Primary-Key mit sequenziellen Disk-Reads (cache-freundlich). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Zugriffsmuster: sequenzielle Forward-Iteration ohne Predicate-Evaluation, O(n) Summen-Scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage-Reihenfolge = Insert-Order, KEIN Index (Baseline). Lookup mittels O(n) Full-Scan mit Predicate-Evaluation pro Record (datenabhängiger Branch, kein früher Abbruch). Standard: PostgreSQL Heap-Tabellen, HBase. Zugriffsmuster: ungeordneter sequenzieller Scan mit Vergleichslast und Branch-Misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Daten direkt in Index-Leaf-Pages eingebettet (Oracle IOT, SQL Server CLUSTERED INDEX style). Spart Pointer-Indirektion vs. NonClustered. Äquivalent zu B+Tree-Leaves mit eingebetteten Daten oder Trie-Knoten (ART/HOT/Wormhole). Zugriffsmuster: sequenziell wie Clustered, aber mit Zusatzlast durch Daten im Leaf (zwei Felder pro Record).	Srinivasan et al., VLDB 2000 (Oracle8i IOT) [SDF ⁺ 00]
NonClusteredIndex Organization	Index-Reihenfolge $\neq$ Storage-Reihenfolge, mehrere Secondary-Indizes pro Tabelle. Lookup über Index + Pointer-Hop zur Daten-Row (Random-Stride, Cache-Misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Zugriffsmuster: Index-navigation gefolgt von zufälligen Storage-Hops (LCG-determiniert, REAL cache-Miss-Latenz durch TLB/L2-Invalidierung), anschließend ein Residual-Predicate-Vergleich pro geholtem Record (datenabhängiger Branch auf der geholten Daten-Row nach dem Pointer-Hop).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Beitragende Quellen: Bayer & McCreight 1972, Comer 1979, Srinivasan et al. 2000, Garcia-Molina, Ullman & Widom (Textbook) [BM72, Com79, SDF⁺00, GMUW09].

D.13 T12 I/O-Dispatch-Achse

Diese Achse ist das *I/O-Dispatch-Organ* (T12) des Lebewesens SearchAlgorithm (vgl. Abschnitt 3.2).

Modellierung und Auswahl von I/O-Strategien für die Persistierung und den Speicherzugriff im Index: Speicherung im RAM-only (Baseline), gepuffert via OS-Page-Cache (Standard), direkter I/O mit O_DIRECT-Flag (NVMe-Optimierung), oder Memory-Mapped File I/O (Persistent Memory). Diese Achse ermöglicht die Entkopplung der Such- und Baum-Strukturen von der Persistierungs-Implementierung und gestattet A/B-Messungen der Dispatch-Overhead-Profile unter identischen Workloads.

Statische Sub-Achsen

ID	Name	Beschreibung
IO1	Persistierungsmodell	Klassifiziert die Speicherart: RAM-only (keine Persistierung, reine Mess-Baseline), oder Disk/Persistent-Memory (Datei-gesicherte Indizes mit verschiedenen Caching-Strategien).
IO2	Caching-Strategie	Charakterisiert die OS-Puffer-Nutzung: Buffered (OS Page-Cache, read-ahead + write-back, Standard), Direct I/O (<code>O_DIRECT</code> , Page-Cache-Bypass, 512B/4KB-Alignment), oder Memory-Mapped (file-backed <code>mmap</code> mit automatischer OS-Pagination und page-fault-getriebenem Zugriff).
IO3	Schreib-Dauerhaftigkeit	Legt die Konsistenz-Garantien fest: keine <code>fsync</code> (höchste Performance, keine Absicherung), <code>fsync</code> nach Batch (Kompromiss, Batch-Commits), oder atomare Schreibvorgänge (höchste Sicherheit, niedrigste Performance). In dieser Implementierung hauptsächlich als konzeptionelle Sub-Achse vorhanden; operative Umsetzung erfolgt über die Caching-Strategie.

Bausteine

Variante	Beschreibung	Quelle
<code>InMemoryOnly</code>	Rein RAM-basierter Index ohne Persistierung. Direkter, sequentieller Record-Zugriff ohne Dispatch-Overhead. Schnellster Pfad und Vergleichs-Nullpunkt der Achse. Verwendet für Mess-Baseline und Performance-Referenzierung unter Bedingungen ohne I/O-Latenz.	Code/Baseline
<code>BufferedIo</code>	Standard <code>read()/write()</code> via OS-Page-Cache mit automatischem read-ahead und write-back. Default für allzweck-DBMS. Trade-off: bequem, aber Double-Buffering (App-Cache + OS-Cache führt zu erhöhter Speichernutzung). Papier-Anker: Stonebraker CACM 1981 (Konzept-/Position-Paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Confidence: medium (Engineering-Pattern, kein Algo-Code).	P-Stonebraker-1981
<code>DirectIo</code>	Bypass des OS-Page-Cache mittels <code>O_DIRECT</code> -Flag (<code>open(2)</code>), NVMe-SSD-optimal. 512B/4KB-Alignment erforderlich. Ermöglicht DBMS-eigene Cache-Strategien (RocksDB, MySQL). Höhere CPU-Kosten, aber keine Cache-Pollution durch Background-Activity. Keine dedizierte Papier-Quelle (Linux-API seit 2.4.10); wird als bewusste Engineering-Baseline behandelt.	Code/Baseline
<code>MmapIo</code>	Memory-Mapped File I/O (<code>mmap</code>) mit Persistent Memory oder Read-Heavy Workloads. OS pagiert automatisch, kein Copy zwischen User- und Kernel-Space. Trade-off: page-fault-Latenz-Spitzen vs. Reuse-Freundlichkeit. Papier-Anker: Crotty et al. CIDR 2022 (Anti-Pattern-Baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Bewusst als Anti-Pattern-Referenz zitiert, um Grenzen und Overhead zu dokumentieren. OSS-Referenz MIT-lizenziert.	P-Crotty-2022

Beitragende Quellen: Stonebraker-1981, Crotty-2022 [Sto81, CLP22]

D.14 T13 Migrations-Strategie-Achse

Diese Achse ist das *Migrations-Organ* (T13) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Die Migrations-Strategie-Achse (T13, `axis_migration`) kapselt Entscheidungslogiken zur Daten-Platzierung across mehrschichtige Speicher-Hierarchien (RAM/SSD/HDD, Hot/Cold-Tiers). Sie modelliert, wann und wie häufig zugegriffene Daten zu schnellerem Storage promoviert bzw. selten zugegriffene

demoviert werden, ohne echte Tier-Migration durchzuführen (Entscheidungslogik-Messung nur). Unterstützt vier konkrete Strategien: static (Baseline), Heat/Cold-Separation, Multi-Tier-Hierarchie, ML-driven Adaptive.

Statische Sub-Achsen

ID	Name	Beschreibung
MG1	Trigger-Mechanismus (MG1)	Bestimmt, wann eine Migrations-Entscheidung auslöst wird: keine (static), Schwellwert-basiert (Threshold), beobachtungsgesteuert (Observation). Compile-Time Variante pro Strategy.
MG2	Migrations-Richtung (MG2)	Bestimmt die Richtung von Daten-Bewegung: keine (static), aufwärts (RAM→SSD), abwärts (SSD→RAM), bidirektional. Compile-Time Variante.
MG3	Migrations-Granularität (MG3)	Spezifiziert die Skalierungseinheit für Migrations-Entscheidungen: keine (static), Page (4KB), Block (64KB), Object (einzelnes Item). Compile-Time Kategorie.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
D1	Entscheidungs-Schwellwert (Decision Threshold)	Laufzeit-Parameter: Hot/Cold-Klassifikations-Grenzwert (z.B. Recency-Counter-Limit für LRU-basierte Promotions/Demotions). Benutzer-konfigurierbar.
D2	Gewichtungen für Feature-Scoring	Laufzeit-Parameter für AdaptiveMigration: Gewichte (w_{freq} , w_{rec}) für Häufigkeits- und Recency-Features in der ML-Linearkombination. Self-tuning möglich.
D3	Tier-Konfiguration	Laufzeit: Anzahl Speicher-Tiers (z.B. $kTiers=3$ für RAM/SSD/HDD in Tier BasedMigration), Kapazitäten pro Tier, Promotions/Demotion-Schwellen.

Bausteine

Variante	Beschreibung	Quelle
NoMigration	Statische Platzierung ohne Migration (Baseline). Daten werden einmalig positioniert, keine Bewegung zwischen Tiers. Benchmark-Nullpunkt für Vergleich mit aktiven Strategien.	Code/Baseline
HotColdMigration	Heat/Cold-Separation via LRU + probabilistisches Recency-Counting. Häufig zugegriffene Daten (hot) werden zu schnellerem Cache promoviert, selten zugegriffene (cold) zu langsamem Storage demoviert. Standard für LRU-K und LFU-Cache-Systeme.	Anti-Caching (PVLDB 2013) [DPT ⁺ 13]
TierBased Migration	Multi-Ebenen-Migration RAM→NVM→SSD→HDD mit Modulo-basiertem Tier-Mapping. Jedes Datenitem wird anhand eines Feldwertes modulo-auf Ziel-Tier gemappt. Standard für RocksDB, LSM-Trees und Tiered-Cache-Systeme mit konfigurierbaren Schwellwerten.	Managing NVM in DBMSs (SIGMOD 2018) [RLK ⁺ 18]
AdaptiveMigration	ML-driven adaptive Migration mit Online-Learning (Cachelib/ LeCaR-Stil). Gewichtete Linearkombination von Häufigkeits- und Recency-Features berechnet adaptiven Score; gleitender Lern-Akkumulator modelliert Online-Learning. Höherer Overhead vs. optimale Hit-Rate.	ML-based LeCaR (USENIX HotStorage 2018) [VRM ⁺ 18]

Beitragende Quellen: Anti-Caching (PVLDB 2013), LeCaR (HotStorage 2018), Managing NVM (SIGMOD 2018) [DPT⁺13, VRM⁺18, RLK⁺18].

D.15 T14 Filter-Achse

Diese Achse ist das *Filter-Organ* (T14) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Die Filter-Achse T14 bietet wählbare probabilistische und deterministische Membership-Query-Filter (Punkt- und Bereichsabfragen). Sie deckt klassische Bloom-Filter mit k Hash-Funktionen, den deletable-Cuckoo-Filter mit Bucket-Hashing und Fingerprints, den succinct Range-Filter SuRF mit LOUDS-Trie-Navigation sowie den platzsparenden Xor-Filter mit konstanter 3er-XOR-Latenz ab. Zweck: Trade-offs zwischen Raum, Abfragelatenz, Mutability und Falsch-Positiv-Rate direkt im Cache-Engine-Experiment messbar machen.

Statische Sub-Achsen

ID	Name	Beschreibung
FT1	Abfrage-Typ	Unterscheidung zwischen Punkt-Abfragen (membership: key enthalten?) und Bereichs-Abfragen (range: keys in [a,b]?). Punkt-Filter sind Bloom/Cuckoo/Xor; nur RangeSurfFilter unterstützt Range-Semantik.
FT2	Mutierbarkeit	Unterteilung in unveränderbare Filter (Bloom, RangeSurf, Xor nach Build immutable) vs. Mutable (Cuckoo mit delete-Operation). Bestimmt Design des Wrapper und Cachingmöglichkeiten während Laufzeit.
FT3	Fehler-Profil	Charakterisierung der fehlertoleranten Semantik: False-Positive-Only (Bloom, Cuckoo, Xor — Filter sagt „ja“ falsch), vs. lossless (theoretisch deterministische Antwort). Bloom: approx. $k \cdot n$ Hash-Tests; Xor: exakt 3 XOR-Operationen (branch-frei).

Bausteine

Variante	Beschreibung	Quelle
BloomFilter	Klassischer Bloom-Filter nach Bloom 1970: m -Bit-Bitmap, k unabhängige Hash-Funktionen (double-hashing nach Kirsch/Mitzenmacher 2006). Punkt-Abfragen nur; False-Positives möglich, keine Deletes. Implementierung mit echter persistenter m -Bit-Bitmap (8 KiB, $m=65536$, $k=4$ double-hashing), die im Out-of-Process-Mess-Pfad über die tatsächlich eingefügten Schlüssel geprobt wird; die ehrlich deklarierte Pseudo-Bitmap-Probe ($k=4$ Hash-Tests pro Query, charakteristische early-exit-Latenz bei Mismatch) verbleibt nur im In-Process-Segment-Timing-Scan.	Bloom 1970 (CACM 13:422-426)
CuckooFilter	Cuckoo Filter nach Fan et al. CoNEXT 2014: Bucketed Cuckoo-Hashing mit 1-Byte-Fingerprints. Unterstützt Deletes und Lookup-Counting (Vorteil vs Bloom). Probe-Charakteristik: 2 Bucket-Lookups + Fingerprint-Vergleich (partielle-Schlüssel-Cuckoo nach CoNEXT 2014 §3: $i2 = i1 \text{ XOR hash}(fp)$). Höhere Insert-Kosten, aber flexible Mutierbarkeit.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)

Variante	Beschreibung	Quelle
RangeSurfFilter	SuRF (Succinct Range Filter) nach Zhang et al. SIGMOD 2018: LOUDS-Bitmap-Trie für Range-Abfragen. Einziger Filter mit Range-Semantik (lower_bound/upper_bound auf Byte-Prefix-Ebene). Probe-Charakteristik: mehrstufiger Trie-Abstieg mit succinct-LOUDS-Navigation (bis kMaxDepth=6), nicht exakt 1 Hash-Test, daher latenz-variabel je Prefix-Länge. Praktisch in RocksDB SST-Optimierungen.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor Filter nach Graf+Lemire 2020 (JEA 25): XOR-basierter Filter mit ca. 9 bits/key (vs. 10 bits/key Bloom bei gleicher FP-Rate). Immutable nach Build. Probe-Charakteristik: exakt 3 unabhängige Tabellen-Zugriffe (3 disjunkte Tabellen-Drittel h0, h1, h2) + 3er-XOR, branch-frei, konstante Latenz ohne early-exit. Sicherheit gegen Wegoptimierung, kleinster Speicherfootprint aller Filter.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)

Beitragende Quellen: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020 [Blo70, FAKM14, ZLL⁺18, GL20].

D.16 T15 Pufferung Q1 (Operations-Puffer)

Diese Achse ist das *Pufferungs-Organ (Q1)* (T15) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Die Achse *axis_q1_queueing* realisiert verschiedene Pufferungsstrategien für Operationen im Cache-Engine-Kontext. Sie umfasst Lehrbuch-ADTs (FIFO, LIFO, Append-Only, Priority-Heap), spezialisierte Versioning-Techniken (Delta-Chain, Copy-on-Write, Epoch, Tombstone) sowie moderne Lock-Free-Queues (SPSC/MPMC nach Lamport und Vyukov). Die Strategien werden übersetzungszeitlich über die Achsen-Registry (*mp_list* der W2-Recherche, Enable-Flag-gefiltert) pro Permutations-Binary selektiert (kein Runtime-Switch) und sind klassifizierbar nach Zugriffsmuster (sequenziell, sortiert, zyklisch, versioniert, batched, lock-free).

Statische Sub-Achsen

ID	Name	Beschreibung
QS1	Sequenzieller Zugriff	FIFO/LIFO/Append-Only: Zugriff am Anfang oder Ende, typisch Write-Coalescing + MemTable-Pufferung
QS2	Sortierter Zugriff	Skip-List/Priority-Heap: Sortierter Drain, typisch LSM-Compact und Hot-Key-Eviction
QS3	Zyklischer Zugriff	Bounded Ring-Buffer: Feste Kapazität, Disruptor-Pattern, wrap-around mit Overflow-Handling
QS4	Versionierter Zugriff	Delta-Chain/Tombstone/CoW/Epoch: Multi-Version Concurrency Control, Snapshot-Isolation, Reclamation-Windows
QS5	Batch-Operationen	Batched-Insert-Buffer: Sammlung von Operationen zu Sub-Batches, SIMD-Vectorization, Cache-Locality-Optimierung
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS-Loop oder wait-free Lamport-Modell, echte Lock-Freedom ohne Mutex

Bausteine

Variante	Beschreibung	Quelle
NoBuffer	Passthrough/Identität (Baseline): <code>put()</code> ist no-op, <code>get()</code> liefert immer <code>std::nullopt</code> . Sinnvoll als Vergleichspunkt und in klassischen B+/ART-Szenarien mit direktem Leaf-Zugriff. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-Only Linear Buffer (LSM-MemTable, Bw-Tree-Delta): monoton wachsendes <code>std::vector</code> ohne mittiges Löschen. Optimiert für reine Append + Bulk-Drain. Family Q02, QS1 <code>sequential_access</code> . Literatur: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O'Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO-Queue (<code>std::deque</code>): First-In-First-Out via <code>deque::push_back/deque::pop_front</code> . Unbounded, kann mittig effizient wachsen. Lehrbuch-ADT, typisch für Write-Coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO-Stack (<code>std::vector</code>): Last-In-First-Out via <code>vector::push_back/vector::pop_back</code> . Unbounded, optimiert für Hot-Path-Wiederverwendung und Versions-Tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline
BoundedRingBuffer	Bounded Ring-Buffer (Disruptor-Pattern): Feste Kapazität, zyklischer Zugriff mit wrap-around. Overflow: <code>drop_oldest</code> (default). <code>iterable_aspect_t</code> : Capacity als Laufzeit-Loop. Family Q05, QS3 <code>cyclic_access</code> . Literatur: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority-Heap (<code>std::priority_queue</code>): Max-Heap, <code>get()</code> liefert höchste Priorität zuerst. Erste Strategie mit <code>supports_priority_ordering()</code> =true. Unbounded, Lehrbuch-ADT (Williams 1964). Anwendung: LRU-Approximation, Hot-Key-Promotion, Eviction-Queues. Family Q06, QS2 <code>ordered_access</code> .	Code/Baseline
DeltaChainBuffer	Delta-Chain Buffer (Bw-Tree, Levandoski 2013): Erste versioned-Strategie (<code>is_versioned()</code> =true). Jeder <code>put()</code> hängt Delta-Record mit aufsteigender Version-ID an Spitze der Chain an. <code>get()</code> LIFO-Drain mit Versions-ID. Append-Versioning-Paradigma. Family Q07, QS4 <code>versioned_access</code> .	Bw-Tree (Levandoski et al. ICDE 2013) [LLS13]
SkiplistBuffer	Skip-List Buffer (sortierter Buffer, RocksDB-Stil): <code>put()</code> fügt in sortierter Reihenfolge ein via <code>std::set.get()</code> ASCENDING-drain (minimum first), typisch für LSM-Compact-Output. Unterschied zu <code>PriorityHeapBuffer</code> : Skiplist=ascending, Heap=descending. Literatur: Pugh CACM 1990. Family Q08, QS2 <code>ordered_access</code> .	Pugh 1990, RocksDB
TombstoneBuffer	Tombstone-Marker Buffer (LSM + MVCC): <code>put()</code> erzeugt Live-Eintrag, deleted-Einträge bleiben als Tombstones (<code>std::nullopt</code>) bis Compact-Lauf. <code>get()</code> überspringt Tombstones FIFO-artig, <code>is_versioned()</code> =true (Erase-Versioning, anders als Delta-Chain Append-Versioning). <code>size()</code> zählt nur Live-Slots. Family Q09, QS4 <code>versioned_access</code> . Literatur: O'Neil LSM-tree 1996.	O'Neil 1996
CopyOnWriteBuffer	Copy-on-Write Snapshot-Buffer (Persistent Data Structures): <code>put()/get()</code> inkrementiert Snapshot-Version, frühere Snapshots bleiben logisch erhalten via <code>shared_ptr</code> . Snapshot-Versioning-Paradigma (anders als Delta-Chain Append, Tombstone Erase). <code>is_versioned()</code> =true. Family Q10, QS4 <code>versioned_access</code> . Literatur: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	Driscoll et al. 1989
EpochBuffer	Epoch-based Buffer / QSBR (Quiescent-State-Based-Reclamation): Einträge in aktueller Epoche akkumuliert. Nach <code>epoch_threshold</code> Übergang => frühere Epoche kann reclaimed werden. <code>iterable_aspect_t</code> : <code>epoch_threshold</code> als Laufzeit-Parameter. <code>is_versioned()</code> =true (Reclamation-Window-Versioning). Family Q11, QS4 <code>versioned_access</code> . Literatur: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012

Variante	Beschreibung	Quelle
BatchedInsert Buffer	Batched-Insert Buffer (OLAP-Indexing, ART-Bulk-Insert): <code>put()</code> puffert in Sub-Batches der Größe <code>batch_size_</code> , vollständige Batches werden übergeben (bulk-Drain). Ziel: amortisierter Insert-Cost durch Bulk-Operations, SIMD-Vectorization, Cache-Locality. Erste QS5-Strategie. Family Q12, QS5 <code>batched_access</code> . Literatur: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFree SPSCBuffer	Lock-Free SPSC Ring-Queue (Lamport 1977): Single-Producer/Single-Consumer. <code>put()</code> wait-free (1 <code>atomic.store</code> + 1 <code>atomic.load</code>), <code>get()</code> wait-free. Kein CAS nötig dank disjoint Modified-Sets. <code>ProgressGuarantee</code> : <code>:LockFree</code> (sogar wait-free). <code>iterable_aspect_t</code> : Capacity. Overflow: dropping. Pflicht-Vertrag: genau 1 Producer + 1 Consumer. Family Q13a, QS6 <code>lock_free_access</code> .	Lamport 1977
LockFree MPMCBuffer	Lock-Free Bounded MPMC Queue (Vyukov 2010–11): Multi-Producer/Multi-Consumer via per-cell atomic Sequence-Number und CAS-Loop. <code>Cell.sequence == pos</code> (Producer schreibt), dann <code>pos+1</code> (Consumer liest). Retries bei Contention (wait-free pro Op NICHT garantiert). Capacity MUST Power-of-2. <code>iterable_aspect_t</code> . Family Q13b, QS6 <code>lock_free_access.is_original_eligible=true</code> (Vyukov BSD-2).	Vyukov 1024cores 2010–11, Michael/Scott PODC 1996
OriginalLockFree MpmcConcurrent Queue	<code>moodycamel::ConcurrentQueue</code> (Cameron Desrochers, Block-Based MPMC): Habich-Compliance-Wrapper mit Original-Code-Linking (2/6 Methoden original: <code>put</code> , <code>get</code> ; 4/6 Lücken: <code>emplace</code> , <code>peek_front</code> , <code>peek_back</code> , <code>clear</code> sind Re-Impl). <code>is_original_module()</code> = false wegen Lückenfüllungen. Header-only, BSD-2/BSL-1.0 dual. Analog <code>LockFreeMPMCBuffer</code> , aber mit echter Submodule-Bindung. Family Q15, QS6 <code>lock_free_access.is_original_eligible=true</code> (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

Beitragende Quellen: LMAX 2011, P01 (Leis ART ICDE 2013), Bw-Tree (Levandowski et al. ICDE 2013), P29 (RCU OLS 2001), Lamport 1977, Vyukov 1024cores 2010–11, Michael/Scott PODC 1996, Pugh CACM 1990, O’Neil LSM-tree 1996, Driscoll et al. 1989, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014 [TFB⁺11, LKN13, LLS13, McK01, Lam77, Vyu11, MS96, Pug90, OCGO96, DSST89, MKM12, LZS⁺23, Wil64, Des14].

D.17 T16 Pufferung Q2 (Flush-Policy)

Diese Achse ist das *Pufferungs-Organ (Q2)* (T16) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Die Achse Q2 definiert automatische Spülstrategien für Schreibpuffer. Sie entscheidet, WANN ein gefüllter Buffer sein Ausgabeergebnis an die nächste Stufe propagiert — nach je einer Operation (Eager), gar nicht (Lazy), bei Schwellwertüberschuss (Watermark), nach Zeitfenster (Timed) oder adaptiv nach Workload-Häufigkeit (EWMA-gesteuert LSM-Stil). Dies ist zentral für den Speicher-/Durchsatz-Tradeoff und die Batching-Effizienz in Cache-Engines.

Statische Sub-Achsen

ID	Name	Beschreibung
FS1	Ereignisgetriggert (Event-Triggered)	Flush wird durch explizite Ereignisse (put/get-Operation oder Eviction) ausgelöst. Umfasst EagerFlush (nach jeder Operation) und LazyFlush (nur bei Eviction).
FS2	Schwellwertgetriggert (Threshold-Triggered)	Flush wird ausgelöst, wenn die Pufferfüllung einen Schwellwert (Default 75%) überschreitet. Reduziert unnötige Spülungen bei sparsamem Traffic.
FS3	Zeitgetriggert (Time-Triggered)	Flush wird periodisch nach einem Zeitfenster (Micro-Batching) ausgelöst, unabhängig von Pufferfüllung. Klassisches Hadoop/Spark/Kafka-Pattern für bounded-latency Guarantees.
FS4	Adaptiv (Adaptive-Triggered)	Flush-Schwellwert wird adaptiv aus Workload-Häufigkeit via EWMA gelernt. Hohe Write-Rate → früher spülen (60% Schwelle); niedrige Rate → später spülen (95% Schwelle). Erste Q2-Strategie mit self-tuning.

Bausteine

Variante	Beschreibung	Quelle
EagerFlush	Pessimistische Write-Through-Policy: spült nach JEDER <code>put()</code> -Operation vollständig (FullFlush). Niedrigste Latenz, aber maximale Spülfrequenz und höchster Durchsatzaufwand — klassisches Synchronous-Write-Pattern.	Lehrbuch Write-Through Policy / Code Baseline
LazyFlush	Optimistische Write-Back-Policy: spült NIE automatisch, nur bei expliziter Eviction aus dem Buffer. Maximales Batching, minimale Spülfrequenz — klassisches Deferred-Write-Pattern. <code>should_flush()</code> gibt IMMER NoFlush zurück.	Lehrbuch Write-Back Policy / Code Baseline
WatermarkFlush	Schwellwert-gesteuerte Flush-Policy (Default 75% Kapazität). Iterable Aspekt mit 5 Varianten (50%, 65%, 75%, 85%, 95%) für Laufzeit-Permutation. Verwandt mit LSM-Tree Memtable-Trigger (O’Neil 1996).	LSM-Tree O’Neil 1996 (verwandt) / Code Implementation
TimedFlush	Zeit-Fenster-basierte Micro-Batching-Policy (iterable Aspekt: 10/100/1000/10000 ms). Inspiriert von Kafka (NetDB 2011) und Spark D-Streams (SOSP 2013) für Bounded-Latency-Guarantees. Unabhängig von Pufferfüllung.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation
AdaptiveLsmFlush	Selbst-tunende adaptive Watermark-Policy mit EWMA über Burst-Rate. Threshold interpoliert zwischen 60% (hohe Write-Rate) und 95% (niedrige Rate). Konzeptuell RocksDB Dynamic Leveled Compaction + O’Neil LSM 1996; EWMA-Logik ist CE-eigen. Erste Q2-Strategie mit <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O’Neil LSM-Tree 1996 / Code Implementation

Beitragende Quellen: O’Neil et al. 1996 (LSM-Tree), Kafka (NetDB 2011), Spark D-Streams (SOSP 2013) [OCGO96, KNR11, ZDL⁺13].

D.18 T17 Persistenz-Ziel (Persistence-Target)

Diese Achse ist das *Persistenz-Ziel-Organ* (T17) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2); sie ist mit der Achsen-Neuordnung als achtzehnter Organ-Slot hinzugekommen.

Die Achse macht das Ablage-Ziel der Struktur als eigenen Organ-Slot explizit, statt eine In-Memory-Annahme implizit mitzuführen. Im kanonischen Vermessungs-Katalog ist sie auf ihren

In-Memory-Vertreter gepinnt, damit der Permutationsraum bei 2^{17} Binary-Identitäten je System-Permutation bleibt (Abschnitt 3.4.5); jede `binary_id` trägt gleichwohl das achtzehnte Segment `persistence_target=persistence_memory_only`.

Bausteine

Variante	Beschreibung	Quelle
<code>MemoryOnlyTarget</code>	In-Memory-Ablage als Standard-Ziel; einziger freigeschalteter Vertreter des kanonischen Vermessungs-Katalogs. Kein Geräte-Schreibpfad (<code>has_device_writeback_path()=false</code>).	Code/Baseline
<code>DiskWritebackTarget</code>	Platten-Rückschreibe-Ziel; als Baustein vollständig vorhanden (Typ, Registry, Sub-Achsen PT1–PT2), im kanonischen Katalog jedoch per Option abgeschaltet, sodass die Achse dort einelementig bleibt.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.19 System-Sphäre: Ziel-ISA (`target_isa`)

Diese Größe war in der Neunzehner-Zählung das *ISA-Organ* (damals T12); seit der Achsen-Neuordnung ist sie die System-Haupt-Achse `target_isa` — selbst eine Komplex-Achse (vgl. Abschnitte 3.4.1 und 3.4.2).

Die Achse spezifiziert die Ziel-CPU-Instruktionssatz-Architektur als Compile-Time-Konstante und ermöglicht damit Cross-Platform-Optimierungen für vier dominante Serverarchitekturen: x86-64 (Intel/AMD), AArch64 (ARM), Power-ISA (IBM) und RISC-V (Open-Source). Jede Variante definiert eine hardware-spezifische SIMD-Baseline (SSE2, NEON, VSX, skalar) und charakterisiert damit die Vektorbeschleunigungspotenziale für Cache-Engine-Operationen wie `simd_field_sum` (Akkumulation von 32-bit-Wort-Feldern).

Statische Sub-Achsen

ID	Name	Beschreibung
IS1	Wortbreite (Word-Size)	CPU-Datenpfadbreite: 32-bit, 64-bit oder 128-bit. In der Cache-Engine auf 64-bit standardisiert für alle vier ISA-Profile.
IS2	CPU-Hersteller/Familie (Vendor)	Instruktionssatz-Familie: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Bestimmt SIMD-Extension-Kompatibilität: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).
IS3	ZIH-Cluster-Zugehörigkeit	Produktionsmaschinen an TU Dresden: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids, x86-64), Capella (AMD EPYC 9334, x86-64), Grace Hopper (NVIDIA Grace Hopper ARMv9 Neoverse V2, AArch64), N/A (Forschungsboards, RISC-V/Power). Ermöglicht Hardware-spezifische Experimente mit echten Produktionscluster-Profilen.

Über diese drei Entwurfs-Dimensionen hinaus führt die bau-treibende System-Registry vier Unter-System-Achsen am Komplex-Wrapper: zwei übersetzungszeitliche (`target_isa_complex` und `scheduling`, je `stage=ct` mit `kind=fixed_enum_tuple`) sowie zwei dynamische (`numa_`

node und page). Die erste bündelt die benannten Rekombinationen der Zielmaschinen (complex_count=2) mit je drei festen Gliedern (Owner OD-2).

Rekombination	Glied	Deklaration
prod1_zen5 (isa=x86_64)	ram_frequency_mhz	declared=false — keine deklarierte Zahl.
prod1_zen5	cas_latency_cl	declared=false — keine deklarierte Zahl.
prod1_zen5	cpu_fabrication	declared=true, value=AuthenticAMD/26/68/0 (Hersteller/Familie/Modell/Stepping).
prod2_raptor_lake (isa=x86_64)	ram_frequency_mhz	declared=true, value=4800, declaration_ source=dmidecode 2026-07.
prod2_raptor_lake	cas_latency_cl	declared=false — keine deklarierte Zahl.
prod2_raptor_lake	cpu_fabrication	declared=true, value=GenuineIntel/6/151/2.

Notiz: Die Werte stammen aus kDeclaredMachines, das seinerseits den <machines>-Block der Anwender-XML spiegelt (O-4b). declaration_source konserviert den eingefrorenen Ursprung einer deklarierten Zahl (A2-Notiz, keine Provenienz-Stufe); Laufzeit-Stufen stehen niemals in dieser XML (P3).

Die zweite übersetzungszeitliche Unter-Achse scheduling trägt fünf feste Glieder. Sie war früher eine System-Haupt-Achse und ist mit Paket A3 als sub_axis an den target_isa-Komplex-Wrapper umgehängt worden — ein Umzug, keine Löschung.

Glied	Registry-Wert	Beschreibung
worker_pool_layout	value_ ordinal=0	Aufteilung des Arbeiter-Pools über die Kerne.
simd_worker_count_limit	value=2	Obergrenze gleichzeitiger SIMD-Arbeiter.
hetero_core_dispatch	value_ ordinal=0	Verteilung auf heterogene Kerne (P-/E-Kerne).
co_routine_strategy	value_ ordinal=0	Koroutinen-Strategie der Auftrags-Abwicklung.
batch_granularity	value_ ordinal=0	Körnung der Stapel-Abarbeitung.

Notiz: Die Ordinale sind rohe Enum-Werte ohne übersetzungszeitliches Label-Vorbild; die Kennungs-Zeichenketten sind an die Zugriffs-Aufrufe von concepts::scheduling_strategy.hpp compile-gekoppelt.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
numa_node	NUMA-Knoten	stage=runtime, value_type=token, option_ source=machine_resolved. Spiegel des Organ-Members alloc_ hw.numa_node; die zulässigen Werte sind die Knoten-Kennungen der konkreten Maschine und zur Bauzeit nicht erkennbar — daher eine deklarierte Options-Herkunft statt eines Options-Katalogs.
page	Seitengrößen-Freigabe	stage=runtime, value_type=token, option_ source=machine_resolved. Die Freigabe der Seitengrößen (4k/2m/ lg) als System-Capability; die Durchsetzung liegt organ-seitig im Alloc PageHint-NTTP, diese Achse ist die System-Seite der Freigabe.

Namensfalle (compile-time gesichert): Die Unter-Achse heißt `page` und ist nicht `page_type`. `axis_01_page_type` ist der Baum-Knoten-PageKind der Organ-Welt und eine völlig andere Achse (Bauplan D2.6/IV-BLOCKER-5); der `static_assert` in `target_isa_sub_axes.hpp` hält das fest. Der Baustein-Katalog des Seitentyps steht unverändert in der konservierten Alt-Sektion „Build-PG“.

Bausteine

Variante	Beschreibung	Quelle
<code>Amd64Isa</code>	x86-64 / Intel 64 ISA (AMD/Intel Server+Desktop). Dominante Serverplattform mit SSE2-SIMD-Baseline (4 uint32-Lanes pro 128-bit-Register). Implementiert echte SSE2-Vektorisierung via <code>_mm_loadu_si128 + _mm_add_epi32</code> auf nativen x86-64-Hosts; skalarer Fallback auf Nicht-x86-Build-Hosts. ZIH-Präsenz: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids), Capella (AMD EPYC 9334).	Vendor-Spec (AMD 2000 / Intel fortlaufend)
<code>Aarch64Isa</code>	ARMv8-A 64-bit ISA (ARM Holdings). Wachsende Serverrelevanz (Apple M-Series, AWS Graviton, Ampere Altra). SIMD-Baseline: NEON (128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert NEON-Breite ohne ARM-Intrinsics auf x86-Build-Hosts); echte NEON-Vektorisierung auf nativen ARMv8-Hosts. ZIH-Präsenz: GraceHopper (NVIDIA Grace Hopper, Neoverse V2 + Hopper GPU).	Vendor-Spec (ARM Ltd., ab 2011/2013)
<code>PowerPcIsa</code>	Power ISA (POWER9/10, ppc64le Little-Endian). IBM Power Systems (AC922, IC922, S1022). SIMD-Baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert VSX-Breite ohne Power-Intrinsics auf x86-Build-Hosts). Relevant für HPC-Workloads mit NVLink-GPU-Anbindung (AC922). ZIH-Präsenz: nicht primär, aber für Forschungs-Vergleiche verfügbar.	Vendor/Standards-Spec (IBM/ OpenPOWER, v3.0 2017, v3.1 2020)
<code>RiscVIsa</code>	RISC-V RV64GC Open-Source ISA (RV64 General + Compressed). Wachsende Bedeutung (SiFive, Alibaba Xuantie, Andes, ETH Zürich Cores). SIMD-Baseline: KEINE (RVV ist optional, nicht Teil von RV64GC baseline). Cache-Engine-Implementierung: bewusst plain skalarer Pfad (nicht entrollt) – modelliert fehlende Vektoreinheit der Basis-ISA und erzeugt damit reale, langsamere Laufzeit gegenüber x86-64 und ARM. ZIH-Präsenz: embedded + Forschungs-Boards.	Standards-Spec (RISC-V Intl.; UCB TR ab 2011)

Registry-Ist: Die bau-treibende System-Registry führt derzeit zwei freigeschaltete Bausteine — `x86_64` (nativ, ohne Triple- und March-Flags) und `aarch64` (Kreuz-Übersetzung mit `triple=-target aarch64-linux-gnu` und `march=-march=armv8-a`). Der oben aufgeführte Vier-ISA-Katalog bleibt als *Angebots*-Katalog erhalten: Power-ISA und RISC-V sind als Bausteine beschrieben, im heutigen Bau-Angebot aber nicht freigeschaltet.

Beitragende Quellen: —

D.20 System-Sphäre: Betriebssystem (`operating_system`)

Diese Größe ist die zweite System-Haupt-Achse und damit Teil des *Bluts* und keines Organs (vgl. Abschnitt 3.4.1); in der kanonischen, code-gewachten Ordnung steht sie an Platz zwei von drei.

Die Haupt-Achse trägt ausschließlich die *Klassen*-Identität: genau drei Betriebssystem-Familien (Ausprägungs-Katalog OP-10, baustein_count=3), dazu stage=ct und binary_id=never. Die *Instanz* — Distribution, Release, Kernel und Patch-Stand — liegt dagegen vollständig in den drei Unter-Achsen; eine Distribution als vierte Ausprägung der Haupt-Achse einzutragen, verfehlte diesen Schnitt. Die reale Flotte (Windows 11 und Windows Server 2022, die Linux-Docker-Matrix, zwei macOS-Maschinen) fällt vollständig in die drei Familien.

Dynamische Sub-Achsen

ID	Name	Beschreibung
os_version	Distributions-/ Release-Identität	Die Release-Identität der Betriebssystem-Instanz innerhalb ihrer Familie: unter Linux ID und VERSION_ID aus /etc/os-release, unter Windows die Produktlinie, unter macOS die Produktversion.
kernel	Kernel-Release	Das Kernel-Release der Instanz: unter Linux und macOS das uname-Release, unter Windows das NT-Kernel-Tripel.
build	Build- und Patch-Stand	Der Build- bzw. Patch-Stand der Instanz <i>einschließlich</i> des Update-Zustands (Owner-Entscheid E-12, wörtlich: „Wir mergen den Update Zustand in Build“). Ein separates update_zustand ist damit ausdrücklich <i>keine</i> vierte Unter-Achse, sondern Bestandteil dieser.

Notiz: Alle drei sind stage=runtime mit value_type=token und option_source=machine_resolved: Sie haben keinen übersetzungszeitlichen Options-Katalog, weil ihre zulässigen Werte an der konkreten Maschine hängen und zur Bauzeit nicht erkennbar sind. Die Laufzeit-Erhebung ist ein eigener Bau-Schritt (Paket OS-U3, operating_system_probe_{linux,macos,windows}.hpp); die Einhängung selbst ist nur das Angebot. Die Namen stehen seit dem Owner-Entscheid E3 vom 02.08.2026 fest (Paket A14/OS-U1).

Bausteine

Variante	Beschreibung	Quelle
linux	Linux-Familie (LinuxOperatingSystem, posix=true). Deckt die gesamte Docker-Distributions-Matrix ab; die einzelne Distribution ist Sache der Unter-Achse os_version, nicht dieser Ausprägung.	Code/Baseline
windows	Windows-Familie (WindowsOperatingSystem, posix=false). Deckt Desktop- und Server-Produktlinien ab (Windows 11, Windows Server 2022).	Code/Baseline
macos	macOS-Familie (MacosOperatingSystem, posix=true). Deckt die Apple-Silicon- und Intel-Maschinen der Flotte ab.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

Stempel-Neutralität (A-15): Die drei Unter-Achsen sind Laufzeit-Unter-Achsen und stehen deshalb niemals im Binary-Stempel. Ihre Einhängung ändert weder die Stempel-Zeilen noch den Bau-Versions-Suffix, und sie darf den Achsen-Code-Stand von operating_system nicht anheben (Bump-Verbots-Wache): Ein solcher Sprung verschöbe den SHA512 aller Neubauten gegen den Bestand und erzwänge genau den Voll-Neubau aller Alt-Binaries, den der Owner-Entscheid ausschließt.

D.21 System-Sphäre: Erweiterungs-Hardware (`external_utils`)

Diese Größe ist die dritte System-Haupt-Achse; sie ist ausschließlich der Sammel-Hub der Hardware-Meta-Achsen — SIMD- und Vektor-Erweiterungen, GPU, FPGA und NPU sowie externe Beschleuniger (vgl. Abschnitt 3.4.1).

Der Hub trägt *nur* System-Meta-Metas; das Last-Rahmenwerk gehört ausdrücklich nicht dazu. Weil die Meta-Meta-Achsen keine eigene Stempel-Zeile bekommen, sondern als Klammer-Anhang an das Ende der jeweiligen Realm-Zeile treten, schließt dieser Hub den System-Realm ab: Der Anhang wird typgetrieben aus der Glied-Typliste des Hubs gefaltet (`ExternalUtilsHub` → `SimdExternalUtilsFamily`), die Ebene steckt in der Klammer-Tiefe und nicht im Namen (Owner-Entscheid E2 und Owner-Antwort Q1 vom 02.08.2026). Im bau-treibenden Angebot führt die Achse heute `baustein_count=1`, `stage=ct` und `binary_id=never`.

Dynamische Sub-Achsen

ID	Name	Beschreibung
<code>simd</code>	Vektor-Erweiterungs-Stufe	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=3</code> . Die Familien-Kennung ist zugleich das Unter-Achsen-Label (Schreibweise <code>external_utils.simd=...</code>); die je Dialekt reflektierten Übersetzer-Flags liefert die Option, nicht die Familie.

Die drei Optionen der Unter-Achse mit ihren Flags je Übersetzer-Dialekt:

Option	Flag (gpp und clang)	Flag (msvc)
<code>no_extension</code>	— (leer)	— (leer)
<code>avx2</code>	<code>-mavx2</code>	<code>/arch:AVX2</code>
<code>avx512</code>	<code>-mavx512f</code>	<code>/arch:AVX512</code>

Notiz (Q2/K2): `simd` materialisiert als Flag der Übersetzungs-Naht und erscheint damit im Bau-Versions-Suffix (vgl. Abschnitt 3.4.4) — niemals als Laufzeit-Typ-Umschalter und niemals in der `binary_id`.

Zur Unter-Achse gehört ein feiner Merkmals-Katalog `simd_feature_catalog` mit `count=23` Merkmalen, gestuft nach `tier`: vierzehn Merkmale der Stufe `avx512`, vier der Stufe `avx256`, drei Begleit-Merkmale (`companion`) und zwei skalare (`scalar`). Die Flags sind für `gpp` und `clang` durchgängig identisch.

Merkmal (cpuinfo)	Flag (gpp und clang)	Stufe (tier)
<code>avx512f</code>	<code>-mavx512f</code>	<code>avx512</code>
<code>avx512cd</code>	<code>-mavx512cd</code>	<code>avx512</code>
<code>avx512vl</code>	<code>-mavx512vl</code>	<code>avx512</code>
<code>avx512dq</code>	<code>-mavx512dq</code>	<code>avx512</code>
<code>avx512bw</code>	<code>-mavx512bw</code>	<code>avx512</code>
<code>avx512ifma</code>	<code>-mavx512ifma</code>	<code>avx512</code>
<code>avx512vbmi</code>	<code>-mavx512vbmi</code>	<code>avx512</code>
<code>avx512_vbmi2</code>	<code>-mavx512vbmi2</code>	<code>avx512</code>
<code>avx512_vnni</code>	<code>-mavx512vnni</code>	<code>avx512</code>
<code>avx512_bitalg</code>	<code>-mavx512bitalg</code>	<code>avx512</code>

Merkmal (cpuinfo)	Flag (gpp und clang)	Stufe (tier)
avx512_vpopcntdq	-mavx512vpopcntdq	avx512
avx512_vp2intersect	-mavx512vp2intersect	avx512
avx512_bf16	-mavx512bf16	avx512
avx512_fp16	-mavx512fp16	avx512
avx2	-mavx2	avx256
fma	-mfma	avx256
f16c	-mf16c	avx256
avx_vnni	-mavxvnni	avx256
gfni	-mgfni	companion
vaes	-mvaes	companion
vpclmulqdq	-mvpclmulqdq	companion
bmi2	-mbmi2	scalar
popcnt	-mpopcnt	scalar

Maschinen-Signaturen (*count=3*): Die Registry hält je Zielmaschine die Teilmenge der tatsächlich vorhandenen Merkmale fest — `prod1_zen5` (`host_isa=x86_64`) mit 22 Merkmalen, `prod2_raptor_lake` mit 9 und `odroid_gracemont` mit 9. Nur die Zen-5-Maschine trägt die `avx512`-Stufe; die beiden anderen enden bei `avx256` samt Begleit- und Skalar-Merkmalen.

Bausteine

Variante	Beschreibung	Quelle
<code>simd</code>	Vektor-Erweiterungs-Familie (<code>SimdExternalUtilsFamily</code> , <code>family_id=simd</code> , freigeschaltet). Einziges heute freigeschaltetes Glied des Hubs; die Familien für GPU, FPGA und NPU sind im Zusanchnitt vorgesehen, aber nicht Teil des bau-treibenden Angebots.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.22 Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse (Hardware-Beschleuniger)

Konservierter Angebots- und Historien-Katalog: Die Alt-Achsen-Kennung `axis_09b` ist durch die Haupt-Achse `external_utils` mit ihrer Unter-Achse `simd` abgelöst. Der folgende Acht-Bausteine-Katalog bleibt als Angebots- und Historien-Katalog unter dieser Haupt-Achse vollständig erhalten, ebenso die Sub-Achsen-Tabelle SE1–SE3 als historischer Block.

Die Achse `axis_09b_simd_extension` modelliert die verfügbaren SIMD-/Accelerator-Erweiterungen der CPU-Instruktionssatz-Architektur als Sub-Achse zu `axis_09` (Haupt-ISA). Sie ermöglicht Compile-Time-Auswahl zwischen verschiedenen Vektor-Erweiterungen (SSE2/AVX2/AVX-512 für x86; NEON/SVE2 für ARM; RVV für RISC-V) sowie Baseline (Skalar) und GPU-Beschleunigung (CUDA), mit rückwärts-kumulativer Schichthierarchie und hardwarespezifischer Topologie (SIMD-Units pro Sockel, P/E-Core-Zugriff).

Statische Sub-Achsen

ID	Name	Beschreibung
SE1	Vector-Width (Vektorbreite)	Compile-Time-bekannte maximale Vektorbreite der SIMD-Extension in Bits: 0 (keine), 128, 256, 512, oder -1 (skalierbar). Bestimmt Lane-Breite und Parallelisierungsgrad.
SE2	Accelerator-Type (Beschleuniger-Typ)	Kategorisiert die Beschleuniger-Klasse: none (Baseline), simd (Vektor-SIMD), matrix (Matrix-Engine), gpu (GPU), npu (Neural Processing Unit). Influenziert Komposition mit hardware-Topologie (<code>axis_12</code>).
SE3	Compat-Family (Kompatibilitäts-Familie)	Constraint zur Haupt-ISA (<code>axis_09</code>): none (universell), x86, arm, riscv, cuda. Sichert Compile-Time-Validierung: z.B. <code>Sse2SimdExtension</code> kompatibel nur mit <code>Amd64Isa</code> .

Bausteine

Variante	Beschreibung	Quelle
<code>NoSimdExtension</code>	Baseline-Variante ohne SIMD-Beschleunigung. Skalare Fallback-Strategie für alle Permutationen ohne Accelerator. Universal kompatibel zu allen Haupt-ISAs (x86/ARM/RISC-V/PowerPC). Pflicht-Vergleichspunkt.	Code/Baseline
<code>Sse2SimdExtension</code>	Intel SSE2 128-bit Vektor-Extension (x86_64 ABI-Pflicht seit 2003). Nur kompatibel mit <code>Amd64Isa</code> . Baseline auf x86_64; bietet 4x uint32-Lanes/Iteration. Standard auf Cluster-Knoten Barnard/Capella.	Intel ISA Ext Ref 319433-060
<code>Avx2SimdExtension</code>	Intel AVX2 256-bit Vektor-Extension (Haswell+ 2013). Nur x86_64. Rückwärts-kumulativ: umfasst alle SSE+AVX. Default-Optimierungs-Stufe für Server-DBMS. 2 AVX2-Units/Sockel. ZIH-Standard auf Barnard (Intel Sapphire Rapids) und Capella (AMD EPYC 9334).	Intel ISA Ext Ref 319433
<code>Avx512SimdExtension</code>	Intel AVX-512 512-bit Vektor-Extension (Skylake-X+, Ice Lake Server, Zen 4+). Rückwärts-kumulativ: alle SSE/AVX/AVX2 + 15 separate Flags (VNNI für Vector-Indexes, VPOPCNTDQ für Bitmap-Indexes). Nicht auf allen x86_64-CPU's (z.B. Haswell/Raptor Lake disabled). 1 AVX-512-Unit/Sockel; E-Cores kein Zugriff (Intel Hybrid).	Intel ISA Ext Ref 319433-060
<code>NeonSimdExtension</code>	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI-Pflicht ARMv8+). Kompatibel nur mit <code>Aarch64Isa</code> . Baseline auf allen AArch64-CPU's (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON-Unit/Sockel; alle Cores zugänglich (big.LITTLE-Architektur).	ARM ARM DDI 0487 / Cortex-A8 TRM
<code>Sve2SimdExtension</code>	ARM SVE2 (Scalable Vector Extension 2) ARMv9-Pflicht (skalierbar 128–2048 bit). Kompatibel nur mit <code>Aarch64Isa</code> . ZIH-Cluster Grace Hopper (NVIDIA Grace = ARM Neoverse V2 mit 128-bit SVE2). Skalierbare Lane-Breite; 1 Unit/Sockel; alle Cores.	SVE [SBB ⁺ 17] (IEEE Micro 2017); SVE2/ Armv9 per ARM ARM [Arm24]
<code>RvvSimdExtension</code>	RISC-V Vector Extension (RVV v1.0, skalierbar VLEN 128–65536 bit). Kompatibel nur mit <code>RiscVIsa</code> . Optional (nicht Baseline wie SSE2/NEON). Wachsende Relevanz: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). Forschungs-Boards an TU Dresden/ZIH.	RVV-v1.0-Spec [RIS21]; Vitruvius+ (TACO 2023) [MPU ⁺ 23]
<code>CudaGh200SimdExtension</code>	NVIDIA CUDA auf Hopper GH200 (Grace Hopper Superchip GPU-Beschleuniger). ZIH-Cluster Alpha Centauri mit NVIDIA HGX. Kompatibel mit <code>Aarch64Isa</code> (Grace CPU) UND <code>Amd64Isa</code> (Hopper-Host). NVLink-C2C 900 GB/s zwischen Grace-CPU und Hopper-GPU. Massive parallel (96 GB HBM3 GPU; ~16.896 CUDA Cores). <code>units_per_socket()</code> = -1 (GPU ist separate Device).	NVIDIA Tesla [LNOM08] (IEEE Micro 2008) + NVIDIA GH200 Whitepaper

Quellen: ISA-Spezifikationen und Whitepaper (Intel, ARM, IEEE Micro, ACM TACO, NVIDIA); Einzel-

nachweise in der Quellenspalte der Bausteine.

D.23 System-Sphäre: Äußere Komplex-Klammer (`build_target_complex`) und Bau-Werkzeugkette (`build_toolchain`)

Diese Größe ist der Befehls-Klammer-Wrapper über der festen Rekombination der drei System-Haupt-Achsen (`member_count=3`: `target_isa`, `operating_system`, `external_utils`) und schließt damit den System-Realm-Block ab (vgl. Abschnitt 3.4.2).

Als Klammer ist er ausdrücklich *keine* vierte Haupt-Achse: Die kanonische System-Achsen-Ordnung bleibt bei dreien. Drei übersetzungszeitliche Abgangs-Wachen (`static_assert`) halten das fest — `compiler`, `scheduling` und `load_framework` dürfen nicht als Haupt-Achse zurückkehren; ihre Abgänge waren Umzüge, keine Löschungen. Das Last-Rahmenwerk ist auch kein *Glied* dieser Klammer: Es lebt seit Paket A3 im Mess-Realm, während die System-Meta-Metas unter dem Hub `external_utils` fahren. Die Achse führt `stage=ct` und `binary_id=never`.

Statische Sub-Achsen

An der Klammer hängt die Unter-Achsen-Gruppe `build_toolchain` (`sub_axis_count=3`). Die Registry nennt die drei Glieder flach; die Verschachtelung darunter zeigt die tatsächliche Eltern-Kette.

Glied	Eltern-Bezug	Beschreibung
<code>compiler</code>	<code>build_target_complex</code>	<code>stage=ct</code> , <code>baustein_count=2</code> . Die Übersetzer-Wahl selbst; sie war früher eine System-Haupt-Achse und ist mit Paket A3 zur Unter-Achsen-Gruppe der äußeren Klammer geworden.
<code>opt_level</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=5</code> . Optimierungsstufe je Übersetzer-Dialekt.
<code>atomic128</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=2</code> . Freigabe der 128-Bit-Vergleichs-Tausch-Instruktion.

Notiz: Die Gruppe hängt an `build_target_complex` und nicht an `target_isa`, weil sie den *Bau* aus der Rekombination aller drei Haupt-Achsen beschreibt und nicht eine Eigenschaft der Ziel-ISA allein (O-1r).

Bausteine

Variante	Beschreibung	Quelle
<code>gcc</code>	GNU-Übersetzer (<code>GccCompilerAxis</code> , Treiber <code>g++-16</code> , <code>supports_fno_gnu_unique=true</code>).	Code/Baseline
<code>clang</code>	LLVM-Übersetzer (<code>ClangCompilerAxis</code> , Treiber <code>clang++-22</code> , <code>supports_fno_gnu_unique=false</code>).	Code/Baseline

Die fünf Optionen der Unter-Achse `opt_level` mit ihren Flags je Dialekt; die Spalte `ieee754_deterministic` weist aus, ob die Stufe die Gleitkomma-Semantik unangetastet lässt:

Option	Flag (gpp und clang)	Flag (msvc)	ieee754_deterministic
00	-O0	/Od	true
01	-O1	/O1	true
02	-O2	/O2	true
03	-O3	/O2	true
Ofast	-Ofast	/O2	false

Die zwei Optionen der Unter-Achse `atomic128`:

Option	Flag (gpp und clang)	Flag (msvc)
<code>no_cx16</code>	— (leer)	— (leer)
<code>cx16</code>	<code>-mcx16</code>	— (leer)

Bau-Versionskennung: `opt_level`, `simd` und `atomic128` materialisieren als Flags der Übersetzungs-Naht und erscheinen damit im Bau-Versions-Suffix der Binary — einer geordneten Acht-Segment-Folge, in der ein leeres Glied schlicht kein Segment erzeugt (vgl. Abschnitt 3.4.4). In der `binary_id` stehen sie niemals.

Versions-Flag-Grammatik v2 (Owner-KERN vom 07.08.2026, löst die Q3-Grammatik vollständig ab): Die Version eines jeden Achsen-Algorithmus ist einheitlich dreistellig; ein `v`-Präfix gibt es seit der Flag-Grammatik v2 nicht mehr, Kurzformen bleiben verboten. Übersetzungszeitliche Wachen setzen die Form durch:

```

version := UINT '.' UINT '.' UINT [ '.' flag ]*
flag     := basis [ '{' sub [ '.' sub ]* '}' ] | companion
basis    := 'c' | 'g' | 'f' | 'n' | 'x128' | 'x256' | 'x512'
sub      := token [ '{' sub [ '.' sub ]* '}' ] (rekursiv)

```

Vor jedem Flag steht ein Punkt, auch vor dem ersten (`1.0.0.c`, nie `1.0.0c`); die Basis hängt dabei direkt an ihrer öffnenden Klammer, ohne eigenen Punkt dazwischen (`c{p.e}`, nie `c.{p.e}`). Das Suffix `e` bedeutet seither *Efficiency Core* und ist ein Sub-Flag wie jedes andere unter der Basis `c`; das frühere Konzept eines experimentell eingebrachten Prüflings-Algorithmus ist ersatzlos entfallen. Beispielform: `memory_layout=SoaMemoryLayout@1.0.0.c{p.e}.x512{f.vl.bw.dq}.gfnl`.

D.24 Mess-Sphäre: Mess-Kategorien (`measurement_category`)

Diese Größe gehört zum dritten Realm — den Mess-Achsen, dem *Blut des Messens* (vgl. Abschnitt 3.4.1). Sie führt `stage=ct`, `binary_id=never` und `baustein_count=16`. Ihre Werte manifestieren sich als Spalten der Ergebnis-Tabellen und als Einstellungs-Etikett eines Laufs, niemals in der Binary-Identität. Im Kapitel-Modell sind die sechzehn Kategorien die *Unter-Achse* des Mess-Toolings; die Mess-Registry führt sie zusätzlich als eigenen Angebots-Block, weil das Tooling seine Kategorien nicht selbst erfindet, sondern aus diesem Katalog wählt. Die Spalte `regime_ordinal` trennt das zeit- und observer-basierte Regime (0) vom zählerbasierten PMC-Regime (1).

Bausteine

Kategorie	category_ ordinal	Regime (regime_ordinal)
CLU	0	0 — Zeit/Observer
CACHE_MISS_L1	1	1 — PMC
CACHE_MISS_L2	2	1 — PMC
CACHE_MISS_L3	3	1 — PMC
DTLB_MISS	4	1 — PMC
MEMORY_FOOTPRINT	5	0 — Zeit/Observer
BRANCH_MISS	6	1 — PMC
IPC_CPI	7	1 — PMC
LATENCY_MEAN	8	0 — Zeit/Observer
LATENCY_P50	9	0 — Zeit/Observer
LATENCY_P95	10	0 — Zeit/Observer
LATENCY_P99	11	0 — Zeit/Observer
LATENCY_P999	12	0 — Zeit/Observer
THROUGHPUT	13	0 — Zeit/Observer
ENERGY_J	14	1 — PMC
FILL_BUFFER_OCCUPANCY	15	0 — Zeit/Observer

Regime-Zweiteilung: Neun Kategorien liegen im Zeit- und Observer-Regime (CLU, MEMORY_FOOTPRINT, die fünf Latenz-Kategorien, THROUGHPUT, FILL_BUFFER_OCCUPANCY), sieben im PMC-Regime (die drei Cache-Miss-Stufen, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J). Die Aufteilung ist deckungsgleich mit der Zweiteilung „neun/sieben“ aus Abschnitt 3.4.1.

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.25 Mess-Sphäre: Mess-Tooling/Collector (collector)

Diese Größe ist die Instrumenten-Achse des Mess-Realms: Sie hält fest, mit welchem Werkzeug eine Kategorie erhoben wird (vgl. Abschnitt 3.4.1).

Sie führt `stage=ct`, `binary_id=never` und `baustein_count=3`. Jeder Baustein trägt sein Regime und die Menge der Kategorien, die er bedient; die Vereinigung der drei Mengen deckt zehn der sechzehn Kategorien ab. Von den übrigen sechs werden die vier Latenz-Perzentil-Kategorien (LATENCY_P50 bis LATENCY_P999) nicht von einem Erhebungs-Werkzeug, sondern von der nachgelagerten Auswertung bedient (nearest-rank über den Rohwerten, ergänzt um das HDR-Histogramm, Abschnitt 4.5); MEMORY_FOOTPRINT und FILL_BUFFER_OCCUPANCY sind im Angebots-Katalog beschrieben, aber noch keinem Werkzeug zugeordnet.

Bausteine

Variante	Regime	Bediente Kategorien
WallClockSystemAxis	0	LATENCY_MEAN, THROUGHPUT
ObserverSnapshot	0	CLU
SystemAxis		
PmcSystemAxis	1	CACHE_MISS_L1, CACHE_MISS_L2, CACHE_MISS_L3, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.26 Mess-Sphäre: Telemetrie-Bausteine (konservierter Katalog)

Diese Größe war in der Neunzehner-Zählung das *Telemetrie-Organ* (damals T10); seit der Achsen-Neuordnung ist sie eine *zweigeteilte* Haupt-Größe — Sweep-Unter-Achse des Mess-Toolings im Planer und Übersetzungszeit-Hauptgröße im Builder (vgl. Abschnitt 3.4.1). Ihre Baustein-Katalogisierung bleibt hier als konservierter Katalog erhalten.

Definiert Strategien zur Messung von Index-Operationen (Insert, Lookup, Erase) mit verschiedenen Overhead-Profilen. Unterstützt Density-Tracking für adaptive Node-Typ-Wahl, globale Insert-Counter für Durchsatz-Messungen, HDR-Histogramme für Latenz-Quantile und leaf-only-Counter zur Vermeidung von Cache-Line-False-Sharing in den Messzyklen.

Statische Sub-Achsen

ID	Name	Beschreibung
TM1	Scope (Geltungsbereich)	Gültigkeitsbereich der Zählerverwaltung: leaf-only (nur Blatt-Knoten), per-node (alle Knoten), oder global (einzelner Akkumulator für die ganze Struktur). Compile-Time-Wahl, beeinflusst Overhead durch Konkurrenz-Druck auf gemeinsame Cache-Linien.
TM2	Metrik-Typ (Messgröße)	Art der gemessenen Metrik: counter (Zähler für Ereignisse), histogram (Verteilung mit Quantilen), oder density (Slot-Belegung pro Knoten). Beeinflusst den Speicher-Footprint und die Mess-Granularität.
TM3	Overhead-Level (Messlast)	Größenordnung des Mess-Overheads: low (atomare Operationen), medium (per-node-Datenstrukturen), high (Histogramm-Logging). Bestimmt die Eignung für verschiedene Workload-Szenarien (Benchmarking vs. Produktion).

Bausteine

Variante	Beschreibung	Quelle
LeafOnly Counter	Zähler NUR in Blatt-Knoten (vermeidet Cache-Line-Ping-Pong durch Verzicht auf Inner-Node-Updates). Niedriger Overhead bei leaf-only Scope. Entspricht dem kohärenz-schonenden Zähler-Konzept aus Kuehn DaMoN 2023, obwohl das Original-Paper kein Leaf-Counter-Algorithmus ist — CE-Baseline für False-Sharing-Vermeidung.	P28 (Kuehn DaMoN 2023, Kontext; Code/Baseline)
DensityTracker	Per-Node Density/Slot-Occupancy-Tracking für adaptive Node-Type-Wahl (4→16→48→256). Kontextuelle Telemetrie zur ART-Struktur (Leis ICDE 2013), misst Slot-Belegung aller Knoten. Höherer Overhead als LeafOnly, liefert aber Adaption-Signal für dynamische Rebalancing-Entscheidungen.	P01 (ART/Leis ICDE 2013, Kontext; Code/Baseline)
InsertCounter	Globaler atomarer Insert-Zähler mit minimalem Overhead (1 atomic increment pro Insert). Kontextuelle Telemetrie zur HOT-Struktur (Binna SIGMOD 2018), global Scope. Ausreichend für Durchsatz-Mess-Reihen, keine Latenz-Granularität.	P02 (HOT/Binna SIGMOD 2018, Kontext; Code/Baseline)
Latency Histogram	HDR-Histogramm zur Lookup-Latenz-Verteilung (p50/p95/p99 Quantile). Basiert auf HdrHistogram (Gil Tene), CC0-1.0 Lizenz. Höchster Overhead (pro Lookup), liefert aber vollständige Latenz-Statistik für SLA-orientierte und research-grade Messreihen. Einziger is_original-Kandidat der Achse.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_c, CC0-1.0, is_original)

Variante	Beschreibung	Quelle
Probability-Hints	Wahrscheinlichkeits-Hinweise der cache-oblivious Bender-Linie (P16/P17): verteilungs- bzw. wahrscheinlichkeitsbezogene Belegungs-Hinweise, die als Eingabe der Layout-Entscheidungen dienen. Paper-Baustein der Achsen-Sezierung (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P16/P17 (Bender 2000/2005) [BDFC00, BDFC05]

Beitragende Quellen: HdrHistogram (Gil Tene) [Ten12], P16/P17 (Bender) [BDFC00, BDFC05].

Verwandte Literatur: P01, P02, P28 [LKN13, BZP⁺18, Kue23].

D.27 Mess-Sphäre: Last-Rahmenwerk (load_framework) — Meta-Meta-Achse

Diese Größe ist die Meta-Meta-Haupt-Achse des Mess-Realms und steht deshalb an dessen Ende (vgl. Abschnitt 3.4.1).

Sie führt `category=measurement_meta_meta`, `stage=ct`, `binary_id=never` und `baustein_count=1`. Ihre Umzugs-Geschichte ist Teil ihrer Identität: Sie hat die System-Welt mit Paket A3 *ersatzlos* Richtung Mess-Realm verlassen und ist dort die Meta-Meta-Haupt-Achse des Planers; unter den Hub der System-Meta-Metas gehört sie ausdrücklich nicht. Im Stempel erhält sie *keine* eigene Meta-Meta-Zeile: Ihre Kennung tritt als dynamischer Klammer-Anhang an das *Ende* der bestehenden Mess-Realm-Zeile (Owner-Entscheid E2 und Owner-Antwort Q1 vom 02.08.2026). Der Speiseweg ist hier eintragsgetrieben — die gewählte Kennung samt Version kommt aus der Rahmenwerk-Registry des Mess-Realms und steht deshalb nicht im Typ, anders als beim typgetriebenen System-Hub.

Dynamische Sub-Achsen

Die Registry führt sechs Sweep-Dimensionen (`resource_control_version=1`); alle sind `stage=runtime`.

Dimension	Herkunft (source)	Wert-Typ und Bedeutung
workload	measurement:load_framework	token — das Lastprofil; Einzelquelle ist das Unter-Achsen-Etikett des Last-Rahmenwerks selbst.
thread_count	resource_control_pod	uint — Zahl der Arbeiter-Fäden.
prefetch_distance	resource_control_pod	uint — Vorhol-Abstand.
pool_budget_bytes	resource_control_pod	uint — Speicher-Budget des Pools.
batch_size	resource_control_pod	uint — Stapelgröße je Operation.
inline_threshold_bytes	resource_control_pod	uint — Schwelle der Inline-Ablage.

Ehrlichkeits-Notiz: Die Wiederholungszahl eines Laufs und die Mess-Modi sind bewusst *nicht* in dieser Liste — aus zwei *verschiedenen* Gründen. Die Wiederholungszahl ist weiterhin eine reine Etikett-Dimension ohne übersetzungszeitlichen Anker und wird erst nach ihrer Typisierung reflektierbar. Die Modi dagegen sind inzwischen typisiert: es sind *vier* (Debug-, Mess-, Release- und Vergleichs-Durchführung), getragen von einer übersetzungszeitlichen Registry ohne Laufzeit-Verzweigung, wobei die Vergleichs-Durchführung heute die Wählbarkeit trägt und noch nicht den eigenen Ablauf. Sie fehlen hier dennoch,

weil die Auffächerung dieser Unter-Achse in die Reflexions-Matrix aussteht. Ein handgeschriebener Eintrag wäre in beiden Fällen ein Phantom — die honest-0-Doktrin verbietet ihn (offener Punkt W2-B).

Bausteine

Variante	Beschreibung	Quelle
ycsb	Das YCSB-Lastprofil-Rahmenwerk; freigeschaltet, mit dem Unter-Achsen-Etikett <code>workload</code> . Einziger heute freigeschalteter Baustein der Achse.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.28 Build-PG Seitentyp-Achse (PAGE-TYPE)

Abgrenzung und Konservierung: Diese Sektion beschreibt den `PageKind` der Organ-Welt (`axis_01_page_type`) und ist von der System-Unter-Achse `page` der Ziel-ISA (Seitengrößen-Freigabe 4k/2m/1g) verschieden — beide Größen tragen ähnliche Namen, aber weder gemeinsame Bausteine noch gemeinsame Semantik. Der Katalog bleibt als konservierter Angebots- und Historien-Katalog vollständig erhalten.

Die Seitentyp-Achse definiert die Struktur- und Indexierungsstrategien von Branch-Seiten im Trie-Index: von dicht-indiziertem Direktadressierungsindex (DenseByte), über mehrstufige Dichte-Varianten (ExtendedDense, SparsePatricia) bis zu Pfadkollaps-Optimierungen (Redirect, CustomCache) und B+-artige sortierte Seiten (BPlus). Alle Seitentypen sind Cache-Line-bewusst und implementieren das Concept-Interface `PageTypeStrategy` mit garantierter Austauschbarkeit (F15-Permutation).

Statische Sub-Achsen

ID	Name	Beschreibung
PG1	Struktur-Rolle (Branching)	Klassifizierung der Verzweigungsseiten-Architektur: Branch-Seite mit voller Verzweigung, kollabierter Single-Path-Knoten (Redirect) oder spezielle Cache-optimierte Variante.
PG2	Dichte-Klasse (Density)	Grad der Slot-Auslastung und daraus resultierende Indexierungsstrategie: dicht besiedelt (0-256 Slots), erweitert (>256 Slots) oder sparse mit Patricia-Kompression.
PG3	Pfad-Kollaps-Strategie	Handhabung eindeutiger Restpfade: keine Kompression (Standard), oder Kollaps durch Redirect/CoCo-Technik zur Speicherminimierung.

Bausteine

Variante	Beschreibung	Quelle
DenseBytePageType	Dichte byte-indizierte Verzweigungsseite mit direkter 256-Slot-Adressierung (direct-addressed). Seitentyp 1: direkte Abbildung von Schlüssel-Byte auf Slot-Index ohne Umleitung. Quelle: Adaptive Radix Tree (ART), Leis et al. 2013.	P01

Variante	Beschreibung	Quelle
ExtendedDensePageType	Erweiterte dichte Seite für >256 Einträge mit mehrstufigem Indexierungsschema (z.B. 2-Level-Lookup oder variable k-Byte-Span). Seitentyp 2: Dichte-Klasse für komplexere Slot-Anordnungen. Quelle: Adaptive Radix Tree Erweiterungen, PRT-ART.	P01 (extended)
SparsePatriciaPageType	Spärlich besetzte Seite mit Patricia-Pfadkompression (Single-Bit-Split). Seitentyp 3: Dichte-Klasse mit Bit-Vektor-Traversal für niedrige Branchingness. Quelle: HOT (Height Optimized Trie, Binna et al. 2018, basierend auf Patricia-Trie, Morrison 1968).	P02
RedirectPageType	Kollabierter eindeutiger Restpfad (Single-Path-Knoten). Seitentyp 4: Pfad-Kollaps-Strategie zur Speicheroptimierung bei monopol eindeutigen Nachfolgersequenzen. Weder klassischer Branch noch Leaf, speichert nur das Suffix bis zum nächsten echten Verzweigungs- oder Wert-Knoten. Quelle: CoCo-Trie (Boffa et al. 2024).	P04
CustomCachePageType	Cache-Line-orientierte Custom-Seite mit explizitem Layout-Design für 64-Byte-Alignment. Seitentyp 5: Struktur-Rolle Branch mit Hardware-bewusster Geometrie. Quelle: PRT-ART Eigenentwicklung mit TLB-Offset + Cache-Line-Aligned Memory Layout (Achse 5).	PRT-ART
BPlusPageType	B+-artige sortierte Verzweigungsseite mit Range-Walk-Traversal. Seitentyp 6: Struktur-Rolle Branch, aber mit sortierter Key-Array-Organisation statt Byte-Indexierung. Quelle: Masstree (Mao et al. 2012), B+-Tree-Familie.	P03

Beitragende Quellen: P01, P02, P03, P04, Morrison 1968, PRT-ART [LKN13, BZP⁺18, MKM12, BFTV24, Mor68].

Verwandte Literatur: P05, P06, P10, P11, P12, P13, P20 [FJKN20, SSL⁺21, ZLL⁺18, RR99, RR00, HP03, MBL25].

D.29 Build-HW — Plattform-Hardware-Strategie

Ist-Status (konservierter Katalog): Diese Achse ist aus dem bau-treibenden Angebot der System-Registry ausgeschlossen. Die Registry hält den Ausschluss wörtlich fest: `extension_hardware_system_axis.hpp` ist eine deprecatede Insel, `hardware_isa_system_axis.hpp` ein reiner Host-Deskriptor, der den Bau nicht treibt — beide gehören nicht zum bau-treibenden Angebot. Die statischen Plattform-Profile mit ihren festen Konstanten für Cache-Zeilen-Größe, Seiten-Größe und SIMD-Bit-Breite sind konzeptionell durch die Laufzeit-Hardware-Erkennung des `CacheEngine` Builder abgelöst — eine Fabrik über ISA und Betriebssystem, die Hardware-Werte erhebt statt sie statisch zu setzen (vgl. Abschnitt 3.4.3). Die fünf Sub-Achsen-Einträge (hw1 bis hw5) und die drei Hardware-Profile bleiben als historischer Katalog vollständig erhalten.

Definiert die Plattform-Hardware-Familie (CPU-Architektur, SIMD-Capabilities, Cache-Topologie, Memory-Paging) als Compile-Time-Konstanten für alle Cache-Engine-Algorithmen. Die Achse erfasst, welche Plattformen zur Build-Zeit konfigurierbar sind, und liefert statische Properties (Cache-Line-Größe, Page-Größe, SIMD-Bit-Breite, NUMA-Fähigkeit, HugePage-Fähigkeit) für algorithmengerechte Datenstrukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
hw1_simd_family	12.1 SIMD-Familie	Welche Vector-Instruction-Set wird aktiv genutzt? (AVX2, AVX-512, NEON, SVE2, scalar/none). Bestimmt die maximale Vektorbreite für parallele Operationen innerhalb eines Algorithmus.
hw2_cache_level_targeting	12.2 Cache-Level-Targeting	Welcher Cache-Level wird optimiert (L1-aware, L2-aware, L3-aware, HBM-aware)? Entscheidend für Prefetch-Strategien und Speicher-Layout-Entscheidungen.
hw3_numa_strategy	12.3 NUMA-Strategie	Wie wird NUMA-Topologie berücksichtigt (single-node, multi-node, NUMA-aware-pinning, NUMA-balanced)? Beeinflusst die Allocator-Lokalität und Thread-Pinning-Strategie.
hw4_prefetch_hardware	12.4 Prefetch-Hardware-Instruktionen	Welche expliziten Hardware-Prefetch-Instruktionen sind verfügbar (PREFETCH, PREFETCHNTA, PREFETCHW, none)? Kernbestandteil von Distance-Estimator und path-oriented Prefetch.
hw5_atomic_instruction_family	12.5 Atomic-Befehl-Familie	Welche atomaren Read-Modify-Write-Befehle sind verfügbar (CAS, LL-SC, RMW-extended via HTM/XBEGIN/XEND, none)? Fundamentale Basis für Lock-Free-Concurrency und Optimistic-Lock-Coupling.

Bausteine

Variante	Beschreibung	Quelle
X86_64HardwareProfile	Moderne x86-64 Desktop/Server-Plattform (Intel/AMD). Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=256 (AVX2 konservativ), numa_capable=true, huge_page_capable=true. Basis für desktop- und server-seitige Experimente.	Code/Baseline
Aarch64HardwareProfile	ARM64/AArch64 Server und Mobile Plattformen (Ampere Altra, Apple Silicon M-Serie, Linux ARM). Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=128 (NEON Standard), numa_capable=true, huge_page_capable=true. Ermöglicht Cross-Platform-Vergleiche.	Code/Baseline
GenericHardwareProfile	Konservatives Fallback-Profil für plattform-agnostische Konfigurationen. Build-Time-Konstanten: cache_line_size=64, memory_page_size=4096, simd_width_bits=0 (scalar-only), numa_capable=false, huge_page_capable=false. Lowest-Common-Denominator wenn kein spezialisierter Adapter passt.	Code/Baseline

Quellen: Plattform-Profile als Code/Baseline (keine externen Paper).

D.30 A-Korpus Allokator-Korpus (A01-A23)

Die Allokator-Achse dokumentiert 25 speicherverwaltungs-Strategien (Kennungen A01–A21, die drei Standardbibliotheks-Baselines A22a–A22c sowie A23), die das dynamische Speichermanagement in Such-Workloads beeinflussen. Sie umfasst klassische Lehrbuch-Allokatoren (Buddy, Slab), industrielle Standards (dlmalloc, jemalloc, TCMalloc, mimalloc, snmalloc), spezialisierte Varianten (NUMA-aware, PIM-Malloc, Cache-Aware) und moderne gehärtete Implementierungen (StarMalloc). Alle 25 Wrapper erfüllen das `AllocatorStrategy`-Konzept und ermöglichen systematische Messung des Speichermanagement-Einflusses auf Suchbaum-Performance (F15-Operativität).

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Struktur der freien Speicher-Blöcke: Per-Page-Sharding (mimalloc), Per-Thread-Caching (snmalloc, rpmalloc, TCMalloc), Buddy-Baum (Buddy-Allocator), Single-Global-Free-List (dlmalloc), Per-Heap-Locks (Hoard).
AA2	Size-Class-Schema	Klassifizierung von Block-Größen: Slab-pro-Objektgröße (Slab), dlmalloc Bins, jemalloc Size-Classes, scallo Spans, Buddy-Power-of-2.
AA3	Thread-Lokalität	Grad der Thread-Isolation: tcmalloc thread-local-caches, rpmalloc per-thread-spanning, snmalloc message-passing, Hoard per-heap-locks.
AA4	Synchronisation	Concurrency-Mechanismus: Lock-Free CAS (Michael, LRMalloc), Per-Heap-Locks (Hoard), Wait-Free (Crystalline), Mutex-basiert.
AA5	Allokations-Richtlinie	Routing von Allokationsanfragen: NUMA-origin-aware (NUMAalloc), Cache-Set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc).
AA6	Rückgewinnung	Freigabe gelöschter Blöcke: Magazine-Cache (Slab), Page-Heap-Release (TCMalloc), Lock-Free-Reclamation (LRMalloc), Deferred-Free (mimalloc).
AA7	Fragmentierungsstrategie	Vermeidung von Fragmentierung: dlmalloc Coalescing, Object-Coloring (Slab), Buddy-Splitting, Page-Local-Sharding (mimalloc), Span-Pooling (scallo).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierung. Drei-Ebenen-Struktur: Thread-lokale Heaps, globale Heap für Cross-Thread-Migration, False-Sharing-Reduktion durch segregierte Allocations-Anfragen.	A01 [BMBW00] (Berger/McKinley/Blumofe/Wilson, ASPLOS 2000, 10.1145/378993.379232) + Code/Baseline
SlabAllocator (A02)	Object-Caching Kernel-Allocator. Slab-pro-Objektgröße mit Magazine-Cache pro CPU (Bonwick 1994/2001). Klassische Solaris-Kernel-Implementierung für vorallokierte Objekt-Pools.	A02 [Bon94] (Bonwick, USENIX 1994) + Code/Baseline
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allocator nach Michael 2004. Hazard-Pointer-geschützte Lock-Free-Datenstrukturen, LGPL-Re-Implementierung mit IBM-Patent-Hintergrund.	A03 [Mic04b] (Michael, PLDI 2004, 10.1145/996841.996848) + Code/Baseline
MimallocAllocator (A04)	Free-List-Sharding (3 Free-Lists pro Page). Microsoft Research 2019: deferred_free Hooks, temporal Cadence, Page-lokale Sharding. is_original: MIT-Direktbindung.	A04 [LZM19] (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
JemallocAllocator (A05)	Arena-based Size-Class-Allocator. Evans 2006 / Free BSD. Pro-CPU Arenas mit Tcache-Layer für Thread-lokale Caches. Facebook-Standard. is_original: BSD-2-Clause-Direktbindung.	A05 [Eva06] (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_original (github.com/jemalloc/jemalloc, BSD-2-Clause)
TCMallocAllocator (A06)	Thread-Cache + Central-Cache + Page-Heap. Google gperftools 2005. Später TEMERAIRE (OSDI 2021) mit Warehouse-Scale-Optimierungen.	A06 [HKT ⁺ 21] (TCMalloc 2005; TEMERAIRE, OSDI 2021) + Code/Baseline

Variante	Beschreibung	Quelle
SnmallocAllocator (A07)	Message-Passing-Allocator. Paul Liétar et al. ISMM 2019: Free-Operationen via Lock-Free-Queue an Allocating-Thread, Cross-Thread-Lock-Contention-Vermeidung. is_original: MIT-Direktbindung.	A07 [LDPB19] (Liétar, ISMM 2019, 10.1145/3315573.3329980) + is_original (github.com/microsoft/snmalloc, MIT)
ScallocAllocator (A08)	Span-basiert, Virtual-Spans-Technik. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-skalierbar mit niedriger Fragmentierung.	A08 [AKLS15] (Aigner et al., OOPSLA 2015) + Code/Baseline
NUMAAllocator (A09)	NUMA-Origin-Aware Allocator. UTSASRG ISMM 2023. Knoten-Hinweis für lokale NUMA-Allokation, Multi-Knoten-Systeme optimiert.	A09 [YZZ ⁺ 23] (Yang et al., ISMM 2023) + Code/Baseline
RPmallocAllocator (A10)	Per-Thread Span-Caching. Mattias Jansson 2017, OSS ohne Paper. Sonderfall: Pflicht-Init (rpmalloc_initialize, rpmalloc_thread_initialize). is_original: Public-Domain/MIT-Direktbindung.	Code/Baseline (github.com/mjansson/rpmalloc, Public-Domain/MIT) + is_original
LRmallocAllocator (A11)	Lock-Free + Hazard-Pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Moderner Lock-Free-Allocator mit ABA-Protection. is_original: MIT-Direktbindung.	A11 [LR18] (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + is_original (github.com/ricleite/lrmalloc, MIT)
CAMAAllocator (A12)	Cache-Aware Memory Allocator. Herter/Backes/Hauptenthal/Reineke ECRTS 2011 (Saarland). Vorhersagbare Cache-Behavior, keine öffentliche Code-Referenz.	A12 [HBHR11] (Herter et al., ECRTS 2011) + Code/Baseline
StarMalloc Allocator (A13)	Formal Verified Hardened Allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel-formale Verifikation, Apache-2.0-Lizenz.	A13 [RFP24a] (Reitz et al., OOPSLA 2024) + Code/Baseline
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hupage-aware Variante, Marketing-Variante ohne eigenständiges Artefakt.	A14 [HKT ⁺ 21, ZGV ⁺ 24] (Hunter OSDI 2021 / Zhou ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, Scalable, Lock-Free Memory Allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid-Ansatz für Skalierbarkeit, keine öffentliche Code-Referenz.	A15 [LYTL19] (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00064) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-In-Memory Allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Speicherverteilung über Rechenknoten, PIM-Hardware-optimiert.	A16 [VIA26] (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline
Crystalline Reclamation (A17)	Wait-free bzw. lock-/wait-free Safe-Memory-Reclamation-Familie für nicht-blockierende Datenstrukturen. Kein eigenständiger malloc/free-Allokator, sondern Reclamation-Policy.	A17 [NR21, NR24] (Nikolaev/Ravindran, DISC 2021 / PLDI 2024) + Code/Baseline
ExgenAllocator (A18)	Single-Threaded Specialized Allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). „Old is Gold“-Optimierungen für Einthread-Workloads, keine öffentliche Code-Referenz.	A18 [LJY25] (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy System (Power-of-2-Splitting). Knuth TAOCP Vol.1 (1968), CACM 1965. Klassischer Lehrbuch-Allocator, Binärbaum-Splitting ohne OSS-Code-Referenz.	A19 [Kno65] (Knuth/Carlson, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea Malloc (Bins-based). Lea 1987–2012, Public-Domain (CC0). Klassiker seit 1987, Bins-Size-Class-Schema, älteste produktive Implementierung. is_original: Public-Domain-Direktbindung.	A20 [Lea96] (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + is_original (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	ptmalloc2 (glibc malloc). Wolfram Gloger 1990er-2000er. LGPL-2.1+ Copyleft, Linking blockiert (no is_original). Standard-Basis für glibc-basierte Systeme.	Code/Baseline (malloc.de/en/) + License restriction

Variante	Beschreibung	Quelle
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis), portabler Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen ohne Vendor-Linking.	Code/Baseline (C++ Standard Library) + Baseline
PmrResource Allocator (A22b)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources Standard-Bibliothek, Fallback auf new/delete-Allokator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22c)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Eigener Size-Class-Pool, F15-operativ (erste Wrapper mit eigenem Speicherhaltungsverhalten ohne Vendor-Linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative
VmemMagazines Allocator (A23)	Vmem + Magazines (Slab-Erweiterung). Bonwick 2001 USENIX ATC. Per-CPU-Magazine für Skalierung, Kernel-Klassiker ohne öffentliche Code-Referenz.	A23 [BA01] (Bonwick, USENIX ATC 2001, usenix.org/.../bonwick.html) + Code/Baseline

Beitragende Quellen: je Allokator A01–A23 einzeln in der Quellenspalte der Bausteine-Tabelle nachgewiesen.

D.31 Achsen-Inventar aus den generierten Registries

Die folgende Tabelle wird nicht von Hand gepflegt: der Anhang-Generator (`Code/08_appendix_generator`) leitet sie aus den drei generierten Achsen-Registries des Codes ab — Organ-, System- und Mess-Realm — und schreibt sie je Lauf nach `anhang/de/tabellen/`. Sie ist damit eine Ableitung aus der übersetzungszeitlichen Reflektion und kein zweiter, gepflegter Stand neben den Registries. Enthalten die Registries keine Achse, entsteht bewusst keine Datei; es wird keine leere Tabelle erfunden.

(Das Achsen-Inventar erscheint nach dem nächsten Lauf des Anhang-Generators.)

E Architekturentscheidungen

Dieser Anhang sammelt die tragenden Entwurfsentscheidungen der Arbeit als kompakte *Architecture Decision Records* (ADRs): je Entscheidung der Kontext, in dem sie fiel, die Entscheidung selbst und ihre wichtigste Konsequenz. Ziel ist die schnelle Nachvollziehbarkeit — *warum* der Apparat so gebaut ist, wie ihn die Kapitel beschreiben; die ausführliche Herleitung und Begründung liegt jeweils im per *Beleg* referenzierten Abschnitt des Haupttextes und wird hier nicht wiederholt.

ADR-1 — C-stabile Modulgrenze statt C++-Modul-ABI

Kontext: Jede Permutation wird als eigenständige Tier-Binary kompiliert und zur Laufzeit geladen; C++23-Module strukturieren zwar die Übersetzungseinheiten, garantieren für sich genommen aber keine portable ABI-Stabilität [Xu24]. **Entscheidung:** Eine einheitliche C-Modulgrenze (`comdare_create_anatomy() → IAnatomyBase`) exportiert je Permutation genau ein Lebewesen; dahinter liegt die dreischichtige Abstraktionskette `CacheEngine → ExecutionEngine → SearchEngine`. **Konsequenz:** Die ABI-Stabilität stellt die C-Grenze her, nicht das Modulsystem; der ABI-Vertrag wird je Modul prüfbar (opt-in ABI-Verify). **Beleg:** Abschnitte 1.4, 3.3.3 und 4.3.1.

ADR-2 — Compile-Time-Permutation über Typlisten statt Laufzeit-Adaption

Kontext: Achtzehn Organ-Kompositions-Achsen zählen schon im kanonischen Vermessungskatalog $2^{17} = 131072$ Binary-Identitäten je System-Permutation — eine davon, das Persistenz-Ziel, auf ihren In-Memory-Vertreter gepinnt —, und der rohe Varianten-Produktraum über alle Achsen-Kataloge samt Unter-Achsen liegt Größenordnungen darüber; der Hot-Path darf weder `virtual`-Aufrufe noch Laufzeit-Switches tragen. **Entscheidung:** Die Permutation läuft übersetzungszeitlich über Boost.MP11-Typlisten (`mp_list`) und concept-geprüftes `if constexpr (resolve_baustein)` — kein Runtime-Tag, kein `std::variant`; der literale `std::variant`-Katalog besteht nur als historische Elf-Achsen-Vorstufe (`baustein_variants`-Fallback). **Konsequenz:** Die Achsen-Abstraktion bleibt eine zero-cost abstraction. **Beleg:** Abschnitte 1.4, 3.3.3, 4.3 und 3.4.5.

ADR-3 — Drei-Ebenen-Modell als die *eine* Architektur

Kontext: Anatomie-Metapher und technische Engine-Sicht drohen zu parallelen Modellbäumen auseinanderzulaufen. **Entscheidung:** Es gibt genau eine Hierarchie ohne Parallel-Bäume — Gattung (Außen-Interface, das Prüf-Dock) → Gattungs-Unterklasse (fester Achsen-Satz) → Achsen-Organe — unter `IExecutionEngine` als gemeinsamer messbarer Wurzel; eine eigene Gattung entstünde erst, wenn sich der Basis-Achsen-Satz grundlegend unterschiede. **Konsequenz:** „SearchEngine“ ist nur die ABI-Laufzeit-Sicht desselben Lebewesens, kein zweites Modell. **Beleg:** Abschnitte 3.3, 3.3.2 und 4.3.1.

ADR-4 — Durchreich-Doktrin und erzwungene Vollbelegung

Kontext: Nicht-baumartige Verfahren (etwa die Hash-Gegenprobe) tragen baum- und trie-spezifische Achsen wie Pfadkompression oder Knotentyp nicht sinnvoll. **Entscheidung:** Solche Achsen fallen nicht weg, sondern werden als Durchreich-Varianten (None) belegt; das `IsComposition`-Concept erzwingt übersetzungszeitlich die Belegung aller achtzehn Organe, sodass keine Composition mit offener Achse übersetzt. **Konsequenz:** Nicht-baumartige Verfahren erfordern keine eigene Gattung, sondern ein eingeschränktes Achsen-Subset derselben Gattung. **Beleg:** Abschnitte 3.2, 3.3.2 und 4.3.

ADR-5 — Schichtenmodell E4–E0 mit eingefrorenen Verträgen und Contract-Tests

Kontext: Zwei Implementierungs-Inkrementen über mehrere Ebenen gleichzeitig mussten vollständig zurückgenommen werden; bei einem Produktraum, der jede vollständige Materialisierung ausschließt, sind Mehr-Ebenen-Änderungen praktisch nicht abnehmbar. **Entscheidung:** Die Experiment-Pipeline gliedert sich in die Schichten E4–E0 (plus Querschnitt M); jede Schicht wird einzeln fertiggestellt, mit explizit eingefrorenem Interface-Vertrag zur Nachbar-Ebene, und gegen einen übersetzungszeitlichen Ersatz (Fake) der Nachbar-Ebene contract-getestet. **Konsequenz:** Jede Ebene ist einzeln abnehmbar, bevor die nächste beginnt; die Grenze E3→E2 ist bereits golden-getestet. **Beleg:** Abschnitte 3.3.4 und 4.4.

ADR-6 — honest-0-Doktrin: ehrliche Null bzw. ehrliche Leere

Kontext: Nicht wirklich erhebbare Werte (Zähler ohne Hardware-Zugriff, Achsen ohne echten Laufzeit-Parameter) laden zu plausiblen Phantomwerten ein, die die Heuristik-Ableitung systematisch vergiften würden. **Entscheidung:** Das System meldet ehrlich ungültig oder leer statt eines Phantomwerts — per-Ereignis-Gültigkeits-Flags je Messung; die Wall-Clock-Quelle verweigert es etwa, ihren Mittelwert als Perzentil zu etikettieren. **Konsequenz:** Lücken bleiben sichtbar und gezielt schließbar; die Doktrin ist in den Bausteinen einprogrammiert statt nur dokumentiert. **Beleg:** Abschnitte 3.4 und 4.5.

ADR-7 — Drei-Repository-Split mit orthogonalen Subsystemen

Kontext: Mess-Apparat, Prüfling und Auswertung haben getrennte Verantwortlichkeiten und Lebenszyklen. **Entscheidung:** Drei Repositorien — `comdare-cache-engine` (wie gemessen wird), `comdare-prt-art` (der Prüfling) und `probst-Diplomarbeit-cache-engine` (was getestet wird) —, `comdare-cache-engine` dabei mit Pitchfork-Quellbaum-Layout; die vier Subsysteme (Mess-Treiber, CacheEngineBuilder, CacheEngine, Prüfling) liegen orthogonal zu dieser Aufteilung. **Konsequenz:** Builder und CacheEngine teilen sich ein Repo als ausführbare Datei bzw. Bibliothek; die Rollen bleiben klar getrennt. **Beleg:** Abschnitt 4.1.

ADR-8 — Familienweise Adapter-Fallbacks für eigenständig lauffähige Builds

Kontext: Fremd-Code (Allokator-Vendors, SOTA-Bausteine) ist nicht auf jeder Plattform und in jedem Build verfügbar oder aktiviert. **Entscheidung:** Übersetzungszeitliche Flag-Familien schalten Adapter zu oder ab, jeweils mit sicherem, familienweise gewähltem Fallback: Allokator-Adapter ersetzen den fehlenden Vendor durch `std::malloc/std::free`, Datenstruktur-Adapter durch einen Stub, der bei Benutzung kontrolliert eine `std::runtime_error`-Ausnahme wirft. **Konsequenz:** Der Build bleibt ohne jede Fremd-Aktivierung eigenständig lauffähig. **Beleg:** Abschnitt 4.3.

ADR-9 — Mess-Gating und vier Auslieferungsvarianten

Kontext: Mess- und Observer-Kopplungen kosten Zeit und dürfen den Produktiv-Pfad nicht belasten. **Entscheidung:** Die Mess-Kopplungen des Laufzeit-Adapters sind über `COMDARE_MEASUREMENT_ON` gegated; daraus entstehen vier Auslieferungsvarianten derselben Permutation (Debug mit Observer, Messung mit Aggregator, Release ohne Mess-Kopplungen sowie Compare als bereits wählbares, bau-gleich zu Release gehaltenes Etikett für den künftigen Replay-Vergleich), und den Mess-Aggregator führt der `ExperimentDriver` host-seitig. **Konsequenz:** Der Produktiv-Pfad bleibt mess-frei und trägt keinen Mess-Overhead. **Beleg:** Abschnitte 4.1 und 4.3.2.

ADR-10 — `std::map`-Vertrag als gemeinsamer Nenner mit Konformitäts-Gatter

Kontext: Strukturell verschiedene Suchalgorithmen sind nur über eine gemeinsame, gut verstandene Außen-Schnittstelle fair vergleichbar. **Entscheidung:** Alle Verfahren werden auf den C++-Vertrag der geordneten Map `std::map<Key, Value>` abgebildet (parallel `std::vector` als Container-Vergleichsfall); ein Konformitäts-Gatter prüft diesen Vertrag vor jeder Messung. **Konsequenz:** Nur Vergleichbares wird verglichen; die Achsen beschreiben ausschließlich das innere Verhalten hinter der festen Schnittstelle. **Beleg:** Abschnitt 3.4.

ADR-11 — Entwurfsmuster-Auswahl mit Negativliste; Command nur im Hybrid-Modus

Kontext: Der Apparat ist bewusst eine Komposition benannter Entwurfsmuster; unklare Muster-Rollen würden die Architektur verwässern. **Entscheidung:** Verwendet werden u. a. *Strategy*, *Abstract Factory*, *Composite*, *Iterator*, *Adapter*, *Observer*, *Decorator*, *Memento* und CRTP; nicht verwendet werden *Singleton*, *Flyweight* und *Bridge*, und das *Command*-Muster tritt ausschließlich als Befehls-Kanal des Hybrid-Modus auf. **Konsequenz:** Jedes Muster hat genau eine benannte Rolle in der Verkabelung des Apparats. **Beleg:** Abschnitte 3.3.3 und 3.6.2.

ADR-12 — Mess-Regime-Doppel: immutables Talos und root-Linux

Kontext: Produktions-Treue und Mess-Tiefe stehen im Konflikt: Ein produktionsnahes System verwehrt gerade den Hardware-Zähler-Zugriff, den tiefe Messungen brauchen. **Entscheidung:** Auf den Produktions-Zielmaschinen wird jede Messung unter zwei Betriebssystem-Regimes erhoben: einem immutablen Betriebssystem (Talos) als Produktiv-Spiegel und einem root-Linux mit vollem Hardware-Zähler-Zugriff (`perf/MSR`). **Konsequenz:** Die zeit- und observer-basierten Kategorien laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit; die PMC-Kategorien werden nur unter dem privilegierten Regime erhoben. **Vollzugsstand:** Die Entscheidung gilt, ist aber erst zur Hälfte eingelöst — die berichteten Messungen stammen sämtlich vom root-Linux-Regime; der Talos-Spiegel bleibt Anforderung und wird nach der Abgabe erhoben. **Beleg:** Abschnitte 5.1 und 1.4.

F Vergleichsinterfaces: `std::map` und `std::vector`

Dieser Anhang führt — ergänzend zu Abschnitt 3.4 — die vollständige Auflistung aller Standard-Interface-Funktionen der beiden Vergleichshüllen `std::map` (geordnet-assoziativ) und `std::vector` (Sequenz) nach dem Stand von C++23 mit ihrer offiziell erwarteten Semantik und ihrer Komplexitäts-Garantie gemäß dem ISO-C++-Standard [ISO24]. Er dient zugleich als zentrale Referenz für die Weiterentwicklung der Cache-Engine: Da die Suchalgorithmus-Hülle ihr Innenleben durch dynamisch austauschbare Achsen-Algorithmen ersetzt (Abschnitt 3.4), wird je Standard-Funktion erklärungsbedürftig, welches Verhalten der äußere Vertrag garantiert.

Variadische Abbildung. Ein Template-Parameter stellt die `std::vector`-API bereit, zwei Parameter die `std::map`-API und $N > 2$ eine `std::map<K, std::tuple<...>` (Abschnitt 4.3.1); die geteilten Funktionen (Iteration, `size/empty/clear`, `swap`, `get_allocator`) tragen in beiden Hüllen dieselbe Semantik.

Zusammengesetzte Funktionen. Einige Funktionen ergeben sich „unter der Haube“ als Komposition primitiverer Operationen — etwa `operator[]` als `lower_bound` mit anschließendem Default-Einfügen bei fehlendem Schlüssel oder `at` als `find` mit Wurf einer `std::out_of_range`-Ausnahme bei Fehlen —; solche Fälle sind in der Spalte „Anmerkung“ gekennzeichnet. Die Spalte „seit“ nennt die einführende Standard-Fassung. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Konformitäts-Teilmenge. Das Konformitäts-Gatter (Abschnitt 3.4) prüft vor jeder Messung eine Kern-Teilmenge der `std::map`-Operationen, damit nur Vergleichbares verglichen wird; sie ist in der `std::map`-Tabelle als solche gekennzeichnet.

F.1 `std::map` (geordnet-assoziativ)

Tabelle F.1 listet die vollständige Schnittstelle von `std::map<Key, T, Compare, Allocator>` nach C++23. Ein † markiert die Kern-Operationen, die das Konformitäts-Gatter (Abschnitt 3.4) vor jeder Messung prüft, damit nur Vergleichbares verglichen wird. Komplexitäten beziehen sich auf $n = \text{size}()$; bei Bereichs-Operationen ist N die Zahl der Eingabe-Elemente.

Tabelle F.1: Vollständige `std::map`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			

Funktion	Offizielle Semantik	Komplexität	seit
(Konstruktor)	leer; aus <code>Compare/Allocator</code> ; Bereich; Kopie/Move (auch allokatoren-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ bis $O(N \log N)$	C++98
(Destruktor)	zerstört alle Elemente und Knoten	$O(n)$	C++98
<code>operator=</code>	Kopier-/Move- <code>/initializer_list</code> -Zuweisung mit Allokator-Propagation	$O(n)$	C++98
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
<code>at</code> †	Referenz auf den Mapped-Wert; wirft <code>std::out_of_range</code> bei Fehlen	$O(\log n)$	C++11
<code>operator[]</code> †	Referenz; fügt bei Fehlen wertinitialisiert ein (wirft nie)	$O(\log n)$	C++98
Iteratoren			
<code>begin/end</code> †	Iterator auf erstes Element bzw. Ende, in Schlüsselordnung (<code>cbegin/cend</code> ab C++11)	$O(1)$	C++98
<code>rbegin/rend</code>	Rückwärts-Iteratoren (<code>crbegin/crend</code> ab C++11)	$O(1)$	C++98
Kapazität			
<code>empty</code> †	leer-Test (<code>begin() == end()</code>); <code>[nodiscard]</code> ab C++20	$O(1)$	C++98
<code>size</code>	Anzahl der Elemente	$O(1)$	C++98
<code>max_size</code>	theoretische Maximalgröße	$O(1)$	C++98
Modifikatoren			
<code>clear</code> †	alle Elemente entfernen	$O(n)$	C++98
<code>insert</code> †	Element(e) ohne Überschreiben (Wert, P&&, Hint, Bereich, <code>initializer_list</code> , Node-Handle ab C++17)	$O(\log n)$; Hint amort. $O(1)$	C++98
<code>insert_range</code>	Eingabe-Bereich einfügen, keine Überschreibung	$O(N \log(n+N))$	C++23
<code>insert_or_assign</code> †	einfügen oder vorhandenen Wert zuweisen; <code>bool</code> meldet Einfügung	$O(\log n)$	C++17
<code>emplace</code> †	in-place konstruieren, nur falls Schlüssel fehlt (sonst verworfen)	$O(\log n)$	C++11

Funktion	Offizielle Semantik	Komplexität	seit
<code>emplace_hint</code>	wie <code>emplace</code> , positionsnah zum Hint	$O(\log n)$; amort. $O(1)$	C++11
<code>try_emplace†</code>	wie <code>emplace</code> , rührt die Argumente bei vorhandenem Schlüssel nicht an	$O(\log n)$	C++17
<code>erase†</code>	nach Iterator/Bereich/Schlüssel (Schlüssel: 0 oder 1 entfernt)	Iter. amort. $O(1)$; Key $O(\log n)$	C++98
<code>swap</code>	Inhalt und Comparator tauschen; Iteratoren bleiben gültig	$O(1)$	C++98
<code>extract</code>	Element als Node-Handle entnehmen (ohne Kopie/Move)	Iter. amort. $O(1)$; Key $O(\log n)$	C++17
<code>merge</code>	Knoten aus Quell-Map splicen; vorhandene Schlüssel bleiben dort	$O(N \log(n+N))$	C++17
Suche			
<code>find†</code>	Iterator auf den Schlüssel oder <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	Anzahl (0 oder 1)	$O(\log n)$	C++98
<code>contains†</code>	Mitgliedschaft (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	erster Schlüssel $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	erster Schlüssel $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	Paar <code>[lower_bound, upper_bound)</code>	$O(\log n)$	C++98
Beobachter			
<code>key_comp†</code>	Kopie des Schlüssel-Comparators	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (vergleicht Paare über den Schlüssel)	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und paarweise gleiche Elemente	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich; synthetisiert <code><=>=></code>	$O(n)$	C++20
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions-Leitfäden	CTAD aus Bereich/ <code>initializer_list</code> / <code>from_range</code>	— (Compile-Zeit)	C++17

Die vor C++20 eigenständigen Vergleichsoperatoren `<`, `<=`, `>`, `>=` werden seit C++20 aus `operator<=` synthetisiert, `!=` aus `operator==` umgeschrieben. Die heterogenen Überladungen von `operator[]`, `at`, `try_emplace` und `insert_or_assign` (transparenter Schlüssel) gehören erst zu C++26 und bleiben hier ausgeklammert; die C++23-Neuzugänge sind genau der `from_range`-Konstruktor, `insert_range` und die transparenten Schlüssel-Überladungen von `erase/extract`.

F.2 `std::vector` (Sequenz)

Tabelle F.2 listet die vollständige Schnittstelle von `std::vector<T, Allocator>` nach C++23 ($n = \text{size}()$). Sämtliche Member sind seit C++20 `constexpr`.

Tabelle F.2: Vollständige `std::vector`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			
(Konstruktor)	<code>leer/Allocator; n × Default/Wert;</code> Bereich; Kopie/Move (auch allokator-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ oder $O(n)$	C++98
(Destruktor)	zerstört alle Elemente, gibt Speicher frei	$O(n)$	C++98
<code>operator=</code>	Kopie/Move/ <code>initializer_list</code>	$O(n)$	C++98
<code>assign</code>	Inhalt durch $n \times$ Wert/Bereich/ <code>initializer_list</code> ersetzen	$O(n)$	C++98
<code>assign_range</code>	Inhalt durch Eingabe-Bereich ersetzen	$O(n)$	C++23
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
<code>at</code>	geprüfter Zugriff; wirft <code>std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	ungeprüfter Zugriff (UB außerhalb des Bereichs)	$O(1)$	C++98
<code>front/back</code>	Referenz auf erstes/letztes Element (UB bei leer)	$O(1)$	C++98
<code>data</code>	Zeiger auf den zusammenhängenden Speicher (nicht bei <code>vector<bool></code>)	$O(1)$	C++11

Iteratoren

Funktion	Offizielle Semantik	Komplexität	seit
<code>begin/end</code> , <code>rbegin/rend</code>	Vorwärts-/Rückwärts-Iteratoren in Indexordnung (<code>c...</code> -Varianten ab C++11)	$O(1)$	C++98
Kapazität			
<code>empty/size/</code> <code>max_size</code>	leer-Test, Elementanzahl, Maximalgröße	$O(1)$	C++98
<code>reserve</code>	Kapazität $\geq n$; realloziert und invalidiert; wirft <code>length_error</code> bei $n > \text{max_size}()$	$O(n)$	C++98
<code>capacity</code>	aktuell allozierte Kapazität	$O(1)$	C++98
<code>shrink_to_fit</code>	nicht-bindende Reduktion der Kapazität auf <code>size()</code>	$O(n)$	C++11
Modifikatoren			
<code>clear</code>	alle Elemente entfernen (Kapazität bleibt)	$O(n)$	C++98
<code>insert</code>	vor Position: Wert/ $n \times$ /Bereich/ <code>initializer_list</code> ; verschiebt den Schwanz	$O(n)$	C++98
<code>insert_range</code>	Eingabe-Bereich vor Position einfügen	$O(n)$	C++23
<code>emplace</code>	Element in-place vor Position konstruieren	$O(n)$	C++11
<code>erase</code>	Position/Bereich entfernen, Schwanz nachrücken	$O(n)$	C++98
<code>push_back</code>	Element anhängen (Kopie/Move)	amort. $O(1)$	C++98
<code>emplace_back</code>	in-place anhängen; Referenz-Rückgabe ab C++17	amort. $O(1)$	C++11
<code>append_range</code>	Eingabe-Bereich anhängen	amort. $O(n)$	C++23
<code>pop_back</code>	letztes Element entfernen (UB bei leer)	$O(1)$	C++98
<code>resize</code>	auf n kürzen oder mit Default/Wert auffüllen	$O(n)$	C++98
<code>swap</code>	Inhalt und Kapazität mit anderem Vektor tauschen	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und elementweise Gleichheit	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich	$O(n)$	C++20

Funktion	Offizielle Semantik	Komplexität	seit
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase</code>	entfernt alle Elemente gleich dem Wert; gibt die Anzahl zurück	$O(n)$	C++20
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions- Leitfäden	CTAD aus Bereich/ <code>from_range</code>	— (Compile-Zeit)	C++17

Die Spezialisierung `std::vector<bool>` packt Wahrheitswerte bitweise: Sie besitzt kein `data()` und keine Zusammenhangs-Garantie, liefert über `operator[]/at/front/back` einen Proxy-Typ `reference` statt `bool&` und bietet zusätzlich `flip()` (auf Vektor- und auf `reference`-Ebene), ein statisches `swap(reference, reference)` sowie eine `std::hash`-Spezialisierung. Die Vergleichsoperatoren verhalten sich wie bei `std::map`.

F.3 Zusammengesetzte Funktionen und Konformitäts-Teilmenge

Mehrere Schnittstellen-Funktionen sind „unter der Haube“ Kompositionen primitiverer Operationen (Tabelle F.3); die Tabelle beschreibt den ISO-Standardvertrag. Die eigene Mess-Hülle weicht an zwei Stellen bewusst davon ab, weil der ABI-Konformitätsvertrag `noexcept` verlangt und eine werfende Hülle als nicht-konform gilt: `operator[]` ist dort ein nicht-mutierendes `std::optional`-Lesen (der Schreibpfad läuft über `set()/insert_or_assign`), und `at` wirft nicht (`optional`-/Status-Rückgabe). Für diese Arbeit ist das die zentrale Beobachtung: Sämtliche Map-Zugriffe und -Modifikatoren ruhen auf einem einzigen Ordnungs-Such-Primitiv (`lower_bound/find`); eine Achsen-Implementierung muss daher nur eine korrekte Ordnungs-Suche samt Knoten-Verkettung bereitstellen, der Rest ergibt sich als feste Komposition — genau die Austauschbarkeit, deren Vergleich der zentrale Mess-Akt ist (Abschnitt 3.4). Das Konformitäts-Gatter prüft die in Tabelle F.1 mit † markierte Kern-Teilmenge.

Tabelle F.3: Zusammengesetzte Funktionen („unter der Haube“)

Funktion	Zusammensetzung	Anmerkung
<code>map::</code>	<code>lower_bound</code> + Default-Einfügen	mutiert bei Fehlen;
<code>operator[]</code>	bei Fehlen → Referenz	wirft nie
<code>map::at</code>	<code>find</code> + Wurf	einzigster werfender
	<code>std::out_of_range</code> bei Fehlen	Zugriff; mutiert nie

Funktion	Zusammensetzung	Anmerkung
<code>map::insert / emplace</code>	Ordnungs-Suche + Knoten verlinken nur bei fehlendem Schlüssel	kein Überschreiben; <code>emplace</code> kann Argumente bei Treffer dennoch konsumieren
<code>map::try_ emplace</code>	wie <code>emplace</code> , aber Argumente bei Treffer unangetastet	bewahrt move-only- Argumente
<code>map::insert_ or_assign</code>	Ordnungs-Suche + Einfügen oder Zuweisen	überschreibt (anders als <code>insert</code>)
<code>map::count / contains / equal_range</code>	reduzieren auf <code>find</code> bzw. <code>lower_bound+upper_bound</code>	bei Unikat-Map stets 0 oder 1
<code>vector:: push_back / emplace_back</code>	ggf. Wachstum (Realloc + Umzug) + Konstruktion am Ende	versteckte Kosten + Invalidierung beim Realloc
<code>vector::insert / emplace</code>	ggf. Wachstum + Schwanz verschieben + Einsetzen	zusätzlicher $O(n)$ -Shift
<code>vector::resize</code>	anhängen (Default/Wert) oder Schwanz kürzen	bidirektional; verkleinert die Kapazität nicht
<code>vector::assign</code>	<code>clear</code> + Bereich/ $n \times$ einfügen	Kapazität wird ggf. wiederverwendet

Funktionen, die beide Hüllen dem Namen nach *teilen* — `size`, `empty`, `clear`, die Iterator-Familien, `swap`, `get_allocator` sowie `insert/emplace/erase` —, tragen in der Map Schlüssel- und im Vektor Positions-/Index-Semantik; `operator[]` etwa fügt in der Map ein, ist im Vektor hingegen ein ungeprüfter Indexzugriff. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) hinter diesen Verträgen dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Danksagung

Mein Dank gilt zuerst apl. Prof. Dr.-Ing. Dirk Habich — einem bescheidenen, disziplinierten Menschen — für die engagierte Betreuung dieser Arbeit: Trotz stets knapper Zeit hat er sich immer Zeit genommen für präzise Rückfragen, klare Prioritäten und den Freiraum, diese Arbeit weitgehend eigenständig zu entwickeln, mit punktueller Unterstützung dort, wo sie gebraucht wurde. Dr.-Ing. Alexander Krause danke ich für die Begleitung dieser Arbeit als zweiter Betreuer. Schließlich danke ich dem Team von UltiHash: Die lehrreichen Jahre in diesem Startup haben viele der Erfahrungen und Fragestellungen geprägt, die diese Arbeit nun technisch umsetzt.

Erklärungen zum Urheberrecht

Sperrvermerk (eingeschränkt)

Diese Arbeit selbst unterliegt keinem Sperrvermerk. Ein eingeschränkter Sperrvermerk betrifft allein den Software-Kern der `comdare-cache-engine`: Er ist zur Begutachtung einsehbar und testbar; eine geschäftliche Nutzung oder Weitergabe über die Begutachtung hinaus bedarf einer gesonderten Lizenz der Rechteinhaberin, der BEP Venture UG (haftungsbeschränkt), der Inhaberin der eingetragenen Marke Comdare. Ausgenommen ist die Forschung: Für Forschungszwecke ist der Software-Kern frei nutzbar. Die in dieser Arbeit beschriebenen Experimente verwenden die Cache-Engine als Anwender über frei wählbare Testparameter und sind von der Einschränkung nicht berührt.

Lizenzen und Copyright

Der Code des Diplomarbeits-Repositories (Mess-Treiber und Auswertungs-Pipeline) sowie der Prüfling PRT-ART stehen unter Apache-2.0. Der Software-Kern der `comdare-cache-engine` steht unter einem dualen Lizenzmodell: Für Forschung und Lehre ist der Programmcode frei nutzbar; jede geschäftliche Nutzung bedarf einer gesonderten Lizenz der Rechteinhaberin. Die Rechte am Software-Kern liegen bei der BEP Venture UG (haftungsbeschränkt), Marke Comdare. Der Text dieser Arbeit unterliegt dem privaten Copyright von Benjamin-Elias Probst.

